운영체제

- 문장별 근거 · 인쇄 점검

목차

검증 범위와 인쇄 점검 3
부 도입 4
제1장. 서론 5
제2장. 운영체제 구조 26
제3장. 프로세스 28
제4장. 스레드와 병행성 50
제5장. CPU 스케줄링 68
제6장. 동기화 도구들 98
제7장. 동기화 예제 112
제8장. 교착 상태 124
제9장. 메인 메모리 136
제10장. 가상 메모리 152
제11장. 대용량 저장장치 구조 174
제12장. 입출력 시스템 196
제13장. 파일 시스템 인터페이스 215
제14장. 파일 시스템 구현 234
제15장. 파일 시스템 내부구조 252
제7부 도입. 보안과 보호 265

검증 범위와 인쇄 점검

문장·목록·표의 항목·코드·수식 3327개에 근거 번호와 원문의 절·인쇄 쪽수·PDF 면 번호를 연결했습니다. 문장의 원문 위치는 기존 상세 원고의 절별 근거를 유지했으며, 추가한 정의·요약·치트·복습 답은 원문과 다시 대조했습니다. 같은 번호는 세 PDF에서 재사용합니다.

코드·수식·그림과 의심 표기는 원문 스캔을 우선했습니다. 의심스러운 내용을 외부 지식으로 교정하거나 미제공 부분을 채우지 않았습니다.

전체 페이지 렌더링 점검 후 결과를 반영합니다.

부 도입

번호	문장·표 항목·원문 코드·수식	원문 위치
U00-001	운영체제는 사용자와 하드웨어 사이의 중개자이며, 프로그램을 편리하고 효율적으로 실행할 환경을 제공한다.	(부 도입, p.1 / PDF p.6)
U00-002	하드웨어를 관리하면서 시스템의 정상 동작을 지키고, 설계 목표와 각 구성 부분의 입력·출력·동작을 명확히 해야 한다.	(부 도입, p.1 / PDF p.6)
U00-003	병행 또는 병렬로 실행되는 스레드가 데이터를 공유할 때 접근을 제어하지 않으면 경쟁 조건으로 값이 손상될 수 있다.	(부 도입, p.281 / PDF p.226)
U00-004	동기화 도구는 공유 데이터 접근을 제어하지만, 잘못 사용하면 교착 상태나 성능 저하를 일으킬 수 있다.	(부 도입, p.281 / PDF p.226)
U00-005	실행 중인 프로그램과 접근하는 데이터의 적어도 일부는 메인 메모리에 있어야 한다.	(부 도입, p.383 / PDF p.324)
U00-006	여러 프로세스를 메모리에 유지하는 관리 기법의 효과는 상황에 따라 다르며, 선택은 하드웨어 설계와 지원의 영향을 받는다.	(부 도입, p.383 / PDF p.324)
U00-007	대용량 저장장치는 파일과 데이터를 영구 저장한다.	(부 도입, p.489 / PDF p.429)
U00-008	장치는 전송 단위·접근 방식·동기성·공유 여부·읽기와 쓰기·속도가 다양하다.	(부 도입, p.489 / PDF p.429)
U00-009	입출력 서브시스템은 이를 제어하면서 단순한 인터페이스를 제공하고, 병목을 줄이도록 입출력을 최적화한다.	(부 도입, p.489 / PDF p.429)
U00-010	파일은 작성자가 정의한 관련 정보의 모음이다.	(부 도입, p.577 / PDF p.516)
U00-011	파일 시스템은 파일을 물리적 저장장치에 매핑하고 사용자·프로그램의 접근과 조작 방법을 제공한다.	(부 도입, p.577 / PDF p.516)
U00-012	느릴 수 있는 물리 장치의 효율적인 접근과 공유·원격 접근 요구를 함께 고려한다.	(부 도입, p.577 / PDF p.516)

제1장. 서론

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-001	운영체제는 컴퓨터 하드웨어를 관리하고 응용 프로그램 실행 환경을 제공하는 소프트웨어이다.	(1.12, p.56 / PDF p.60)
U01-002	인터럽트는 하드웨어가 운영체제와 상호 작용하는 주요 방법이며 인터럽트 핸들러에 의해 관리된다.	(1.12, p.56 / PDF p.60)
U01-003	가상화에는 컴퓨터 하드웨어를 여러 가지 실행 환경으로 추상화하는 작업이 포함된다.	(1.12, p.57 / PDF p.61)
U01-004	운영체제는 컴퓨터 하드웨어를 관리하고 응용 프로그램의 실행 기반을 제공하며 사용자와 하드웨어 사이를 중재하는 소프트웨어이다.	(1.1, p.4-6 / PDF p.8-10)
U01-005	하드웨어는 CPU·메모리·입출력 장치로 기본 계산 자원을 제공한다.	(1.1, p.4-6 / PDF p.8-10)
U01-006	응용 프로그램은 이 자원을 사용해 사용자의 문제를 해결하며, 운영체제는 여러 사용자와 프로그램 사이의 하드웨어 사용을 제어하고 조정한다.	(1.1, p.4-6 / PDF p.8-10)
U01-007	사용자 관점	(1.1, p.4-6 / PDF p.8-10)
U01-008	개인용 컴퓨터에서는 사용의 편리함이 주요 목표이다. 모바일 장치는 터치·음성 인터페이스를 제공한다. 내장형 컴퓨터는 사용자 개입 없이 동작하기도 한다.	(1.1, p.4-6 / PDF p.8-10)
U01-009	시스템 관점	(1.1, p.4-6 / PDF p.8-10)
U01-010	자원 할당자로서 CPU 시간·메모리·저장공간·입출력 장치의 상충하는 요청을 효율적이고 공정하게 처리한다. 제어 프로그램으로서 부적절한 사용을 방지한다.	(1.1, p.4-6 / PDF p.8-10)
U01-011	운영체제에 포함되는 요소에는 보편적으로 합의된 경계가 없다.	(1.1, p.4-6 / PDF p.8-10)
U01-012	원문은 항상 실행되는 프로그램인 커널, 관리에 쓰이는 시스템 프로그램, 개발자에게 추가 서비스를 제공하는 미들웨어를 구별한다.	(1.1, p.4-6 / PDF p.8-10)
U01-013	커널과 관련된 프로그램이라도 반드시 커널의 일부인 것은 아니다.	(1.1, p.4-6 / PDF p.8-10)
U01-014	모바일 운영체제에는 데이터베이스·멀티미디어·그래픽 지원 프레임워크도 포함된다.	(1.1, p.4-6 / PDF p.8-10)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-015	범용 컴퓨터는 하나 이상의 CPU, 공유 메모리, 공통 버스로 연결된 장치 컨트롤러로 구성된다.	(1.2, p.7 / PDF p.11)
U01-016	컨트롤러는 특정 유형의 장치를 담당하고 로컬 버퍼와 특수 목적 레지스터를 가진다.	(1.2, p.7 / PDF p.11)
U01-017	장치 드라이버는 컨트롤러의 특성을 알고 운영체제의 다른 부분에 일관된 인터페이스를 제공한다.	(1.2, p.7 / PDF p.11)
U01-018	CPU와 장치 컨트롤러가 병렬로 작동하므로 공유 메모리 접근은 동기화되어야 한다.	(1.2, p.7 / PDF p.11)
U01-019	인터럽트는 하드웨어가 CPU의 처리가 필요한 이벤트를 알리는 핵심 수단이다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-020	드라이버가 컨트롤러 레지스터에 값을 적재하면 컨트롤러가 작업을 시작하고, 완료 후 인터럽트로 알린다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-021	CPU는 하던 실행을 중단하고 서비스 루틴으로 제어를 옮긴 뒤 중단된 연산을 재개한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-022	CPU는 명령어 실행을 마칠 때 인터럽트 요청 라인을 검사한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-023	인터럽트 번호를 인터럽트 벡터의 인덱스로 사용해 해당 핸들러로 이동한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-024	핸들러는 변경될 상태를 저장하고 원인을 확인하여 필요한 처리를 수행한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-025	상태를 복원하고 인터럽트 복귀 명령으로 이전 실행을 재개한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-026	인터럽트 벡터는 서비스 루틴 주소의 배열이다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-027	중간 검색 루틴을 거치지 않고 적절한 루틴을 호출하여 처리 시간을 줄인다.	(1.2.1, p.7-12 / PDF p.11-16)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-028	장치 수가 벡터 항목 수보다 많으면 각 항목에 핸들러 리스트를 연결하는 인터럽트 체인을 사용하고 요청을 처리할 핸들러를 찾을 때까지 호출한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-029	마스크 불가능 인터럽트	(1.2.1, p.7-12 / PDF p.11-16)
U01-030	복구할 수 없는 메모리 오류 같은 이벤트를 위해 예약한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-031	마스크 가능 인터럽트	(1.2.1, p.7-12 / PDF p.11-16)
U01-032	중요한 명령 시퀀스 실행 전에 잠시 끌 수 있으며 장치의 서비스 요청에 사용된다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-033	우선순위 레벨	(1.2.1, p.7-12 / PDF p.11-16)
U01-034	낮은 우선순위 처리를 연기하고 높은 우선순위 인터럽트가 낮은 우선순위 실행을 선점하게 한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-035	인터럽트 서비스는 빈번하고 시간에 민감하므로 효율적이어야 한다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-036	복귀 주소와 변경되는 레지스터 상태를 보존하지 않으면 원래 실행을 올바르게 재개할 수 없다.	(1.2.1, p.7-12 / PDF p.11-16)
U01-037	실행할 프로그램은 먼저 메인 메모리에 적재되어야 한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-038	일반적인 메인 메모리는 DRAM으로 구현되는 휘발성 저장장치이다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-039	전원을 켤 때 운영체제를 적재하는 부트스트랩 프로그램은 RAM에 영구 보관할 수 없으므로 EEPROM 등 비휘발성 펌웨어를 사용한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-040	적재(load)는 메인 메모리의 바이트 또는 워드를 CPU 레지스터로 옮기고, 저장(store)은 반대로 옮긴다.	(1.2.2, p.12-15 / PDF p.16-19)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-041	명령 실행은 명령 인출·해독·피연산자 인출·연산·결과 저장으로 진행될 수 있다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-042	메모리는 주소가 생성된 방법이나 그 내용이 명령인지 데이터인지를 구별하지 않고 주소의 연속으로 취급한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-043	메인 메모리는 모든 프로그램과 데이터를 영구 저장하기에는 작고 전원 손실 시 내용을 잃는다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-044	보조저장장치는 이를 확장해 대량의 데이터를 영구 보존한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-045	HDD와 NVM은 프로그램이 메모리에 적재되기 전 저장소이며, 처리 중 데이터의 원천과 목적지가 되기도 한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-046	휘발성 저장장치	(1.2.2, p.12-15 / PDF p.16-19)
U01-047	전원 제거 시 내용을 잃는다. 안전하게 보존할 데이터는 비휘발성 장치에 기록해야 한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-048	기계적 비휘발성 저장장치	(1.2.2, p.12-15 / PDF p.16-19)
U01-049	HDD·광 디스크·자기 테이프 등. 일반적으로 전기적 장치보다 용량이 크고 바이트당 비용이 낮다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-050	전기적 비휘발성 저장장치(NVM)	(1.2.2, p.12-15 / PDF p.16-19)
U01-051	플래시·FRAM·NRAM·SSD 등. 일반적으로 더 빠르지만 비싸고 용량이 작다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-052	3차 저장장치	(1.2.2, p.12-15 / PDF p.16-19)
U01-053	백업·드물게 쓰는 데이터·장기 보관을 위한 느리고 큰 저장장치이다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-054	저장 계층은 속도·용량·비용·휘발성을 함께 고려한다.	(1.2.2, p.12-15 / PDF p.16-19)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-055	작고 빠른 메모리는 CPU에 가깝게 두며, 접근 시간이나 전송 속도 차이가 큰 두 수준 사이에는 캐시를 둘 수 있다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-056	원문은 1바이트=8비트, 워드를 구조 본연의 데이터 단위로 설명한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-057	저장장치의 바이트 단위와 네트워킹의 비트 단위를 구별한다.	(1.2.2, p.12-15 / PDF p.16-19)
U01-058	그림 1.6 저장장치 계층 구조	(1.2.2, p.14 / PDF p.18)
U01-059	소량 데이터에는 인터럽트 구동 입출력이 적합하지만 대량 전송에서 바이트마다 인터럽트를 발생시키면 오버헤드가 커진다.	(1.2.3, p.15-16 / PDF p.19-20)
U01-060	직접 메모리 접근(DMA)은 버퍼·포인터·입출력 카운트를 설정한 후 컨트롤러가 CPU 개입 없이 메모리와 장치 버퍼 사이에서 블록 전체를 전송하게 한다.	(1.2.3, p.15-16 / PDF p.19-20)
U01-061	블록 전송이 끝날 때 인터럽트를 발생시키므로 그동안 CPU는 다른 작업을 할 수 있다.	(1.2.3, p.15-16 / PDF p.19-20)
U01-062	스위치 구조는 공유 버스 경쟁 없이 여러 구성요소의 동시 통신을 허용하여 DMA를 더 효과적으로 만들 수 있다.	(1.2.3, p.15-16 / PDF p.19-20)
U01-063	원문은 단일 처리 코어를 가진 범용 CPU가 하나인 경우를 단일 처리기 시스템으로 설명한다.	(1.3.1, p.16-17 / PDF p.20-21)
U01-064	디스크·키보드·그래픽 컨트롤러의 전용 처리기는 제한된 명령어만 실행하고 사용자 프로세스를 실행하지 않는다.	(1.3.1, p.16-17 / PDF p.20-21)
U01-065	그러한 전용 처리기가 있다고 해서 다중 처리기 시스템이 되는 것은 아니다.	(1.3.1, p.16-17 / PDF p.20-21)
U01-066	코어는 명령을 실행하고 로컬 데이터용 레지스터를 갖는 기본 계산 단위이다.	(1.3.1, p.16-17 / PDF p.20-21)
U01-067	다중 처리기 시스템은 여러 처리기가 버스와 메모리·주변 장치 등을 공유한다.	(1.3.2, p.17-20 / PDF p.21-24)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-068	처리기를 늘리면 처리량이 증가하지만 협력 오버헤드와 공유 자원 경합 때문에 N개 처리기의 속도 향상이 반드시 N배가 되지는 않는다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-069	대칭형 다중 처리(SMP)에서는 각 CPU가 운영체제 기능과 사용자 프로세스를 포함한 모든 작업을 수행한다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-070	각 CPU는 자체 레지스터와 로컬 캐시를 가지며 물리 메모리를 공유한다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-071	한 CPU가 유휴이고 다른 CPU가 과부하인 상태를 줄이려면 작업과 자료구조를 적절히 공유해야 한다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-072	다중 코어는 한 칩 안에 여러 계산 코어를 두는 방식이다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-073	칩 내부 통신의 효율과 낮은 전력 소모가 이점이며 로컬 캐시와 공유 캐시를 결합할 수 있다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-074	운영체제에는 N개 코어가 N개 CPU처럼 보인다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-075	공유 버스 경합이 확장성의 병목이 되면 NUMA 구조에서 각 CPU 또는 CPU 그룹에 로컬 메모리를 둔다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-076	모든 CPU는 하나의 물리 주소 공간을 공유하지만 로컬 메모리보다 원격 메모리 접근 지연이 크다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-077	운영체제의 스케줄링과 메모리 관리가 이 단점을 줄여야 한다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-078	블레이드 서버는 각 보드가 독립적으로 부팅하고 자신의 운영체제를 실행한다는 점에서 전통적인 공유형 다중 처리기와 다르다.	(1.3.2, p.17-20 / PDF p.21-24)
U01-079	클러스터형 시스템은 둘 이상의 독립적인 노드를 네트워크로 연결한 약결합 시스템이다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-080	일반적으로 저장장치를 공유하고 LAN이나 고속 상호 연결망을 사용한다.	(1.3.3, p.20-22 / PDF p.24-26)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-081	노드가 서로를 감시하다가 고장 노드의 저장장치 소유권을 넘겨받고 응용을 다시 실행함으로써 높은 가용성을 제공한다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-082	비대칭형 클러스터	(1.3.3, p.20-22 / PDF p.24-26)
U01-083	다른 호스트가 응용을 실행하는 동안 긴급 대기 호스트는 감시만 하다가 고장 시 활성 서버가 된다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-084	대칭형 클러스터	(1.3.3, p.20-22 / PDF p.24-26)
U01-085	여러 호스트가 응용을 실행하며 서로 감시한다. 모든 하드웨어를 이용하지만 여러 응용이 실행 가능해야 효율적이다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-086	남은 하드웨어 수준에 비례해 서비스를 계속하는 것이 우아한 성능 저하이며, 결함허용 시스템은 단일 구성요소 오류에도 계속 작동하도록 감지·진단·수정 기법을 둔다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-087	고성능 계산에 클러스터를 사용하려면 응용을 병렬화하여 각 노드가 일부 문제를 해결한 뒤 결과를 합쳐야 한다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-088	공유 데이터의 병렬 접근에는 충돌을 막는 접근 제어·잠금이 필요하며 원문은 분산 잠금 관리자(DLM)를 예로 든다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-089	SAN은 여러 호스트가 여러 저장장치에 연결되도록 한다.	(1.3.3, p.20-22 / PDF p.24-26)
U01-090	부팅 시 펌웨어의 부트스트랩 프로그램이 CPU 레지스터·컨트롤러·메모리를 초기화하고 커널을 찾아 메모리에 적재한다.	(1.4, p.23-24 / PDF p.27-28)
U01-091	커널이 실행되면 서비스를 제공하고, 일부 서비스는 부팅 때 적재되어 계속 실행되는 시스템 데몬이 맡는다.	(1.4, p.23-24 / PDF p.27-28)
U01-092	원문은 Linux의 systemd를 다른 데몬을 시작하는 시스템 프로그램으로 제시한다.	(1.4, p.23-24 / PDF p.27-28)
U01-093	실행할 일이 없으면 시스템은 이벤트를 기다린다.	(1.4, p.23-24 / PDF p.27-28)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-094	하드웨어 인터럽트 외에 트랩(예외)은 0으로 나누기·유효하지 않은 메모리 접근 등의 오류나 사용자 프로그램의 특정 요청으로 발생하는 소프트웨어 인터럽트이다.	(1.4, p.23-24 / PDF p.27-28)
U01-095	사용자 요청은 시스템 콜을 통해 운영체제 서비스를 요구한다.	(1.4, p.23-24 / PDF p.27-28)
U01-096	원문의 Hadoop 보충 설명은 분산 파일 시스템·YARN의 자원 및 작업 관리·MapReduce의 병렬 처리가 클러스터에서 결합됨을 보여 준다.	(1.4, p.23-24 / PDF p.27-28)
U01-097	다중 프로그래밍은 여러 프로세스를 메모리에 유지하여 한 프로세스가 입출력 등을 기다릴 때 다른 프로세스를 실행한다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-098	실행 가능한 일이 있는 동안 CPU를 유휴로 두지 않아 이용률을 높이는 것이 목적이다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-099	다중 태스킹은 이를 확장하여 프로세스 전환을 자주 수행함으로써 사용자에게 빠른 응답을 제공한다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-100	여러 프로세스를 동시에 유지하려면 메모리 관리가, 준비된 여러 프로세스 중 다음 대상을 고르려면 CPU 스케줄링이 필요하다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-101	가상 메모리는 일부만 메모리에 적재된 프로세스도 실행하게 하여 논리 메모리를 물리 메모리와 분리한다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-102	파일·저장장치 관리, 자원 보호, 통신·동기화와 교착 상태 처리도 여러 프로세스의 질서 있는 실행과 연결된다.	(1.4.1, p.24-26 / PDF p.28-30)
U01-103	운영체제와 사용자 코드가 자원을 공유하므로 하드웨어는 사용자 모드와 커널 모드를 구별한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-104	원문의 모드 비트 예에서는 커널 모드가 0, 사용자 모드가 1이다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-105	부팅은 커널 모드에서 시작하고 사용자 프로그램으로 제어를 넘길 때 사용자 모드로 전환한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-106	인터럽트·트랩·시스템 콜이 발생하면 하드웨어가 커널 모드로 바꾼다.	(1.4.2, p.26-28 / PDF p.30-32)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-107	특권 명령은 커널 모드에서만 허용한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-108	입출력 제어·타이머 관리·인터럽트 관리 등이 그 예이다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-109	사용자 모드에서 실행하려 하면 불법 명령으로 트랩을 발생시켜 운영체제로 제어를 넘긴다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-110	모드는 둘 이상으로 확장될 수 있으며 원문은 Intel 보호 링, ARMv8의 여러 모드, 가상 머신 관리자를 위한 모드를 소개한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-111	사용자 프로세스가 운영체제의 작업 수행을 요청한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-112	트랩과 인터럽트 벡터를 통해 서비스 루틴으로 제어가 이동하고 커널 모드가 된다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-113	커널이 호출 종류와 인수의 정확성·합법성을 검사하여 요청을 수행한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-114	사용자 모드로 바꾸고 시스템 콜 다음 명령으로 복귀한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-115	프로그램이 불법 명령이나 자신의 영역 밖 메모리에 접근하면 하드웨어가 오류를 검출한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-116	운영체제는 적절한 오류 메시지를 제공하고 비정상 종료시킬 수 있으며, 원문은 조사·수정을 위한 메모리 덤프도 설명한다.	(1.4.2, p.26-28 / PDF p.30-32)
U01-117	그림 1.13 사용자 모드에서 커널 모드로의 전환	(1.4.2, p.26 / PDF p.30)
U01-118	타이머는 사용자 프로그램의 무한 루프나 서비스 미호출 때문에 운영체제가 CPU 제어를 잃는 일을 방지한다.	(1.4.3, p.28-29 / PDF p.32-33)
U01-119	지정한 시간 후 인터럽트를 발생시키며 가변 타이머는 고정률 클록과 계수기로 구현할 수 있다.	(1.4.3, p.28-29 / PDF p.32-33)
U01-120	운영체제가 계수기를 설정하고 매 틱마다 감소시켜 0일 때 인터럽트를 발생시킨다.	(1.4.3, p.28-29 / PDF p.32-33)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-121	제어를 사용자에게 넘기기 전에 설정되어야 하며 타이머 값 변경은 특권 명령이다.	(1.4.3, p.28-29 / PDF p.32-33)
U01-122	원문 Linux 보충 설명에서 HZ는 초당 타이머 인터럽트 빈도, jiffies는 부팅 이후 인터럽트 누적 개수이다.	(1.4.3, p.28-29 / PDF p.32-33)
U01-123	프로세스는 실행 중인 프로그램의 인스턴스이며 CPU 시간·메모리·파일·입출력 장치와 초기화 데이터를 필요로 한다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-124	프로그램은 디스크에 놓인 수동적 개체이지만 프로세스는 다음 명령을 가리키는 프로그램 카운터를 갖는 능동적 개체이다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-125	동일 프로그램을 실행해도 별도의 프로세스가 될 수 있다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-126	단일 스레드에는 한 실행 순서가, 다중 스레드에는 각 스레드의 프로그램 카운터가 있다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-127	사용자·시스템 프로세스를 생성하고 제거한다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-128	프로세스와 스레드에 CPU를 스케줄하고 중지·재개한다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-129	프로세스 동기화와 통신을 위한 기법을 제공한다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-130	종료 시 재사용 가능한 자원을 회수한다.	(1.5.1, p.29-30 / PDF p.33-34)
U01-131	메인 메모리는 CPU와 입출력 장치가 공유하는 바이트 배열이며 각 바이트에는 주소가 있다.	(1.5.2, p.30-31 / PDF p.34-35)
U01-132	프로그램을 실행하려면 메모리에 적재하고 절대 주소로 매핑해야 한다.	(1.5.2, p.30-31 / PDF p.34-35)
U01-133	여러 프로그램을 메모리에 유지하면 CPU 이용률과 응답성을 높일 수 있지만 메모리 관리가 필요하다.	(1.5.2, p.30-31 / PDF p.34-35)
U01-134	운영체제는 사용 중인 영역과 소유 프로세스를 추적하고, 공간을 할당·회수하며, 적재하거나 제거할 프로세스 또는 그 일부를 결정한다.	(1.5.2, p.30-31 / PDF p.34-35)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-135	관리 기법의 효율은 상황에 따라 달라지고 필요한 하드웨어 지원의 영향을 받는다.	(1.5.2, p.30-31 / PDF p.34-35)
U01-136	운영체제는 물리 저장 매체의 차이를 감추고 파일이라는 논리 저장 단위를 정의한다.	(1.5.3, p.31-32 / PDF p.35-36)
U01-137	파일은 생성자가 정의한 관련 정보의 집합이며 프로그램과 데이터를 표현할 수 있다.	(1.5.3, p.31-32 / PDF p.35-36)
U01-138	운영체제는 파일을 물리 매체로 매핑하고 디렉터리로 조직하며 사용자별 접근을 통제한다.	(1.5.3, p.31-32 / PDF p.35-36)
U01-139	파일·디렉터리 생성과 제거, 조작 프리미티브, 보조저장장치 매핑, 안정적인 매체로의 백업이 책무이다.	(1.5.3, p.31-32 / PDF p.35-36)
U01-140	HDD·NVM 같은 보조저장장치는 적재 전 프로그램과 데이터의 주요 온라인 저장소이다.	(1.5.4, p.32 / PDF p.36)
U01-141	운영체제는 마운팅·언마운팅, 가용 공간 관리, 저장장소 할당, 디스크 스케줄링, 분할과 보호를 맡는다.	(1.5.4, p.32 / PDF p.36)
U01-142	매우 빈번하게 사용되므로 장치와 알고리즘의 효율이 전체 시스템 속도에 영향을 줄 수 있다.	(1.5.4, p.32 / PDF p.36)
U01-143	백업·장기 보존용 3차 저장장치는 덜 빈번하게 쓰이더라도 장치 할당·반환과 데이터 이주 등의 관리가 필요하다.	(1.5.4, p.32 / PDF p.36)
U01-144	캐싱은 정보를 더 빠른 저장장치에 일시 복사하는 원리이다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-145	먼저 캐시를 검사하여 있으면 직접 쓰고, 없으면 원래 저장장치에서 가져와 사용하면서 캐시에 둔다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-146	캐시는 제한된 크기이므로 크기와 교체 정책이 중요하다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-147	CPU 레지스터는 메인 메모리의 고속 캐시처럼 볼 수 있으며 레지스터 할당은 프로그래머·컴파일러가 정한다.	(1.5.5, p.33-34 / PDF p.37-38)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-148	하드웨어만으로 관리되는 캐시는 이 책의 운영체제 제어 대상과 구별한다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-149	한 데이터의 복사본이 디스크·메인 메모리·캐시·레지스터에 동시에 존재하면 값의 갱신 시점이 서로 달라진다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-150	다중 태스킹에서는 각 프로세스가 최신 값을 보도록 해야 한다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-151	다중 처리기에서는 여러 로컬 캐시의 복사본을 일치시키는 캐시 일관성 문제가 생기며 통상 하드웨어에서 처리한다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-152	분산 환경의 파일 복제본 갱신은 더욱 복잡하다.	(1.5.5, p.33-34 / PDF p.37-38)
U01-153	입출력 서브시스템은 장치의 구체적인 특성을 사용자와 운영체제의 나머지 부분에서 숨긴다.	(1.5.6, p.34-35 / PDF p.38-39)
U01-154	버퍼링·캐싱·스풀링을 포함하는 메모리 관리 부분, 일반 장치 드라이버 인터페이스, 특정 하드웨어 드라이버로 구성된다.	(1.5.6, p.34-35 / PDF p.38-39)
U01-155	해당 장치의 특성을 직접 아는 것은 그 장치 드라이버이다.	(1.5.6, p.34-35 / PDF p.38-39)
U01-156	보호(protection)	(1.6, p.35-36 / PDF p.39-40)
U01-157	프로그램·프로세스·사용자의 자원 접근을 제어한다. 허가된 사용과 허가되지 않은 사용을 구분하고 제어 명세와 강제 시행 방법을 제공한다.	(1.6, p.35-36 / PDF p.39-40)
U01-158	보안(security)	(1.6, p.35-36 / PDF p.39-40)
U01-159	시스템 외부와 내부의 공격을 방어한다. 보호 기능이 작동해도 인증 정보가 도난되면 자료가 복사·삭제될 수 있다.	(1.6, p.35-36 / PDF p.39-40)
U01-160	운영체제는 사용자 이름과 유일한 사용자 식별자를 연결하고 로그인 인증에서 이를 결정한다.	(1.6, p.35-36 / PDF p.39-40)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-161	식별자는 사용자의 프로세스·스레드에 연관된다.	(1.6, p.35-36 / PDF p.39-40)
U01-162	그룹 식별자는 여러 사용자에게 공통 접근 권한을 부여할 때 사용한다.	(1.6, p.35-36 / PDF p.39-40)
U01-163	제한된 작업에 권한 상승이 필요할 수 있으며 원문은 UNIX setuid를 통해 파일 소유자의 유효 사용자 식별자로 실행하는 경우를 제시한다.	(1.6, p.35-36 / PDF p.39-40)
U01-164	보호와 보안은 사용자를 구별하는 기반에 의존하지만 동일한 개념은 아니다.	(1.6, p.35-36 / PDF p.39-40)
U01-165	가상화는 한 컴퓨터의 하드웨어를 여러 실행 환경으로 추상화하여 각 환경이 독립된 컴퓨터에서 실행되는 듯하게 만드는 기술이다.	(1.7, p.36-38 / PDF p.40-42)
U01-166	VMM은 게스트 운영체제를 실행하고 자원 사용을 관리하며 게스트 사이를 보호한다.	(1.7, p.36-38 / PDF p.40-42)
U01-167	원문은 호스트 운영체제 위의 응용 형태 VMM과 호스트 자체 역할을 하는 VMM을 모두 소개한다.	(1.7, p.36-38 / PDF p.40-42)
U01-168	원문의 구별에서 에뮬레이션은 소스와 대상 CPU가 다를 때 하드웨어를 소프트웨어로 모의하고 명령을 번역하므로 비용이 크다.	(1.7, p.36-38 / PDF p.40-42)
U01-169	가상화에서는 동일 CPU용으로 컴파일된 운영체제를 다른 운영체제 환경에서 실행한다.	(1.7, p.36-38 / PDF p.40-42)
U01-170	활용 사례는 다른 운영체제의 응용 사용, 여러 플랫폼용 개발·시험·디버깅, 데이터 센터의 자원 관리이다.	(1.7, p.36-38 / PDF p.40-42)
U01-171	분산 시스템은 물리적으로 떨어진 이기종 컴퓨터가 네트워크로 연결된 집합이다.	(1.8, p.38-39 / PDF p.42-43)
U01-172	자원 공유는 계산 속도·기능·데이터 가용성·신뢰성을 높일 수 있다.	(1.8, p.38-39 / PDF p.42-43)
U01-173	운영체제는 원격 접근을 파일 접근처럼 일반화하거나 네트워크 기능을 명시적으로 호출하게 한다.	(1.8, p.38-39 / PDF p.42-43)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-174	네트워크는 프로토콜·거리·매체에 따라 다르며 원문은 LAN·WAN·MAN·PAN과 TCP/IP를 소개한다.	(1.8, p.38-39 / PDF p.42-43)
U01-175	네트워크 운영체제의 컴퓨터는 다른 컴퓨터와 파일·메시지를 공유하지만 서로 자율적으로 동작한다.	(1.8, p.38-39 / PDF p.42-43)
U01-176	분산 운영체제는 컴퓨터들이 밀접하게 통신하여 하나의 운영체제가 전체 네트워크를 제어하는 것처럼 보이게 하므로 자율성이 더 낮다.	(1.8, p.38-39 / PDF p.42-43)
U01-177	배열은 각 원소에 직접 접근한다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-178	리스트는 데이터를 순서로 나타내며 연결 리스트는 포인터로 항목을 연결한다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-179	단일 연결 리스트는 후속 항목, 이중 연결 리스트는 앞뒤 항목을 가리키고 원형 연결 리스트의 마지막은 첫 항목을 가리킨다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-180	가변 수 항목의 삽입·삭제는 쉽지만 길이 n인 리스트의 특정 항목을 찾는 최악 성능은 O(n)이다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-181	스택	(1.9.1, p.39-41 / PDF p.43-45)
U01-182	후입선출(LIFO), push·pop	(1.9.1, p.39-41 / PDF p.43-45)
U01-183	함수 인수·지역 변수·복귀 주소를 보관하고 복귀 시 제거한다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-184	큐	(1.9.1, p.39-41 / PDF p.43-45)
U01-185	선입선출(FIFO)	(1.9.1, p.39-41 / PDF p.43-45)
U01-186	인쇄 작업이나 CPU 실행 대기 태스크를 순서대로 조직한다.	(1.9.1, p.39-41 / PDF p.43-45)
U01-187	트리는 부모·자식 관계로 계층을 나타낸다.	(1.9.2, p.41 / PDF p.45)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-188	일반 트리는 임의 수의 자식, 이진 트리는 최대 두 자식을 가진다.	(1.9.2, p.41 / PDF p.45)
U01-189	이진 탐색 트리는 값의 순서 관계를 추가하지만 형태에 따라 최악의 탐색이 O(n)이 될 수 있다.	(1.9.2, p.41 / PDF p.45)
U01-190	균형 이진 탐색 트리는 깊이를 제한하여 최악의 탐색 O(lg n)을 보장한다.	(1.9.2, p.41 / PDF p.45)
U01-191	원문은 Linux CPU 스케줄링의 red-black 트리를 예로 든다.	(1.9.2, p.41 / PDF p.45)
U01-192	해시 함수는 데이터에 산술 연산을 수행해 테이블 인덱스로 쓸 수 있는 수를 반환한다.	(1.9.3, p.41-42 / PDF p.45-46)
U01-193	구현에 따라 인출 성능이 O(1)만큼 좋아질 수 있다.	(1.9.3, p.41-42 / PDF p.45-46)
U01-194	서로 다른 입력이 같은 인덱스로 매핑되는 해시 충돌은 각 항목에 연결 리스트를 두어 처리할 수 있지만 충돌이 늘면 효율이 떨어진다.	(1.9.3, p.41-42 / PDF p.45-46)
U01-195	해시 맵은 키와 값을 연결하며 키에 해시 함수를 적용해 값을 찾는다.	(1.9.3, p.41-42 / PDF p.45-46)
U01-196	비트맵은 n개 자원의 상태를 n개 비트로 나타낸다.	(1.9.4, p.42-43 / PDF p.46-47)
U01-197	원문의 예에서는 0이 사용 가능, 1이 사용 불가능을 뜻하지만 반대로 정할 수도 있다.	(1.9.4, p.42-43 / PDF p.46-47)
U01-198	i번째 비트는 i번째 자원에 대응한다.	(1.9.4, p.42-43 / PDF p.46-47)
U01-199	한 자원에 8비트 부울 값을 쓰는 것보다 공간을 절약하므로 디스크 블록처럼 많은 자원의 가용성을 표시할 때 유용하다.	(1.9.4, p.42-43 / PDF p.46-47)
U01-200	웹 포털·무선 연결·모바일 접근이 사무실과 가정의 전통적인 계산 환경 경계를 넓힌다.	(1.10.1, p.43-44 / PDF p.47-48)
U01-201	thin client는 웹 기반 계산을 이해하는 터미널로 보안과 유지보수 측면에서 워크스테이션을 대체하기도 한다.	(1.10.1, p.43-44 / PDF p.47-48)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-202	배치 시스템은 미리 정해진 입력을 대량 처리하고 대화형 시스템은 사용자 입력을 기다린다.	(1.10.1, p.43-44 / PDF p.47-48)
U01-203	시분할의 타이머·스케줄링 기법은 현대 워크스테이션과 서버에서도 프로세스에 계산 시간을 빈번하게 제공하는 데 사용된다.	(1.10.1, p.43-44 / PDF p.47-48)
U01-204	모바일 컴퓨팅은 가볍고 휴대 가능한 스마트폰·태블릿 환경이다.	(1.10.2, p.44-45 / PDF p.48-49)
U01-205	GPS·가속도계·자이로스코프를 활용하는 응용과 증강현실을 지원한다.	(1.10.2, p.44-45 / PDF p.48-49)
U01-206	무선망이나 휴대 전화 데이터망으로 온라인 서비스에 접근한다.	(1.10.2, p.44-45 / PDF p.48-49)
U01-207	원문은 PC에 비해 메모리·처리 능력이 제한되고 전력 소모가 중요한 설계 조건임을 설명하며 iOS와 Android를 예로 든다.	(1.10.2, p.44-45 / PDF p.48-49)
U01-208	이 설명과 수치는 원문 집필 시점의 환경이다.	(1.10.2, p.44-45 / PDF p.48-49)
U01-209	클라이언트가 생성한 요청을 서버가 충족시키는 분산 구조이다.	(1.10.3, p.45-46 / PDF p.49-50)
U01-210	계산 서버는 작업 요청 인터페이스를 제공하고 수행 결과를 돌려준다.	(1.10.3, p.45-46 / PDF p.49-50)
U01-211	파일 서버는 파일 생성·갱신·읽기·제거 인터페이스를 제공하며 웹 서버가 브라우저에 파일을 전달하는 경우가 예이다.	(1.10.3, p.45-46 / PDF p.49-50)
U01-212	피어 간(P2P) 시스템에서는 클라이언트와 서버가 고정적으로 구별되지 않고 각 노드가 요청·제공 상황에 따라 두 역할을 맡는다.	(1.10.4, p.46-47 / PDF p.50-51)
U01-213	서비스가 여러 노드로 분산되어 전통적인 서버 병목을 줄일 수 있다.	(1.10.4, p.46-47 / PDF p.50-51)
U01-214	서비스 발견에는 중앙 검색 서비스에 등록하는 방법과 요청을 방송하고 제공자가 응답하는 방법이 있다.	(1.10.4, p.46-47 / PDF p.50-51)
U01-215	원문은 Napster·Gnutella·Skype로 이 구성을 설명한다.	(1.10.4, p.46-47 / PDF p.50-51)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-216	클라우드 컴퓨팅은 계산·저장·응용을 네트워크 서비스로 제공한다.	(1.10.5, p.47-49 / PDF p.51-53)
U01-217	가상화를 기반으로 이용할 수 있으며 사용자는 사용한 자원에 따라 비용을 지불할 수 있다.	(1.10.5, p.47-49 / PDF p.51-53)
U01-218	공중·사유·혼합형이라는 운영 범위의 구별과 SaaS·PaaS·IaaS라는 제공 자원의 구별은 서로 독립적인 배타 분류가 아니다.	(1.10.5, p.47-49 / PDF p.51-53)
U01-219	공중·사유·혼합형	(1.10.5, p.47-49 / PDF p.51-53)
U01-220	누구나 비용을 지불하고 사용하는 서비스 / 한 회사용 / 두 방식을 포함하는 방식	(1.10.5, p.47-49 / PDF p.51-53)
U01-221	SaaS	(1.10.5, p.47-49 / PDF p.51-53)
U01-222	인터넷으로 사용하는 응용	(1.10.5, p.47-49 / PDF p.51-53)
U01-223	PaaS	(1.10.5, p.47-49 / PDF p.51-53)
U01-224	응용에 맞게 준비된 소프트웨어 스택	(1.10.5, p.47-49 / PDF p.51-53)
U01-225	IaaS	(1.10.5, p.47-49 / PDF p.51-53)
U01-226	인터넷으로 사용하는 서버·저장장치	(1.10.5, p.47-49 / PDF p.51-53)
U01-227	클라우드 하부구조에는 전통적 운영체제, 가상 머신을 관리하는 VMM, VMM과 클라우드 자원·인터페이스를 관리하는 도구가 결합될 수 있다.	(1.10.5, p.47-49 / PDF p.51-53)
U01-228	원문 그림 1.24는 IaaS 공중 클라우드의 고객 인터페이스·관리 서비스·가상 머신·저장장치·부하 분배 관계를 보여 준다.	(1.10.5, p.47-49 / PDF p.51-53)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-229	내장형 시스템은 특정 작업을 수행하며 사용자 인터페이스가 거의 없을 수 있다.	(1.10.6, p.48-49 / PDF p.52-53)
U01-230	범용 운영체제와 전용 응용, 제한된 기능의 내장형 운영체제, 운영체제 없이 ASIC만 사용하는 형태가 모두 존재한다.	(1.10.6, p.48-49 / PDF p.52-53)
U01-231	실시간 시스템은 데이터 흐름이나 처리에 엄격한 시간 제약이 있어 정해진 제한 시간 안에 올바른 결과를 내야 정확하게 동작한 것으로 간주된다.	(1.10.6, p.48-49 / PDF p.52-53)
U01-232	단순히 빠른 응답이 바람직한 일반 시스템과 이 조건을 구별한다.	(1.10.6, p.48-49 / PDF p.52-53)
U01-233	무료·공개 소스 운영체제는 소스 코드 형태로 제공되어 분석·수정·컴파일·실행을 통한 학습을 가능하게 한다.	(1.11, p.50 / PDF p.54)
U01-234	원문의 자유 소프트웨어는 소스 공개뿐 아니라 사용·재배포·수정을 허용하는 라이선스를 요구한다.	(1.11, p.50 / PDF p.54)
U01-235	따라서 자유 소프트웨어는 공개 소스이지만 모든 공개 소스가 같은 자유를 보장하는 것은 아니다.	(1.11, p.50 / PDF p.54)
U01-236	반면 비공개 소스는 컴파일된 이진 코드만 제공할 수 있다.	(1.11, p.50 / PDF p.54)
U01-237	이진 코드에서 역공학으로 원래 소스와 주석을 완전히 복원하는 것은 어렵다.	(1.11, p.50 / PDF p.54)
U01-238	초기에는 사용자 집단과 제조업체가 프로그램을 소스 코드와 함께 교환·배포했지만 소프트웨어의 사용 대상을 제한하고 경쟁자로부터 아이디어를 보호하려는 목적에서 이진 파일만 배포하는 관행이 확산되었다.	(1.11.1, p.51 / PDF p.55)
U01-239	원문은 이러한 변화가 자유·공개 소스 운동의 배경임을 설명한다.	(1.11.1, p.51 / PDF p.55)
U01-240	Richard Stallman의 GNU와 자유 소프트웨어 운동에서 자유는 가격보다 사용자의 권리를 뜻한다.	(1.11.2, p.51-52 / PDF p.55-56)
U01-241	실행, 분석·수정, 수정 없이 배포·판매, 수정해서 배포·판매할 자유가 핵심이다.	(1.11.2, p.51-52 / PDF p.55-56)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-242	카피레프트는 재배포 후에도 이러한 자유를 유지하게 한다.	(1.11.2, p.51-52 / PDF p.55-56)
U01-243	원문은 GPL에서 바이너리와 함께 소스를 배포하고 수정본을 포함한 사본에 동일 GPL을 적용한다고 설명한다.	(1.11.2, p.51-52 / PDF p.55-56)
U01-244	GNU가 개발한 컴파일러·도구와 Linus Torvalds의 Linux 커널이 결합하여 GNU/Linux를 이룬다.	(1.11.3, p.52-53 / PDF p.56-57)
U01-245	원문은 커널만을 Linux, GNU 도구까지 포함한 전체를 GNU/Linux로 구별한다.	(1.11.3, p.52-53 / PDF p.56-57)
U01-246	배포판은 기능·유틸리티·응용·하드웨어 지원·인터페이스·목적에서 차이가 난다.	(1.11.3, p.52-53 / PDF p.56-57)
U01-247	라이브 CD·DVD는 설치 없이 부팅하여 사용할 수 있고 가상 머신 이미지는 기존 호스트에서 별도의 운영체제를 학습하게 한다.	(1.11.3, p.52-53 / PDF p.56-57)
U01-248	원문에 수록된 다운로드 주소와 설치 지시는 학습 사례로만 검토하였다.	(1.11.3, p.52-53 / PDF p.56-57)
U01-249	BSD UNIX는 AT&T UNIX에서 출발하였고 원문은 4.4BSD-lite 및 FreeBSD·NetBSD·OpenBSD·DragonFlyBSD를 소개한다.	(1.11.4, p.53-54 / PDF p.57-58)
U01-250	소스 트리와 버전 제어를 이용하면 파일 변경 이력과 동시 수정 충돌을 관리할 수 있다.	(1.11.4, p.53-54 / PDF p.57-58)
U01-251	macOS의 공개 소스 핵심인 Darwin은 BSD UNIX에 기반하고 커널 패키지 이름은 xnu이다.	(1.11.4, p.53-54 / PDF p.57-58)
U01-252	원문은 Solaris의 BSD UNIX와 System V 기반 이력, OpenSolaris 공개, 그 기반에서 발전한 Illumos를 설명한다.	(1.11.5, p.54 / PDF p.58)
U01-253	이 이력은 제공된 본문의 설명 범위이며 이후 프로젝트의 현황을 외부 자료로 보충하지 않는다.	(1.11.5, p.54 / PDF p.58)
U01-254	소스가 공개되면 동작 관찰이나 문헌만으로 추측하던 구현을 직접 검토할 수 있다.	(1.11.6, p.54-55 / PDF p.58-59)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-255	프로그램을 수정·시험하고 다양한 목표·라이선스·구현을 비교할 수 있다.	(1.11.6, p.54-55 / PDF p.58-59)
U01-256	가상화와 하드웨어 시뮬레이터는 전용 기계 없이 여러 운영체제나 역사적 시스템을 탐구하게 한다.	(1.11.6, p.54-55 / PDF p.58-59)
U01-257	원문은 소스 공개의 학습 가치와 다양성을 강조한다.	(1.11.6, p.54-55 / PDF p.58-59)
U01-258	원문 요약은 하드웨어 관리, 인터럽트, 저장 계층, 다중 프로그래밍·태스킹, 실행 모드, 프로세스·메모리·파일 관리, 보호·가상화·계산 환경·공개 소스를 연결한다.	(1.12, p.56-59 / PDF p.60-63)
U01-259	장말 연습문제 1.1~1.11은 운영체제 목적, 실시간 제약, 구성 범위, 특권 명령, 캐시, 클라이언트·서버와 P2P를 점검한다.	(1.12, p.56-59 / PDF p.60-63)
U01-260	추가 자료와 참고문헌은 존재와 주제를 검토했으며 그 외부 문헌의 내용이나 새 풀이를 포함하지 않는다.	(1.12, p.56-59 / PDF p.60-63)
U01-261	목적	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)
U01-262	실행할 일이 있으면 CPU를 계속 사용	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)
U01-263	프로세스를 빈번하게 전환하여 빠른 응답	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)
U01-264	관계	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)
U01-265	여러 프로그램을 메모리에 유지	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-266	다중 프로그래밍의 논리적 확장	(다중 프로그래밍과 다중 태스킹, p.24-26 / PDF p.28-30)
U01-267	메인 메모리는 프로세서가 직접 액세스할 수 있는 유일한 대용량 저장장소이다.	(1.12, p.56 / PDF p.60)
U01-268	비휘발성 저장장치는 메인 메모리의 확장이며 대량의 데이터를 영구적으로 보유할 수 있다.	(1.12, p.56 / PDF p.60)
U01-269	다중 프로그래밍은 여러 작업을 동시에 메모리에 적재하여 CPU가 실행할 작업이 있게 한다.	(1.12, p.56 / PDF p.60)
U01-270	다중 태스킹은 CPU 스케줄링 알고리즘이 프로세스 간에 빠르게 전환하여 빠른 응답 시간을 제공하는 다중 프로그래밍의 확장이다.	(1.12, p.56 / PDF p.60)
U01-271	특권을 가진 명령어는 커널 모드에서만 실행될 수 있다.	(1.12, p.56 / PDF p.60)
U01-272	저장장치 시스템은 속도와 비용에 따른 계층 구조로 구성되며, 계층이 내려갈수록 일반적으로 비트당 비용은 감소하고 액세스 시간은 증가한다.	(1.12, p.56 / PDF p.60)
U01-273	다중 프로그래밍	(1.12, p.56 / PDF p.60)
U01-274	다중 태스킹	(1.12, p.56 / PDF p.60)

제2장. 운영체제 구조

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-001	제공 PDF는 인쇄 59쪽 다음에 117쪽으로 이어진다.	(2, PDF p.1)
U02-002	제2장 본문은 원문에서 확인 불가.	(2, PDF p.1)
U02-003	아래는 목차에서 확인한 절 이름이며, 해당 개념의 정의나 작동 원리를 요약한 것으로 표시하지 않는다.	(2, PDF p.1)
U02-004	2.1	(2, PDF p.1)
U02-005	운영체제 서비스	(2, PDF p.1)
U02-006	2.2	(2, PDF p.1)
U02-007	사용자와 운영체제 인터페이스	(2, PDF p.1)
U02-008	2.3	(2, PDF p.1)
U02-009	시스템 콜	(2, PDF p.1)
U02-010	2.4	(2, PDF p.1)
U02-011	시스템 서비스	(2, PDF p.1)
U02-012	2.5	(2, PDF p.1)
U02-013	링커와 로더	(2, PDF p.1)
U02-014	2.6	(2, PDF p.1)
U02-015	응용 프로그램이 운영체제마다 다른 이유	(2, PDF p.1)
U02-016	2.7	(2, PDF p.1)
U02-017	운영체제 설계 및 구현	(2, PDF p.1)
U02-018	2.8	(2, PDF p.1)
U02-019	운영체제 구조	(2, PDF p.1)
U02-020	2.9	(2, PDF p.1)
U02-021	운영체제 빌딩과 부팅	(2, PDF p.1)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-022	2.10	(2, PDF p.1)
U02-023	운영체제 디버깅	(2, PDF p.1)
U02-024	2.11	(2, PDF p.1)
U02-025	요약	(2, PDF p.1)

제3장. 프로세스

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-001	프로세스는 실행 중인 프로그램이며 현재 활동 상태는 프로그램 카운터와 다른 레지스터로 나타낸다.	(3.9, p.169 / PDF p.116)
U03-002	공유 메모리를 사용하는 프로세스 간 통신에서는 두 개 이상의 프로세스가 동일한 메모리 영역을 공유한다.	(3.9, p.170 / PDF p.117)
U03-003	원격 프로시저 호출(RPC)은 다른 컴퓨터의 프로세스에서 함수를 호출할 수 있는 방식으로 함수 호출 개념을 추상화한다.	(3.9, p.170 / PDF p.117)
U03-004	프로세스는 실행 중인 프로그램이다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-005	현재 활동은 프로그램 카운터와 프로세서 레지스터의 내용으로 표현된다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-006	프로그램은 명령어를 저장한 수동적 파일이지만 프로세스는 다음 명령과 자원의 집합을 가진 능동적 개체이다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-007	실행 파일이 메모리에 적재되면 프로세스가 된다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-008	같은 프로그램에 연관되어도 각 프로세스의 실행 순서와 데이터·힙·스택은 별개일 수 있다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-009	텍스트	(3.1.1, p.118-120 / PDF p.65-67)
U03-010	실행 코드. 실행 중 크기는 고정된다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-011	데이터	(3.1.1, p.118-120 / PDF p.65-67)
U03-012	전역 변수. 실행 중 크기는 고정된다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-013	힙	(3.1.1, p.118-120 / PDF p.65-67)
U03-014	실행 중 동적으로 할당되는 메모리. 할당·반환에 따라 변한다.	(3.1.1, p.118-120 / PDF p.65-67)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-015	스택	(3.1.1, p.118-120 / PDF p.65-67)
U03-016	함수 매개변수·복귀 주소·지역 변수 등 임시 데이터. 호출 시 활성화 레코드를 넣고 복귀 시 제거한다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-017	스택과 힙은 동적으로 커지더라도 서로 겹치지 않아야 한다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-018	원문의 C 메모리 배치 보충 설명은 전역 데이터를 초기화된 데이터와 bss로 구별하고 argc·argv 저장 영역도 보인다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-019	size 명령의 text·data·bss와 그 합계를 나타내는 dec·hex 필드를 설명한다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-020	JVM처럼 프로세스 자체가 다른 프로그램의 실행 환경이 될 수도 있다.	(3.1.1, p.118-120 / PDF p.65-67)
U03-021	새로운(new)	(3.1.2, p.121 / PDF p.68)
U03-022	생성 중이다.	(3.1.2, p.121 / PDF p.68)
U03-023	실행(running)	(3.1.2, p.121 / PDF p.68)
U03-024	명령어를 실행하고 있다.	(3.1.2, p.121 / PDF p.68)
U03-025	대기(waiting)	(3.1.2, p.121 / PDF p.68)
U03-026	입출력 완료나 신호 등 이벤트를 기다린다.	(3.1.2, p.121 / PDF p.68)
U03-027	준비(ready)	(3.1.2, p.121 / PDF p.68)
U03-028	처리기 할당을 기다린다.	(3.1.2, p.121 / PDF p.68)
U03-029	종료(terminated)	(3.1.2, p.121 / PDF p.68)
U03-030	실행이 끝났다.	(3.1.2, p.121 / PDF p.68)
U03-031	이름과 세부 구분은 운영체제마다 다를 수 있다.	(3.1.2, p.121 / PDF p.68)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-032	한순간 한 처리기 코어에서 실행되는 프로세스는 하나이지만 준비·대기 상태에는 여러 프로세스가 있을 수 있다.	(3.1.2, p.121 / PDF p.68)
U03-033	준비와 대기는 각각 CPU와 이벤트를 기다린다는 차이가 있다.	(3.1.2, p.121 / PDF p.68)
U03-034	그림 3.2 프로세스 상태 다이어그램	(3.1.2, p.121 / PDF p.68)
U03-035	프로세스 제어 블록(PCB)은 운영체제에서 각 프로세스를 표현하는 커널 자료구조이다.	(3.1.3, p.121-122 / PDF p.68-69)
U03-036	실행을 시작하거나 다시 시작하는 데 필요한 상태와 회계 정보를 저장한다.	(3.1.3, p.121-122 / PDF p.68-69)
U03-037	프로세스 상태와 다음 명령의 주소인 프로그램 카운터	(3.1.3, p.121-122 / PDF p.68-69)
U03-038	인터럽트 후 복원을 위해 보존할 CPU 레지스터와 상태 코드	(3.1.3, p.121-122 / PDF p.68-69)
U03-039	우선순위·스케줄링 큐 포인터·매개변수	(3.1.3, p.121-122 / PDF p.68-69)
U03-040	기준·한계 레지스터, 페이지·세그먼트 테이블 등 메모리 관리 정보	(3.1.3, p.121-122 / PDF p.68-69)
U03-041	CPU 시간·경과 시간·제한·계정·프로세스 식별 정보	(3.1.3, p.121-122 / PDF p.68-69)
U03-042	할당된 입출력 장치와 열린 파일 목록	(3.1.3, p.121-122 / PDF p.68-69)
U03-043	단일 실행 스레드를 가진 프로세스는 한 번에 하나의 실행 흐름을 수행한다.	(3.1.4, p.122-123 / PDF p.69-70)
U03-044	여러 스레드를 허용하면 입력 처리와 철자 검사처럼 한 프로세스의 여러 일을 함께 수행할 수 있다.	(3.1.4, p.122-123 / PDF p.69-70)
U03-045	다중 처리기에서는 스레드가 병렬 실행될 수 있다.	(3.1.4, p.122-123 / PDF p.69-70)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-046	스레드 지원을 위해 PCB에 스레드별 정보를 포함하는 등 시스템 전체의 수정이 필요하며 상세 내용은 제4장에 이어진다.	(3.1.4, p.122-123 / PDF p.69-70)
U03-047	프로세스 스케줄러는 실행 가능한 프로세스 가운데 CPU 코어에 할당할 대상을 선택한다.	(3.2, p.123 / PDF p.70)
U03-048	다중 프로그래밍은 CPU 이용을 최대화하고 시분할은 빈번한 교체로 사용자 상호 작용을 지원한다.	(3.2, p.123 / PDF p.70)
U03-049	메모리에 있는 프로세스 수는 다중 프로그래밍 정도이다.	(3.2, p.123 / PDF p.70)
U03-050	입출력에 더 많은 시간을 쓰는 I/O 바운드와 계산에 더 많은 시간을 쓰는 CPU 바운드의 일반적 동작을 함께 고려해야 한다.	(3.2, p.123 / PDF p.70)
U03-051	준비 큐는 CPU에서 실행되기를 기다리는 프로세스를, 대기 큐는 입출력 완료 등 특정 이벤트를 기다리는 프로세스를 담는다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-052	준비 큐는 PCB를 연결한 리스트로 구현할 수 있다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-053	새 프로세스는 준비 큐에서 디스패치를 기다린다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-054	입출력을 요청하면 해당 대기 큐에서 완료를 기다린다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-055	자식을 생성한 뒤 종료를 기다리면 자식 종료 대기 큐로 간다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-056	타임 슬라이스 만료나 인터럽트로 코어에서 제거되면 준비 큐로 돌아갈 수 있다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-057	이벤트 완료 후 대기에서 준비로 돌아와 다시 스케줄된다. 종료하면 큐에서 제거되고 PCB와 자원이 반환된다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-058	Linux 보충 설명에서 task_struct는 프로세스 상태·스케줄링·부모와 자식·열린 파일·주소 공간을 표현한다.	(3.2.1, p.123-126 / PDF p.70-73)
U03-059	활성 프로세스는 이중 연결 리스트로 조직되고 current가 현재 실행 프로세스를 가리킨다.	(3.2.1, p.123-126 / PDF p.70-73)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-060	CPU 스케줄러는 준비 큐에서 하나를 골라 CPU 코어를 할당한다.	(3.2.2, p.126 / PDF p.73)
U03-061	I/O 바운드는 짧게 실행한 뒤 대기하고 CPU 바운드는 긴 계산을 요구하므로 스케줄러는 자주 실행된다.	(3.2.2, p.126 / PDF p.73)
U03-062	스와핑은 메모리 부족 등의 상황에서 프로세스를 디스크로 내보내 다중 프로그래밍 정도를 낮추고 나중에 상태를 복원해 중단 지점부터 재개하는 기법이다.	(3.2.2, p.126 / PDF p.73)
U03-063	문맥은 PCB에 표현되는 CPU 레지스터 값·프로세스 상태·메모리 관리 정보 등이다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-064	인터럽트 후 재개하려면 현재 상태를 저장하고 복원해야 한다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-065	다른 프로세스로 CPU를 넘기는 문맥 교환은 이전 프로세스의 문맥을 PCB에 저장하고 다음 프로세스의 문맥을 복구한다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-066	교환 중에는 유용한 사용자 작업을 하지 못하므로 순수한 오버헤드이다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-067	메모리 속도, 저장할 레지스터 수, 특수 명령, 레지스터 집합 지원 및 메모리 관리 기법의 복잡도에 따라 시간이 달라진다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-068	여러 레지스터 집합이 있으면 집합 포인터 변경으로 교환할 수 있으나 활성 프로세스가 더 많으면 메모리 복사가 다시 필요하다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-069	모바일 다중 태스킹 보충 설명은 전경·백그라운드와 Android의 사용자 인터페이스 없는 서비스를 구별한다.	(3.2.3, p.126-128 / PDF p.73-75)
U03-070	프로세스를 생성하는 쪽은 부모, 새로 생성되는 쪽은 자식이다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-071	반복 생성으로 프로세스 트리를 이룬다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-072	운영체제는 유일한 정수 pid로 프로세스를 식별한다.	(3.3.1, p.129-134 / PDF p.76-81)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-073	원문의 Linux 사례에서는 systemd가 pid 1인 루트 부모이며 ps·pstree로 프로세스 및 계층을 살펴본다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-074	자식은 운영체제로부터 직접 자원을 받거나 부모 자원의 일부만 사용하도록 제한될 수 있다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-075	부모가 자원을 나누거나 공유하고 초기화 데이터를 넘길 수도 있다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-076	부모는 자식과 병행 실행하거나 일부·전체 자식 종료를 기다린다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-077	주소 공간은 부모의 복사본일 수도, 새 프로그램으로 적재될 수도 있다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-078	UNIX fork()는 부모 주소 공간의 복사본인 새 프로세스를 만들며 두 프로세스는 호출 다음부터 실행한다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-079	부모에는 자식 pid가, 자식에는 0이 반환된다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-080	exec()는 현재 주소 공간을 새 프로그램으로 덮어쓰며 오류가 없으면 돌아오지 않는다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-081	부모의 wait()는 자식 종료까지 기다린다.	(3.3.1, p.129-134 / PDF p.76-81)

번호	문장·표 항목·원문 코드·수식	원문 위치

U03-082	그림 3.8 fork()로 별도 프로세스 생성 (원문 코드 발췌)
	#include <sys/types.h>
	#include <stdio.h>
	#include <unistd.h>
	
	int main()
	{
	pid_t pid;
	
	/* 새 프로세스를 생성한다(fork) */
	pid = fork();
	
	if (pid < 0) { /* 오류가 발생했음 */
	fprintf(stderr,"Fork Failed");
	return 1;
	}
	else if (pid == 0) { /* 자식 프로세스 */
	execlp("/bin/ls","ls",NULL);
	}
	else { /* 부모 프로세스 */
	/* 부모가 자식이 완료되기를 기다릴 것임 */
	wait(NULL);
	printf("Child Complete");
	}
	
	return 0;
	}
		(3.3.1, p.132 / PDF p.79)

U03-083	Windows CreateProcess()는 생성과 함께 지정된 프로그램을 자식 주소 공간에 적재한다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-084	STARTUPINFO는 새 프로세스의 특성을, PROCESS_INFORMATION은 프로세스·스레드 핸들과 식별자를 제공한다.	(3.3.1, p.129-134 / PDF p.76-81)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-085	원문 mspaint.exe 예제는 구조체 초기화 후 생성하고 WaitForSingleObject()로 종료를 기다린 뒤 핸들을 닫는다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-086	fork()처럼 부모 주소 공간을 먼저 복사하는 방식과 구별한다.	(3.3.1, p.129-134 / PDF p.76-81)
U03-087	프로세스는 마지막 문장을 끝내고 exit()로 삭제를 요청하며 부모에게 종료 상태를 반환할 수 있다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-088	운영체제는 메모리·열린 파일·입출력 버퍼를 회수한다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-089	부모는 자원 초과·불필요한 작업·자신의 종료 등으로 자식을 종료시킬 수도 있다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-090	일부 시스템은 부모 종료 시 모든 후손을 종료하는 연쇄식 종료를 시행한다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-091	좀비	(3.3.2, p.134-136 / PDF p.81-83)
U03-092	종료했지만 부모가 wait()하지 않아 종료 상태를 담은 프로세스 테이블 항목이 남아 있다. 부모가 상태를 수집하면 pid와 항목도 반환된다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-093	고아	(3.3.2, p.134-136 / PDF p.81-83)
U03-094	부모가 wait() 대신 먼저 종료한 자식이다. 전통적 UNIX는 init가 새 부모가 되어 종료 상태를 수집한다. 원문 Linux에서는 systemd 외 프로세스의 상속도 허용한다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-095	Android는 자원이 부족하면 중요도가 낮은 프로세스부터 종료한다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-096	원문의 중요도 순서는 전경 → 가시적 → 서비스 → 백그라운드 → 빈 프로세스이다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-097	여러 조건을 만족하면 가능한 높은 등급을 지정한다.	(3.3.2, p.134-136 / PDF p.81-83)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-098	수명주기 지침은 종료 전에 상태를 보존하고 사용자가 돌아왔을 때 재개하도록 한다.	(3.3.2, p.134-136 / PDF p.81-83)
U03-099	다른 프로세스에 영향을 주거나 받는 협력 프로세스에는 정보 공유·계산 가속화·모듈성의 이점이 있다.	(3.4, p.137-139 / PDF p.84-86)
U03-100	계산을 나누어 실제 속도를 높이려면 여러 처리 코어가 필요하다.	(3.4, p.137-139 / PDF p.84-86)
U03-101	협력을 위한 IPC에는 공유 메모리와 메시지 전달이라는 두 기본 모델이 있다.	(3.4, p.137-139 / PDF p.84-86)
U03-102	정보 교환	(3.4, p.137-139 / PDF p.84-86)
U03-103	공유 영역을 읽고 쓴다.	(3.4, p.137-139 / PDF p.84-86)
U03-104	메시지를 보내고 받는다.	(3.4, p.137-139 / PDF p.84-86)
U03-105	운영체제 관여	(3.4, p.137-139 / PDF p.84-86)
U03-106	공유 영역 설정 후 일반 메모리 접근으로 수행한다.	(3.4, p.137-139 / PDF p.84-86)
U03-107	통상 시스템 콜을 이용해 커널 개입 비용이 든다.	(3.4, p.137-139 / PDF p.84-86)
U03-108	원문의 특징	(3.4, p.137-139 / PDF p.84-86)
U03-109	설정 후 빠른 통신이 가능하지만 접근 조정은 프로세스가 책임진다.	(3.4, p.137-139 / PDF p.84-86)
U03-110	적은 데이터 교환에 유용하고 분산 시스템에서 구현하기 쉽다.	(3.4, p.137-139 / PDF p.84-86)
U03-111	Chrome 사례는 브라우저·렌더러·플러그인 프로세스를 분리하여 한 사이트의 고장이 다른 사이트 전체로 퍼지는 영향을 줄이고 렌더러의 디스크·네트워크 접근을 샌드박스로 제한하는 구성을 보여 준다.	(3.4, p.137-139 / PDF p.84-86)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-112	이 사례는 원문이 기술한 구조의 설명이다.	(3.4, p.137-139 / PDF p.84-86)
U03-113	통신 프로세스는 공유 세그먼트를 만들고 각자의 주소 공간에 추가한다.	(3.5, p.139-141 / PDF p.86-88)
U03-114	데이터의 형식·위치는 프로세스가 정하며 동일 위치의 동시 쓰기를 방지할 책임도 프로세스에 있다.	(3.5, p.139-141 / PDF p.86-88)
U03-115	생산자·소비자에서는 생산자가 채우고 소비자가 소모할 공유 버퍼와 동기화가 필요하다.	(3.5, p.139-141 / PDF p.86-88)
U03-116	무한 버퍼에서 생산자는 계속 생산할 수 있지만 소비자는 항목을 기다릴 수 있다.	(3.5, p.139-141 / PDF p.86-88)
U03-117	유한 버퍼는 비었으면 소비자가, 가득 찼으면 생산자가 대기한다.	(3.5, p.139-141 / PDF p.86-88)
U03-118	원문의 원형 배열에서 in은 다음 빈 위치, out은 다음 소비 위치이다.	(3.5, p.139-141 / PDF p.86-88)
U03-119	in == out이면 비었고 (in + 1) % BUFFER_SIZE == out이면 가득 찬 것으로 본다.	(3.5, p.139-141 / PDF p.86-88)

U03-120	그림 3.12 공유 메모리 생산자 (원문 코드 발췌)
	item next_produced;
	
	while (true) {
	/* produce an item in next_produced */
	
	while (((in + 1) % BUFFER_SIZE) == out)
	; /* do nothing */
	
	buffer[in] = next_produced;
	in = (in + 1) % BUFFER_SIZE;
	}
		(3.5, p.141 / PDF p.88)

번호	문장·표 항목·원문 코드·수식	원문 위치

U03-121	그림 3.13 공유 메모리 소비자 (원문 코드 발췌)
	item next_consumed;
	
	while (true) {
	while (in == out)
	; /* do nothing */
	
	next_consumed = buffer[out];
	out = (out + 1) % BUFFER_SIZE;
	
	/* consume the item in next_consumed */
	}
		(3.5, p.141 / PDF p.88)

U03-122	이 구현은 BUFFER_SIZE개의 배열에 최대 BUFFER_SIZE – 1개 항목만 담는다.	(3.5, p.139-141 / PDF p.86-88)
U03-123	배열 전체를 사용하는 새 해결책은 원문 연습 대상으로 남겨져 있으므로 추가하지 않는다.	(3.5, p.139-141 / PDF p.86-88)
U03-124	이 절의 예제만으로 병행 접근의 동기화 문제가 해결되는 것은 아니며 원문은 이를 제6·7장으로 연결한다.	(3.5, p.139-141 / PDF p.86-88)
U03-125	메시지 전달은 동일 주소 공간을 공유하지 않고 통신·동기화하게 하며 네트워크로 분리된 프로세스에 유용하다.	(3.6, p.141-142 / PDF p.88-89)
U03-126	최소 연산은 send(message)와 receive(message)이다.	(3.6, p.141-142 / PDF p.88-89)
U03-127	고정 길이는 시스템 구현이 단순하지만 프로그램 작성을 어렵게 하고, 가변 길이는 프로그램 작성은 편하지만 시스템 구현이 복잡하다.	(3.6, p.141-142 / PDF p.88-89)
U03-128	통신 링크에는 명명·동기화·버퍼링의 설계 선택이 있다.	(3.6, p.141-142 / PDF p.88-89)
U03-129	직접 통신에서는 송신자와 수신자가 상대 프로세스를 명시한다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-130	통신을 원하는 두 프로세스 사이에 링크가 자동 구성되고 각 쌍에 하나의 연결이 생긴다.	(3.6.1, p.142-145 / PDF p.89-92)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-131	대칭 주소 지정은 둘 다 상대 이름을, 비대칭은 송신자만 수신자 이름을 지정한다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-132	이름을 코드에 직접 넣으면 이름 변경 시 모든 참조를 바꿔야 하므로 모듈성이 제한된다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-133	간접 통신은 고유 식별자를 가진 메일박스·포트로 송수신한다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-134	공유 메일박스가 있을 때만 연결되고 하나의 링크가 여러 프로세스와 관련되거나 한 쌍 사이에 여러 링크가 있을 수 있다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-135	여러 수신자가 같은 메일박스를 공유하면 링크당 두 프로세스로 제한하거나 한 번에 하나만 수신하게 하거나 시스템이 한 수신자를 선택하는 규칙이 필요하다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-136	프로세스 소유 메일박스는 소유자가 수신하고 종료하면 사라진다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-137	운영체제 소유 메일박스는 독립적으로 존재하며 생성·송수신·삭제 기법이 필요하다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-138	생성자의 소유권과 수신 권한은 적절한 시스템 콜로 이전할 수 있다.	(3.6.1, p.142-145 / PDF p.89-92)
U03-139	봉쇄형 보내기	(3.6.2, p.145-146 / PDF p.92-93)
U03-140	수신 프로세스나 메일박스가 메시지를 받을 때까지 송신자가 기다린다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-141	비봉쇄형 보내기	(3.6.2, p.145-146 / PDF p.92-93)
U03-142	보낸 후 자신의 실행을 계속한다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-143	봉쇄형 받기	(3.6.2, p.145-146 / PDF p.92-93)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-144	메시지가 사용 가능할 때까지 기다린다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-145	비봉쇄형 받기	(3.6.2, p.145-146 / PDF p.92-93)
U03-146	유효한 메시지 또는 널을 반환받는다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-147	보내기와 받기가 모두 봉쇄형이면 송신자와 수신자의 랑데부가 된다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-148	생산자는 메시지가 전달될 때까지, 소비자는 메시지가 도착할 때까지 기다리는 방식으로 생산자·소비자를 표현할 수 있다.	(3.6.2, p.145-146 / PDF p.92-93)
U03-149	무용량	(3.6.3, p.146 / PDF p.93)
U03-150	대기 메시지를 보관하지 못하므로 수신자가 받을 때까지 기다린다.	(3.6.3, p.146 / PDF p.93)
U03-151	유한 용량	(3.6.3, p.146 / PDF p.93)
U03-152	빈자리가 있으면 넣고 계속하지만 가득 차면 공간을 기다린다.	(3.6.3, p.146 / PDF p.93)
U03-153	무한 용량	(3.6.3, p.146 / PDF p.93)
U03-154	메시지를 계속 보관할 수 있어 송신자는 버퍼 때문에 봉쇄되지 않는다.	(3.6.3, p.146 / PDF p.93)
U03-155	무용량은 버퍼 없는 시스템으로, 나머지는 자동 버퍼링으로 부른다.	(3.6.3, p.146 / PDF p.93)
U03-156	직접·간접 통신이라는 명명 방식과 버퍼 용량이라는 조건을 서로 다른 설계 기준으로 구별한다.	(3.6.3, p.146 / PDF p.93)
U03-157	shm_open()으로 이름 있는 공유 메모리 객체를 만들거나 연다. 이름·열기 플래그·접근 허가권을 지정하고 파일 설명자를 받는다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-158	ftruncate()로 바이트 단위 크기를 설정한다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-159	mmap()으로 객체를 프로세스의 주소 공간에 매핑하고 반환 포인터로 읽거나 쓴다.	(3.7.1, p.147-149 / PDF p.94-96)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-160	소비자는 사용을 마친 뒤 shm_unlink()로 이름 있는 공유 메모리를 제거한다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-161	원문 생산자는 OS라는 4096바이트 객체에 Hello와 World!	(3.7.1, p.147-149 / PDF p.94-96)
U03-162	문자열을 쓰며 포인터를 기록한 바이트 수만큼 이동한다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-163	MAP_SHARED는 공유하는 프로세스가 변경 값을 접근하도록 지정한다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-164	아래 코드는 원본 그림으로 보존하였다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-165	별도 실행 출력은 원문에 실행 결과 미제시.	(3.7.1, p.147-149 / PDF p.94-96)
U03-166	그림 3.16 POSIX 공유 메모리 생산자 코드	(3.7.1, p.148 / PDF p.95)
U03-167	그림 3.17 POSIX 공유 메모리 소비자 코드	(3.7.1, p.149 / PDF p.96)
U03-168	원문 확인 필요: 소비자 그림은 shm_open()에 O_RDONLY를 쓰면서 mmap() 보호 인수에는 PROT_READ | PROT_WRITE를 표시한다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-169	원본의 플래그를 임의로 고치지 않고 그림 그대로 남겼다.	(3.7.1, p.147-149 / PDF p.94-96)
U03-170	Mach의 태스크 간 통신은 대부분 포트로 메시지를 주고받는다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-171	포트는 크기가 정해진 단방향 메일박스이며 여러 송신자와 하나의 수신자를 가질 수 있다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-172	응답은 별도 포트로 보낸다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-173	포트 권한은 태스크에 속해 같은 태스크의 스레드가 공유한다.	(3.7.2, p.149-153 / PDF p.96-100)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-174	생성자는 수신 권한을 갖고 다른 태스크에 송신 권한을 부여할 수 있다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-175	Task Self·Notify 포트와 부트스트랩 서버는 커널 통신·이벤트 알림·포트 발견을 지원한다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-176	mach_msg()의 메시지는 고정 헤더와 가변 본체로 구성된다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-177	헤더에는 크기·소스·대상 포트 등의 메타데이터가 있다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-178	단순 메시지는 해석되지 않는 데이터, 복잡한 메시지는 out-of-line 메모리 포인터나 포트 권한을 포함할 수 있다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-179	같은 송신자의 메시지는 FIFO로 넣지만 서로 다른 송신자 사이의 절대 순서는 보장하지 않는다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-180	큐가 가득 차면 송신자는 무한 대기·제한 시간 대기·즉시 복귀·일시 캐시 중에서 선택할 수 있다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-181	캐시 선택은 다른 요청을 계속 처리해야 하는 서버의 일회성 응답에 쓰이며 송신 스레드당 하나만 커널에 보관한다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-182	같은 시스템 내부에서는 가상 메모리 매핑을 이용해 실제 데이터 복사를 피할 수 있지만 다른 시스템 사이에는 이 기법이 적용되지 않는다.	(3.7.2, p.149-153 / PDF p.96-100)
U03-183	ALPC(고급 로컬 프로시저 호출)는 동일 기계의 프로세스 사이 통신에 최적화된 메시지 전달 설비이다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-184	서버가 공개 연결 포트를 만들면 클라이언트가 연결을 요청하고 서버는 한 쌍의 사적 통신 포트로 채널을 만든다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-185	콜백은 응답 대기 중에도 다른 요청을 처리하도록 한다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-186	256바이트 이하	(3.7.3, p.153-154 / PDF p.100-101)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-187	포트 메시지 큐를 중간 저장소로 사용하여 복사한다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-188	큰 메시지	(3.7.3, p.153-154 / PDF p.100-101)
U03-189	채널에 연관된 공유 메모리인 섹션 객체와 그 포인터·크기를 보낸다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-190	섹션 객체보다 더 큼	(3.7.3, p.153-154 / PDF p.100-101)
U03-191	서버가 클라이언트 주소 공간을 직접 읽고 쓰는 API를 사용할 수 있다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-192	ALPC는 Windows API의 직접 사용 가능한 부분이 아니다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-193	응용은 표준 RPC를 호출하고 같은 시스템의 RPC가 간접적으로 ALPC를 통해 처리된다.	(3.7.3, p.153-154 / PDF p.100-101)
U03-194	파이프는 쓰기 종단의 생산자와 읽기 종단의 소비자를 연결한다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-195	일반 파이프는 단방향이므로 양방향 전송에는 서로 반대 방향의 파이프 두 개가 필요하다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-196	UNIX의 pipe(fd)는 fd[0]을 읽기, fd[1]을 쓰기 종단으로 만들며 read()·write()로 접근한다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-197	일반적으로 부모가 만든 뒤 fork()로 자식이 열린 파이프를 상속한다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-198	원문의 Greetings 예제에서 부모는 쓰고 자식은 읽으며 양쪽은 쓰지 않는 종단을 닫는다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-199	쓰는 쪽이 닫았을 때 읽는 쪽이 EOF를 검출하도록 하는 데 이 절차가 중요하다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-200	Windows의 익명 파이프도 부모·자식 사이의 단방향 통신이다.	(3.7.4.1, p.155-160 / PDF p.102-107)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-201	CreatePipe() 이후 상속 허용과 표준 입력·출력 재지정을 명시하며 ReadFile()·WriteFile()로 읽고 쓴다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-202	양쪽 운영체제의 일반 파이프는 동일 기계의 관련 프로세스 사이에서 사용한다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-203	파이프를 비교할 때 방향, 반이중·전이중, 프로세스 관계 요구, 네트워크 지원을 구별한다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-204	반이중은 한순간 한 방향, 전이중은 양방향 동시 전송이다.	(3.7.4.1, p.155-160 / PDF p.102-107)
U03-205	관계·수명	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-206	부모·자식일 필요가 없고 파일 시스템에서 명시적으로 제거할 때까지 존재	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-207	부모·자식일 필요가 없으며 여러 프로세스가 사용 가능	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-208	전송	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-209	반이중, 바이트 단위. 양방향에는 보통 두 FIFO 사용	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-210	전이중, 바이트·메시지 단위	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-211	범위	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-212	같은 기계	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-213	같거나 다른 기계	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-214	UNIX는 mkfifo()로 만들고 일반 파일처럼 open()·read()·write()·close()한다.	(3.7.4.2, p.160-161 / PDF p.107-108)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-215	원문 셸 사례 ls | less와 dir | more는 앞 명령의 출력을 뒤 명령의 입력으로 전달하여 한 화면씩 표시하는 생산자·소비자 관계를 보여 준다.	(3.7.4.2, p.160-161 / PDF p.107-108)
U03-216	소켓은 통신의 극점이며 네트워크 통신에는 각 프로세스에 하나씩 두 소켓이 필요하다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-217	IP 주소와 포트 번호의 조합으로 구별하고 연결은 유일한 소켓 쌍으로 구성된다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-218	서버는 지정 포트에서 요청을 기다리고 클라이언트의 연결을 수락한다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-219	원문은 well-known 서비스 포트와 클라이언트에게 부여하는 포트를 구별한다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-220	Java의 Socket은 연결 기반 TCP, DatagramSocket은 비연결 UDP, MulticastSocket은 여러 수신자에게 보내는 소켓이다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-221	Date 서버 예제는 포트 6013에서 accept()로 기다리고 연결되면 PrintWriter로 날짜·시간을 전송한 뒤 소켓을 닫는다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-222	클라이언트는 127.0.0.1:6013에 연결하여 스트림으로 읽고 닫는다. 127.0.0.1은 자기 기계를 가리키는 루프백 주소이다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-223	소켓은 효율적이고 널리 쓰이지만 구조화되지 않은 바이트 스트림만 제공하는 낮은 수준이다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-224	데이터의 구조와 해석은 클라이언트·서버의 책임이며, 더 높은 수준의 대안이 RPC이다.	(3.8.1, p.162-165 / PDF p.109-112)
U03-225	원문의 날짜 출력에 구체적인 실제 실행 값은 원문에 실행 결과 미제시.	(3.8.1, p.162-165 / PDF p.109-112)
U03-226	RPC는 네트워크의 다른 시스템에 있는 프로시저를 지역 프로시저처럼 호출하도록 추상화한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-227	메시지는 원격 서비스 주소·함수 식별자·매개변수를 포함하고 수행 결과는 별도 메시지로 반환한다.	(3.8.2, p.164-168 / PDF p.111-115)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-228	클라이언트와 서버의 스텁이 통신 세부사항을 숨긴다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-229	클라이언트가 원격 프로시저에 대응하는 스텁을 호출하고 인수를 건넨다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-230	스텁이 서버 포트를 찾고 매개변수를 정돈(marshalling)하여 전송한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-231	서버 스텁이 메시지를 받아 풀어내고 실제 프로시저를 호출한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-232	필요한 반환값도 반대 방향으로 정돈·전달한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-233	기계마다 정수의 바이트 순서가 다를 수 있다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-234	XDR 같은 기종 중립 표현은 송신 시 변환하고 수신 시 해당 기종 표현으로 되돌려 차이를 다룬다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-235	원문은 big-endian과 little-endian 가운데 어느 쪽이 더 낫다는 문제가 아니라 구조상의 선택이라고 설명한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-236	네트워크 오류는 요청 손실·중복 실행 문제를 만든다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-237	원문의 최대 한 번 방식은 타임스탬프와 처리 이력으로 중복 요청을 무시한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-238	정확히 한 번 설명은 여기에 수신·실행 ACK와 ACK 전까지의 재전송을 더한다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-239	서버 고장과 ACK 손실의 경우는 장말 연습문제의 검토 대상이므로 별도의 보장이나 풀이를 창작하지 않는다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-240	바인딩은 서버 포트를 찾는 과정이다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-241	고정 포트는 컴파일 시 정하지만 이후 변경이 어렵다.	(3.8.2, p.164-168 / PDF p.111-115)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-242	동적 방식은 고정된 matchmaker에 RPC 이름을 보내 포트를 얻은 뒤 실제 요청을 보낸다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-243	초기에 오버헤드가 있지만 유연하다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-244	원문은 분산 파일 시스템의 read·write·rename·delete·status 요청도 활용 예로 든다.	(3.8.2, p.164-168 / PDF p.111-115)
U03-245	그림 3.29 RPC의 실행	(3.8.2, p.168 / PDF p.115)
U03-246	RPC는 같은 기계의 IPC에도 쓰이며 Android는 바인더 프레임워크로 이를 제공한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-247	서비스는 사용자 인터페이스 없이 백그라운드 연산을 수행한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-248	bindService()가 서비스를 바인드하면 서버의 onBind()가 호출된다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-249	메시지 전달의 경우 onBind()는 Messenger를 반환한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-250	단방향이므로 응답하려면 클라이언트의 Messenger를 Message의 replyTo에 포함한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-251	RPC의 경우 원격 메소드를 정의한 인터페이스를 반환하며 AIDL에서 Java 인터페이스와 스텁을 생성한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-252	바인더가 매개변수 정돈·전달·서비스 호출·반환값 전송을 처리한다.	(3.8.2.1, p.167-169 / PDF p.114-116)
U03-253	프로세스의 메모리·상태·PCB는 스케줄링과 문맥 교환의 기반이다.	(3.9, p.169-173 / PDF p.116-120)
U03-254	생성·종료는 부모·자식 관계와 자원 회수로 이어지고 협력은 IPC를 요구한다.	(3.9, p.169-173 / PDF p.116-120)
U03-255	장말 연습문제 3.1~3.7은 fork 결과·프로세스 수·문맥 교환·공유 상태·RPC 전달 의미를 점검한다.	(3.9, p.169-173 / PDF p.116-120)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-256	문제의 코드도 검토하였으나 원문에 없는 실행 결과나 새 풀이를 작성하지 않았다.	(3.9, p.169-173 / PDF p.116-120)
U03-257	추가 자료·참고문헌의 외부 내용은 사용하지 않았다.	(3.9, p.169-173 / PDF p.116-120)
U03-258	교환 단위	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-259	공유 영역의 데이터	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-260	송수신 메시지	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-261	커널 관여	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-262	영역 설정 후 일반 메모리 접근	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-263	통상 각 통신에 시스템 콜 사용	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-264	책임	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-265	자료 형식과 동시 접근 조정	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-266	이름·봉쇄·버퍼링 규칙 선택	(IPC의 기본 모델, p.137-139 / PDF p.84-86)
U03-267	핵심 조건	(좀비와 고아, p.135-136 / PDF p.82-83)
U03-268	종료한 자식의 상태를 부모가 아직 수집하지 않음	(좀비와 고아, p.135-136 / PDF p.82-83)
U03-269	부모가 자식보다 먼저 종료	(좀비와 고아, p.135-136 / PDF p.82-83)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-270	처리	(좀비와 고아, p.135-136 / PDF p.82-83)
U03-271	부모의 wait가 항목을 회수	(좀비와 고아, p.135-136 / PDF p.82-83)
U03-272	다른 부모가 인계받아 종료 상태를 수집	(좀비와 고아, p.135-136 / PDF p.82-83)
U03-273	프로세스 제어 블록(PCB)은 운영체제의 프로세스를 나타내는 커널 데이터 구조이다.	(3.9, p.170 / PDF p.117)
U03-274	문맥 교환은 한 프로세스 실행에서 다른 프로세스 실행으로 전환할 때 수행된다.	(3.9, p.170 / PDF p.117)
U03-275	메시지 전달에서는 두 프로세스가 서로 메시지를 교환하여 통신한다.	(3.9, p.170 / PDF p.117)
U03-276	파이프는 두 프로세스가 통신할 수 있는 통로를 제공하며 일반 파이프와 지명 파이프의 두 형태가 있다.	(3.9, p.170 / PDF p.117)
U03-277	소켓을 사용하면 다른 시스템의 두 프로세스가 네트워크를 통해 통신할 수 있다.	(3.9, p.170 / PDF p.117)
U03-278	fork()와 CreateProcess() 시스템 콜은 각각 UNIX와 Windows에서 프로세스를 생성하는 데 사용된다.	(3.9, p.170 / PDF p.117)
U03-279	공유 메모리	(3.9, p.170 / PDF p.117)
U03-280	메시지 전달	(3.9, p.170 / PDF p.117)

제4장. 스레드와 병행성

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-001	스레드는 CPU 이용의 기본 단위이며 스레드 ID, 프로그램 카운터, 레지스터 집합과 스택으로 구성된다.	(4.1, p.176 / PDF p.122)
U04-002	병행 시스템은 모든 작업이 진행되게 하여 둘 이상의 작업을 지원한다.	(4.2, p.179 / PDF p.125)
U04-003	암묵적 스레딩은 작업을 식별하고 언어 또는 API 프레임워크가 스레드를 만들고 관리할 수 있게 한다.	(4.8, p.216 / PDF p.162)
U04-004	스레드는 CPU 이용의 기본 단위이며 스레드 ID·프로그램 카운터·레지스터 집합·스택으로 구성된다.	(4.1, p.176-178 / PDF p.122-124)
U04-005	같은 프로세스의 스레드는 코드·데이터·열린 파일·신호 같은 자원을 공유한다.	(4.1, p.176-178 / PDF p.122-124)
U04-006	단일 스레드는 하나의 제어 흐름, 다중 스레드는 같은 프로세스 안의 여러 제어 흐름을 제공한다.	(4.1, p.176-178 / PDF p.122-124)
U04-007	원문은 브라우저의 표시와 네트워크 검색, 문서 편집기의 입력·철자 검사, 이미지 축소판 생성 등을 예로 든다.	(4.1, p.176-178 / PDF p.122-124)
U04-008	웹 서버는 요청을 듣는 스레드와 요청을 처리하는 스레드를 나눌 수 있다.	(4.1, p.176-178 / PDF p.122-124)
U04-009	같은 작업을 위해 프로세스를 매번 새로 만드는 것보다 자원을 공유하는 스레드가 효율적일 수 있다.	(4.1, p.176-178 / PDF p.122-124)
U04-010	커널도 장치·메모리 관리·인터럽트 처리를 담당하는 여러 스레드로 구성될 수 있다.	(4.1, p.176-178 / PDF p.122-124)
U04-011	응답성	(4.1, p.176-178 / PDF p.122-124)
U04-012	일부가 봉쇄되거나 긴 연산을 해도 다른 실행 흐름이 사용자에게 응답할 수 있다.	(4.1, p.176-178 / PDF p.122-124)
U04-013	자원 공유	(4.1, p.176-178 / PDF p.122-124)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-014	프로세스 간에는 명시적 IPC가 필요하지만 스레드는 같은 주소 공간과 자원을 공유한다.	(4.1, p.176-178 / PDF p.122-124)
U04-015	경제성	(4.1, p.176-178 / PDF p.122-124)
U04-016	프로세스보다 생성에 드는 시간·메모리가 적고 스레드 간 문맥 교환이 일반적으로 빠르다.	(4.1, p.176-178 / PDF p.122-124)
U04-017	규모 적응성	(4.1, p.176-178 / PDF p.122-124)
U04-018	다중 처리기에서 스레드를 다른 처리기에 병렬 배치할 수 있다. 단일 스레드는 처리기가 많아도 하나에서만 실행한다.	(4.1, p.176-178 / PDF p.122-124)
U04-019	병행성은 여러 작업이 모두 진행되도록 지원하는 것이고 병렬성은 여러 작업을 같은 순간 수행하는 것이다.	(4.2, p.178-182 / PDF p.124-128)
U04-020	단일 코어에서는 실행을 교대로 끼워 넣어 병행성을 제공할 수 있지만 병렬 실행은 하지 못한다.	(4.2, p.178-182 / PDF p.124-128)
U04-021	다중 코어는 각 코어에 다른 스레드를 배치하여 실제 동시 실행을 가능하게 한다.	(4.2, p.178-182 / PDF p.124-128)
U04-022	Amdahl의 법칙은 순차 부분과 병렬 부분이 있는 응용에 코어를 추가할 때 가능한 성능 이득을 나타낸다.	(4.2, p.178-182 / PDF p.124-128)
U04-023	원문에서 S는 반드시 순차 실행되는 비율이고 N은 처리 코어 수이다.	(4.2, p.178-182 / PDF p.124-128)
U04-024	원문 수식: speedup ≤ 1 / (S + (1 − S) / N); S: 반드시 순차적으로 실행되어야 하는 부분; N: 처리 코어 수; 원문 예는 병렬 75%·순차 25%에서 2코어 약 1.6배, 4코어 2.28배를 제시한다. N이 무한히 커지면 1/S에 수렴하므로 순차 부분이 성능 향상을 제한한다.	(4.2, p.178-182 / PDF p.124-128)
U04-025	Amdahl의 법칙 그래프	(4.2, p.180 / PDF p.126)
U04-026	원문 확인 필요: 같은 보충 설명은 순차 부분이 40%인 경우 최대 2.0배라고 기재한다.	(4.2, p.178-182 / PDF p.124-128)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-027	제시된 1/S 설명과 수치가 일치하지 않으므로 이 문구를 정답 수치로 일반화하지 않고 원문 확인이 필요한 부분으로 남긴다.	(4.2, p.178-182 / PDF p.124-128)
U04-028	임의로 수치를 고치지 않았다.	(4.2, p.178-182 / PDF p.124-128)
U04-029	태스크 인식: 독립적으로 병행 실행할 수 있는 부분을 찾는다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-030	균형: 나누어진 태스크가 전체 작업에 균등하게 기여하게 한다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-031	기여가 작은 작업에 별도 코어를 쓰는 것이 항상 유리하지 않다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-032	데이터 분리: 각 코어의 태스크가 처리할 데이터를 나눈다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-033	데이터 종속성: 한 태스크가 다른 태스크의 데이터에 의존하면 실행을 동기화한다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-034	시험·디버깅: 병렬 실행 경로가 다양하여 단일 스레드보다 검증이 어렵다.	(4.2.1, p.179-181 / PDF p.125-127)
U04-035	데이터 병렬	(4.2.2, p.181-182 / PDF p.127-128)
U04-036	데이터의 부분집합. 각 코어는 같은 연산을 한다.	(4.2.2, p.181-182 / PDF p.127-128)
U04-037	배열의 앞·뒤 부분을 다른 코어가 각각 합산한다.	(4.2.2, p.181-182 / PDF p.127-128)
U04-038	태스크 병렬	(4.2.2, p.181-182 / PDF p.127-128)
U04-039	각자 다른 연산을 수행하는 태스크	(4.2.2, p.181-182 / PDF p.127-128)
U04-040	같은 배열에 대해 서로 다른 통계 연산을 수행한다.	(4.2.2, p.181-182 / PDF p.127-128)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-041	두 방식은 상호 배타적이지 않으므로 한 응용이 혼합하여 사용할 수 있다.	(4.2.2, p.181-182 / PDF p.127-128)
U04-042	데이터를 나누는지, 수행할 연산 자체를 나누는지를 구별한다.	(4.2.2, p.181-182 / PDF p.127-128)
U04-043	사용자 스레드는 커널 위에서 라이브러리가 관리하고 커널 스레드는 운영체제가 직접 관리한다.	(4.3, p.182-185 / PDF p.128-131)
U04-044	CPU에서 실행되려면 두 수준 사이의 매핑이 필요하다.	(4.3, p.182-185 / PDF p.128-131)
U04-045	다대일	(4.3, p.182-185 / PDF p.128-131)
U04-046	여러 사용자 스레드 → 커널 스레드 하나. 사용자 공간 관리가 효율적이다.	(4.3, p.182-185 / PDF p.128-131)
U04-047	하나의 봉쇄형 시스템 콜이 전체를 봉쇄한다. 여러 코어에서 병렬 실행할 수 없다.	(4.3, p.182-185 / PDF p.128-131)
U04-048	일대일	(4.3, p.182-185 / PDF p.128-131)
U04-049	사용자 스레드마다 커널 스레드 하나. 한 스레드 봉쇄 중 다른 스레드가 실행되고 병렬 실행 가능하다.	(4.3, p.182-185 / PDF p.128-131)
U04-050	사용자 스레드 생성마다 커널 스레드가 필요하여 과도한 개수는 부담이 된다.	(4.3, p.182-185 / PDF p.128-131)
U04-051	다대다	(4.3, p.182-185 / PDF p.128-131)
U04-052	많은 사용자 스레드 → 같거나 더 적은 커널 스레드. 커널이 다른 실행 스레드를 스케줄할 수 있다.	(4.3, p.182-185 / PDF p.128-131)
U04-053	융통성이 있지만 구현이 어렵다.	(4.3, p.182-185 / PDF p.128-131)
U04-054	두 수준	(4.3, p.182-185 / PDF p.128-131)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-055	다대다를 기본으로 하면서 특정 사용자 스레드를 특정 커널 스레드에 고정할 수 있다.	(4.3, p.182-185 / PDF p.128-131)
U04-056	사용자·커널 수준의 협력이 필요하다.	(4.3, p.182-185 / PDF p.128-131)
U04-057	원문은 Linux와 Windows의 일대일 모델 및 작업을 하부 스레드에 매핑하는 현대 병행 라이브러리를 대비한다.	(4.3, p.182-185 / PDF p.128-131)
U04-058	커널 스레드 수를 제한하는 필요성과 병렬성·구현 복잡도를 함께 고려한다.	(4.3, p.182-185 / PDF p.128-131)
U04-059	스레드 라이브러리는 생성·관리를 위한 API를 제공한다.	(4.4, p.185-186 / PDF p.131-132)
U04-060	사용자 공간만의 라이브러리는 지역 함수 호출로 구현되고, 커널 지원 라이브러리는 호출이 커널 시스템 콜로 이어진다.	(4.4, p.185-186 / PDF p.131-132)
U04-061	원문은 Pthreads·Windows·Java를 설명한다.	(4.4, p.185-186 / PDF p.131-132)
U04-062	Pthreads는 사용자 또는 커널 수준 구현이 가능하며 Windows 라이브러리는 커널 수준이다.	(4.4, p.185-186 / PDF p.131-132)
U04-063	Java는 보통 호스트 운영체제의 라이브러리를 사용하지만 JVM 명세가 구체적 매핑을 정하지는 않는다.	(4.4, p.185-186 / PDF p.131-132)
U04-064	비동기 스레딩은 부모가 자식을 만든 뒤 독립적으로 계속 실행한다.	(4.4, p.185-186 / PDF p.131-132)
U04-065	동기 스레딩은 자식들이 모두 종료하여 조인할 때까지 부모가 기다린 뒤 결과를 결합한다.	(4.4, p.185-186 / PDF p.131-132)
U04-066	원문의 합산 예제는 동기 방식을 사용한다.	(4.4, p.185-186 / PDF p.131-132)
U04-067	Pthreads와 Windows의 전역 변수는 같은 프로세스 스레드가 공유하지만 Java는 전역 데이터 개념 대신 명시적으로 공유를 조정한다.	(4.4, p.185-186 / PDF p.131-132)
U04-068	Pthreads는 POSIX가 정한 스레드 생성·동기화 동작의 명세이며 그 자체가 하나의 구현은 아니다.	(4.4.1, p.186-188 / PDF p.132-134)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-069	원문 합산 프로그램의 main과 runner는 sum을 공유한다.	(4.4.1, p.186-188 / PDF p.132-134)
U04-070	pthread_attr_init()으로 기본 속성을 설정하고 pthread_create()에 식별자·속성·시작 함수·인수를 전달한다.	(4.4.1, p.186-188 / PDF p.132-134)
U04-071	pthread_join()은 합산 스레드 종료를 기다리며 runner는 pthread_exit()로 종료한다.	(4.4.1, p.186-188 / PDF p.132-134)
U04-072	여러 스레드의 조인은 반복문에서 각각 기다릴 수 있다.	(4.4.1, p.186-188 / PDF p.132-134)

번호	문장·표 항목·원문 코드·수식	원문 위치

U04-073	그림 4.11 Pthreads 합산 프로그램 (1/2) (원문 코드 발췌)
	#include <pthread.h>
	#include <stdio.h>
	#include <stdlib.h>
	
	int sum; /* this data is shared by the thread(s) */
	void *runner(void *param); /* threads call this function */
	
	int main(int argc, char *argv[])
	{
	pthread_t tid; /* the thread identifier */
	pthread_attr_t attr; /* set of thread attributes */
	
	/* set the default attributes of the thread */
	pthread_attr_init(&attr);
	/* create the thread */
	pthread_create(&tid, &attr, runner, argv[1]);
	/* wait for the thread to exit */
	pthread_join(tid,NULL);
	
	printf("sum = %d\n",sum);
	}
	
	/* The thread will execute in this function */
	void *runner(void *param)
	{
	int i, upper = atoi(param);
	sum = 0;
	
	for (i = 1; i <= upper; i++)
		(4.4.1, p.187 / PDF p.133)

U04-074	그림 4.11 Pthreads 합산 프로그램 (2/2) (원문 코드 발췌)
	sum += i;
	
	pthread_exit(0);
	}
		(4.4.1, p.187 / PDF p.133)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-075	Windows의 합산 예제는 전역 Sum을 공유하고 Summation 함수에서 계산한다.	(4.4.2, p.188-190 / PDF p.134-136)
U04-076	CreateThread()에 보안 속성·스택 크기·시작 함수·인수·생성 플래그·식별자 수신 위치를 전달한다.	(4.4.2, p.188-190 / PDF p.134-136)
U04-077	부모는 WaitForSingleObject()로 완료를 기다리고 핸들을 닫은 뒤 결과를 출력한다.	(4.4.2, p.188-190 / PDF p.134-136)
U04-078	여러 스레드는 WaitForMultipleObjects()에 객체 수·핸들 배열·모든 객체 대기 여부·타임아웃을 지정하여 기다린다.	(4.4.2, p.188-190 / PDF p.134-136)
U04-079	Java는 Thread를 상속하여 run()을 재정의하거나 Runnable 인터페이스를 구현한다.	(4.4.3, p.190-192 / PDF p.136-138)
U04-080	Runnable 객체를 전달해 Thread를 만들고 start()를 호출하면 새 스레드가 초기화되어 run()을 실행할 자격을 얻는다.	(4.4.3, p.190-192 / PDF p.136-138)
U04-081	원문은 run()을 직접 호출하지 말 것을 명시한다.	(4.4.3, p.190-192 / PDF p.136-138)
U04-082	부모는 join()으로 완료를 기다릴 수 있다.	(4.4.3, p.190-192 / PDF p.136-138)
U04-083	Runnable을 람다 표현식으로 작성하는 방법도 소개한다.	(4.4.3, p.190-192 / PDF p.136-138)
U04-084	Executor는 Runnable을 execute()에 전달하여 태스크 실행과 명시적인 Thread 생성·start 호출을 분리한다.	(4.4.3.1, p.192-194 / PDF p.138-140)
U04-085	태스크를 생산하고 실행 스레드가 소비하는 모델이다.	(4.4.3.1, p.192-194 / PDF p.138-140)
U04-086	Callable은 결과를 반환할 수 있으며 submit()이 반환하는 Future의 get()으로 결과를 얻는다.	(4.4.3.1, p.192-194 / PDF p.138-140)
U04-087	execute()는 결과를 반환하지 않는다는 차이가 있다.	(4.4.3.1, p.192-194 / PDF p.138-140)
U04-088	원문 합산 예제는 newSingleThreadExecutor()를 만들고 Callable 작업을 submit()한 뒤 Future.get()으로 결과를 기다린다.	(4.4.3.1, p.192-194 / PDF p.138-140)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-089	이 구성은 스레드 생성과 결과를 분리하므로 반드시 스레드의 최종 종료까지가 아니라 결과가 사용 가능해질 때까지 기다리도록 한다.	(4.4.3.1, p.192-194 / PDF p.138-140)
U04-090	수백·수천 개 스레드를 직접 만들고 관리하는 부담을 줄이기 위해 생성·관리 책임을 컴파일러와 실행시간 라이브러리로 옮긴다.	(4.5, p.194 / PDF p.140)
U04-091	암묵적 스레딩에서 개발자는 병렬 가능한 작업을 식별하고 라이브러리가 이를 하부 스레드에 매핑한다.	(4.5, p.194 / PDF p.140)
U04-092	정확성·자원 관리의 어려움 자체가 사라지는 것은 아니며 제6·8장의 문제와 연결된다.	(4.5, p.194 / PDF p.140)
U04-093	미리 만든 스레드들은 작업을 기다린다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-094	요청이 오면 새 스레드 대신 작업을 풀에 제출한다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-095	사용 가능한 스레드가 작업을 처리하고, 없으면 작업이 기다린다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-096	작업을 마친 스레드는 풀로 돌아가 다음 작업을 기다린다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-097	기존 스레드 재사용은 생성 시간을 절약하고 동시 스레드 수를 제한하여 자원 고갈을 막는다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-098	작업과 실행 방법을 분리하면 지연 실행·주기 실행도 가능하다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-099	풀 크기는 CPU 수·메모리·최대 동시 요청 등에 따라 정하거나 활용도에 따라 동적으로 조절한다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-100	풀 제출 작업을 비동기 실행할 수 있는 경우에 잘 작동한다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-101	원문은 Windows QueueUserWorkItem()과 Android AIDL의 스레드 풀을 사례로 든다.	(4.5.1, p.195-197 / PDF p.141-143)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-102	Java의 newSingleThreadExecutor()는 하나, newFixedThreadPool(size)는 지정된 수, newCachedThreadPool()은 재사용 가능한 무제한 풀을 만든다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-103	shutdown() 이후 새 작업은 거부하고 이미 제출된 작업이 완료되면 종료한다.	(4.5.1, p.195-197 / PDF p.141-143)
U04-104	fork-join은 부모가 자식 작업을 나누고 완료를 기다린 뒤 결과를 결합하는 동기 모델이다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-105	암묵적 방식에서는 실제 스레드를 직접 만들지 않고 병렬 작업을 식별하며 라이브러리가 스레드 수와 배정을 정한다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-106	Java의 ForkJoinPool은 분할 정복 작업을 받는다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-107	충분히 작으면 직접 풀고, 그렇지 않으면 하위 문제를 fork한 뒤 join하여 결과를 합친다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-108	ForkJoinTask의 하위 클래스인 RecursiveTask는 결과를 반환하고 RecursiveAction은 반환하지 않는다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-109	원문 SumTask 예제는 배열의 범위를 둘로 나누고 임계값 THRESHOLD=1000 아래에서 직접 합산한다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-110	언제 충분히 작다고 볼지는 실제 타이밍 시험이 필요하다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-111	작업 수가 스레드 수보다 훨씬 많을 수 있다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-112	각 작업 스레드는 자신의 큐를 유지하며 큐가 비면 work stealing으로 다른 큐의 작업을 가져와 부하를 분산한다.	(4.5.2, p.197-201 / PDF p.143-147)
U04-113	OpenMP는 C·C++·FORTRAN의 API와 컴파일러 디렉티브 집합으로 공유 메모리 병렬 프로그래밍을 돕는다.	(4.5.3, p.201-203 / PDF p.147-149)
U04-114	개발자가 병렬 영역을 표시하면 실행시간 라이브러리가 스레드 실행을 관리한다.	(4.5.3, p.201-203 / PDF p.147-149)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-115	원문은 parallel 영역과 parallel for로 반복을 분배하는 예를 제시한다.	(4.5.3, p.201-203 / PDF p.147-149)
U04-116	스레드 수와 데이터의 공유·개별 사용 여부도 선택할 수 있다.	(4.5.3, p.201-203 / PDF p.147-149)

U04-117	OpenMP 병렬 for (원문 코드 발췌)
	#pragma omp parallel for
	for (i = 0; i < N; i++) {
	c[i] = a[i] + b[i];
	}
		(4.5.3, p.202 / PDF p.148)

U04-118	GCD는 macOS·iOS의 실행시간 라이브러리·API·언어 확장이다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-119	태스크를 디스패치 큐에 넣고 큐에서 꺼낼 때 관리 중인 풀의 가용 스레드에 배정한다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-120	직렬 큐는 FIFO로 꺼낸 하나가 끝나야 다음 작업을 실행한다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-121	병행 큐도 FIFO로 제거하지만 여러 태스크를 동시에 실행할 수 있다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-122	사용자 대화형	(4.5.4, p.203-204 / PDF p.149-150)
U04-123	즉각적인 사용자 인터페이스·이벤트 반응. 작업량이 매우 작아야 한다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-124	사용자 시작	(4.5.4, p.203-204 / PDF p.149-150)
U04-125	파일·URL 열기처럼 사용자가 시작하여 완료를 기다리는 비교적 긴 작업	(4.5.4, p.203-204 / PDF p.149-150)
U04-126	유틸리티	(4.5.4, p.203-204 / PDF p.149-150)
U04-127	데이터 가져오기처럼 오래 걸리지만 즉각적인 결과가 필요하지 않은 작업	(4.5.4, p.203-204 / PDF p.149-150)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-128	백그라운드	(4.5.4, p.203-204 / PDF p.149-150)
U04-129	색인·백업처럼 사용자에게 보이지 않고 시간에 민감하지 않은 작업	(4.5.4, p.203-204 / PDF p.149-150)
U04-130	작업은 C 계열의 블록 또는 Swift closure로 나타낸다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-131	GCD는 POSIX 스레드 풀의 크기를 응용 수요와 시스템 능력에 맞추어 적극적으로 늘리거나 줄인다.	(4.5.4, p.203-204 / PDF p.149-150)
U04-132	Intel TBB는 특별한 컴파일러나 언어 지원 없이 C++ 병렬 응용을 돕는 템플릿 라이브러리이다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-133	개발자는 태스크를 지정하고 스케줄러가 하부 스레드에 배정하며 부하 균형·캐시 인지를 제공한다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-134	병렬 루프·원자적 연산·상호 배제 잠금 및 병행 자료구조를 포함한다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-135	parallel_for는 반복 공간을 청크로 나누어 작업을 만들고 가용 스레드에 배정한다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-136	처리 코어 수에 맞춰 직접 배열을 분할하는 방식보다 하드웨어별 분할의 세부사항을 라이브러리에 맡길 수 있다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-137	원문은 apply(v[i])를 각 배열 원소에 적용하는 직렬 루프와 다음 템플릿 호출을 대비한다.	(4.5.5, p.204-206 / PDF p.150-152)
U04-138	TBB parallel_for (원문 코드 발췌) parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});	(4.5.5, p.205 / PDF p.151)
U04-139	다중 스레드에서 fork()가 모든 스레드를 복제할지 호출한 스레드만 복제할지 고려해야 한다.	(4.6.1, p.206 / PDF p.152)
U04-140	원문은 두 버전을 제공하는 UNIX를 설명한다.	(4.6.1, p.206 / PDF p.152)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-141	곧 exec()를 할 경우 전체 프로세스가 대체되므로 모든 스레드 복제는 불필요하고, 복사본을 그대로 실행할 경우 모든 스레드가 필요하다.	(4.6.1, p.206 / PDF p.152)
U04-142	어느 스레드가 exec()를 호출하든 지정한 새 프로그램은 모든 스레드를 포함한 전체 프로세스를 대체한다.	(4.6.1, p.206 / PDF p.152)
U04-143	UNIX 신호는 특정 이벤트에서 생성되고 프로세스에 전달되어 처리된다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-144	동기식 신호는 불법 메모리 접근·0으로 나누기처럼 실행 연산에서 생겨 그 연산을 한 스레드에 전달되어야 한다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-145	비동기 신호는 외부의 종료 요청·타이머 만료처럼 발생한다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-146	커널의 기본 처리기를 사용자 정의 처리기로 대체할 수 있고 신호에 따라 무시하거나 종료할 수 있다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-147	다중 스레드에서는 적용 스레드·모든 스레드·일부·전담 스레드 중 전달 대상을 정해야 한다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-148	비동기 신호는 허용된 스레드에 전달될 수 있지만 한 번만 처리해야 한다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-149	kill()은 프로세스, pthread_kill()은 특정 스레드를 지정한다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-150	Windows의 APC는 지정된 스레드에 이벤트 처리 함수를 호출하므로 UNIX의 프로세스 전체 신호 처리와 다르다.	(4.6.2, p.206-208 / PDF p.152-154)
U04-151	비동기 취소	(4.6.3, p.208-210 / PDF p.154-156)
U04-152	목적 스레드를 즉시 종료	(4.6.3, p.208-210 / PDF p.154-156)
U04-153	공유 자료 갱신 중단과 자원 회수 누락 문제가 생길 수 있다.	(4.6.3, p.208-210 / PDF p.154-156)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-154	지연 취소	(4.6.3, p.208-210 / PDF p.154-156)
U04-155	취소 요청을 표시하고 목적 스레드가 안전한 시점에 확인	(4.6.3, p.208-210 / PDF p.154-156)
U04-156	질서 있게 정리할 기회를 준다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-157	pthread_cancel()은 취소 요청을 표시할 뿐이며 실제 동작은 목적 스레드의 설정에 달린다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-158	취소 비활성 상태에서는 요청이 보류될 수 있다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-159	기본 지연 취소는 취소 점에 도달할 때 발생한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-160	봉쇄형 read() 등이 취소 점이며 pthread_testcancel()로 직접 검사할 수 있다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-161	요청이 있으면 이 함수는 복귀하지 않고 종료한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-162	정리 핸들러는 종료 전에 자원을 해제하도록 한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-163	원문은 Pthreads의 비동기 취소를 권장하지 않는다고 설명한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-164	Java는 interrupt()로 인터럽트 상태를 설정하고 스레드가 isInterrupted()로 검사하는 지연 취소와 유사한 정책을 사용한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-165	요청을 보냈다는 사실과 실제 종료했다는 사실을 구별해야 한다.	(4.6.3, p.208-210 / PDF p.154-156)
U04-166	TLS는 스레드마다 자신만 접근할 고유 데이터를 제공한다.	(4.6.4, p.210-211 / PDF p.156-157)
U04-167	지역 변수는 한 함수 호출 동안만 보이지만 TLS는 여러 함수 호출에 걸쳐 보존된다.	(4.6.4, p.210-211 / PDF p.156-157)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-168	정적 데이터와 비슷하지만 각 스레드마다 고유하다는 차이가 있다.	(4.6.4, p.210-211 / PDF p.156-157)
U04-169	생성 과정을 직접 제어하지 못하는 풀에서도 스레드별 식별자 등을 저장할 수 있다.	(4.6.4, p.210-211 / PDF p.156-157)
U04-170	원문은 Java ThreadLocal, Pthreads 키, C# ThreadStatic, gcc __thread를 소개한다.	(4.6.4, p.210-211 / PDF p.156-157)
U04-171	다대다·두 수준 모델에서는 사용자 라이브러리와 커널이 협력하여 필요한 커널 스레드 수를 조정해야 한다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-172	LWP(경량 프로세스)는 사용자 라이브러리에 가상 처리기처럼 보이고 각 LWP는 커널 스레드 하나에 연결된다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-173	커널 스레드가 봉쇄되면 해당 LWP와 그 위의 사용자 스레드도 봉쇄된다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-174	커널이 스레드가 봉쇄됨과 식별자를 라이브러리에 알리고 새 가상 처리기를 제공한다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-175	upcall 처리기가 상태를 저장하고 다른 실행 가능 스레드를 스케줄한다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-176	대기 이벤트가 끝나면 커널이 다시 알린다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-177	처리기가 이전 스레드를 실행 가능으로 표시하고 가용 가상 처리기에서 실행하도록 한다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-178	입출력 중심 응용은 동시에 봉쇄되는 호출을 감당할 충분한 LWP가 필요할 수 있다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-179	원문 예에서 파일 읽기 5개에 LWP가 4개뿐이면 다섯 번째 요청은 한 LWP가 돌아올 때까지 기다린다.	(4.6.5, p.211-212 / PDF p.157-158)
U04-180	Windows는 사용자 스레드 하나에 커널 스레드 하나를 대응시킨다.	(4.7.1, p.213-214 / PDF p.159-160)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-181	스레드는 식별자·레지스터·프로그램 카운터·사용자 및 커널 스택·개별 저장 영역을 가진다.	(4.7.1, p.213-214 / PDF p.159-160)
U04-182	ETHREAD는 소속 프로세스·시작 루틴·KTHREAD 포인터, KTHREAD는 스케줄링·동기화·커널 스택·TEB 포인터를 갖는다.	(4.7.1, p.213-214 / PDF p.159-160)
U04-183	둘은 커널 공간이다.	(4.7.1, p.213-214 / PDF p.159-160)
U04-184	TEB는 사용자 공간에서 식별자·사용자 스택·TLS 배열을 제공한다.	(4.7.1, p.213-214 / PDF p.159-160)
U04-185	그림 4.21 Windows 스레드의 자료구조	(4.7.1, p.214 / PDF p.160)
U04-186	Linux는 프로세스와 스레드를 모두 태스크로 다룬다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-187	clone()의 플래그가 부모·자식의 공유 범위를 결정한다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-188	CLONE_FS는 파일 시스템 정보, CLONE_VM은 메모리 공간, CLONE_SIGHAND는 신호 처리기, CLONE_FILES는 열린 파일 집합을 공유한다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-189	여러 자료구조를 공유하면 스레드처럼, 공유하지 않으면 fork()에 가까운 프로세스처럼 동작한다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-190	task_struct가 데이터 자체가 아니라 관련 자료구조의 포인터를 가지므로 필요한 부분만 같은 구조를 가리키게 할 수 있다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-191	원문은 clone의 다른 플래그가 하나의 Linux 커널 아래 격리된 시스템인 컨테이너 생성에도 연결된다고 설명한다.	(4.7.2, p.214-215 / PDF p.160-161)
U04-192	스레드의 공유 이점과 병렬화의 어려움은 함께 고려해야 한다.	(4.8, p.215-218 / PDF p.161-164)
U04-193	매핑 모델과 스레딩 도구는 생성·관리 책임을 어디에 두는지 구별한다.	(4.8, p.215-218 / PDF p.161-164)
U04-194	장말 연습문제 4.1~4.7은 성능 이점·Amdahl·병렬 유형·사용자와 커널 스레드·문맥 교환·생성 자원·LWP를 검토한다.	(4.8, p.215-218 / PDF p.161-164)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-195	새 사례나 계산 풀이를 추가하지 않았고 추가 자료·참고문헌의 외부 내용은 사용하지 않았다.	(4.8, p.215-218 / PDF p.161-164)
U04-196	분할 대상	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-197	데이터 부분집합	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-198	서로 다른 작업	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-199	코어별 연산	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-200	동일	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-201	고유 연산	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-202	관계	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-203	두 방법을 혼합할 수 있다.	(데이터 병렬과 태스크 병렬, p.181-182 / PDF p.127-128)
U04-204	시점	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)
U04-205	즉시 종료	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-206	안전한 취소 점에서 요청을 확인	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)
U04-207	자원·공유 상태	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)
U04-208	갱신 중 중단·자원 미반환 위험	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)
U04-209	정리 기회를 제공	(비동기 취소와 지연 취소, p.208-210 / PDF p.154-156)
U04-210	병렬 시스템은 둘 이상의 작업을 동시에 수행할 수 있다.	(4.2, p.179 / PDF p.125)
U04-211	사용자 스레드는 커널 위에서 지원되며 커널의 지원 없이 관리된다.	(4.3, p.182 / PDF p.128)
U04-212	커널 스레드는 운영체제에 의해 직접 지원되고 관리된다.	(4.3, p.182 / PDF p.128)
U04-213	데이터 병렬 처리는 동일한 데이터의 부분 집합을 다른 컴퓨팅 코어에 분산시키고 각 코어에서 동일한 연산을 수행한다.	(4.8, p.216 / PDF p.162)
U04-214	작업 병렬 처리는 여러 코어에 작업을 분산시키며 각 작업은 고유한 연산을 수행한다.	(4.8, p.216 / PDF p.162)
U04-215	동일한 프로세스에 속한 스레드는 코드와 데이터를 포함하여 많은 프로세스 자원을 공유한다.	(4.8, p.215 / PDF p.161)
U04-216	다중 스레드 응용 프로그램의 네 가지 주요 이점은 응답성, 자원 공유, 경제성 및 확장성이다.	(4.8, p.215 / PDF p.161)
U04-217	사용자 수준 스레드는 궁극적으로 CPU에서 실행되도록 커널 스레드에 매핑되어야 한다.	(4.8, p.216 / PDF p.162)
U04-218	데이터 병렬 처리	(4.8, p.216 / PDF p.162)
U04-219	작업 병렬 처리	(4.8, p.216 / PDF p.162)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-220	Amdahl의 법칙: speedup ≤ 1 / (S + (1 − S) / N); S는 반드시 순차 실행되는 부분, N은 처리 코어 수이다.	(4.2, p.180 / PDF p.126)

제5장. CPU 스케줄링

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-001	CPU 스케줄링은 준비 큐에서 대기 프로세스를 선택하고 CPU를 할당하는 작업이다.	(5.9, p.274 / PDF p.220)
U05-002	스케줄링 알고리즘은 CPU 이용률, 처리량, 총처리 시간, 대기 시간 및 응답 시간의 다섯 기준으로 평가할 수 있다.	(5.9, p.274 / PDF p.220)
U05-003	Rate-monotonic 실시간 스케줄링은 선점과 정적 우선순위 정책을 사용하여 주기적 작업을 스케줄한다.	(5.9, p.275 / PDF p.221)
U05-004	다중 프로그래밍은 한 프로세스가 입출력 등을 기다리는 동안 다른 프로세스에 CPU를 할당하여 이용률을 높인다.	(5.1, p.219-220 / PDF p.165-166)
U05-005	코어가 하나면 한순간 하나만 실행하고 다중 코어에서는 이 개념을 각 코어에 적용한다.	(5.1, p.219-220 / PDF p.165-166)
U05-006	현대 운영체제에서 실제 스케줄 대상은 보통 커널 스레드이다.	(5.1, p.219-220 / PDF p.165-166)
U05-007	원문은 일반적 논의에서는 프로세스 스케줄링이라는 표현을 쓰고 스레드에 특화된 경우 이를 구별한다.	(5.1, p.219-220 / PDF p.165-166)
U05-008	프로세스 실행은 CPU 버스트와 I/O 버스트가 교대로 나타나는 사이클이다.	(5.1.1, p.220-222 / PDF p.166-168)
U05-009	마지막 CPU 버스트는 다음 입출력 대신 종료 요청으로 끝난다.	(5.1.1, p.220-222 / PDF p.166-168)
U05-010	관측상 짧은 CPU 버스트는 많고 긴 버스트는 적다.	(5.1.1, p.220-222 / PDF p.166-168)
U05-011	I/O 중심 프로그램은 짧은 CPU 버스트가 많고 CPU 중심 프로그램은 긴 버스트를 가질 수 있으므로 이 분포가 알고리즘 선택에 중요하다.	(5.1.1, p.220-222 / PDF p.166-168)
U05-012	CPU 스케줄러는 준비된 메모리 내 프로세스 가운데 다음 실행 대상을 선택한다.	(5.1.2, p.220-221 / PDF p.166-167)
U05-013	준비 큐는 반드시 FIFO가 아니며 우선순위 큐·트리·순서 없는 연결 리스트도 가능하다.	(5.1.2, p.220-221 / PDF p.166-167)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-014	큐의 레코드는 일반적으로 PCB이다.	(5.1.2, p.220-221 / PDF p.166-167)
U05-015	준비 상태라는 의미와 구현 자료구조를 혼동하지 않아야 한다.	(5.1.2, p.220-221 / PDF p.166-167)
U05-016	실행 → 대기	(5.1.3, p.221-223 / PDF p.167-169)
U05-017	현재 프로세스가 CPU를 내놓았으므로 다른 준비 프로세스를 선택해야 한다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-018	실행 → 준비	(5.1.3, p.221-223 / PDF p.167-169)
U05-019	현재 프로세스의 계속 실행과 다른 프로세스 사이에서 선택할 수 있다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-020	대기 → 준비	(5.1.3, p.221-223 / PDF p.167-169)
U05-021	새로 준비된 프로세스와 현재 실행 프로세스를 비교할 수 있다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-022	종료	(5.1.3, p.221-223 / PDF p.167-169)
U05-023	실행할 다른 준비 프로세스를 선택해야 한다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-024	첫 번째와 네 번째 경우에만 교체하면 비선점(협조적) 스케줄링이다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-025	할당된 CPU는 종료하거나 대기로 전환할 때까지 유지한다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-026	다른 상황에서도 CPU를 뺏을 수 있으면 선점이다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-027	선점은 공유 자료 갱신 도중 다른 실행이 개입하여 경쟁 조건을 만들 수 있다.	(5.1.3, p.221-223 / PDF p.167-169)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-028	비선점형 커널은 구조가 단순해지지만 실시간 응답에 불리하다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-029	선점형 커널은 공유 커널 자료구조를 mutex 등으로 보호해야 한다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-030	인터럽트의 영향을 받는 짧은 코드 부분에서 인터럽트를 잠시 끌 수 있지만 자주 사용하거나 긴 명령 구간에 적용하면 안 된다.	(5.1.3, p.221-223 / PDF p.167-169)
U05-031	디스패처는 스케줄러가 선택한 프로세스에 실제 CPU 제어를 넘긴다.	(5.1.4, p.223-224 / PDF p.169-170)
U05-032	문맥 교환, 사용자 모드 전환, 사용자 프로그램의 적절한 재개 위치로 이동하는 작업을 수행한다.	(5.1.4, p.223-224 / PDF p.169-170)
U05-033	하나를 멈추고 다음 실행을 시작할 때까지의 시간이 디스패치 지연이므로 빠르게 수행되어야 한다.	(5.1.4, p.223-224 / PDF p.169-170)
U05-034	원문 Linux 예제는 vmstat로 시스템의 문맥 교환을, /proc의 프로세스 상태 파일로 개별 횟수를 관찰한다.	(5.1.4, p.223-224 / PDF p.169-170)
U05-035	자발적 교환은 현재 사용할 수 없는 자원을 요청하여 CPU를 포기한 경우이고 비자발적 교환은 시간 할당량 만료·높은 우선순위 선점 등으로 빼앗긴 경우이다.	(5.1.4, p.223-224 / PDF p.169-170)
U05-036	CPU 이용률	(5.2, p.225-226 / PDF p.171-172)
U05-037	CPU가 바쁘게 일하는 비율	(5.2, p.225-226 / PDF p.171-172)
U05-038	최대화	(5.2, p.225-226 / PDF p.171-172)
U05-039	처리량	(5.2, p.225-226 / PDF p.171-172)
U05-040	단위 시간에 완료한 프로세스 수	(5.2, p.225-226 / PDF p.171-172)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-041	총처리 시간	(5.2, p.225-226 / PDF p.171-172)
U05-042	제출부터 완료까지. 준비 대기·CPU 실행·입출력 시간을 포함	(5.2, p.225-226 / PDF p.171-172)
U05-043	최소화	(5.2, p.225-226 / PDF p.171-172)
U05-044	대기 시간	(5.2, p.225-226 / PDF p.171-172)
U05-045	준비 큐에서 보낸 시간의 합	(5.2, p.225-226 / PDF p.171-172)
U05-046	응답 시간	(5.2, p.225-226 / PDF p.171-172)
U05-047	요청 제출부터 첫 응답이 시작될 때까지	(5.2, p.225-226 / PDF p.171-172)
U05-048	첫 응답까지와 응답을 모두 출력하는 데 걸리는 시간은 다르다.	(5.2, p.225-226 / PDF p.171-172)
U05-049	평균뿐 아니라 최댓값·최솟값이나 응답 시간의 편차가 중요한 환경도 있다.	(5.2, p.225-226 / PDF p.171-172)
U05-050	원문의 기본 수치 예제는 단일 코어, 프로세스당 하나의 CPU 버스트, 밀리초 단위 및 평균대기 시간이라는 단순화 조건을 사용한다.	(5.2, p.225-226 / PDF p.171-172)
U05-051	이 조건을 실제 모든 실행의 성능 보장으로 확대하지 않는다.	(5.2, p.225-226 / PDF p.171-172)
U05-052	FCFS는 CPU를 먼저 요청한 순서로 할당한다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-053	준비 큐의 꼬리에 PCB를 넣고 머리에서 선택하는 FIFO로 구현할 수 있어 단순하다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-054	비선점이므로 현재 버스트를 마치거나 대기할 때까지 CPU를 점유한다.	(5.3.1, p.226-228 / PDF p.172-174)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-055	긴 작업 뒤에 짧은 작업이 몰리면 평균대기 시간이 길어진다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-056	CPU 버스트(ms)	(5.3.1, p.226-228 / PDF p.172-174)
U05-057	24	(5.3.1, p.226-228 / PDF p.172-174)
U05-058	3	(5.3.1, p.226-228 / PDF p.172-174)
U05-059	P1→P2→P3일 때 대기	(5.3.1, p.226-228 / PDF p.172-174)
U05-060	0	(5.3.1, p.226-228 / PDF p.172-174)
U05-061	27	(5.3.1, p.226-228 / PDF p.172-174)
U05-062	P2→P3→P1일 때 대기	(5.3.1, p.226-228 / PDF p.172-174)
U05-063	6	(5.3.1, p.226-228 / PDF p.172-174)
U05-064	첫 순서는 평균 (0 + 24 + 27)/3 = 17ms, 두 번째는 (6 + 0 + 3)/3 = 3ms이다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-065	같은 버스트 집합도 도착 순서에 따라 크게 달라진다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-066	아래는 원문 첫 순서의 간트차트이다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-067	FCFS의 원문 간트차트	(5.3.1, p.227 / PDF p.173)
U05-068	호위 효과는 긴 CPU 중심 프로세스가 CPU를 보유하는 동안 여러 I/O 중심 프로세스가 기다리는 현상이다.	(5.3.1, p.226-228 / PDF p.172-174)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-069	이때 장치는 쉬고, CPU 중심 작업이 입출력으로 옮기면 짧은 작업들이 빨리 실행·재대기하여 이번에는 CPU가 쉬게 될 수 있다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-070	짧은 프로세스를 먼저 처리할 때보다 CPU·장치 이용률이 낮아지는 이유이다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-071	대화형 시스템에서 규칙적인 CPU 몫을 제공하기 어렵다는 한계도 있다.	(5.3.1, p.226-228 / PDF p.172-174)
U05-072	SJF는 전체 작업 길이가 아니라 다음 CPU 버스트가 가장 짧은 프로세스를 고른다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-073	길이가 같으면 FCFS로 정한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-074	주어진 작업 집합에서 짧은 버스트를 앞당기면 짧은 작업의 대기 감소가 긴 작업의 증가보다 커서 평균대기 시간을 최소화한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-075	그러나 실제 다음 버스트를 미리 알 수 없으므로 예측이 필요하다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-076	원문 버스트(ms)	(5.3.2, p.228-230 / PDF p.174-176)
U05-077	6	(5.3.2, p.228-230 / PDF p.174-176)
U05-078	8	(5.3.2, p.228-230 / PDF p.174-176)
U05-079	7	(5.3.2, p.228-230 / PDF p.174-176)
U05-080	3	(5.3.2, p.228-230 / PDF p.174-176)
U05-081	SJF 대기(ms)	(5.3.2, p.228-230 / PDF p.174-176)
U05-082	16	(5.3.2, p.228-230 / PDF p.174-176)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-083	9	(5.3.2, p.228-230 / PDF p.174-176)
U05-084	0	(5.3.2, p.228-230 / PDF p.174-176)
U05-085	동시 도착한 위 예의 실행 순서는 P4→P1→P3→P2이고 구간 경계는 0·3·9·16·24이다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-086	평균대기는 (3 + 16 + 9 + 0)/4 = 7ms이며 원문 FCFS 비교값은 10.25ms이다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-087	원문 수식: $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n$, $0 \le \alpha \le 1$; t_n: n번째 실제 CPU 버스트 길이; τ_n: 앞서 축적한 예측값; τ_{n+1}: 다음 CPU 버스트 예측; α: 최근 실제값과 과거 예측에 줄 상대 가중치; α=0이면 과거 예측만, α=1이면 가장 최근 버스트만 반영한다. α=1/2이면 둘에 같은 무게를 준다. 원문 그림은 α=1/2, 초기 예측 10을 사용한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-088	재귀적으로 대입하면 오래된 버스트에 대한 가중치는 (1−α)의 거듭제곱으로 감소한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-089	원문의 전개식은 아래에 그대로 보존한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-090	지수 평균의 원문 전개식	(5.3.2, p.230 / PDF p.176)
U05-091	선점형 SJF는 새로 도착한 버스트와 현재 남은 시간을 비교하여 더 짧은 쪽을 실행하는 최소 잔여 시간 우선(SRTF)이다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-092	비선점형은 현재 버스트가 끝나도록 한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-093	도착(ms)	(5.3.2, p.228-230 / PDF p.174-176)
U05-094	1	(5.3.2, p.228-230 / PDF p.174-176)
U05-095	2	(5.3.2, p.228-230 / PDF p.174-176)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-096	버스트(ms)	(5.3.2, p.228-230 / PDF p.174-176)
U05-097	4	(5.3.2, p.228-230 / PDF p.174-176)
U05-098	5	(5.3.2, p.228-230 / PDF p.174-176)
U05-099	원문 예에서 P1이 시간 0에 시작하지만 시간 1에 남은 7보다 짧은 4의 P2가 도착하여 선점한다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-100	실행 순서는 P1(0~1)→P2(1~5)→P4(5~10)→P1(10~17)→P3(17~26)이다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-101	평균대기는 [(10−1)+(1−1)+(17−2)+(5−3)]/4 = 6.5ms이고 비선점 비교값은 7.75ms이다.	(5.3.2, p.228-230 / PDF p.174-176)
U05-102	선점형 SJF의 원문 간트차트	(5.3.2, p.230 / PDF p.176)
U05-103	RR은 FCFS에 시간 할당량 q와 선점을 추가한다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-104	준비 큐를 원형으로 돌며 각 프로세스에 한 번씩 q를 준다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-105	q 이전에 버스트가 끝나면 자발적으로 넘기고, 길면 타이머 인터럽트 후 큐 꼬리로 돌아간다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-106	실행 가능한 프로세스가 하나가 아니면 연속 두 몫을 받지 않는다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-107	원문의 P1=24, P2=P3=3 및 q=4 예에서는 P1(0~4)→P2(4~7)→P3(7~10) 후 P1이 나머지를 수행한다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-108	대기는 6·4·7이고 평균은 17/3 = 5.66ms로 제시된다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-109	원문의 시간 경계를 보존한 차트는 아래와 같다.	(5.3.3, p.230-233 / PDF p.176-179)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-110	RR의 원문 간트차트(q=4ms)	(5.3.3, p.231 / PDF p.177)
U05-111	원문 수식: 다음 몫까지의 대기 상한: (n − 1)q; n: 준비 큐의 프로세스 수; q: 각 시간 할당량; 각 프로세스가 최대 q 덩어리로 CPU의 1/n을 나누어 받는 원문 조건이다. 예를 들어 n=5, q=20ms이면 매 100ms마다 최대 20ms를 받는다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-112	q가 매우 크면 FCFS와 같아지고 너무 작으면 문맥 교환 오버헤드가 커진다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-113	따라서 q는 문맥 교환 시간보다 충분히 커야 하지만 과도하게 커서도 안 된다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-114	q 증가가 평균 총처리 시간을 반드시 개선하는 것은 아니다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-115	많은 버스트가 한 몫 안에 끝나면 개선될 수 있다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-116	원문은 CPU 버스트의 약 80%가 q보다 짧아야 한다는 경험 기준을 제시한다.	(5.3.3, p.230-233 / PDF p.176-179)
U05-117	우선순위 스케줄링은 가장 높은 우선순위 프로세스에 CPU를 준다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-118	같은 순위는 FCFS 또는 RR로 처리할 수 있다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-119	예상 다음 버스트의 역을 우선순위로 쓰는 SJF는 이 방식의 특수한 경우이다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-120	숫자가 크거나 작을 때 어느 쪽이 높은 우선순위인지는 시스템마다 다르며 원문의 기본 예는 낮은 숫자가 높은 순위를 뜻한다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-121	내부 우선순위는 시간 제한·메모리 요구·열린 파일 수·I/O 대 CPU 비율처럼 측정 가능한 값에서, 외부 우선순위는 중요성·비용 등의 외부 기준에서 정한다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-122	선점형은 더 높은 순위 도착 시 현재 실행을 뺏고 비선점형은 다음 선택 때 반영한다.	(5.3.4, p.233-235 / PDF p.179-181)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-123	주요 한계는 낮은 순위가 끝없이 기다리는 기아(무한 봉쇄)이다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-124	노화(aging)는 오래 대기한 프로세스의 우선순위를 점차 높여 이를 완화한다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-125	같은 순위에 RR을 쓰면 우선순위 집단 안에서도 몫을 나눈다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-126	원문은 최고 순위가 먼저 끝나고 다음 동일 순위 쌍이 q=2로 교대하는 예로 두 정책의 결합을 설명한다.	(5.3.4, p.233-235 / PDF p.179-181)
U05-127	다단계 큐는 순위나 프로세스 유형별로 준비 큐를 나눈다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-128	각 큐는 자체 알고리즘을 사용할 수 있다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-129	원문은 대화형 전경 큐에 RR, 배치 배경 큐에 FCFS를 쓰는 경우를 든다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-130	전형적으로 프로세스는 진입할 때 정해진 큐에 남는다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-131	큐 내부뿐 아니라 큐 사이에도 정책이 필요하다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-132	고정 우선순위를 쓰면 상위 큐가 비어야 하위 큐를 실행하고 상위 작업 도착 시 하위를 선점한다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-133	다른 방식은 큐별로 CPU 비율을 나누는 것으로 원문은 전경 80%·배경 20%를 예로 든다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-134	높은 큐의 절대 우선권과 각 큐의 시간 몫은 다른 선택이다.	(5.3.5, p.235-237 / PDF p.181-183)
U05-135	다단계 피드백 큐는 프로세스가 큐 사이를 이동할 수 있다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-136	CPU를 많이 쓰면 낮은 큐로 내려가고 I/O 중심·대화형처럼 짧은 버스트는 높은 큐에 머문다.	(5.3.6, p.237-239 / PDF p.183-185)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-137	오래 대기한 하위 큐 작업을 올리는 노화로 기아를 줄일 수 있다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-138	새 프로세스는 큐 0에서 8ms를 받는다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-139	끝나지 않으면 큐 1의 꼬리로 내려가고 큐 0이 비었을 때 16ms를 받는다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-140	또 끝나지 않으면 큐 2로 간다. 큐 0·1이 비었을 때 FCFS로 실행한다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-141	상위 큐에 새 프로세스가 오면 하위 큐의 실행을 선점한다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-142	그림 5.9 다단계 피드백 큐	(5.3.6, p.238 / PDF p.184)
U05-143	정의해야 할 매개변수는 큐 수, 각 큐 알고리즘, 승격 시기, 강등 시기, 새로 서비스가 필요할 때 들어갈 큐이다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-144	특정 시스템에 맞게 매우 유연하지만 이 매개변수의 적절한 선택이 필요하여 복잡하다.	(5.3.6, p.237-239 / PDF p.183-185)
U05-145	PCS(프로세스 경쟁 범위)	(5.4.1, p.239-240 / PDF p.185-186)
U05-146	같은 프로세스의 사용자 스레드들이 가용 LWP를 놓고 경쟁한다. 스레드 라이브러리가 결정한다.	(5.4.1, p.239-240 / PDF p.185-186)
U05-147	SCS(시스템 경쟁 범위)	(5.4.1, p.239-240 / PDF p.185-186)
U05-148	시스템 전체의 커널 스레드가 물리 CPU 코어를 놓고 경쟁한다. 커널이 결정한다.	(5.4.1, p.239-240 / PDF p.185-186)
U05-149	사용자 스레드가 LWP에 스케줄되었다고 해서 곧바로 물리 CPU에서 실행되는 것은 아니다.	(5.4.1, p.239-240 / PDF p.185-186)
U05-150	해당 커널 스레드도 CPU에 배정되어야 한다.	(5.4.1, p.239-240 / PDF p.185-186)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-151	일대일 Windows·Linux는 SCS를 사용한다.	(5.4.1, p.239-240 / PDF p.185-186)
U05-152	PCS는 통상 프로그래머가 정한 우선순위를 따르고 더 높은 순위로 선점할 수 있지만 동일 순위의 타임 슬라이스를 보장하지는 않는다.	(5.4.1, p.239-240 / PDF p.185-186)
U05-153	PTHREAD_SCOPE_PROCESS는 PCS, PTHREAD_SCOPE_SYSTEM은 SCS를 선택한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-154	다대다에서 SYSTEM을 쓰면 사용자 스레드에 LWP를 만들고 연결하여 일대일에 가까운 방식이 된다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-155	pthread_attr_getscope()는 현재 범위를 읽고 pthread_attr_setscope()는 속성에 범위를 지정한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-156	오류 시 0이 아닌 값을 반환한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-157	원문은 Linux·macOS에서 SYSTEM만 허용한다고 설명한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-158	원문 그림 5.10은 기본 속성을 얻고 현재 범위를 출력한 뒤 SYSTEM을 설정하여 다섯 스레드를 생성·조인한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-159	원문 확인 필요: 인쇄 240쪽 설명 중 PROCESS로 지정한다는 문장과 실제 그림의 SYSTEM 설정이 일치하지 않는다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-160	이 노트는 실제 코드의 SYSTEM을 기준으로 목적을 설명하고 불일치를 명시한다.	(5.4.2, p.240-241 / PDF p.186-187)
U05-161	그림 5.10의 경쟁 범위 설정 부분 (원문 코드 발췌) /* set the scheduling algorithm to PCS or SCS */ pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);	(5.4.2, p.241 / PDF p.187)
U05-162	여러 CPU는 병렬 실행과 부하 공유를 가능하게 하지만 스케줄링을 복잡하게 한다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-163	비대칭 다중 처리는 한 마스터가 스케줄링·입출력 등 시스템 활동을 맡아 자료 공유를 단순화하지만 마스터가 병목이 될 수 있다.	(5.5.1, p.242-243 / PDF p.188-189)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-164	SMP는 각 처리기가 스스로 스케줄한다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-165	공통 큐	(5.5.1, p.242-243 / PDF p.188-189)
U05-166	유휴 처리기가 곧바로 작업을 선택할 수 있다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-167	같은 스레드 중복 선택·누락을 막는 동기화가 필요하며 락 경합이 병목이 될 수 있다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-168	처리기별 큐	(5.5.1, p.242-243 / PDF p.188-189)
U05-169	공유 큐 접근 경합을 줄이고 캐시 선호도를 활용한다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-170	큐마다 부하가 달라질 수 있어 균등화가 필요하다.	(5.5.1, p.242-243 / PDF p.188-189)
U05-171	한 칩의 여러 코어는 운영체제에 개별 논리 CPU처럼 보인다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-172	코어는 메모리 데이터가 준비되기를 기다리는 메모리 스톨로 계산 시간을 잃을 수 있다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-173	하드웨어 다중 스레딩은 한 코어에 여러 하드웨어 실행 상태를 두고 한 스레드가 기다릴 때 다른 스레드로 전환한다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-174	각 하드웨어 스레드는 명령 포인터·레지스터 등 상태를 가지므로 운영체제에는 논리 CPU로 보인다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-175	거친 다중 스레딩은 긴 지연 이벤트에서 전환하여 파이프라인 정리·재충전 비용이 크다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-176	세밀한 다중 스레딩은 명령 주기 같은 짧은 경계에서 전환하도록 회로를 제공하여 전환 비용을 낮춘다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-177	물리 코어의 캐시·파이프라인 자원은 하드웨어 스레드가 공유한다.	(5.5.2, p.243-246 / PDF p.189-192)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-178	운영체제가 논리 CPU인 하드웨어 스레드에 실행할 소프트웨어 스레드를 배정한다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-179	코어가 실제 실행할 하드웨어 스레드를 선택한다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-180	두 결정은 무관하지 않다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-181	소프트웨어 스레드를 같은 코어의 하드웨어 스레드 둘에 넣으면 자원을 나누므로 서로 다른 코어에 배정하는 것보다 느릴 수 있다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-182	운영체제가 자원 공유 구조를 알면 더 적절한 배정을 할 수 있다.	(5.5.2, p.243-246 / PDF p.189-192)
U05-183	부하 균등화는 어떤 처리기는 쉬고 다른 처리기에는 대기 스레드가 쌓이는 불균형을 줄인다.	(5.5.3, p.247 / PDF p.193)
U05-184	주로 처리기별 큐를 사용하는 경우 필요하다.	(5.5.3, p.247 / PDF p.193)
U05-185	push 이주는 주기적으로 부하를 검사하여 바쁜 처리기에서 덜 바쁜 곳으로 옮기고, pull 이주는 유휴 처리기가 바쁜 쪽에서 작업을 가져온다.	(5.5.3, p.247 / PDF p.193)
U05-186	둘은 함께 사용할 수 있다.	(5.5.3, p.247 / PDF p.193)
U05-187	같은 스레드 수가 항상 같은 부하를 뜻하지 않는다.	(5.5.3, p.247 / PDF p.193)
U05-188	우선순위나 다른 목표도 고려할 수 있으며 단순한 균등 기준은 스케줄링 목표와 충돌할 수 있다.	(5.5.3, p.247 / PDF p.193)
U05-189	원문 확인 필요: 이 항목의 소제목 번호는 인쇄 247쪽에서 5.5.2로 표시되어 앞의 다중 코어 절 번호와 중복된다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-190	본문의 다른 참조는 5.5.4를 사용하므로 정리 순서를 구별하고 실제 인쇄 번호를 제목에 병기하였다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-191	같은 처리기에서 계속 실행하면 최근 데이터가 남은 warm cache를 활용한다.	(5.5.4, p.247-249 / PDF p.193-195)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-192	다른 곳으로 이주하면 캐시를 무효화·재충전하는 비용이 든다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-193	동일 처리기를 선호하되 보장하지 않는 것이 약한 선호도, 실행 가능한 CPU 집합을 지정하는 것이 강한 선호도이다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-194	NUMA에서는 로컬 메모리가 원격 메모리보다 빠르므로 CPU 스케줄러와 메모리 배치가 협력해 실행 CPU에 가까운 메모리를 써야 한다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-195	부하 균등화를 위해 이주하면 캐시·NUMA의 이점을 잃을 수 있다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-196	따라서 균등화와 메모리 접근 지연 최소화는 상충하는 목표이며 함께 조정해야 한다.	(5.5.4, p.247-249 / PDF p.193-195)
U05-197	HMP는 같은 명령어 집합을 실행하되 속도·전력 관리 특성이 다른 코어를 결합한다.	(5.5.5, p.249 / PDF p.195)
U05-198	작업이 모든 코어에서 실행 가능하므로 특정 마스터만 시스템 작업을 하는 비대칭 다중 처리와 다르다.	(5.5.5, p.249 / PDF p.195)
U05-199	원문의 ARM big.LITTLE에서 짧고 높은 성능을 요구하는 대화형 작업은 big, 긴 배경 작업은 효율적인 LITTLE에 배정할 수 있다.	(5.5.5, p.249 / PDF p.195)
U05-200	전력 절약 모드에서는 big을 끄는 방법도 설명한다.	(5.5.5, p.249 / PDF p.195)
U05-201	연성 실시간	(5.6, p.250 / PDF p.196)
U05-202	중요 실시간 프로세스가 일반 프로세스보다 우선권을 가진다. 특정 시점 실행은 보장하지 않는다.	(5.6, p.250 / PDF p.196)
U05-203	경성 실시간	(5.6, p.250 / PDF p.196)
U05-204	태스크가 마감시간 안에 서비스를 받아야 한다. 기한 뒤 결과는 서비스를 받지 못한 것과 같은 문제가 된다.	(5.6, p.250 / PDF p.196)
U05-205	이벤트 지연은 이벤트 발생부터 대응 서비스까지이다.	(5.6.1, p.250-252 / PDF p.196-198)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-206	인터럽트 지연은 인터럽트 발생부터 서비스 루틴 시작까지로, 현재 명령 완료·종류 식별·상태 저장 등이 포함된다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-207	커널의 인터럽트 불능 시간이 영향을 주므로 매우 짧게 제한해야 한다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-208	경성 실시간에서는 단순히 짧을 뿐 아니라 정해진 한계보다 작아야 한다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-209	디스패치 지연은 현재 프로세스를 멈추고 다음 실행을 시작하는 시간이다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-210	선점형 커널이 이를 줄이는 데 효과적이다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-211	충돌 단계에는 커널 실행의 선점과 높은 순위가 필요한 자원을 낮은 순위가 해제하기까지의 대기가 포함되고, 이후 높은 순위를 CPU에 스케줄한다.	(5.6.1, p.250-252 / PDF p.196-198)
U05-212	실시간은 필요 시 빠르게 응답하도록 선점 우선순위 스케줄링을 요구한다.	(5.6.2, p.252-253 / PDF p.198-199)
U05-213	하지만 높은 순위만 주는 것으로는 연성 기능에 그치며 경성은 기한을 지킬 추가 기법이 필요하다.	(5.6.2, p.252-253 / PDF p.198-199)
U05-214	원문은 주기 태스크의 수행 시간 t, 마감시간 d, 주기 p를 정의한다.	(5.6.2, p.252-253 / PDF p.198-199)
U05-215	원문 수식: $0 \le t \le d \le p$, 실행 빈도 = 1/p; t: 주기마다 필요한 고정 수행 시간; d: 서비스를 완료해야 하는 마감시간; p: 반복 주기; 원문 주기 태스크 모델의 관계이다. 승인 제어는 기한 내 완료 가능한 요청을 허용하고 그렇지 않으면 거절한다.	(5.6.2, p.252-253 / PDF p.198-199)
U05-216	Rate-monotonic(RM)은 주기가 짧은 태스크에 높은 정적 우선순위를 주는 선점 정책이다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-217	주기 태스크는 각 버스트에서 고정된 수행 시간을 요구한다.	(5.6.3, p.253-255 / PDF p.199-201)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-218	정적 우선순위 알고리즘 가운데 원문이 제시한 의미에서 최적이지만 모든 CPU 부하를 항상 기한 안에 처리하지는 못한다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-219	주기 p	(5.6.3, p.253-255 / PDF p.199-201)
U05-220	50	(5.6.3, p.253-255 / PDF p.199-201)
U05-221	100	(5.6.3, p.253-255 / PDF p.199-201)
U05-222	수행 t	(5.6.3, p.253-255 / PDF p.199-201)
U05-223	20	(5.6.3, p.253-255 / PDF p.199-201)
U05-224	35	(5.6.3, p.253-255 / PDF p.199-201)
U05-225	이용률 t/p	(5.6.3, p.253-255 / PDF p.199-201)
U05-226	0.40	(5.6.3, p.253-255 / PDF p.199-201)
U05-227	0.35	(5.6.3, p.253-255 / PDF p.199-201)
U05-228	총 75% 이용률이라도 P2를 먼저 실행하면 P1이 시간 55에 끝나 기한 50을 놓친다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-229	RM에서는 P1이 0~20, P2가 20~50 실행한 뒤 다음 P1이 선점하여 50~70 실행하고 P2가 70~75에 끝나 두 기한을 만족한다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-230	원문 수식: N(2^(1/N) − 1); N: 스케줄할 주기 프로세스 수; 원문이 제시한 최악의 경우 보장 가능한 CPU 이용률 경계이다. 한 프로세스는 100%, 두 프로세스는 약 83%, 수가 커지면 약 69%로 내려간다. 경계 초과는 모든 경우 실패한다는 뜻이 아니라 보장할 수 없다는 뜻이다.	(5.6.3, p.253-255 / PDF p.199-201)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-231	두 번째 예 p1=50·t1=25, p2=80·t2=35에서는 총 이용률이 원문 반올림으로 약 0.94이다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-232	P1의 고정 우선권으로 P2가 시간 80의 기한을 놓친다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-233	원문 차트로 그 실패 시점을 보존하였다.	(5.6.3, p.253-255 / PDF p.199-201)
U05-234	그림 5.23 RM이 기한을 놓치는 예	(5.6.3, p.255 / PDF p.201)
U05-235	EDF는 마감시간이 빠를수록 높은 동적 우선순위를 준다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-236	프로세스가 실행 가능해질 때 기한을 알려야 하며 새로운 기한에 맞춰 순위를 재조정한다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-237	RM과 달리 주기적일 필요도 고정 CPU 할당량일 필요도 없다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-238	RM이 실패한 p1=50·t1=25, p2=80·t2=35 예에서 EDF는 P1을 0~25에 실행한 뒤 P2를 실행한다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-239	시간 50에도 P2의 기한 80이 새 P1의 100보다 빠르므로 선점하지 않고 60까지 완료한다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-240	P1은 60~85에 실행하여 두 번째 기한을 지킨다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-241	이후 시간 100에는 P1의 150이 P2의 160보다 빨라 선점한다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-242	그림 5.24 EDF 스케줄링	(5.6.4, p.256 / PDF p.202)
U05-243	원문은 EDF를 이론적으로 최적이며 가능한 부하에서 CPU를 최대한 활용할 수 있는 방식으로 설명한다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-244	실제는 문맥 교환·인터럽트 비용 때문에 100%의 유효 이용이 불가능하다는 한계를 함께 둔다.	(5.6.4, p.255-256 / PDF p.201-202)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-245	기한을 지킬 수 없는 임의의 부하까지 보장한다는 뜻으로 확대하지 않는다.	(5.6.4, p.255-256 / PDF p.201-202)
U05-246	원문 수식: 할당받는 CPU 시간 비율 = N/T; T: 전체 시간 몫; N: 응용에 할당한 몫; 원문 예는 T=100에서 A=50·B=15·C=20을 주어 각각 50%·15%·20%를 보장한다. 승인 제어는 남은 몫 안의 요청만 허용한다.	(5.6.5, p.256-257 / PDF p.202-203)
U05-247	위에서 이미 85몫을 배정했으므로 새 D가 30몫을 요구하면 거절한다.	(5.6.5, p.256-257 / PDF p.202-203)
U05-248	몫을 보장하려면 자원이 충분할 때만 새 응용을 받아들이는 승인 제어가 함께 필요하다.	(5.6.5, p.256-257 / PDF p.202-203)
U05-249	SCHED_FIFO	(5.6.6, p.257-258 / PDF p.203-204)
U05-250	가장 높은 우선순위의 FIFO 스레드가 종료·봉쇄할 때까지 실행한다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-251	SCHED_RR	(5.6.6, p.257-258 / PDF p.203-204)
U05-252	같은 순위에 시간 할당량을 제공하는 점을 제외하면 FIFO와 유사하다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-253	SCHED_OTHER	(5.6.6, p.257-258 / PDF p.203-204)
U05-254	구체적 구현은 정의하지 않아 시스템별로 다르다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-255	POSIX.1b의 pthread_attr_getschedpolicy()와 pthread_attr_setschedpolicy()는 스레드 속성의 정책을 읽거나 설정한다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-256	오류 시 0이 아닌 값을 반환한다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-257	원문 그림 5.25는 현재 정책을 확인한 뒤 FIFO로 설정하고 다섯 스레드를 생성·조인한다.	(5.6.6, p.257-258 / PDF p.203-204)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-258	PCS·SCS라는 경쟁 범위와 FIFO·RR이라는 실행 정책을 서로 다른 속성으로 구별한다.	(5.6.6, p.257-258 / PDF p.203-204)
U05-259	원문은 전통적 UNIX 변형, O(1) 스케줄러, 커널 2.6.23부터의 CFS로 이어지는 역사를 설명한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-260	다음 내용은 제공된 책이 다루는 구현 범위이다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-261	스케줄링 클래스마다 순위와 알고리즘을 두고 가장 높은 클래스의 태스크를 선택한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-262	기본 CFS와 실시간 클래스를 구별한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-263	CFS는 고정 길이의 순위별 시간 할당량 대신 nice 값에 따른 CPU 비율을 제공한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-264	nice는 −20~19이고 작은 값이 높은 순위이며 기본은 0이다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-265	목적 지연시간은 모든 실행 가능 태스크가 한 번씩 실행할 시간 간격이고 태스크 수가 많아지면 늘어날 수 있다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-266	vruntime은 우선순위를 반영한 가상 실행 시간이다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-267	nice=0이면 실제 시간과 같고 낮은 우선순위는 더 빨리, 높은 우선순위는 더 느리게 증가한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-268	다음 실행 대상은 가장 작은 vruntime이다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-269	같은 nice에서 I/O 중심은 짧게 쓰고 대기하므로 CPU 중심보다 vruntime이 작아져 깨어날 때 우선할 수 있다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-270	실행 가능 태스크는 vruntime을 키로 하는 적·백 트리에 들어가며 봉쇄되면 빠진다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-271	가장 왼쪽이 최소값이고 그 위치를 rb_leftmost에 캐시하여 다음 선택을 효율화한다.	(5.7.1, p.259-262 / PDF p.205-208)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-272	원문은 실시간의 FIFO·RR을 일반 태스크보다 높은 전역 순위에 배치한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-273	부하 균등화는 스레드 수만이 아니라 우선순위·평균 CPU 이용을 결합해 부하를 정의한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-274	캐시·NUMA 불이익을 줄이기 위해 자원을 공유하는 코어를 스케줄링 도메인으로 계층화하고 가까운 도메인부터 균형을 맞춘다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-275	다른 NUMA 노드로의 먼 이주는 심각한 불균형에서 선택하며 전체가 바쁠 때는 로컬 접근 이점을 보존하려 한다.	(5.7.1, p.259-262 / PDF p.205-208)
U05-276	Windows는 우선순위 기반 선점으로 높은 스레드를 우선 실행한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-277	디스패처는 32단계 순위를 사용한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-278	가변 클래스는 1~15, 실시간은 16~31이며 0은 메모리 관리용이다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-279	각 순위 큐를 높은 곳부터 조사하고 준비 스레드가 없으면 idle 스레드를 실행한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-280	프로세스의 IDLE·BELOW_NORMAL·NORMAL·ABOVE_NORMAL·HIGH·REALTIME 클래스와 스레드의 상대 순위를 결합해 실제 순위를 정한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-281	가변 스레드는 몫 만료 시 기본 순위 아래로는 내려가지 않게 순위를 낮추고, I/O 대기 해제 시 종류에 따라 높인다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-282	대화형 응답과 계산 중심 작업의 CPU 독점을 함께 고려하는 방식이다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-283	전경 NORMAL 프로세스에는 더 긴 시간 몫을 주는 정책도 설명한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-284	UMS는 커널과 독립적인 사용자 모드 생성·관리를 지원한다.	(5.7.2, p.263-266 / PDF p.209-212)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-285	앞선 fiber는 같은 TEB를 공유하는 제약이 있었지만 UMS는 각 실행 문맥을 따로 보관한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-286	원문은 직접 사용자 스케줄러를 만들기보다 Concurrency Runtime 같은 상위 도구가 이를 활용하도록 설계됨을 설명한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-287	다중 처리기에서는 같은 코어의 하드웨어 스레드를 SMT 집합으로 묶고 최근 실행·선호도를 고려한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-288	각 스레드의 이상적인 처리기를 분산하여 부하를 여러 물리 코어에 나누며 프로세스마다 시작 시드 값을 달리하여 첫 코어 쏠림을 피한다.	(5.7.2, p.263-266 / PDF p.209-212)
U05-289	Solaris는 TS(시분할)·IA(대화형)·RT(실시간)·SYS(시스템)·FSS(공평 공유)·FP(고정 우선순위)라는 여섯 클래스를 전역 순위로 매핑한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-290	클래스별 알고리즘과 우선순위가 다르며 가장 높은 전역 순위 스레드를 선택한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-291	기본 TS는 다단계 피드백으로 순위를 동적으로 바꾸고 높은 순위에 짧은 몫, 낮은 순위에 긴 몫을 준다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-292	몫을 다 쓰면 순위를 낮추고 수면에서 복귀하면 높여 대화형 응답과 CPU 중심 처리량을 조정한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-293	IA는 같은 정책을 쓰되 창 응용의 순위를 높인다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-294	RT는 일반 클래스보다 우선하며 SYS는 커널 스레드용이므로 커널 모드에 들어간 사용자 프로세스와 동일하지 않다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-295	FP는 순위를 동적으로 바꾸지 않는다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-296	FSS는 프로젝트라는 프로세스 집합에 CPU 공유 권리를 부여한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-297	인터럽트 서비스용 스레드는 일반 클래스 밖의 가장 높은 범위를 사용한다.	(5.7.3, p.266-269 / PDF p.212-215)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-298	선택된 스레드는 봉쇄·몫 만료·더 높은 순위 선점까지 실행한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-299	원문은 Solaris 9부터 일대일 스레드 모델로 바뀐 것도 함께 설명한다.	(5.7.3, p.266-269 / PDF p.212-215)
U05-300	모든 환경에 최선인 알고리즘은 없다.	(5.8, p.268-269 / PDF p.214-215)
U05-301	먼저 이용률·처리량·응답·총처리 시간 가운데 상대 중요성과 제약을 정해야 한다.	(5.8, p.268-269 / PDF p.214-215)
U05-302	원문은 최대 응답 300ms를 만족하면서 이용률을 높이는 식의 목표를 예로 든다.	(5.8, p.268-269 / PDF p.214-215)
U05-303	그런 뒤 동일 기준으로 후보 알고리즘을 평가한다.	(5.8, p.268-269 / PDF p.214-215)
U05-304	결정론적 모델링은 미리 정한 정확한 작업 부하에 각 알고리즘을 적용한다.	(5.8.1, p.269-271 / PDF p.215-217)
U05-305	단순하고 빠르며 정확한 비교값을 주지만 그 입력에만 적용된다.	(5.8.1, p.269-271 / PDF p.215-217)
U05-306	반복되는 동일 프로그램의 요구량을 정확히 측정할 수 있을 때 선택에도 쓸 수 있다.	(5.8.1, p.269-271 / PDF p.215-217)
U05-307	버스트(ms)	(5.8.1, p.269-271 / PDF p.215-217)
U05-308	10	(5.8.1, p.269-271 / PDF p.215-217)
U05-309	29	(5.8.1, p.269-271 / PDF p.215-217)
U05-310	3	(5.8.1, p.269-271 / PDF p.215-217)
U05-311	7	(5.8.1, p.269-271 / PDF p.215-217)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-312	12	(5.8.1, p.269-271 / PDF p.215-217)
U05-313	이 입력의 원문 결과는 FCFS 평균 28ms, 비선점 SJF 평균 13ms, q=10인 RR 평균 23ms이다.	(5.8.1, p.269-271 / PDF p.215-217)
U05-314	모든 프로세스와 버스트를 시간 0에 안다는 특정 조건에서 비교한 것으로, 이후의 알 수 없는 작업 부하에 그대로 일반화하지 않는다.	(5.8.1, p.269-271 / PDF p.215-217)
U05-315	작업이 날마다 달라지면 고정 입력보다 도착·CPU·I/O 버스트의 분포를 측정하거나 추정한다.	(5.8.2, p.271-272 / PDF p.217-218)
U05-316	CPU와 각 I/O 장치를 큐를 가진 서버로 표현하고 도착률·서비스율로 이용률·평균 길이·대기를 계산하는 것이 큐잉 네트워크 분석이다.	(5.8.2, p.271-272 / PDF p.217-218)
U05-317	원문 수식: n = λ × W; n: 서비스 중인 프로세스를 제외한 평균 큐 길이; λ: 큐에 도착하는 평균률; W: 큐의 평균대기 시간; Little의 공식은 원문의 안정 상태 조건에서 도착 수와 이탈 수가 같음을 이용한다. 두 변수를 알면 나머지를 구한다. 원문 예 λ=7개/초, n=14개에서 W=2초이다.	(5.8.2, p.271-272 / PDF p.217-218)
U05-318	분석 가능한 분포·알고리즘이 제한적이고 실제와 다른 단순 분포 및 독립 가정이 필요할 수 있다.	(5.8.2, p.271-272 / PDF p.217-218)
U05-319	따라서 계산값은 실제 시스템의 근사이며 정확성에 한계가 있다.	(5.8.2, p.271-272 / PDF p.217-218)
U05-320	모의실험은 주요 구성요소를 자료구조로 만들고 모형 시간을 증가시키면서 프로세스·장치·스케줄러의 활동을 반영해 통계를 수집한다.	(5.8.3, p.272-273 / PDF p.218-219)
U05-321	수학적·경험적 분포의 난수로 입력을 만들 수 있지만 빈도만으로는 실제 이벤트의 순서 관계를 보존하지 못한다.	(5.8.3, p.272-273 / PDF p.218-219)
U05-322	추적 파일은 실제 실행 이벤트의 순서를 기록하여 같은 입력으로 알고리즘들을 비교하게 한다.	(5.8.3, p.272-273 / PDF p.218-219)
U05-323	상세할수록 정확해질 수 있지만 실행 시간·저장공간·설계와 디버깅 비용이 커진다.	(5.8.3, p.272-273 / PDF p.218-219)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-324	원문 그림은 하나의 실제 추적을 FCFS·SJF·RR 모의실험에 각각 넣고 성능 통계를 비교한다.	(5.8.3, p.272-273 / PDF p.218-219)
U05-325	실제 운영체제에 알고리즘을 구현해 실행하면 실제 환경을 평가할 수 있지만 코드·자료구조 변경과 시험 비용이 크다.	(5.8.4, p.273-274 / PDF p.219-220)
U05-326	회귀 시험으로 새 버그와 예전 버그 재발을 살펴야 한다.	(5.8.4, p.273-274 / PDF p.219-220)
U05-327	환경 자체도 프로그램 변화와 사용자의 정책 적응에 따라 변하므로 같은 작업을 반복하는 도구·스크립트와 지속적인 측정이 필요하다.	(5.8.4, p.273-274 / PDF p.219-220)
U05-328	관리자가 매개변수를 조정하거나 API로 우선순위를 바꾸는 유연성이 도움이 되지만 특정 응용에 맞춘 최적화가 일반 환경에서도 개선을 주는 것은 아니다.	(5.8.4, p.273-274 / PDF p.219-220)
U05-329	선택 기준·입력 대표성·실제 오버헤드를 함께 검토해야 한다.	(5.8.4, p.273-274 / PDF p.219-220)
U05-330	원문 요약은 알고리즘 선택 규칙·단일 및 다중 코어·실시간 ·Linux/Windows/Solaris와 평가를 연결한다.	(5.9, p.274-279 / PDF p.220-225)
U05-331	연습문제 5.1~5.10은 선점, 간트차트와 대기·총처리 시간, 큐 매개변수, I/O 중심 우대, PCS·SCS 및 우선순위 재계산을 점검한다.	(5.9, p.274-279 / PDF p.220-225)
U05-332	문제의 입력·공식도 검토하였으나 새로운 계산 결과나 풀이를 만들지 않았다.	(5.9, p.274-279 / PDF p.220-225)
U05-333	추가 자료와 참고문헌은 외부 지식을 도입하는 근거로 사용하지 않았다.	(5.9, p.274-279 / PDF p.220-225)
U05-334	FCFS	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-335	도착·요청 순	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-336	비선점·호위 효과	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-337	SJF/SRTF	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-338	짧은 다음 버스트/남은 시간	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-339	예측이 필요함	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-340	RR	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-341	같은 q를 순환 배정	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-342	q와 문맥 교환의 절충	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-343	우선순위	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-344	가장 높은 순위	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-345	숫자 방향 확인·기아와 노화	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-346	다단계 큐	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-347	상위 큐 우선 또는 큐별 몫	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-348	소속이 일반적으로 고정	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-349	피드백 큐	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-350	실행 특성·대기에 따라 이동	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-351	승격·강등 등 매개변수 선택 필요	(기본 CPU 스케줄링 선택 규칙, p.226-239 / PDF p.172-185)
U05-352	우선순위	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-353	주기 짧을수록 높음·정적	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-354	기한 빠를수록 높음·동적	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-355	대상 조건	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-356	고정 수행 시간의 주기 태스크	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-357	주기·고정 수행 시간을 반드시 요구하지 않음	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-358	필요 정보	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-359	주기와 수행 시간	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-360	실행 가능 시 마감시간 통지	(실시간 알고리즘의 순위, p.253-256 / PDF p.199-202)
U05-361	결정론적	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-362	정확한 고정 입력·빠르고 명확	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-363	해당 입력에 국한	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-364	큐잉	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-365	확률 분포로 평균 계산	(평가 방법의 범위, p.269-274 / PDF p.215-220)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-366	분포·독립 가정의 근사	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-367	모의실험	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-368	모형과 추적 파일로 실제 순서 반영	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-369	시간·저장·모형 개발 비용	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-370	구현	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-371	실제 시스템 환경에서 실행	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-372	코드 변경·회귀 시험·환경 변화 비용	(평가 방법의 범위, p.269-274 / PDF p.215-220)
U05-373	선입 선처리(FCFS) 스케줄링은 가장 간단하지만 짧은 프로세스가 매우 긴 프로세스를 기다리게 할 수 있다.	(5.9, p.274 / PDF p.220)
U05-374	최단 작업 우선(SJF) 스케줄링은 최적이며 평균대기 시간이 가장 작지만 다음 CPU 버스트의 길이를 예측하기 어려워 구현이 어렵다.	(5.9, p.274-275 / PDF p.220-221)
U05-375	라운드 로빈(RR) 스케줄링은 CPU를 시간 할당량 동안 프로세스에 할당하며, 만료 전에 CPU를 포기하지 않으면 선점한다.	(5.9, p.275 / PDF p.221)
U05-376	우선순위 스케줄링은 각 프로세스에 우선순위를 배정하고 CPU를 우선순위가 가장 높은 프로세스에 할당한다.	(5.9, p.275 / PDF p.221)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-377	다단계 큐 스케줄링은 프로세스를 우선순위에 따라 여러 큐로 분할하고 가장 높은 큐에서 실행하며, 큐마다 다른 알고리즘을 사용할 수 있다.	(5.9, p.275 / PDF p.221)
U05-378	EDF 스케줄링은 마감시한에 따라 우선순위를 지정하며 마감시한이 빠를수록 우선순위가 높다.	(5.9, p.275 / PDF p.221)
U05-379	다단계 피드백 큐는 프로세스가 다른 큐 간에 이주할 수 있다는 점을 제외하면 다단계 큐와 유사하다.	(5.9, p.275 / PDF p.221)
U05-380	Rate-monotonic	(5.9, p.275 / PDF p.221)
U05-381	EDF	(5.9, p.275 / PDF p.221)
U05-382	Rate-monotonic의 이용률 경계: N(2^(1/N) – 1); N은 스케줄할 프로세스 수이다. 원문의 최악의 경우 CPU 이용률 경계이다.	(5.6.3, p.255 / PDF p.201)

제6장. 동기화 도구들

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-001	경쟁 조건은 프로세스가 공유 데이터에 병행하게 접근할 때 발생하며 최종 결과는 병행 접근의 특정 순서에 따라 다르다.	(6.10, p.315 / PDF p.259)
U06-002	임계구역은 공유 데이터가 조작될 수 있으며 경쟁 조건이 발생할 수 있는 코드 영역이다.	(6.10, p.315 / PDF p.259)
U06-003	모니터는 프로세스 동기화의 높은 수준 형태를 제공하는 추상 데이터 유형이다.	(6.10, p.315 / PDF p.259)
U06-004	협력적 프로세스는 시스템 안의 다른 프로세스 실행에 영향을 주거나 영향을 받는다.	(6.1, p.283-285 / PDF p.227-229)
U06-005	논리 주소 공간을 직접 공유하거나 공유 메모리·메시지 전달로 데이터를 공유할 수 있다.	(6.1, p.283-285 / PDF p.227-229)
U06-006	병행 실행에서는 어느 명령 지점에서도 인터럽트와 실행 전환이 일어날 수 있고, 병렬 실행에서는 여러 코어의 명령 흐름이 동시에 진행된다.	(6.1, p.283-285 / PDF p.227-229)
U06-007	공유 데이터에 대한 접근을 질서 있게 조정하지 않으면 데이터의 일관성이 손상될 수 있다.	(6.1, p.283-285 / PDF p.227-229)
U06-008	원문의 유한 버퍼 예는 채워진 버퍼 수를 나타내는 정수 count를 추가한다.	(6.1, p.283-285 / PDF p.227-229)
U06-009	생산자는 항목을 넣고 count를 증가시키며 소비자는 항목을 꺼내고 감소시킨다.	(6.1, p.283-285 / PDF p.227-229)
U06-010	각 코드는 따로 실행하면 올바르지만 count++와 count--는 각각 읽기·레지스터 계산·저장으로 나뉠 수 있다.	(6.1, p.283-285 / PDF p.227-229)
U06-011	초기 count가 5일 때 두 연산을 병행하면 4, 5, 6이 될 수 있지만 올바른 값은 5이다.	(6.1, p.283-285 / PDF p.227-229)
U06-012	생산자가 count=5를 자신의 레지스터로 읽고 6으로 증가시킨다.	(6.1, p.283-285 / PDF p.227-229)
U06-013	소비자가 아직 5인 count를 다른 레지스터로 읽고 4로 감소시킨다.	(6.1, p.283-285 / PDF p.227-229)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-014	생산자가 6을 count에 저장한 뒤 소비자가 4를 저장한다. 실제 채워진 버퍼는 5개지만 count는 4이다.	(6.1, p.283-285 / PDF p.227-229)
U06-015	경쟁 상황(race condition)은 여러 프로세스가 같은 자료를 병행하게 접근·조작하고 최종 결과가 접근의 특정 순서에 의존하는 상황이다.	(6.1, p.283-285 / PDF p.227-229)
U06-016	원문 예에서는 한순간에 하나의 프로세스만 count를 조작하도록 보장해야 하며, 이를 위해 프로세스 동기화가 필요하다.	(6.1, p.283-285 / PDF p.227-229)
U06-017	임계구역(critical section)은 적어도 하나의 다른 프로세스와 공유하는 데이터를 접근·갱신하는 코드 영역이다.	(6.2, p.286-288 / PDF p.230-232)
U06-018	한 프로세스가 그 영역을 수행하는 동안 다른 프로세스는 자신의 해당 임계구역에서 실행할 수 없어야 한다.	(6.2, p.286-288 / PDF p.230-232)
U06-019	임계구역 문제는 협력적으로 데이터를 공유하는 프로세스의 활동을 동기화할 프로토콜을 설계하는 문제이다.	(6.2, p.286-288 / PDF p.230-232)
U06-020	진입 구역(entry section)에서 임계구역 진입 허가를 요청한다.	(6.2, p.286-288 / PDF p.230-232)
U06-021	임계구역에서 공유 데이터를 조작한다.	(6.2, p.286-288 / PDF p.230-232)
U06-022	퇴출 구역(exit section)을 수행하고 나머지 구역(remainder section)을 실행한다.	(6.2, p.286-288 / PDF p.230-232)
U06-023	상호 배제	(6.2, p.286-288 / PDF p.230-232)
U06-024	한 프로세스가 임계구역을 실행하면 다른 프로세스는 그들의 임계구역을 실행할 수 없다.	(6.2, p.286-288 / PDF p.230-232)
U06-025	진행	(6.2, p.286-288 / PDF p.230-232)
U06-026	임계구역이 비어 있고 진입 희망자가 있다면 나머지 구역에서 실행 중이지 않은 프로세스들만 다음 진입자 결정에 참여하며, 결정을 무한정 연기할 수 없다.	(6.2, p.286-288 / PDF p.230-232)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-027	한정된 대기	(6.2, p.286-288 / PDF p.230-232)
U06-028	한 프로세스의 진입 요청 이후 허가될 때까지 다른 프로세스의 임계구역 진입 횟수에 한계가 있어야 한다.	(6.2, p.286-288 / PDF p.230-232)
U06-029	각 프로세스가 0이 아닌 속도로 실행된다고 가정하지만 프로세스 사이의 상대 속도는 가정하지 않는다.	(6.2, p.286-288 / PDF p.230-232)
U06-030	커널의 열린 파일 리스트, 다음 사용 가능한 PID, 메모리 할당·프로세스 리스트·인터럽트 자료구조도 경쟁 조건에 노출된다.	(6.2, p.286-288 / PDF p.230-232)
U06-031	단일 코어에서 공유 변수를 수정하는 동안 인터럽트를 금지할 수 있지만, 다중 처리기에 이를 적용하면 모든 처리기로 전달하는 비용과 효율 저하, 클록 갱신의 영향을 고려해야 한다.	(6.2, p.286-288 / PDF p.230-232)
U06-032	비선점형	(6.2, p.286-288 / PDF p.230-232)
U06-033	커널 모드에서 실행 중인 프로세스는 커널을 나가거나 봉쇄되거나 자발적으로 CPU를 양보할 때까지 계속 실행한다. 원문은 한순간 커널 실행자가 하나라는 관점에서 커널 자료구조의 경쟁 조건을 설명한다.	(6.2, p.286-288 / PDF p.230-232)
U06-034	선점형	(6.2, p.286-288 / PDF p.230-232)
U06-035	커널 모드에서도 선점을 허용하므로 공유 자료구조를 신중히 보호해야 한다. SMP에서는 여러 처리기의 프로세스가 커널 모드에 있을 수 있다. 더 민첩한 응답과 실시간 프로세스의 선점에 유리하다.	(6.2, p.286-288 / PDF p.230-232)
U06-036	Peterson의 해결안은 임계구역과 나머지 구역을 번갈아 실행하는 두 프로세스에 한정된다.	(6.3, p.288-291 / PDF p.232-235)
U06-037	turn은 진입 순번, flag 배열은 각 프로세스가 진입할 준비가 되었는지를 나타낸다.	(6.3, p.288-291 / PDF p.232-235)
U06-038	두 프로세스가 함께 들어가려 하면 turn에 대한 마지막 배정이 먼저 들어갈 프로세스를 결정한다.	(6.3, p.288-291 / PDF p.232-235)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-039	Peterson의 해결안: 프로세스 P_i의 구조 (원문 코드 발췌)
	while (true) {
	flag[i] = true;
	turn = j;
	while (flag[j] && turn == j)
	;
	
	/* critical section */
	
	flag[i] = false;
	
	/*remainder section */
	}
		(6.3, p.289 / PDF p.233)

U06-040	원문의 증명은 두 프로세스가 동시에 임계구역에 있다면 flag가 모두 true여야 하지만 turn은 동시에 두 값을 가질 수 없다는 점으로 상호 배제를 보인다.	(6.3, p.288-291 / PDF p.232-235)
U06-041	진입을 막는 것은 상대가 진입 의사가 있고 상대 차례인 경우뿐이다.	(6.3, p.288-291 / PDF p.232-235)
U06-042	상대가 나가면 flag를 false로 바꾸고, 다시 들어가려 해도 turn을 넘겨주므로 기다리던 프로세스가 무한정 양보하지 않는다.	(6.3, p.288-291 / PDF p.232-235)
U06-043	적용 한계: 현대 컴파일러와 프로세서는 데이터 종속성이 없는 읽기·쓰기를 재정렬할 수 있다.	(6.3, p.288-291 / PDF p.232-235)
U06-044	원문은 x와 flag를 공유하는 두 스레드에서 x=100보다 flag=true가 먼저 보이면 기대한 100 대신 0을 출력할 수 있음을 보인다.	(6.3, p.288-291 / PDF p.232-235)
U06-045	Peterson의 진입 구역 첫 두 배정 순서가 바뀌면 두 스레드가 동시에 임계구역에 있을 수 있다.	(6.3, p.288-291 / PDF p.232-235)
U06-046	따라서 논리적 증명만으로 현대 구조의 실행까지 보장한다고 해석하면 안 된다.	(6.3, p.288-291 / PDF p.232-235)
U06-047	원문은 메모리 장벽, 원자적 하드웨어 명령, 원자적 변수를 제시한다.	(6.4, p.291-297 / PDF p.235-241)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-048	이들은 직접 동기화에 쓰이거나 더 높은 수준 도구를 만드는 기본 구성요소가 된다.	(6.4, p.291-297 / PDF p.235-241)
U06-049	메모리 모델은 컴퓨터 구조가 프로그램의 메모리 접근에 보장하는 사항을 뜻한다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-050	원문은 한 처리기의 변경 결과가 다른 모든 처리기에 즉시 보이는 강한 순서와 즉시 보이지 않는 약한 순서를 구별한다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-051	구조마다 모델이 다르므로 공유 메모리 다중 처리기의 가시성을 임의로 가정할 수 없다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-052	메모리 장벽(memory barrier, memory fence) 명령이 실행되면 뒤의 적재·저장 전에 앞의 적재·저장이 완료되도록 한다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-053	저장된 변경이 다른 처리기에 보이도록 보장하여 재정렬의 문제를 다룬다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-054	원문의 x/flag 예에서는 읽는 쪽에서 flag 적재 뒤 x 적재 전에, 쓰는 쪽에서는 x 배정 뒤 flag 배정 전에 장벽을 둔다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-055	매우 낮은 수준의 연산이며 원문은 주로 커널 개발자의 특수 동기화 코드에 사용된다고 설명한다.	(6.4.1, p.292-293 / PDF p.236-237)
U06-056	원자적 명령은 검사·변경 또는 교환을 인터럽트되지 않는 하나의 단위로 수행한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-057	test_and_set()은 원래 값을 반환하면서 대상을 true로 설정한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-058	false로 초기화한 lock을 이에 사용하면 먼저 도착한 실행자만 진입하고 나갈 때 false로 바꿀 수 있다.	(6.4.2, p.293-296 / PDF p.237-240)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-059	원자적 compare_and_swap()의 정의 (원문 코드 발췌)
	int compare_and_swap(int *value, int expected, int new_value) {
	int temp = *value;
	
	if (*value == expected)
	*value = new_value;
	
	return temp;
	}
		(6.4.2, p.294 / PDF p.238)

U06-060	CAS 상호 배제 예에서 lock은 0으로 초기화되고 compare_and_swap(&lock, 0, 1)을 반복한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-061	첫 성공자는 원래 값 0을 받아 임계구역에 들어가고 퇴출 시 lock=0으로 바꾼다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-062	그러나 단순 반복은 상호 배제를 만족해도 한정된 대기를 만족하지 않는다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-063	원문의 별도 알고리즘은 waiting 배열을 false, lock을 0으로 초기화하고 퇴출 시 대기자를 순환 탐색한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-064	다음 대기자에게 진입을 넘겨 최대 n−1번만 양보하도록 한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-065	원문의 Intel x86 설명은 cmpxchg가 CAS를 구현하고 대상 피연산자가 갱신되는 동안 lock 접두어로 버스를 잠가 원자적 실행을 강제한다고 기술한다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-066	여기서는 이 설명을 다른 구조의 세부 동작으로 확장하지 않는다.	(6.4.2, p.293-296 / PDF p.237-240)
U06-067	원자적 변수는 정수·부울 같은 기본 자료형에 원자적 연산을 제공한다.	(6.4.3, p.296-297 / PDF p.240-241)
U06-068	CAS는 흔히 직접 사용되기보다 이런 도구의 기초가 된다.	(6.4.3, p.296-297 / PDF p.240-241)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-069	원문의 증가 함수는 현재 값을 읽고 그 값이 유지되어 있을 때만 1 증가시키는 CAS를 시도하며 충돌하면 반복한다.	(6.4.3, p.296-297 / PDF p.240-241)
U06-070	한 변수의 갱신이 원자적이라는 사실만으로 전체 알고리즘의 경쟁 조건이 사라지지는 않는다.	(6.4.3, p.296-297 / PDF p.240-241)
U06-071	유한 버퍼에서 count가 원자적이어도 두 소비자가 count>0을 기다리다가 생산자가 항목 하나만 추가했을 때 함께 대기를 빠져나올 수 있다.	(6.4.3, p.296-297 / PDF p.240-241)
U06-072	원문은 원자적 변수를 주로 카운터·시퀀스 생성기 같은 공유 데이터 한 개의 갱신에 적합한 도구로 제한하여 설명한다.	(6.4.3, p.296-297 / PDF p.240-241)
U06-073	Mutex 락은 임계구역 진입 전에 acquire()로 획득하고 나올 때 release()로 반환하는 도구이다.	(6.5, p.297-299 / PDF p.241-243)
U06-074	available이라는 불린 값이 사용 가능 여부를 나타낸다.	(6.5, p.297-299 / PDF p.241-243)
U06-075	획득·반환 호출은 원자적이어야 하며 CAS로 구현할 수 있다.	(6.5, p.297-299 / PDF p.241-243)
U06-076	원문의 단순 구현은 사용 불가능한 동안 반복 검사하는 바쁜 대기를 한다.	(6.5, p.297-299 / PDF p.241-243)
U06-077	이 유형을 스핀락이라고 부른다.	(6.5, p.297-299 / PDF p.241-243)
U06-078	하나의 CPU 코어를 여러 프로세스가 나누는 환경에서는 다른 프로세스가 쓸 CPU 주기를 낭비한다.	(6.5, p.297-299 / PDF p.241-243)
U06-079	반면 대기하면서 문맥 교환을 하지 않는 이점이 있어 다중 코어에서 임계구역이 짧을 때 유리할 수 있다.	(6.5, p.297-299 / PDF p.241-243)
U06-080	원문이 제시하는 짧은 기간의 기준은 락 유지 시간이 대기와 재개를 위한 두 번의 문맥 교환 시간보다 짧은 경우이다.	(6.5, p.297-299 / PDF p.241-243)
U06-081	락을 얻으려다 봉쇄되면 경합, 즉시 얻을 수 있으면 비경합으로 본다.	(6.5, p.297-299 / PDF p.241-243)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-082	많은 스레드가 경쟁하는 높은 경합은 응용의 전체 성능을 떨어뜨린다.	(6.5, p.297-299 / PDF p.241-243)
U06-083	세마포 S는 정수 변수이며 초기화를 제외하고 원자적인 wait()와 signal()만으로 접근한다.	(6.6, p.299-303 / PDF p.243-247)
U06-084	원문의 고전적 wait는 S>0이 될 때까지 바쁘게 기다린 뒤 감소시키고 signal은 증가시킨다.	(6.6, p.299-303 / PDF p.243-247)
U06-085	검사와 감소까지 인터럽트되지 않는 하나의 동작이어야 한다.	(6.6, p.299-303 / PDF p.243-247)
U06-086	카운팅 세마포	(6.6, p.299-303 / PDF p.243-247)
U06-087	유한 개 자원의 접근을 제어한다. 가용 자원 수로 초기화하고 획득 때 감소, 반환 때 증가시킨다.	(6.6, p.299-303 / PDF p.243-247)
U06-088	이진 세마포	(6.6, p.299-303 / PDF p.243-247)
U06-089	0과 1의 값만 가지며 mutex 락과 유사하게 상호 배제를 제공할 수 있다.	(6.6, p.299-303 / PDF p.243-247)
U06-090	세마포는 실행 순서도 조정한다.	(6.6, p.299-303 / PDF p.243-247)
U06-091	원문은 P_1의 S_1 뒤에 P_2의 S_2가 실행되어야 할 때 synch를 0으로 초기화하고, S_1 뒤에 signal(synch), S_2 앞에 wait(synch)를 둔다.	(6.6, p.299-303 / PDF p.243-247)
U06-092	이 순서 제약은 하나의 임계구역 보호와 구별되는 용도이다.	(6.6, p.299-303 / PDF p.243-247)
U06-093	바쁜 대기를 줄이는 구현에서는 세마포마다 정수 value와 프로세스 리스트 list를 둔다.	(6.6, p.299-303 / PDF p.243-247)
U06-094	wait에서 기다려야 하면 프로세스를 대기 상태로 바꾸고 CPU 스케줄러가 다른 프로세스를 선택한다.	(6.6, p.299-303 / PDF p.243-247)
U06-095	signal이 깨우면 준비 완료 상태·준비 큐로 돌아간다.	(6.6, p.299-303 / PDF p.243-247)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-096	대기 큐를 사용하는 세마포 wait() (원문 코드 발췌)
	wait(semaphore *S) {
	S->value--;
	if (S->value < 0) {
	add this process to S->list;
	sleep();
	}
	}
		(6.6, p.301 / PDF p.245)

U06-097	대기 큐를 사용하는 세마포 signal() (원문 코드 발췌)
	signal(semaphore *S) {
	S->value++;
	if (S->value <= 0) {
	remove a process P from S->list;
	wakeup(P);
	}
	}
		(6.6, p.302 / PDF p.246)

U06-098	이 구현에서는 음수 값의 절댓값이 대기 프로세스 수를 나타낸다.	(6.6, p.299-303 / PDF p.243-247)
U06-099	이는 고전적 정의와 달리 감소 후 검사하기 때문이다.	(6.6, p.299-303 / PDF p.243-247)
U06-100	FIFO 큐로 대기자를 관리하면 한정된 대기를 보장하는 방법이 되지만 세마포의 정확한 사용 자체가 특정 큐잉 전략에 의존하는 것은 아니다.	(6.6, p.299-303 / PDF p.243-247)
U06-101	단일 처리기는 연산 동안 인터럽트를 막을 수 있고 SMP는 CAS·스핀락 등의 기법이 필요하다.	(6.6, p.299-303 / PDF p.243-247)
U06-102	바쁜 대기가 완전히 사라진 것은 아니며 짧은 세마포 연산의 임계구역에 국한된다.	(6.6, p.299-303 / PDF p.243-247)
U06-103	세마포는 호출 순서를 잘못 쓰면 발견하기 어려운 타이밍 오류를 낳는다.	(6.7, p.303-310 / PDF p.247-254)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-104	signal을 먼저 하고 wait를 나중에 하면 상호 배제가 깨질 수 있고, 퇴출 시 signal 대신 wait를 하면 영구 봉쇄될 수 있다.	(6.7, p.303-310 / PDF p.247-254)
U06-105	어느 연산을 빠뜨려도 상호 배제 위반 또는 영구 대기로 이어진다.	(6.7, p.303-310 / PDF p.247-254)
U06-106	이런 오류를 줄이기 위해 고급 언어 구조물인 모니터를 도입한다.	(6.7, p.303-310 / PDF p.247-254)
U06-107	모니터는 내부에서 상호 배제가 보장되는 연산 집합을 포함한 추상 자료형이다.	(6.7, p.303-310 / PDF p.247-254)
U06-108	지역 변수는 내부 함수만 접근하고 다른 프로세스가 직접 표현을 사용하지 못한다.	(6.7, p.303-310 / PDF p.247-254)
U06-109	모니터 안에서는 항상 하나의 프로세스만 활성화된다.	(6.7, p.303-310 / PDF p.247-254)
U06-110	프로그래머가 이 제약을 직접 코딩할 필요가 줄어든다.	(6.7, p.303-310 / PDF p.247-254)
U06-111	조건 변수 x의 x.wait()는 호출자를 일시 중지하고 x.signal()은 기다리는 프로세스 하나를 재개한다.	(6.7, p.303-310 / PDF p.247-254)
U06-112	기다리는 프로세스가 없으면 조건 변수 signal은 효과가 없다.	(6.7, p.303-310 / PDF p.247-254)
U06-113	세마포의 signal은 항상 세마포 상태에 영향을 준다는 점과 다르다.	(6.7, p.303-310 / PDF p.247-254)
U06-114	Signal and wait	(6.7, p.303-310 / PDF p.247-254)
U06-115	신호를 보낸 P가 기다리고 재개된 Q가 모니터를 떠나거나 다른 조건을 기다린다.	(6.7, p.303-310 / PDF p.247-254)
U06-116	Signal and continue	(6.7, p.303-310 / PDF p.247-254)
U06-117	P가 계속하며 Q는 P가 모니터를 떠나거나 다른 조건을 기다릴 때까지 대기한다.	(6.7, p.303-310 / PDF p.247-254)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-118	절충안	(6.7, p.303-310 / PDF p.247-254)
U06-119	P가 signal 직후 모니터를 떠나 Q가 즉시 재개되도록 한다.	(6.7, p.303-310 / PDF p.247-254)
U06-120	계속 실행 정책에서는 Q가 재개할 때 기다리던 논리 조건이 이미 참이 아닐 수 있다.	(6.7, p.303-310 / PDF p.247-254)
U06-121	원문은 Java·C# 등이 모니터 개념을 편입하고 다른 언어에서도 유사한 동기화 지원을 제공한다고 설명한다.	(6.7, p.303-310 / PDF p.247-254)
U06-122	세마포로 signal-and-wait 모니터를 구현할 때 mutex는 1, next는 0으로 초기화한다.	(6.7, p.303-310 / PDF p.247-254)
U06-123	next-count는 신호 후 대기하는 프로세스 수를 센다.	(6.7, p.303-310 / PDF p.247-254)
U06-124	외부 프로시저는 mutex를 획득하고 본체를 수행한 뒤 next 대기자가 있으면 그쪽에 신호하고 아니면 mutex를 반환한다.	(6.7, p.303-310 / PDF p.247-254)
U06-125	조건 x마다 x-sem과 x-count를 0으로 두어 조건 대기자와 신호 후 양보 관계를 연결한다.	(6.7, p.303-310 / PDF p.247-254)
U06-126	수행 재개 순서가 FCFS만으로 충분하지 않으면 x.wait(c)의 정수 c를 우선순위 번호로 저장할 수 있다.	(6.7, p.303-310 / PDF p.247-254)
U06-127	x.signal()은 가장 작은 번호의 프로세스를 재개한다.	(6.7, p.303-310 / PDF p.247-254)
U06-128	원문의 ResourceAllocator는 자원 사용을 희망하는 최대 시간이 가장 짧은 대기자에게 하나의 자원을 준다.	(6.7, p.303-310 / PDF p.247-254)
U06-129	고급 도구라도 잘못된 사용 순서 자체를 보장하지 않는다.	(6.7, p.303-310 / PDF p.247-254)
U06-130	허가 없이 자원 접근·반환, 얻은 자원 미반환, 반환 전에 재요청이 가능하다.	(6.7, p.303-310 / PDF p.247-254)
U06-131	정확성을 위해 호출 순서를 지키는지와 모니터 밖에서 공유 자원을 직접 접근하지 않는지를 확인해야 한다.	(6.7, p.303-310 / PDF p.247-254)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-132	대규모·동적 시스템에서 전수 검사는 비합리적일 수 있다는 한계도 원문에 명시된다.	(6.7, p.303-310 / PDF p.247-254)
U06-133	라이브니스(liveness)는 프로세스가 실행 수명주기 동안 진행하도록 시스템이 충족해야 할 속성이다.	(6.8, p.310-312 / PDF p.254-256)
U06-134	무기한 대기는 진행·한정된 대기를 위반하는 라이브니스 실패이다.	(6.8, p.310-312 / PDF p.254-256)
U06-135	바쁜 대기도 임의의 오랜 시간 지속되면 실패가 될 수 있다.	(6.8, p.310-312 / PDF p.254-256)
U06-136	교착 상태는 집합 안의 모든 프로세스가 그 집합의 다른 프로세스만 일으킬 수 있는 사건을 기다리는 상황이다.	(6.8, p.310-312 / PDF p.254-256)
U06-137	원문은 1로 초기화한 S, Q를 두 프로세스가 서로 반대 순서로 wait하여 각자 상대의 signal을 영원히 기다리는 예를 든다.	(6.8, p.310-312 / PDF p.254-256)
U06-138	우선순위 역전은 높은 우선순위 H가 낮은 우선순위 L이 점유한 자원을 기다리는 동안 중간 우선순위 M이 L을 선점하여 H의 대기까지 길게 하는 문제이다.	(6.8, p.310-312 / PDF p.254-256)
U06-139	원문은 세 단계 이상의 우선순위를 가진 시스템에서 설명한다.	(6.8, p.310-312 / PDF p.254-256)
U06-140	우선순위 상속은 해당 자원을 사용하는 L이 사용 완료까지 H의 우선순위를 물려받게 하고 이후 원래 값으로 돌려, M의 선점을 막는다.	(6.8, p.310-312 / PDF p.254-256)
U06-141	Mars Pathfinder의 반복 리셋 사례는 우선순위 상속을 활성화하여 해결한 원문 사례로 제시된다.	(6.8, p.310-312 / PDF p.254-256)
U06-142	하드웨어 명령은 낮은 수준의 기반이며 CAS로 락 없는 알고리즘도 만들 수 있다.	(6.9, p.312-314 / PDF p.256-258)
U06-143	이런 알고리즘은 오버헤드가 낮고 확장성 장점이 있지만 개발·테스트가 어렵다.	(6.9, p.312-314 / PDF p.256-258)
U06-144	CAS는 먼저 갱신을 시도하고 충돌하면 재시도하는 낙관적 접근, 상호 배제 락은 다른 갱신자가 있다고 가정하고 먼저 획득하는 비관적 접근으로 대비된다.	(6.9, p.312-314 / PDF p.256-258)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-145	없음	(6.9, p.312-314 / PDF p.256-258)
U06-146	양쪽 모두 빠르지만 CAS 보호가 기존 동기화보다 다소 빠를 수 있다.	(6.9, p.312-314 / PDF p.256-258)
U06-147	적당함	(6.9, p.312-314 / PDF p.256-258)
U06-148	CAS는 대개 성공하며 실패해도 재시도가 적어 기존 동기화보다 빠를 수 있다.	(6.9, p.312-314 / PDF p.256-258)
U06-149	심함	(6.9, p.312-314 / PDF p.256-258)
U06-150	반복 충돌이 커지므로 기존 동기화가 기본적으로 CAS보다 빠르다.	(6.9, p.312-314 / PDF p.256-258)
U06-151	한 개 공유 변수 갱신에는 원자적 정수가 가볍다.	(6.9, p.312-314 / PDF p.256-258)
U06-152	임계구역 접근 보호에는 mutex가 이진 세마포보다 간단하고 오버헤드가 적어 선호되지만, 유한 개 자원 제어에는 카운팅 세마포가 적합하다.	(6.9, p.312-314 / PDF p.256-258)
U06-153	여러 reader를 동시에 허용하는 reader-writer 락은 병행성을 높일 수 있다.	(6.9, p.312-314 / PDF p.256-258)
U06-154	모니터·조건 변수는 사용이 간단하지만 구현에 따라 오버헤드와 높은 경합에서의 확장성 한계가 있다.	(6.9, p.312-314 / PDF p.256-258)
U06-155	경쟁 조건은 공유 접근의 순서 의존성에서 발생한다.	(6.10, p.315-317 / PDF p.259-261)
U06-156	임계구역 해결은 상호 배제·진행·한정된 대기를 함께 검토해야 한다.	(6.10, p.315-317 / PDF p.259-261)
U06-157	하드웨어의 가시성과 원자성이 소프트웨어 도구의 토대이며, 세마포·모니터도 올바른 순서와 라이브니스 점검이 필요하다.	(6.10, p.315-317 / PDF p.259-261)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-158	원문 연습 문제는 인터럽트 금지와 클록, 바쁜 대기, 단일·다중 처리기에서의 스핀락, 세마포 연산의 원자성 및 공유 은행 계좌의 경쟁 조건을 점검한다.	(6.10, p.315-317 / PDF p.259-261)
U06-159	새 답안은 만들지 않았다.	(6.10, p.315-317 / PDF p.259-261)
U06-160	상호 배제는 한 프로세스가 자신의 임계구역에서 실행된다면 다른 프로세스는 자신의 임계구역에서 실행될 수 없다는 요구 조건이다.	(6.2, p.286 / PDF p.230)
U06-161	임계구역에서 실행되는 프로세스가 없고 진입하려는 프로세스들이 있으면, 나머지 구역에서 실행 중이지 않은 프로세스들만 다음 진입자를 결정하며 선택은 무한정 연기될 수 없다.	(6.2, p.286 / PDF p.230)
U06-162	한정된 대기는 프로세스가 임계구역 진입을 요청한 후 허용될 때까지 다른 프로세스들의 임계구역 진입 허용 횟수에 한계가 있어야 한다는 요구 조건이다.	(6.2, p.287 / PDF p.231)
U06-163	Mutex 락은 임계구역에 들어가기 전에 락을 획득하고 나올 때 해제하도록 요구하여 상호 배제를 제공한다.	(6.10, p.315 / PDF p.259)
U06-164	세마포는 정수 값을 가지며 상호 배제뿐 아니라 다양한 동기화 문제를 해결하는 데 사용될 수 있다.	(6.10, p.315 / PDF p.259)
U06-165	임계구역 문제의 해결책은 상호 배제, 진행 및 한정된 대기의 세 요구 사항을 충족해야 한다.	(6.10, p.315 / PDF p.259)
U06-166	임계구역 문제의 하드웨어 지원에는 메모리 장벽, compare_and_swap 같은 하드웨어 명령 및 원자적 변수가 포함된다.	(6.10, p.315 / PDF p.259)
U06-167	임계구역 문제의 해결책은 교착 상태를 포함한 라이브니스 문제를 겪을 수 있다.	(6.10, p.315 / PDF p.259)
U06-168	Mutex 락	(6.10, p.315 / PDF p.259)
U06-169	세마포	(6.10, p.315 / PDF p.259)

제7장. 동기화 예제

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-001	고전적인 프로세스 동기화 문제에는 유한 버퍼, reader-writer 및 식사하는 철학자 문제가 포함된다.	(7.6, p.345 / PDF p.288)
U07-002	POSIX API는 mutex 락, 세마포 및 조건 변수를 제공한다.	(7.6, p.345 / PDF p.288)
U07-003	임계구역 문제를 해결하기 위한 대안으로 트랜잭션 메모리, OpenMP 및 함수형 언어가 있다.	(7.6, p.346 / PDF p.289)
U07-004	원문은 새 동기화 방법의 능력을 확인하는 대표 문제로 세 가지 문제를 제시한다.	(7.1, p.319-327 / PDF p.262-270)
U07-005	세마포로 일반 구조를 설명하되 실제 구현에서는 이진 세마포 대신 mutex 락을 사용할 수 있다고 구별한다.	(7.1, p.319-327 / PDF p.262-270)
U07-006	n개의 버퍼는 각각 한 항목을 저장한다.	(7.1.1, p.320-321 / PDF p.263-264)
U07-007	mutex=1은 버퍼 풀의 배타적 접근, empty=n은 빈 버퍼 수, full=0은 채워진 버퍼 수를 나타낸다.	(7.1.1, p.320-321 / PDF p.263-264)
U07-008	자원 수를 기다리는 세마포와 실제 버퍼 조작을 보호하는 세마포의 역할을 분리하여 읽는다.	(7.1.1, p.320-321 / PDF p.263-264)

U07-009	유한 버퍼 생산자 구조 (원문 코드 발췌)
	while (true) {
	. . .
	/* produce an item in next_produced */
	. . .
	wait(empty);
	wait(mutex);
	. . .
	/* add next_produced to the buffer */
	. . .
	signal(mutex);
	signal(full);
	}
		(7.1.1, p.320 / PDF p.263)

번호	문장·표 항목·원문 코드·수식	원문 위치

U07-010	유한 버퍼 소비자 구조 (원문 코드 발췌)
	while (true) {
	wait(full);
	wait(mutex);
	. . .
	/* remove an item from buffer to next_consumed */
	. . .
	signal(mutex);
	signal(empty);
	. . .
	/* consume the item in next_consumed */
	. . .
	}
		(7.1.1, p.321 / PDF p.264)

U07-011	여러 reader가 동시에 읽는 것은 허용하지만 writer는 읽는 스레드나 다른 writer와 동시에 공유 데이터베이스에 접근하면 안 된다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-012	쓰는 동안 배타적 접근을 보장하는 문제가 readers-writers 문제이다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-013	첫 번째 문제	(7.1.2, p.321-323 / PDF p.264-266)
U07-014	writer가 아직 접근 허가를 얻지 못했다면, writer가 기다린다는 이유만으로 새 reader를 기다리게 하지 않는다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-015	writer	(7.1.2, p.321-323 / PDF p.264-266)
U07-016	두 번째 문제	(7.1.2, p.321-323 / PDF p.264-266)
U07-017	writer가 준비되어 기다리면 새 reader의 읽기 시작을 막아 가능한 빨리 쓰게 한다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-018	reader	(7.1.2, p.321-323 / PDF p.264-266)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-019	원문은 첫 번째 문제의 세마포 해법을 제시한다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-020	read_count는 읽는 프로세스 수, mutex는 그 수의 갱신 보호, rw_mutex는 writer의 배타적 접근과 reader 집단의 접근을 조정한다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-021	첫 reader가 rw_mutex를 기다리고 마지막 reader가 반환하며, 중간 reader들은 rw_mutex를 다시 기다리지 않는다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-022	writer가 반환할 때 여러 reader 또는 writer 한 개 중 어느 쪽이 진행할지는 스케줄러에 달린다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-023	Reader-writer 락은 획득할 때 읽기·쓰기 모드를 지정한다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-024	읽기 모드는 여러 프로세스가 동시에 얻을 수 있지만 쓰기는 하나만 얻는다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-025	읽기 전용과 쓰기를 식별하기 쉽고 reader가 writer보다 많은 응용에 유용하다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-026	설정 오버헤드는 mutex·세마포보다 크지만 여러 reader의 병행성으로 이를 상쇄할 수 있다.	(7.1.2, p.321-323 / PDF p.264-266)
U07-027	원형 테이블의 철학자 5명은 생각과 식사를 반복하고 각각 왼쪽·오른쪽 젓가락 두 개가 있어야 식사할 수 있다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-028	이는 교착 상태와 기아 없이 여러 스레드에 여러 자원을 할당하는 문제를 표현한다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-029	각 젓가락을 초기값 1의 세마포로 두고 모두 왼쪽부터 기다리게 하면 이웃의 동시 식사는 막지만 모두 왼쪽만 얻은 채 오른쪽을 기다리는 교착 상태가 가능하다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-030	동시에 테이블에 앉는 철학자를 최대 4명으로 제한할 수 있다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-031	임계구역 안에서 두 젓가락을 모두 집을 수 있을 때에만 집도록 할 수 있다.	(7.1.3, p.323-327 / PDF p.266-270)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-032	홀수는 왼쪽부터, 짝수는 오른쪽부터 집게 하는 비대칭 순서를 사용할 수 있다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-033	모니터 해법은 각 철학자를 THINKING·HUNGRY·EATING 상태로 구별하고 양옆 이웃이 EATING이 아닐 때만 먹게 한다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-034	각자 self 조건 변수로 기다리며 pickup → 식사 → putdown 순서를 지킨다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-035	이 해법은 이웃의 동시 식사와 교착 상태를 막지만 기아 가능성까지 없애지는 않는다.	(7.1.3, p.323-327 / PDF p.266-270)
U07-036	그림 7.7. 모니터 의사 코드 전체. 원문 자체의 표기를 보존하였다.	(7.1.3, p.326 / PDF p.269)
U07-037	Windows는 단일 처리기에서 전역 정보를 접근할 때 관련 인터럽트 핸들러 실행을 잠시 막고, 다중 처리기에서는 짧은 코드에 스핀락을 사용한다.	(7.2, p.327-330 / PDF p.270-273)
U07-038	스핀락 소유 중에는 선점되지 않도록 한다.	(7.2, p.327-330 / PDF p.270-273)
U07-039	커널 밖 스레드 동기화에는 mutex·세마포·event·타이머 등의 dispatcher 객체를 제공한다.	(7.2, p.327-330 / PDF p.270-273)
U07-040	Signaled	(7.2, p.327-330 / PDF p.270-273)
U07-041	객체가 사용 가능하고 획득 시 봉쇄되지 않는다.	(7.2, p.327-330 / PDF p.270-273)
U07-042	Nonsignaled	(7.2, p.327-330 / PDF p.270-273)
U07-043	객체가 사용 불가능하다. 획득하려는 스레드는 대기 상태·객체 대기 큐로 간다.	(7.2, p.327-330 / PDF p.270-273)
U07-044	다시 signaled가 됨	(7.2, p.327-330 / PDF p.270-273)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-045	커널은 객체 종류에 따라 하나 또는 여러 대기 스레드를 준비 상태로 옮긴다. Mutex는 하나, event는 기다리는 모든 스레드가 대상일 수 있다.	(7.2, p.327-330 / PDF p.270-273)
U07-046	Windows의 critical-section 객체는 사용자 모드 mutex이며 커널 개입 없이 획득·반환할 수 있다.	(7.2, p.327-330 / PDF p.270-273)
U07-047	다중 처리기에서는 먼저 회전하고 회전이 길면 커널 mutex를 할당하여 CPU를 양보한다.	(7.2, p.327-330 / PDF p.270-273)
U07-048	경쟁이 있을 때만 커널 mutex 비용을 부담하는 방식이다.	(7.2, p.327-330 / PDF p.270-273)
U07-049	원문이 설명하는 Linux 커널은 선점 가능하며 원자적 정수 atomic_t, mutex, 스핀락·세마포 및 reader-writer 형태를 제공한다.	(7.2, p.327-330 / PDF p.270-273)
U07-050	단순 정수 갱신에는 원자적 연산이 효율적이지만 여러 변수가 관련된 경쟁에는 더 정교한 도구가 필요하다.	(7.2, p.327-330 / PDF p.270-273)
U07-051	mutex_lock()이 락을 얻지 못하면 수면하고 소유자가 반환할 때 깨어난다.	(7.2, p.327-330 / PDF p.270-273)
U07-052	단일 처리기	(7.2, p.327-330 / PDF p.270-273)
U07-053	커널 선점을 불가능하게 하고 끝나면 다시 가능하게 한다.	(7.2, p.327-330 / PDF p.270-273)
U07-054	다중 처리기	(7.2, p.327-330 / PDF p.270-273)
U07-055	스핀락을 획득하고 끝나면 방출한다.	(7.2, p.327-330 / PDF p.270-273)
U07-056	원문의 Linux 스핀락·mutex는 재귀적이지 않다.	(7.2, p.327-330 / PDF p.270-273)
U07-057	같은 스레드가 해제 없이 다시 획득하려 하면 봉쇄된다.	(7.2, p.327-330 / PDF p.270-273)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-058	각 스레드의 preempt_count로 소유 락 수를 추적하며 0보다 크면 커널 선점이 안전하지 않다.	(7.2, p.327-330 / PDF p.270-273)
U07-059	락이 오래 유지되면 스핀락보다 세마포·mutex를 사용하는 것이 적합하다고 설명한다.	(7.2, p.327-330 / PDF p.270-273)
U07-060	POSIX API는 특정 커널의 내부 기능과 달리 사용자 수준 프로그래머가 쓰는 인터페이스이다.	(7.3, p.330-334 / PDF p.273-277)
U07-061	pthread_mutex_t를 pthread_mutex_init()으로 초기화하고, 두 번째 인자로 NULL을 주면 기본 속성을 쓴다.	(7.3, p.330-334 / PDF p.273-277)
U07-062	pthread_mutex_lock()과 pthread_mutex_unlock()으로 임계구역 전후를 보호하며 사용 중인 락을 요청한 스레드는 반환까지 봉쇄된다.	(7.3, p.330-334 / PDF p.273-277)
U07-063	원문은 성공 시 0, 오류 시 0이 아닌 코드가 반환된다고 설명한다.	(7.3, p.330-334 / PDF p.273-277)
U07-064	세마포는 Pthreads 자체의 일부가 아닌 POSIX SEM 확장에 속하지만 많은 구현에서 함께 제공된다.	(7.3, p.330-334 / PDF p.273-277)
U07-065	기명 세마포는 sem_open()으로 생성·열고 관련 없는 프로세스도 이름을 이용해 같은 세마포를 공유한다.	(7.3, p.330-334 / PDF p.273-277)
U07-066	원문의 "SEM", O_CREAT, 0666, 1 예는 없을 때 생성, 다른 프로세스의 읽기·쓰기 권한, 초기값 1을 뜻한다.	(7.3, p.330-334 / PDF p.273-277)
U07-067	기명	(7.3, p.330-334 / PDF p.273-277)
U07-068	이름으로 공통 세마포에 접근한다. sem_open()은 기존 세마포의 디스크립터도 반환한다.	(7.3, p.330-334 / PDF p.273-277)
U07-069	무명	(7.3, p.330-334 / PDF p.273-277)
U07-070	sem_init()에 세마포 포인터·공유 플래그·초기값을 전달한다. 플래그 0은 같은 프로세스의 스레드 사이 공유, 0이 아니면 공유 메모리 영역에 두어 프로세스 사이 공유할 수 있다.	(7.3, p.330-334 / PDF p.273-277)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-071	두 유형 모두 sem_wait()로 획득하고 sem_post()로 반환한다.	(7.3, p.330-334 / PDF p.273-277)
U07-072	Pthread 조건 변수는 pthread_cond_t와 pthread_cond_init()으로 선언·초기화하고 mutex와 연결한다.	(7.3, p.330-334 / PDF p.273-277)
U07-073	C에는 언어 수준 모니터가 없으므로 별도 mutex가 조건 데이터의 무결성을 지킨다.	(7.3, p.330-334 / PDF p.273-277)

U07-074	Pthread 조건 변수로 a == b를 기다림 (원문 코드 발췌)
	pthread_mutex_lock(&mutex);
	while (a != b)
	pthread_cond_wait(&cond_var, &mutex);
	
	pthread_mutex_unlock(&mutex);
		(7.3, p.333 / PDF p.276)

U07-075	공유 데이터를 갱신하는 쪽은 mutex를 얻고 값을 바꾼 다음 pthread_cond_signal()로 대기자 하나에 신호하고 mutex를 해제한다.	(7.3, p.330-334 / PDF p.273-277)
U07-076	signal 호출 자체는 mutex를 해제하지 않는다.	(7.3, p.330-334 / PDF p.273-277)
U07-077	뒤의 pthread_mutex_unlock()으로 락이 풀리면 신호받은 스레드가 락 소유권을 얻고 wait 호출에서 실행을 재개한다.	(7.3, p.330-334 / PDF p.273-277)
U07-078	Java 객체마다 락이 연결된다.	(7.4, p.334-342 / PDF p.277-285)
U07-079	synchronized 메소드는 해당 객체 락을 얻은 스레드만 진입하고 종료하면 반환한다.	(7.4, p.334-342 / PDF p.277-285)
U07-080	다른 스레드는 객체의 진입 집합(entry set)에서 락이 풀리기를 기다린다.	(7.4, p.334-342 / PDF p.277-285)
U07-081	JVM 명세는 선택 순서를 강제하지 않는다고 원문은 설명한다.	(7.4, p.334-342 / PDF p.277-285)
U07-082	공유 데이터를 조작하는 부분만 synchronized 블록으로 감싸면 전체 메소드보다 락 범위(lock scope)를 줄일 수 있다.	(7.4, p.334-342 / PDF p.277-285)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-083	객체에는 별도로 대기 집합(wait set)도 있다.	(7.4, p.334-342 / PDF p.277-285)
U07-084	락을 가진 스레드가 조건을 만족하지 못하면 wait()를 호출하여 락을 해제하고 봉쇄 상태가 되어 대기 집합으로 간다.	(7.4, p.334-342 / PDF p.277-285)
U07-085	notify()는 대기자 하나를 선택하여 진입 집합으로 옮기고 실행 가능 상태로 바꾼다.	(7.4, p.334-342 / PDF p.277-285)
U07-086	이 스레드는 락을 다시 얻은 후에야 wait에서 돌아와 조건을 확인한다.	(7.4, p.334-342 / PDF p.277-285)
U07-087	대기자가 없으면 notify는 무시된다.	(7.4, p.334-342 / PDF p.277-285)
U07-088	생산자가 insert()에서 버퍼가 가득 찼음을 확인하고 wait()하여 락을 놓는다.	(7.4, p.334-342 / PDF p.277-285)
U07-089	소비자가 remove()에 진입해 항목을 제거하고 notify()한다. 이 순간에도 소비자가 락을 소유한다.	(7.4, p.334-342 / PDF p.277-285)
U07-090	생산자는 대기 집합에서 진입 집합으로 이동한다.	(7.4, p.334-342 / PDF p.277-285)
U07-091	소비자가 메소드를 종료하여 락을 반환한다. 생산자가 다시 획득하여 루프에서 빈 공간을 재확인하고 삽입한다.	(7.4, p.334-342 / PDF p.277-285)
U07-092	그림 7.12. 원문 관계도.	(7.4, p.337 / PDF p.280)
U07-093	ReentrantLock은 한 스레드가 소유하며 공정성 설정 등 추가 기능을 제공한다.	(7.4, p.334-342 / PDF p.277-285)
U07-094	호출자가 이미 소유한 락도 다시 획득할 수 있어 재진입이라고 부른다.	(7.4, p.334-342 / PDF p.277-285)
U07-095	사용할 수 없는 락은 unlock()으로 배정 가능해질 때까지 기다린다.	(7.4, p.334-342 / PDF p.277-285)
U07-096	읽기만 하는 스레드의 병행성을 위해 ReentrantReadWriteLock도 제공된다.	(7.4, p.334-342 / PDF p.277-285)

번호	문장·표 항목·원문 코드·수식	원문 위치

U07-097	Java ReentrantLock의 획득·반환 (원문 코드 발췌)
	Lock key = new ReentrantLock();
	
	key.lock();
	try {
	/* critical section */
	}
	finally {
	key.unlock();
	}
		(7.4, p.339 / PDF p.282)

U07-098	Java의 카운팅 세마포는 Semaphore(int value)로 초기화하며 원문은 음수 초기값도 허용한다고 설명한다.	(7.4, p.334-342 / PDF p.277-285)
U07-099	acquire()는 인터럽트되면 InterruptedException을 던질 수 있다.	(7.4, p.334-342 / PDF p.277-285)
U07-100	원문 예는 release()를 finally에 둔다.	(7.4, p.334-342 / PDF p.277-285)
U07-101	조건 변수는 ReentrantLock의 newCondition()에서 얻으며 await()·signal()을 사용한다.	(7.4, p.334-342 / PDF p.277-285)
U07-102	언어 수준 Java 모니터의 wait·notify는 객체의 무명 조건 변수 하나에 연결된다.	(7.4, p.334-342 / PDF p.277-285)
U07-103	명시적 Condition은 기다리는 조건을 나누어 특정 조건의 대기자에게 알릴 수 있다.	(7.4, p.334-342 / PDF p.277-285)
U07-104	원문의 5스레드 순번 예는 각 조건 변수 배열에서 자기 차례를 기다리고 작업 후 다음 turn에 신호한다.	(7.4, p.334-342 / PDF p.277-285)
U07-105	await는 연결된 재진입 락을 해제하지만 signal은 조건에만 알리며 실제 해제는 unlock()이 수행한다.	(7.4, p.334-342 / PDF p.277-285)
U07-106	트랜잭션 메모리의 메모리 트랜잭션은 메모리 읽기·쓰기 연산의 원자적인 순서이다.	(7.5, p.342-345 / PDF p.285-288)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-107	모든 연산이 완수되면 commit하고 그렇지 않으면 수행한 변경을 취소하여 시작 전으로 rollback한다.	(7.5, p.342-345 / PDF p.285-288)
U07-108	원문이 가정한 atomic { S } 구조에서는 개발자 대신 트랜잭션 시스템이 원자성을 보장한다.	(7.5, p.342-345 / PDF p.285-288)
U07-109	락을 쓰지 않는 이 방법에서는 락에 의한 교착 상태가 없고 병행 가능한 읽기를 시스템이 식별할 수 있다고 설명한다.	(7.5, p.342-345 / PDF p.285-288)
U07-110	STM	(7.5, p.342-345 / PDF p.285-288)
U07-111	특수 하드웨어 없이 컴파일러가 검사 코드를 넣어 동시 실행·저수준 락이 필요한 지점을 관리한다.	(7.5, p.342-345 / PDF p.285-288)
U07-112	HTM	(7.5, p.342-345 / PDF p.285-288)
U07-113	캐시 계층과 일관성 프로토콜로 충돌을 관리한다. 코드 계측 오버헤드가 작지만 하드웨어 구조의 변경이 필요하다.	(7.5, p.342-345 / PDF p.285-288)
U07-114	OpenMP는 컴파일러 지시와 API로 공유 메모리 병렬성을 제공한다. #pragma omp critical 뒤 코드 구역은 한순간 하나의 스레드만 실행한다.	(7.5, p.342-345 / PDF p.285-288)
U07-115	여러 임계구역에 이름을 주면 같은 이름을 가진 구역에서 동시에 두 스레드가 활동할 수 없다.	(7.5, p.342-345 / PDF p.285-288)
U07-116	표준 mutex보다 사용이 쉽지만 개발자가 경쟁 가능성을 찾아 보호해야 하며 여러 임계구역이 관여하면 교착 상태가 여전히 가능하다.	(7.5, p.342-345 / PDF p.285-288)
U07-117	함수형 프로그래밍 언어는 명령형 언어의 변경 가능한 상태와 다른 방식을 따른다.	(7.5, p.342-345 / PDF p.285-288)
U07-118	원문은 한 번 값이 배정된 변수는 바뀌지 않는다는 점 때문에 이 장의 공유 상태 경쟁·교착 문제를 피하는 방향으로 설명하고 Erlang·Scala를 언급한다.	(7.5, p.342-345 / PDF p.285-288)
U07-119	이 비교는 원문이 기술하는 범위에 한정하며 다른 언어 특성을 덧붙이지 않는다.	(7.5, p.342-345 / PDF p.285-288)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-120	세 고전 문제는 자원 수 제어·읽기와 쓰기의 병행성·여러 자원 동시 할당을 각각 드러낸다.	(7.6, p.345-347 / PDF p.288-290)
U07-121	커널 내부 동기화와 사용자 수준 POSIX·Java API를 구별하고, 락 소유권과 조건 대기의 이동을 함께 본다.	(7.6, p.345-347 / PDF p.288-290)
U07-122	원문 연습 문제는 운영체제가 여러 락을 제공하는 이유, 철학자 교착, dispatcher 상태, 원자적 정수 연산 등을 확인한다.	(7.6, p.345-347 / PDF p.288-290)
U07-123	외부 API 설명이나 새 계산 결과를 추가하지 않았다.	(7.6, p.345-347 / PDF p.288-290)
U07-124	유한 버퍼 문제에서는 n개의 버퍼로 구성된 풀을 가정하며 각 버퍼는 한 항목을 저장할 수 있다.	(7.1.1, p.320 / PDF p.263)
U07-125	Readers-Writers 문제는 writer가 쓰기 작업 동안 공유 데이터베이스에 대한 배타적 접근 권한을 갖도록 보장하는 동기화 문제이다.	(7.1.2, p.321 / PDF p.264)
U07-126	식사하는 철학자들 문제의 다섯 철학자는 생각과 식사를 반복하며, 식사하려면 자신에게 가장 가까운 두 젓가락을 모두 집어야 한다.	(7.1.3, p.323 / PDF p.266)
U07-127	POSIX의 기명 세마포는 관련 없는 여러 프로세스가 단순히 이름을 참조하여 같은 세마포에 액세스할 수 있다.	(7.6, p.345 / PDF p.288)
U07-128	POSIX의 무명 세마포를 공유하려면 공유 메모리 영역에 세마포를 배치해야 한다.	(7.6, p.345 / PDF p.288)
U07-129	메모리 트랜잭션은 메모리 읽기와 쓰기 연산의 원자적인 연속적 순서이다.	(7.5.1, p.342 / PDF p.285)
U07-130	명령형 혹은 절차형 언어는 상태에 기반을 둔 알고리즘을 구현하는 데 사용되며 상태는 변수와 다른 자료구조를 통해 표현된다.	(7.5.3, p.345 / PDF p.288)
U07-131	함수형 언어에서는 변수가 정의되어 값을 배정받으면 그 값은 변경될 수 없어 상태가 변하지 않는다.	(7.5.3, p.345 / PDF p.288)
U07-132	Windows는 dispatcher 객체와 event를 사용하여 프로세스 동기화 도구를 구현한다.	(7.6, p.345 / PDF p.288)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-133	Linux는 경쟁 조건을 방지하기 위해 원자적 변수, 스핀락 및 mutex 락을 포함한 다양한 접근 방식을 사용한다.	(7.6, p.345 / PDF p.288)
U07-134	기명 세마포	(7.6, p.345 / PDF p.288)
U07-135	무명 세마포	(7.6, p.345 / PDF p.288)
U07-136	유한 버퍼의 초기값: mutex = 1, empty = n, full = 0; n개의 버퍼가 있으며 각 버퍼는 한 항목을 저장한다.	(7.1.1, p.320 / PDF p.263)

제8장. 교착 상태

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-001	집합의 모든 프로세스가 같은 집합의 다른 프로세스에서만 발생할 수 있는 이벤트를 기다리는 경우 그 프로세스 집합에서 교착 상태가 발생한다.	(8.9, p.378 / PDF p.320)
U08-002	교착 상태는 상호 배제, 점유하며 대기, 비선점 및 순환 대기의 네 조건이 모두 존재할 때만 가능하다.	(8.9, p.378 / PDF p.320)
U08-003	안전 상태는 시스템이 안전 순서를 찾을 수 있어, 스레드들이 최대 요구 수를 요구하더라도 교착 상태 없이 차례로 모든 자원을 할당해 줄 수 있는 상태이다.	(8.6.1, p.365 / PDF p.307)
U08-004	자원은 유한하며 여러 유형으로 나뉜다.	(8.1, p.349-351 / PDF p.291-293)
U08-005	같은 유형의 인스턴스들은 서로 동등하여 그중 어느 것을 할당해도 요청을 충족해야 한다.	(8.1, p.349-351 / PDF p.291-293)
U08-006	CPU·파일·입출력 장치뿐 아니라 mutex·세마포도 자원이다.	(8.1, p.349-351 / PDF p.291-293)
U08-007	각 락은 보호하는 자료구조가 다르므로 일반적으로 고유한 자원 클래스로 취급한다.	(8.1, p.349-351 / PDF p.291-293)
U08-008	요청: 즉시 허용되지 않으면 자원을 얻을 때까지 기다린다.	(8.1, p.349-351 / PDF p.291-293)
U08-009	사용: 할당받은 자원을 이용하여 작업한다.	(8.1, p.349-351 / PDF p.291-293)
U08-010	방출: 사용을 마치면 반환한다.	(8.1, p.349-351 / PDF p.291-293)
U08-011	운영체제는 자원의 가용·할당 상태와 소유 스레드를 기록하고 사용할 수 없는 자원 요청자를 대기 큐에 둘 수 있다.	(8.1, p.349-351 / PDF p.291-293)
U08-012	교착 상태는 집합 안의 모든 스레드가 그 집합의 다른 스레드만 일으킬 수 있는 사건을 기다리는 상황이다.	(8.1, p.349-351 / PDF p.291-293)
U08-013	주로 자원 획득·방출을 다루지만 IPC 등 다른 사건도 원인이 될 수 있다.	(8.1, p.349-351 / PDF p.291-293)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-014	원문의 두 Pthread 스레드는 first_mutex와 second_mutex를 서로 반대 순서로 획득한다.	(8.2, p.351-354 / PDF p.293-296)
U08-015	한쪽이 first_mutex, 다른 쪽이 second_mutex를 소유한 후 상대 락을 요청하면 둘 다 기다린다.	(8.2, p.351-354 / PDF p.293-296)
U08-016	그러나 한 스레드가 두 락을 얻고 모두 반환한 다음 다른 스레드가 시작하면 교착은 없다.	(8.2, p.351-354 / PDF p.293-296)
U08-017	가능한 코드 구조와 실제 특정 실행에서의 발생을 구분해야 한다.	(8.2, p.351-354 / PDF p.293-296)
U08-018	특정 스케줄링에서만 드러나므로 검사도 어렵다.	(8.2, p.351-354 / PDF p.293-296)
U08-019	라이브락(livelock)은 스레드가 봉쇄되지는 않지만 실패하는 행동을 계속 재시도하여 진행하지 못하는 장애이다.	(8.2, p.351-354 / PDF p.293-296)
U08-020	원문은 trylock으로 두 번째 락을 얻지 못하면 자기 락을 반환하고 다시 시도하는 두 스레드를 예로 든다.	(8.2, p.351-354 / PDF p.293-296)
U08-021	서로 동시에 반복하면 계속 실패할 수 있다.	(8.2, p.351-354 / PDF p.293-296)
U08-022	원문은 일반적으로 재시도 시간을 무작위화하여 피하는 접근을 설명하고 Ethernet 충돌 후 재전송 대기와 연결한다.	(8.2, p.351-354 / PDF p.293-296)
U08-023	교착 상태	(8.2, p.351-354 / PDF p.293-296)
U08-024	집합 내 다른 스레드만 발생시킬 수 있는 사건을 기다리며 봉쇄된다.	(8.2, p.351-354 / PDF p.293-296)
U08-025	라이브락	(8.2, p.351-354 / PDF p.293-296)
U08-026	실패한 행동을 계속 재시도하여 봉쇄 없이도 앞으로 나아가지 못한다.	(8.2, p.351-354 / PDF p.293-296)
U08-027	상호 배제	(8.3, p.353-358 / PDF p.295-300)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-028	최소 한 자원이 비공유 모드로 점유되어 한 번에 한 스레드만 사용한다.	(8.3, p.353-358 / PDF p.295-300)
U08-029	점유하며 대기	(8.3, p.353-358 / PDF p.295-300)
U08-030	최소 한 자원을 소유하면서 다른 스레드 소유 자원을 추가로 기다린다.	(8.3, p.353-358 / PDF p.295-300)
U08-031	비선점	(8.3, p.353-358 / PDF p.295-300)
U08-032	자원은 강제로 회수할 수 없고 소유자가 작업을 마친 뒤 자발적으로 반환한다.	(8.3, p.353-358 / PDF p.295-300)
U08-033	순환 대기	(8.3, p.353-358 / PDF p.295-300)
U08-034	대기 스레드들의 각자가 다음 스레드가 점유한 자원을 기다리는 고리가 있다.	(8.3, p.353-358 / PDF p.295-300)
U08-035	교착 상태가 가능하려면 네 조건이 동시에 성립해야 한다.	(8.3, p.353-358 / PDF p.295-300)
U08-036	순환 대기는 점유하며 대기를 암시하므로 완전히 독립적인 조건은 아니지만 예방 방법을 설명하기 위해 나누어 본다.	(8.3, p.353-358 / PDF p.295-300)
U08-037	자원 할당 그래프는 스레드를 원, 자원 유형을 사각형, 각 인스턴스를 사각형 내부 점으로 표시한다.	(8.3, p.353-358 / PDF p.295-300)
U08-038	스레드 → 자원은 요청 간선, 인스턴스 → 스레드는 할당 간선이다.	(8.3, p.353-358 / PDF p.295-300)
U08-039	요청이 충족되면 요청 간선을 할당 간선으로 바꾸고 반환하면 할당 간선을 지운다.	(8.3, p.353-358 / PDF p.295-300)
U08-040	사이클 없음	(8.3, p.353-358 / PDF p.295-300)
U08-041	교착 상태가 없다.	(8.3, p.353-358 / PDF p.295-300)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-042	사이클의 자원 유형이 각각 단일 인스턴스	(8.3, p.353-358 / PDF p.295-300)
U08-043	사이클은 교착 상태의 필요충분조건이다.	(8.3, p.353-358 / PDF p.295-300)
U08-044	여러 인스턴스를 가진 자원 유형 포함	(8.3, p.353-358 / PDF p.295-300)
U08-045	사이클은 필요조건이지만 충분조건은 아니다. 다른 스레드의 반환으로 고리를 풀 수 있다.	(8.3, p.353-358 / PDF p.295-300)
U08-046	그림 8.5. 원문 그래프.	(8.3, p.357 / PDF p.299)
U08-047	그림 8.6. 원문 그래프.	(8.3, p.358 / PDF p.300)
U08-048	무시	(8.4, p.358-360 / PDF p.300-302)
U08-049	교착이 발생하지 않는 것으로 취급한다. 원문은 드문 발생에 비해 지속적인 처리 비용이 클 수 있고 수동 복구를 사용할 수 있다고 설명한다.	(8.4, p.358-360 / PDF p.300-302)
U08-050	예방·회피	(8.4, p.358-360 / PDF p.300-302)
U08-051	절대 교착 상태가 되지 않게 제약하거나 현재 할당을 허가할지 판단한다.	(8.4, p.358-360 / PDF p.300-302)
U08-052	탐지 후 회복	(8.4, p.358-360 / PDF p.300-302)
U08-053	교착을 허용하되 상태 조사 알고리즘과 복구 알고리즘을 둔다.	(8.4, p.358-360 / PDF p.300-302)
U08-054	예방은 네 필요조건 중 적어도 하나를 성립하지 않게 하도록 자원 요청 방법을 제한한다.	(8.4, p.358-360 / PDF p.300-302)
U08-055	회피는 스레드의 미래 최대 요구 등 추가 정보를 알고 현재 가용·할당 상태와 함께 요청 허가를 결정한다.	(8.4, p.358-360 / PDF p.300-302)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-056	탐지·회복이 없으면 인식하지 못한 교착이 점차 자원을 묶어 시스템 성능을 낮출 수 있다.	(8.4, p.358-360 / PDF p.300-302)
U08-057	자원 클래스별로 다른 기본 방법을 조합할 수도 있다.	(8.4, p.358-360 / PDF p.300-302)
U08-058	상호 배제	(8.5, p.360-364 / PDF p.302-306)
U08-059	가능한 자원을 공유 가능하게 한다.	(8.5, p.360-364 / PDF p.302-306)
U08-060	읽기 전용 파일과 달리 mutex 같은 자원은 본래 공유할 수 없어 일반적인 해결이 안 된다.	(8.5, p.360-364 / PDF p.302-306)
U08-061	점유하며 대기	(8.5, p.360-364 / PDF p.302-306)
U08-062	실행 전에 모두 요청하거나, 아무 자원도 없을 때만 새 요청을 허용한다.	(8.5, p.360-364 / PDF p.302-306)
U08-063	오래 사용하지 않는 자원을 점유하여 이용률이 낮고 필요한 자원을 한꺼번에 얻지 못하는 기아가 가능하다.	(8.5, p.360-364 / PDF p.302-306)
U08-064	비선점	(8.5, p.360-364 / PDF p.302-306)
U08-065	새 요청을 즉시 못 들어주면 이미 가진 자원을 회수하고 나중에 함께 재획득한다. 또는 대기 중인 다른 스레드에서 선점한다.	(8.5, p.360-364 / PDF p.302-306)
U08-066	상태 저장·복원이 쉬운 CPU 레지스터·트랜잭션 등에 적용되지만 mutex·세마포에는 일반적으로 적용되지 않는다.	(8.5, p.360-364 / PDF p.302-306)
U08-067	순환 대기	(8.5, p.360-364 / PDF p.302-306)
U08-068	모든 자원 유형에 전체 순서를 부여하고 오름차순 요청을 강제한다.	(8.5, p.360-364 / PDF p.302-306)
U08-069	순서 설정만으로는 부족하며 프로그램이 지켜야 한다. 동적 락 획득에도 주의한다.	(8.5, p.360-364 / PDF p.302-306)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-070	순환 대기 예방에서 함수 F는 각 자원 유형에 고유 자연수 번호를 준다.	(8.5, p.360-364 / PDF p.302-306)
U08-071	이미 R_i를 가진 스레드는 더 큰 번호의 자원만 요청한다.	(8.5, p.360-364 / PDF p.302-306)
U08-072	같은 유형의 인스턴스가 여러 개 필요하면 한 번의 요청으로 모두 받아야 한다.	(8.5, p.360-364 / PDF p.302-306)
U08-073	대안은 새 자원 이상 순서의 기존 자원을 모두 먼저 반환하게 하는 것이다.	(8.5, p.360-364 / PDF p.302-306)
U08-074	순환 고리가 있다면 순서가 계속 증가하여 결국 자기보다 작아져야 하는 모순이 생긴다는 것이 원문의 증명이다.	(8.5, p.360-364 / PDF p.302-306)
U08-075	원문의 계좌 이체 예는 from·to 인자에 따라 동적으로 락을 얻으면 두 스레드가 반대 계좌 순서로 호출하여 교착할 수 있음을 보인다.	(8.5, p.360-364 / PDF p.302-306)
U08-076	Linux lockdep은 실행 중 락 획득·반환 패턴과 인터럽트 활성 상태를 검사하여 잠재적 교착을 보고한다.	(8.5, p.360-364 / PDF p.302-306)
U08-077	원문은 개발·수정 검사 도구이며 속도 저하 때문에 실제 운영용으로 쓰지 않는다고 설명한다.	(8.5, p.360-364 / PDF p.302-306)
U08-078	외부 도구 사용법은 덧붙이지 않았다.	(8.5, p.360-364 / PDF p.302-306)
U08-079	예방의 요청 제한은 이용률·처리율을 낮출 수 있다.	(8.6, p.363-371 / PDF p.305-313)
U08-080	회피는 각 스레드의 최대 요구 수 같은 추가 정보를 받아 요청 후의 할당 상태를 판단한다.	(8.6, p.363-371 / PDF p.305-313)
U08-081	상태는 가용 자원 수·현재 할당 수·최대 요구 수로 표현한다.	(8.6, p.363-371 / PDF p.305-313)
U08-082	현재 여유 자원이 충분하다는 사실만으로 요청을 허용하지 않는다.	(8.6, p.363-371 / PDF p.305-313)
U08-083	안전 상태는 모든 스레드가 최대 요구량을 요청해도 어떤 순서로 하나씩 끝낼 수 있는 안전 순서가 있는 상태이다.	(8.6.1, p.365-366 / PDF p.307-308)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-084	순서의 각 스레드에 필요한 추가 자원은 현재 가용량과 앞서 끝난 스레드가 반납할 자원으로 충족해야 한다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-085	즉시 만족할 수 없다면 앞선 스레드의 종료까지 기다리게 한다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-086	안전 상태는 교착 상태가 아니다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-087	교착 상태는 불안전하지만 불안전 상태가 반드시 교착 상태인 것은 아니다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-088	이후 실제 요청에 따라 교착할 수 있다는 뜻이다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-089	원문의 12개 자원 예는 최대 요구 10·4·9, 현재 점유 5·2·2, 가용 3에서 $T_1 \rightarrow T_0 \rightarrow T_2$로 모두 끝낼 수 있다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-090	여기서 T_2에 하나 더 주면 남은 요청을 모두 처리할 안전 순서가 없어 불안전해진다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-091	회피는 항상 안전 상태 안에 머무르게 하므로 가용 자원이 있어도 대기시킬 수 있다.	(8.6.1, p.365-366 / PDF p.307-308)
U08-092	각 자원 유형에 인스턴스가 하나씩인 경우 미래에 요청할 수 있음을 예약 간선(claim edge)이라는 점선으로 표시한다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-093	실행 전 모든 예약을 표시하며 실제 요청 시 요청 간선으로, 반환 시 할당 간선을 예약 간선으로 되돌린다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-094	모든 관련 간선이 예약인 경우에만 새 예약을 추가하도록 조건을 완화할 수 있다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-095	요청 간선을 할당 간선으로 바꾸어도 사이클이 생기지 않는 경우에만 허용한다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-096	생기면 불안전해지므로 자원이 현재 가용해도 기다리게 한다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-097	원문은 사이클 검사에 n^2 차수의 연산이 필요하다고 설명한다.	(8.6.2, p.366-367 / PDF p.308-309)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-098	여러 인스턴스의 경우는 다음 은행원 알고리즘으로 다룬다.	(8.6.2, p.366-367 / PDF p.308-309)
U08-099	스레드는 시작할 때 자원 유형별 최대 개수를 미리 신고하며 총 보유량을 넘을 수 없다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-100	시스템은 요청을 들어준 뒤에도 안전한지 검사한다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-101	아래에서 n은 스레드 수, m은 자원 유형 수이며 벡터 비교는 모든 성분별 비교이다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-102	Available	(8.6.3, p.368-371 / PDF p.310-313)
U08-103	m 벡터. 유형별 현재 가용 개수.	(8.6.3, p.368-371 / PDF p.310-313)
U08-104	Max	(8.6.3, p.368-371 / PDF p.310-313)
U08-105	n×m 행렬. 스레드별 유형별 최대 요구.	(8.6.3, p.368-371 / PDF p.310-313)
U08-106	Allocation	(8.6.3, p.368-371 / PDF p.310-313)
U08-107	n×m 행렬. 현재 할당된 개수.	(8.6.3, p.368-371 / PDF p.310-313)
U08-108	Need	(8.6.3, p.368-371 / PDF p.310-313)
U08-109	n×m 행렬. 앞으로 추가 요청할 수 있는 개수. Need[i][j] = Max[i][j] − Allocation[i][j].	(8.6.3, p.368-371 / PDF p.310-313)
U08-110	8.6.3.1 원문 알고리즘.	(8.6.3, p.369 / PDF p.311)
U08-111	$\text{Request}_i \le \text{Need}_i$인지 검사한다. 아니면 오류로 처리한다.	(8.6.3, p.368-371 / PDF p.310-313)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-112	$Request_i \le Available$인지 검사한다. 아니면 당장 자원이 없어 대기한다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-113	가상으로 할당해 Available을 줄이고 $Allocation_i$를 늘리며 $Need_i$를 줄인다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-114	새 상태가 안전하면 확정하고, 불안전하면 이전 상태로 복원하고 기다리게 한다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-115	8.6.3.3의 최초 Allocation·Max·Available 표.	(8.6.3, p.370 / PDF p.312)
U08-116	원문 최초 Need는 T_0=(7,4,3), T_1=(1,2,2), T_2=(6,0,0), T_3=(0,1,1), T_4=(4,3,1)이다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-117	안전 순서 $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2 \rightarrow T_0$가 있다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-118	T_1의 (1,0,2) 요청을 가상 할당하면 Available=(2,3,0)이 되고 $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_0 \rightarrow T_2$가 안전하므로 즉시 허용한다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-119	이어 T_4의 (3,3,0)은 자원 부족, T_0의 (0,2,0)은 자원이 있어도 불안전하므로 허용하지 않는 것이 원문의 결과이다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-120	새 순서나 계산을 추가하지 않았다.	(8.6.3, p.368-371 / PDF p.310-313)
U08-121	예방·회피를 사용하지 않는 시스템은 교착 발생 여부를 검사하고 발생하면 회복할 방법이 필요하다.	(8.7, p.371-375 / PDF p.313-317)
U08-122	실행 상태 정보 유지·탐지 알고리즘 실행·복구 손실이 모두 오버헤드이다.	(8.7, p.371-375 / PDF p.313-317)
U08-123	각 자원 유형이 하나씩이면 자원 할당 그래프에서 자원 노드를 제거하여 대기 그래프(wait-for graph)를 만든다.	(8.7, p.371-375 / PDF p.313-317)
U08-124	$T_i \rightarrow T_j$는 T_i가 T_j 소유 자원의 반환을 기다림을 뜻한다.	(8.7, p.371-375 / PDF p.313-317)
U08-125	이 경우 사이클이 있는 것과 교착이 동치이다.	(8.7, p.371-375 / PDF p.313-317)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-126	원문은 그래프 사이클 탐지에 $O(n^2)$ 연산이 필요하다고 설명한다.	(8.7, p.371-375 / PDF p.313-317)
U08-127	여러 인스턴스에서는 Available·Allocation과 현재 요청 행렬 Request를 사용한다.	(8.7, p.371-375 / PDF p.313-317)
U08-128	회피의 Need가 미래 최대 추가 요구인 것과 달리 Request는 지금 기다리는 요청이다.	(8.7, p.371-375 / PDF p.313-317)
U08-129	Work=Available로 시작하고 Allocation이 0인 스레드는 Finish=true, 나머지는 false로 둔다.	(8.7, p.371-375 / PDF p.313-317)
U08-130	미완료이면서 $Request_i \le Work$인 스레드를 찾아 $Allocation_i$를 돌려받는다고 가정한다.	(8.7, p.371-375 / PDF p.313-317)
U08-131	더 찾지 못했을 때 Finish=false인 스레드들이 교착에 연루된다.	(8.7, p.371-375 / PDF p.313-317)
U08-132	원문은 $m \times n^2$ 연산을 명시한다.	(8.7, p.371-375 / PDF p.313-317)
U08-133	8.7.2 원문의 4단계 알고리즘.	(8.7, p.373 / PDF p.315)
U08-134	원문의 예에서 A=7, B=2, C=6이고 Available=(0,0,0)이어도 $T_0 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1 \rightarrow T_4$로 끝낼 수 있어 교착이 아니다.	(8.7, p.371-375 / PDF p.313-317)
U08-135	T_2가 C 하나를 더 요청하면 T_0 자원을 회수해도 나머지 요청을 충족하지 못하여 T_1·T_2·T_3·T_4가 교착에 연루된다.	(8.7, p.371-375 / PDF p.313-317)
U08-136	탐지 빈도는 교착 발생 빈도와 보통 연루되는 스레드 수에 달린다.	(8.7, p.371-375 / PDF p.313-317)
U08-137	즉시 만족되지 않는 요청마다 검사하면 원인 시점을 찾기 쉽지만 비용이 크다.	(8.7, p.371-375 / PDF p.313-317)
U08-138	일정 간격이나 CPU 이용률 하락 때만 검사하면 비용은 줄어도 원인 스레드를 특정하기 어려워진다.	(8.7, p.371-375 / PDF p.313-317)
U08-139	원문의 BCC 도구·Java 스레드 덤프·데이터베이스 예는 모두 대기 관계를 관찰하여 사이클을 찾는 의미로 연결된다.	(8.7, p.371-375 / PDF p.313-317)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-140	데이터베이스는 희생 트랜잭션을 중단·rollback하여 락을 풀고 나머지를 재개한다.	(8.7, p.371-375 / PDF p.313-317)
U08-141	탐지 후 운영자에게 알려 수동 처리하거나 시스템이 자동으로 회복할 수 있다.	(8.8, p.376-377 / PDF p.318-319)
U08-142	자동 방법은 한 개 이상의 프로세스·스레드를 종료하여 순환 대기를 깨거나 점유 자원을 선점하는 것이다.	(8.8, p.376-377 / PDF p.318-319)
U08-143	모든 교착 프로세스를 한꺼번에 중지하면 확실하지만 이미 수행한 계산을 잃는다.	(8.8, p.376-377 / PDF p.318-319)
U08-144	한 개씩 중지하면 매번 재탐지하는 비용이 든다.	(8.8, p.376-377 / PDF p.318-319)
U08-145	종료 대상은 우선순위, 지금까지와 남은 실행 시간, 점유한 자원의 종류·수, 앞으로 필요한 자원, 중지해야 할 프로세스 수 등을 고려한다.	(8.8, p.376-377 / PDF p.318-319)
U08-146	파일 갱신 중 종료하면 파일이 부정확해질 수 있고 락을 풀어도 공유 데이터 무결성이 보장되는 것은 아니다.	(8.8, p.376-377 / PDF p.318-319)
U08-147	희생자 선택	(8.8, p.376-377 / PDF p.318-319)
U08-148	점유 자원·실행 시간 등의 비용을 최소화하는 순서를 선택한다.	(8.8, p.376-377 / PDF p.318-319)
U08-149	후퇴(rollback)	(8.8, p.376-377 / PDF p.318-319)
U08-150	자원이 없어진 프로세스는 정상 진행할 수 없어 안전한 상태로 되돌린다. 완전 재시작은 간단하지만 필요한 만큼만 후퇴하려면 더 많은 상태 정보를 유지해야 한다.	(8.8, p.376-377 / PDF p.318-319)
U08-151	기아 방지	(8.8, p.376-377 / PDF p.318-319)
U08-152	같은 프로세스가 계속 희생되지 않도록 한정된 횟수만 선택되게 해야 한다. 비용에 후퇴 횟수를 포함하는 것이 일반적인 해법이다.	(8.8, p.376-377 / PDF p.318-319)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-153	교착 상태는 네 필요조건·요청과 할당 관계·현재와 미래 요구량을 연결하여 판단한다.	(8.9, p.378-381 / PDF p.320-323)
U08-154	사이클의 의미는 자원 인스턴스 수에 따라 다르며 안전·불안전·교착을 구별한다.	(8.9, p.378-381 / PDF p.320-323)
U08-155	원문 연습 문제는 안전 상태, 은행원 알고리즘, 봉쇄와 자원 순서, 탐지·기아 및 자원 선점 정책을 점검한다.	(8.9, p.378-381 / PDF p.320-323)
U08-156	제공된 문제의 새 풀이를 작성하지 않았다.	(8.9, p.378-381 / PDF p.320-323)
U08-157	모든 스레드를 무사히 마칠 수 있는 순서를 찾을 수 없으면 불안전 상태이며, 불안전 상태가 반드시 교착 상태를 뜻하지는 않는다.	(8.6.1, p.365 / PDF p.307)
U08-158	안전 순서에서는 각 스레드의 요청을 현재 남은 자원과 앞서 수행을 마친 스레드가 반환한 자원으로 만족시킬 수 있다.	(8.6.1, p.365 / PDF p.307)
U08-159	은행원 알고리즘은 시스템을 불안전 상태로 이끌 수 있는 자원 요청을 허가하지 않아 교착 상태를 회피한다.	(8.9, p.378 / PDF p.320)
U08-160	교착 상태에 필요한 네 조건 중 하나가 발생하지 않도록 하여 교착 상태를 예방할 수 있다.	(8.9, p.378 / PDF p.320)
U08-161	교착 상태 탐지 알고리즘은 실행 중인 시스템의 프로세스와 자원을 평가하여 프로세스들이 교착 상태에 있는지 결정한다.	(8.9, p.378 / PDF p.320)
U08-162	Need는 각 스레드가 향후 요청할 수 있는 자원 개수를 나타내는 n×m 행렬이다.	(8.6.3, p.368 / PDF p.310)
U08-163	교착 상태 발생 후에는 순환 대기 프로세스 하나를 중단하거나 교착 프로세스에 지정된 자원을 선점하여 복구를 시도할 수 있다.	(8.9, p.378 / PDF p.320)
U08-164	예방	(8.9, p.378 / PDF p.320)
U08-165	회피: 은행원 알고리즘	(8.9, p.378 / PDF p.320)
U08-166	추가 요청 가능 자원: Need[i][j] = Max[i][j] − Allocation[i][j]; Max는 최대 요구, Allocation은 현재 할당 자원의 개수이다.	(8.6.3, p.368 / PDF p.310)

제9장. 메인 메모리

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-001	메모리 관리 장치(MMU)는 논리 주소를 메모리의 물리 주소로 변환한다.	(9.8, p.424 / PDF p.364)
U09-002	페이징의 논리 주소는 페이지 번호와 페이지 오프셋으로 나뉘며, 페이지 번호는 프로세스별 페이지 테이블의 인덱스이다.	(9.8, p.424 / PDF p.364)
U09-003	다중 프로그래밍으로 CPU 이용률·응답성을 높이려면 여러 프로세스를 메모리에 유지해야 한다.	(9.1, p.385-393 / PDF p.325-333)
U09-004	메모리는 고유 주소를 가진 바이트들의 배열이며 CPU는 명령 인출·해독·피연산자 인출·실행·결과 저장을 수행한다.	(9.1, p.385-393 / PDF p.325-333)
U09-005	메모리는 제시된 주소에 따라 읽고 쓸 뿐 그 주소가 생성된 방식이나 내용의 의미를 알지 못한다.	(9.1, p.385-393 / PDF p.325-333)
U09-006	메모리 관리 방식은 하드웨어 설계와 긴밀하게 연결된다.	(9.1, p.385-393 / PDF p.325-333)
U09-007	CPU가 직접 접근하는 범용 저장장치는 메인 메모리와 코어 레지스터이다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-008	명령·데이터가 디스크에 있으면 처리 전 메모리로 옮겨야 한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-009	레지스터와 메인 메모리의 접근 속도 차이 때문에 CPU가 데이터 대기 중 stall될 수 있으며 캐시가 이를 완화한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-010	동시에 운영체제와 다른 사용자 프로세스의 공간을 보호해야 한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-011	각 프로세스에 독립된 합법 주소 영역을 정하고 사용자 모드의 모든 주소를 하드웨어가 검사한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-012	원문의 기준(base) 레지스터는 가장 작은 합법 물리 주소, 상한(limit) 레지스터는 영역 크기를 저장한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-013	기준 이상이며 기준+상한보다 작은 범위인지 검사하는 그림 9.2의 관계가 보호 기준이다.	(9.1.1, p.386-388 / PDF p.326-328)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-014	범위를 벗어나면 운영체제에 트랩한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-015	두 레지스터는 커널 모드에서만 가능한 특권 명령으로 운영체제만 적재한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-016	커널은 사용자 영역과 운영체제 영역을 모두 접근할 수 있어 프로그램 적재, 오류 덤프, 시스템 콜 인수 처리, 문맥 교환 등을 수행한다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-017	보호 검사를 소프트웨어 호출마다 수행하는 것으로 해석하지 않는다.	(9.1.1, p.386-388 / PDF p.326-328)
U09-018	프로그램은 실행 파일에서 메모리로 적재되어 프로세스 문맥에 배치된다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-019	주소는 원시 프로그램의 심볼에서 컴파일러가 만드는 재배치 가능 주소, 링커·로더가 만드는 절대 주소로 바뀔 수 있다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-020	각 바인딩은 한 주소 공간을 다른 주소 공간에 사상한다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-021	컴파일 시간	(9.1.2, p.388-390 / PDF p.328-330)
U09-022	적재 위치를 미리 알면 절대 코드를 만든다. 시작 위치를 변경하려면 다시 컴파일한다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-023	적재 시간	(9.1.2, p.388-390 / PDF p.328-330)
U09-024	컴파일 때 위치를 모르면 재배치 가능 코드를 만든다. 실제 적재 시 바인딩하며 시작 위치가 바뀌면 다시 적재한다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-025	실행 시간	(9.1.2, p.388-390 / PDF p.328-330)
U09-026	실행 중 다른 메모리 위치로 이동할 수 있도록 바인딩을 실행까지 미룬다. 특별한 하드웨어 지원이 필요하다.	(9.1.2, p.388-390 / PDF p.328-330)
U09-027	논리 주소는 CPU가 생성하는 주소이고 물리 주소는 메모리 주소 레지스터에 제시되어 메모리가 취급하는 주소이다.	(9.1.3, p.390-391 / PDF p.330-331)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-028	컴파일·적재 시간 바인딩에서는 두 주소가 같지만 실행 시간 바인딩에서는 다르다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-029	이 책은 논리 주소와 가상 주소를 같은 뜻으로 사용한다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-030	메모리 관리 장치(MMU)는 실행 중 논리 주소를 물리 주소로 바꾼다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-031	단순한 동적 재배치에서는 재배치 레지스터 값을 모든 논리 주소에 더한다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-032	원문의 값 14000과 논리 주소 346은 물리 주소 14346으로 연결된다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-033	사용자 프로그램은 논리 주소를 생성·저장·비교할 수 있지만 실제 메모리 참조 때 하드웨어가 바인딩하므로 실제 물리 위치를 직접 접근하는 것으로 이해하면 안 된다.	(9.1.3, p.390-391 / PDF p.330-331)
U09-034	동적 적재(dynamic loading)는 루틴을 실제로 호출할 때까지 메모리에 올리지 않는다.	(9.1.4, p.391-392 / PDF p.331-332)
U09-035	루틴은 디스크에 재배치 가능 형태로 대기하고 main이 실행되다가 다른 루틴을 부르면 이미 적재됐는지 조사한다.	(9.1.4, p.391-392 / PDF p.331-332)
U09-036	없으면 연결 적재기가 가져오고 테이블을 갱신한 뒤 제어를 넘긴다.	(9.1.4, p.391-392 / PDF p.331-332)
U09-037	오류 처리처럼 실행 빈도가 낮고 큰 루틴에 유리하다.	(9.1.4, p.391-392 / PDF p.331-332)
U09-038	원문은 운영체제의 특별한 지원 없이 프로그램 설계로 가능하고 구현 라이브러리를 제공받을 수 있다고 설명한다.	(9.1.4, p.391-392 / PDF p.331-332)
U09-039	동적 연결은 적재 자체를 미루는 동적 적재와 달리 라이브러리 연결을 실행 시점까지 미룬다.	(9.1.5, p.392-393 / PDF p.332-333)
U09-040	정적 연결은 참조 라이브러리를 각 실행 이미지에 포함하지만 DLL은 실행할 때 찾아 주소를 조정한다.	(9.1.5, p.392-393 / PDF p.332-333)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-041	여러 프로세스가 하나의 물리 사본을 공유할 수 있어 실행 이미지·메모리 공간을 줄인다.	(9.1.5, p.392-393 / PDF p.332-333)
U09-042	라이브러리 갱신 뒤 새 버전을 쓰려면 정적 연결은 재연결이 필요하지만 동적 연결은 새 버전을 사용할 수 있다.	(9.1.5, p.392-393 / PDF p.332-333)
U09-043	단, 여러 버전이 공존할 수 있으므로 프로그램·라이브러리에 버전 정보를 두어 원하는 버전을 구별한다.	(9.1.5, p.392-393 / PDF p.332-333)
U09-044	동적 연결·공유에는 일반적으로 운영체제 지원이 필요하며 각자의 공간이 보호된 프로세스들이 같은 메모리를 이용할 수 있게 해야 한다.	(9.1.5, p.392-393 / PDF p.332-333)
U09-045	메모리는 운영체제 영역과 사용자 영역으로 나뉘며 연속 할당은 각 프로세스에 하나의 연속된 영역을 준다.	(9.2, p.393-397 / PDF p.333-337)
U09-046	재배치 레지스터는 가장 작은 물리 주소, 상한 레지스터는 논리 주소 범위를 유지한다.	(9.2, p.393-397 / PDF p.333-337)
U09-047	MMU는 논리 주소가 범위 안인지 확인하고 재배치 값을 더한다.	(9.2, p.393-397 / PDF p.333-337)
U09-048	디스패처는 문맥 교환 때 두 값을 적재한다.	(9.2, p.393-397 / PDF p.333-337)
U09-049	가변 크기 파티션은 각각 프로세스 하나를 담는다.	(9.2, p.393-397 / PDF p.333-337)
U09-050	운영체제는 사용 중·가용 영역을 기록한다.	(9.2, p.393-397 / PDF p.333-337)
U09-051	프로세스 종료로 생긴 hole을 다시 할당하며 요청보다 크면 분할하고 반환 영역이 인접 hole과 붙으면 병합한다.	(9.2, p.393-397 / PDF p.333-337)
U09-052	공간이 부족하면 거부하거나 대기 큐에 둘 수 있다.	(9.2, p.393-397 / PDF p.333-337)
U09-053	최초 적합	(9.2, p.393-397 / PDF p.333-337)
U09-054	충분히 큰 첫 공간을 고른다. 시작 또는 지난 검색 위치부터 찾고 발견하면 끝낸다.	(9.2, p.393-397 / PDF p.333-337)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-055	최적 적합	(9.2, p.393-397 / PDF p.333-337)
U09-056	요청을 만족하는 가장 작은 공간을 고른다. 크기순이 아니면 전체를 검색하며 작은 나머지를 남긴다.	(9.2, p.393-397 / PDF p.333-337)
U09-057	최악 적합	(9.2, p.393-397 / PDF p.333-337)
U09-058	가장 큰 공간을 고른다. 남은 공간도 다른 용도로 충분히 클 것을 기대한다. 크기순이 아니면 전체 검색이 필요하다.	(9.2, p.393-397 / PDF p.333-337)
U09-059	원문 모의실험 설명에서는 최초·최적 적합이 시간·공간 효율에서 최악 적합보다 좋다.	(9.2, p.393-397 / PDF p.333-337)
U09-060	최초와 최적 중 공간 효율이 항상 우세한 것은 없지만 최초가 일반적으로 빠르다.	(9.2, p.393-397 / PDF p.333-337)
U09-061	이는 이름의 최적을 모든 조건의 최적 성능이라고 해석하지 않아야 하는 이유이다.	(9.2, p.393-397 / PDF p.333-337)
U09-062	외부 단편화	(9.2, p.393-397 / PDF p.333-337)
U09-063	가용 공간 총합은 충분해도 작은 비연속 조각으로 나뉘어 요청을 만족하지 못한다.	(9.2, p.393-397 / PDF p.333-337)
U09-064	내부 단편화	(9.2, p.393-397 / PDF p.333-337)
U09-065	할당 단위 때문에 실제 요청보다 큰 영역을 주어 할당 내부의 일부를 쓰지 못한다.	(9.2, p.393-397 / PDF p.333-337)
U09-066	원문은 최초 적합에서 N개 할당 블록에 대해 0.5N개 블록이 단편화로 손실될 수 있는 50% 규칙을 설명한다.	(9.2, p.393-397 / PDF p.333-337)
U09-067	외부 단편화의 압축(compaction)은 사용 내용을 한쪽으로 모아 큰 가용 영역을 만든다.	(9.2, p.393-397 / PDF p.333-337)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-068	실행 시간의 동적 재배치가 있어야 가능하며 정적 바인딩에서는 할 수 없고 이동 비용도 크다.	(9.2, p.393-397 / PDF p.333-337)
U09-069	논리 공간을 비연속 물리 영역에 놓는 페이징은 다른 해결 방향이다.	(9.2, p.393-397 / PDF p.333-337)
U09-070	프레임은 물리 메모리의 고정 크기 블록, 페이지는 논리 메모리의 같은 크기 블록이다.	(9.3, p.397-408 / PDF p.337-348)
U09-071	페이지를 가용 프레임에 놓으면 프로세스 물리 공간이 연속일 필요가 없어 외부 단편화와 그 압축을 피한다.	(9.3, p.397-408 / PDF p.337-348)
U09-072	논리 주소 공간 크기와 물리 메모리 용량을 분리하며 운영체제·하드웨어가 협력하여 구현한다.	(9.3, p.397-408 / PDF p.337-348)
U09-073	논리 주소는 페이지 번호 p와 페이지 오프셋 d로 나뉜다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-074	페이지 번호가 프로세스별 페이지 테이블의 인덱스이며 테이블에서 찾은 프레임 번호 f와 원래 오프셋으로 물리 주소를 만든다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-075	오프셋은 변하지 않는다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-076	논리 주소에서 p를 추출한다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-077	페이지 테이블에서 p에 대응하는 f를 얻는다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-078	p를 f로 바꾸고 d는 유지하여 물리 주소를 만든다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-079	원문 수식: 논리 주소 공간 2^m, 페이지 크기 2^n 바이트 → 상위 m−n비트: p / 하위 n비트: d; m: 논리 주소의 비트 수; n: 페이지 안 오프셋의 비트 수; 페이지 크기는 하드웨어가 정하며 2의 거듭제곱이므로 비트로 나누어 변환할 수 있다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-080	그림 9.8. 원문 그림.	(9.3.1, p.398 / PDF p.338)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-081	원문의 4 B 페이지·32 B 물리 메모리 예에서 페이지 테이블은 페이지 0→프레임 5, 1→6, 2→1, 3→2이다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-082	논리 주소 0은 20, 3은 23, 4는 24, 13은 9로 사상된다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-083	이들은 원문에 명시된 결과이다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-084	프레임 단위로 할당하므로 외부 단편화는 없지만 마지막 페이지의 내부 단편화는 남는다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-085	원문 예에서 2,048 B 페이지와 72,766 B 요구는 마지막 페이지에 1,086 B를 사용하여 962 B를 낭비한다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-086	크기와 페이지 크기가 무관하면 평균적으로 반 페이지의 내부 단편화가 예상된다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-087	작은 페이지는 이를 줄이지만 페이지 테이블을 키우며 큰 페이지는 디스크 효율에 유리하다는 절충이 있다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-088	운영체제는 가용 프레임을 배정하고 페이지를 적재하여 테이블을 채운다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-089	프로그래머에게는 자기만의 연속 공간처럼 보이지만 실제 프로그램은 여러 프레임에 분산되고 다른 프로세스와 공존한다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-090	시스템의 프레임 테이블은 프레임별 가용 여부·소유 프로세스·페이지를 기록한다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-091	운영체제는 사용자 인수 주소를 변환하고 문맥 교환 때 테이블을 설정해야 하므로 페이징은 문맥 교환 비용도 늘린다.	(9.3.1, p.397-402 / PDF p.337-342)
U09-092	작은 페이지 테이블은 전용 레지스터로 구현할 수 있지만 각 문맥 교환에서 여러 값을 바꿔야 한다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-093	큰 테이블은 메인 메모리에 두고 페이지 테이블 기준 레지스터(PTBR)가 가리키게 하면 프로세스 변경 때 PTBR만 바꾸면 된다.	(9.3.2, p.402-406 / PDF p.342-346)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-094	그러나 데이터를 얻기 위해 테이블과 실제 데이터에 두 번 접근하는 지연이 생긴다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-095	TLB(translation look-aside buffer)는 페이지 테이블 일부를 보관하는 매우 빠른 연관 메모리이다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-096	페이지 번호를 여러 키와 동시에 비교하여 프레임 번호를 얻는다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-097	맞으면 즉시 변환하고, 없으면 테이블을 읽어 변환한 뒤 TLB에 추가한다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-098	가득 차면 LRU·라운드 로빈·무작위 등 정책으로 바꾸며 중요한 커널 항목을 고정할 수 있다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-099	각 TLB 항목의 ASID(address-space identifier)는 어느 프로세스의 주소 공간인지 식별한다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-100	현재 프로세스와 다르면 미스로 처리하여 다른 프로세스 정보가 잘못 사용되는 것을 막는다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-101	ASID가 있으면 여러 프로세스 항목을 함께 보관하고, 없으면 새 테이블 선택 때 TLB를 flush해야 한다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-102	적중률(hit ratio)은 원하는 페이지 번호가 TLB에서 발견되는 비율이다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-103	원문의 계산식은 메모리 접근 10 ns, 테이블 미스의 두 번 접근 20 ns를 사용하여 80% 적중 때 12 ns, 99% 때 10.1 ns를 얻는다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-104	단일 테이블 접근을 가정한 예이며 여러 TLB 계층·여러 테이블 접근이면 추가 정보가 필요하다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-105	인쇄405쪽의 첫 계산식.	(9.3.2, p.405 / PDF p.345)
U09-106	인쇄405쪽의 두 번째 계산식.	(9.3.2, p.405 / PDF p.345)
U09-107	원문 확인 필요: 같은 쪽의 앞 설명 문장에는 TLB 적중 시 "100 ns"가 기재되어 있으나 뒤의 식과 비교 설명은 "10 ns"를 사용한다.	(9.3.2, p.402-406 / PDF p.342-346)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-108	이를 조용히 수정하지 않고 원문 내부 표기 불일치로 남긴다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-109	다단계 TLB의 일반 효과만 취하며 다른 수치를 새로 계산하지 않았다.	(9.3.2, p.402-406 / PDF p.342-346)
U09-110	페이지 테이블의 보호 비트는 읽기·쓰기 또는 읽기 전용 같은 접근 권한을 지정한다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-111	주소 변환과 함께 검사하여 읽기 전용에 쓰면 보호 위반 트랩을 낸다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-112	유효/무효 비트는 그 페이지가 프로세스의 합법 논리 주소 공간에 속하는지 나타내며 접근 허용 여부를 결정한다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-113	페이지 단위 보호는 마지막 페이지 안에서 실제 사용 범위보다 넓은 주소를 유효하게 둘 수 있다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-114	원문의 14비트 주소·2 KB 페이지 예에서 프로그램은 10,468까지 쓰지만 페이지 5 전체가 유효하여 12,287까지 유효로 취급된다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-115	이는 내부 단편화와 연결된다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-116	테이블 크기를 나타내는 PTLR(page table length register)로 유효 테이블 범위를 확인하는 방법도 제시된다.	(9.3.3, p.406-407 / PDF p.346-347)
U09-117	공통 코드는 재진입 코드, 즉 실행 중 자체 수정이 없는 코드이면 여러 프로세스가 동시에 공유할 수 있다.	(9.3.4, p.407-408 / PDF p.347-348)
U09-118	각 프로세스는 레지스터·데이터 저장영역을 따로 가지며 같은 물리 코드 사본을 각자의 테이블에 매핑한다.	(9.3.4, p.407-408 / PDF p.347-348)
U09-119	원문의 libc=2 MB·40프로세스 예는 각각 복사할 때 80 MB 대신 공유 사본 2 MB가 필요하다고 비교한다.	(9.3.4, p.407-408 / PDF p.347-348)
U09-120	읽기 전용 속성은 코드 정확성에만 맡기지 않고 운영체제가 강제해야 한다.	(9.3.4, p.407-408 / PDF p.347-348)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-121	공유 라이브러리뿐 아니라 프로세스 간 공유 메모리 통신도 같은 프레임에 여러 테이블을 연결하는 방식으로 구현될 수 있다.	(9.3.4, p.407-408 / PDF p.347-348)
U09-122	프로세스 간 공유가 각자의 데이터까지 모두 같은 것으로 만든다는 뜻은 아니다.	(9.3.4, p.407-408 / PDF p.347-348)
U09-123	주소 공간이 커지면 페이지 테이블 자체의 물리 메모리 비용이 커진다.	(9.4, p.409-414 / PDF p.349-354)
U09-124	원문은 계층적·해시·역 페이지 테이블과 SPARC Solaris 사례로 테이블 저장·탐색의 절충을 설명한다.	(9.4, p.409-414 / PDF p.349-354)
U09-125	32비트 주소·4 KB 페이지·4 B 항목 예는 페이지 테이블만 프로세스당 4 MB를 요구한다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-126	테이블을 연속 공간에 두는 대신 테이블 자체를 페이징한다. 2단계에서는 20비트 페이지 번호를 바깥 인덱스 10비트와 안쪽 인덱스 10비트로, 오프셋은 12비트로 나눈다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-127	바깥에서 안쪽으로 변환하므로 forward-mapped라고도 부른다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-128	원문의 완전한 64비트 공간·4 KB 가정은 2단계로도 바깥 테이블이 지나치게 커지고 단계를 계속 늘리면 변환의 메모리 접근이 많아지는 문제를 보인다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-129	이는 다음 해시·역 테이블의 동기이다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-130	이 예의 전체 64비트 가정과 뒤에서 설명하는 실제 48비트 주소 사용 사례를 혼동하지 않는다.	(9.4.1, p.409-411 / PDF p.349-351)
U09-131	큰 주소 공간에서 가상 페이지 번호를 해시한 위치에 연결 리스트를 둔다.	(9.4.2, p.411-412 / PDF p.351-352)
U09-132	원소는 가상 페이지 번호·프레임 번호·다음 원소 포인터를 가진다.	(9.4.2, p.411-412 / PDF p.351-352)
U09-133	해시 충돌 리스트를 따라 원하는 가상 페이지 번호와 일치하는 항목을 찾아 물리 프레임을 얻는다.	(9.4.2, p.411-412 / PDF p.351-352)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-134	클러스터 페이지 테이블은 한 항목이 한 페이지가 아니라 여러 페이지를 가리켜 넓게 퍼진 성긴 주소 공간에 유용하다.	(9.4.2, p.411-412 / PDF p.351-352)
U09-135	일반 테이블은 프로세스별 가상 페이지마다 항목을 가지지만 역 페이지 테이블은 시스템 전체에서 물리 프레임마다 하나의 항목을 둔다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-136	항목은 그 프레임의 페이지 주소와 소유 프로세스 ID를 기록한다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-137	원문 IBM RT 설명에서는 <process-id, page-number>를 검색하여 i번째 항목에서 찾으면 물리 주소는 <i, offset>이 된다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-138	물리 메모리 기준 항목만 유지하여 공간이 줄지만 가상 주소로 테이블 전체를 찾는 데 시간이 오래 걸릴 수 있다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-139	해시로 탐색을 줄이고 먼저 TLB를 확인하여 성능을 보완한다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-140	프레임당 항목 하나라 같은 물리 페이지를 여러 가상 주소로 동시에 나타내는 공유 메모리가 복잡해진다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-141	원문은 다른 프로세스가 참조하면 페이지 폴트와 매핑 변경을 거치는 방식의 한계를 설명한다.	(9.4.3, p.412-413 / PDF p.352-353)
U09-142	하드웨어가 TLB에서 변환 테이블 항목(TTE)을 찾는다.	(9.4.4, p.414 / PDF p.354)
U09-143	없으면 메모리의 변환 저장 버퍼(TSB)에서 최근 TTE를 찾고 TLB로 복사한다.	(9.4.4, p.414 / PDF p.354)
U09-144	TSB에도 없으면 인터럽트하여 커널이 해시 테이블을 탐색한다.	(9.4.4, p.414 / PDF p.354)
U09-145	커널이 TTE를 만들어 TSB에 넣고 MMU에 제어를 돌려 변환을 끝낸다.	(9.4.4, p.414 / PDF p.354)
U09-146	커널과 사용자 프로세스용 해시 테이블은 각 항목이 연속 가상 영역의 기준 주소와 범위를 나타내어 페이지마다 항목을 두는 비용을 줄인다.	(9.4.4, p.414 / PDF p.354)
U09-147	하드웨어·운영체제가 함께 다단계 탐색을 줄이는 원문 사례로 읽는다.	(9.4.4, p.414 / PDF p.354)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-148	스와핑은 프로세스나 일부를 메모리에서 백업 저장장치로 내보냈다가 실행을 계속하려고 가져오는 기법이다.	(9.5, p.414-417 / PDF p.354-357)
U09-149	물리 메모리보다 프로세스 총 공간이 커도 동시에 유지할 수 있어 다중 프로그래밍 정도를 높인다.	(9.5, p.414-417 / PDF p.354-357)
U09-150	전체 프로세스의 표준 스와핑은 이미지뿐 아니라 복원할 자료구조·다중 스레드별 정보·메타데이터도 보존해야 한다.	(9.5, p.414-417 / PDF p.354-357)
U09-151	표준 스와핑	(9.5, p.414-417 / PDF p.354-357)
U09-152	전체 프로세스를 이동한다. 유휴 프로세스 메모리를 활성 프로세스에 줄 수 있지만 큰 이동 비용이 든다. 원문은 현대 시스템에서 드물고 Solaris의 극한 부족 상황을 예외로 든다.	(9.5, p.414-417 / PDF p.354-357)
U09-153	페이징에서의 스와핑	(9.5, p.414-417 / PDF p.354-357)
U09-154	프로세스의 일부 페이지만 page-out·page-in하여 전체 복사 비용을 줄인다.	(9.5, p.414-417 / PDF p.354-357)
U09-155	모바일 시스템	(9.5, p.414-417 / PDF p.354-357)
U09-156	원문은 일반적으로 스와핑을 피한다고 설명한다. 플래시 공간·쓰기 횟수·낮은 처리량 제약 때문이다.	(9.5, p.414-417 / PDF p.354-357)
U09-157	원문의 iOS는 부족하면 응용이 메모리를 자발적으로 반환하게 하고 읽기 전용 코드는 제거 후 다시 읽을 수 있지만 수정한 데이터는 그렇게 제거하지 못한다.	(9.5, p.414-417 / PDF p.354-357)
U09-158	충분히 반환하지 못하면 종료될 수 있다.	(9.5, p.414-417 / PDF p.354-357)
U09-159	Android는 비슷하게 프로세스를 종료하되 상태를 플래시에 저장하여 재시작을 돕는다.	(9.5, p.414-417 / PDF p.354-357)
U09-160	이는 제공된 원문의 서술이며 현재 제품 동작으로 확대하지 않는다.	(9.5, p.414-417 / PDF p.354-357)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-161	IA-32의 변환은 논리 주소 → 세그먼테이션 장치 → 선형 주소 → 페이징 장치 → 물리 주소이다.	(9.6, p.417-421 / PDF p.357-361)
U09-162	두 장치가 MMU 역할을 나눈다.	(9.6, p.417-421 / PDF p.357-361)
U09-163	세그먼트 디스크립터는 기준·상한 정보를 가지며 프로세스별 LDT와 공유 GDT를 구분한다.	(9.6, p.417-421 / PDF p.357-361)
U09-164	논리 주소의 selector가 테이블·세그먼트·보호 정보를 선택하고 offset을 검사한 뒤 기준에 더해 32비트 선형 주소를 만든다.	(9.6, p.417-421 / PDF p.357-361)
U09-165	IA-32의 4 KB 페이지는 2단계 페이징이다.	(9.6, p.417-421 / PDF p.357-361)
U09-166	상위 10비트가 CR3이 가리키는 페이지 디렉터리, 다음 10비트가 페이지 테이블, 마지막 12비트가 오프셋이다.	(9.6, p.417-421 / PDF p.357-361)
U09-167	페이지 디렉터리의 Page Size 플래그가 4 MB를 지정하면 하위 테이블 대신 프레임을 직접 가리키고 하위 22비트를 오프셋으로 쓴다.	(9.6, p.417-421 / PDF p.357-361)
U09-168	디스크의 페이지 테이블은 invalid 비트를 이용해 나중에 가져올 수 있다.	(9.6, p.417-421 / PDF p.357-361)
U09-169	PAE(page address extension)는 원문에서 32비트 처리기의 물리 공간을 확장하는 3단계 테이블로 설명된다.	(9.6, p.417-421 / PDF p.357-361)
U09-170	최상위 2비트가 페이지 디렉터리 포인터 테이블을 선택하고 항목을 64비트로 늘려 12비트 오프셋과 합한 36비트 물리 주소·64 GB를 지원한다.	(9.6, p.417-421 / PDF p.357-361)
U09-171	사용하려면 운영체제 지원이 필요하며 원문은 32비트 Windows 데스크톱의 4 GB 제한도 별도 언급한다.	(9.6, p.417-421 / PDF p.357-361)
U09-172	원문의 x86-64는 IA-32 명령 집합 확장에서 출발하며 이론적 64비트 전체와 실제 사용 비트 수를 구분한다.	(9.6, p.417-421 / PDF p.357-361)
U09-173	제시된 구조는 48비트 가상 주소·4단계 테이블·4 KB/2 MB/1 GB 페이지와 52비트 물리 주소를 지원한다.	(9.6, p.417-421 / PDF p.357-361)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-174	이 수치는 제공된 책의 설명으로 한정한다.	(9.6, p.417-421 / PDF p.357-361)
U09-175	그림 9.25. 원문 주소 구조.	(9.6, p.421 / PDF p.361)
U09-176	원문은 64비트 ARMv8를 대상으로 설명하며 실제 사용 가상 주소는 48비트라고 구분한다. 4 KB·16 KB·64 KB의 변환 단위가 있고 페이지뿐 아니라 더 큰 연속 영역 region을 표현한다. 4 KB·16 KB는 최대 4단계, 64 KB는 최대 3단계 페이징을 사용할 수 있다.	(9.7, p.421-423 / PDF p.361-363)
U09-177	4 KB	(9.7, p.421-423 / PDF p.361-363)
U09-178	2 MB, 1 GB	(9.7, p.421-423 / PDF p.361-363)
U09-179	16 KB	(9.7, p.421-423 / PDF p.361-363)
U09-180	32 MB	(9.7, p.421-423 / PDF p.361-363)
U09-181	64 KB	(9.7, p.421-423 / PDF p.361-363)
U09-182	512 MB	(9.7, p.421-423 / PDF p.361-363)
U09-183	4 KB 예에서는 TTBR이 레벨0 테이블을 가리킨다.	(9.7, p.421-423 / PDF p.361-363)
U09-184	레벨1이 1 GB region을 직접 가리키면 하위 30비트, 레벨2가 2 MB region을 직접 가리키면 하위 21비트가 오프셋이다.	(9.7, p.421-423 / PDF p.361-363)
U09-185	네 단계 모두 사용하면 마지막 12비트가 4 KB 페이지 안 오프셋이다.	(9.7, p.421-423 / PDF p.361-363)
U09-186	구조의 다른 레벨·영역을 임의로 덧붙이지 않는다.	(9.7, p.421-423 / PDF p.361-363)
U09-187	ARM TLB는 명령·데이터용 마이크로 TLB와 바깥 단일 메인 TLB로 나뉜다.	(9.7, p.421-423 / PDF p.361-363)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-188	마이크로 TLB는 ASID를 지원한다.	(9.7, p.421-423 / PDF p.361-363)
U09-189	먼저 마이크로, 다음 메인 TLB를 찾고 둘 다 미스이면 하드웨어가 테이블을 순회한다.	(9.7, p.421-423 / PDF p.361-363)
U09-190	주소 생성·보호·사상은 하드웨어와 운영체제의 협력으로 이루어진다.	(9.8, p.423-426 / PDF p.363-366)
U09-191	연속 할당의 외부 단편화를 페이징으로 피하지만 내부 단편화·테이블·변환 비용은 남는다.	(9.8, p.423-426 / PDF p.363-366)
U09-192	TLB와 여러 테이블 구조는 그 비용을 조절한다.	(9.8, p.423-426 / PDF p.363-366)
U09-193	원문 연습 문제는 단편화 구분, 주소의 비트 수, 페이지·오프셋, 배치 전략, 일반·역 테이블 항목 수 등을 확인한다.	(9.8, p.423-426 / PDF p.363-366)
U09-194	새 계산 문제나 답을 만들지 않았다.	(9.8, p.423-426 / PDF p.363-366)
U09-195	기준 레지스터는 가장 작은 유효한 물리 메모리 주소를 저장한다.	(9.8, p.423 / PDF p.363)
U09-196	상한 레지스터는 메모리 범위의 크기를 지정한다.	(9.8, p.423 / PDF p.363)
U09-197	논리 주소는 CPU가 생성한 주소이다.	(9.8, p.424 / PDF p.364)
U09-198	프레임은 물리 메모리를 나눈 고정 크기 블록이다.	(9.8, p.424 / PDF p.364)
U09-199	페이지는 논리 메모리를 나눈 프레임과 같은 크기의 블록이다.	(9.8, p.424 / PDF p.364)
U09-200	TLB(translation look-aside buffer)는 페이지 테이블의 하드웨어 캐시이며, 각 항목은 페이지 번호와 상응하는 프레임을 저장한다.	(9.8, p.424 / PDF p.364)
U09-201	페이지 오프셋은 참조되는 프레임의 특정 위치이다.	(9.8, p.424 / PDF p.364)
U09-202	심볼릭 주소 참조를 실제 물리 주소에 바인딩하는 작업은 컴파일·적재·실행시간에 발생할 수 있다.	(9.8, p.423-424 / PDF p.363-364)
U09-203	다양한 크기의 연속 메모리 파티션은 최초 적합·최적 적합·최악 적합 전략에 따라 할당될 수 있다.	(9.8, p.424 / PDF p.364)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-204	계층적 페이징은 논리 주소를 여러 부분으로 나누며, 계층 수가 커지는 문제를 해결하는 전략으로 해시 페이지 테이블과 역 페이지 테이블이 있다.	(9.8, p.424 / PDF p.364)
U09-205	CPU가 생성한 주소는 논리 주소이며, MMU가 이를 메모리의 물리 주소로 변환한다.	(9.8, p.424 / PDF p.364)
U09-206	기준 레지스터는 가장 작은 유효 물리 주소를 저장하고 상한은 메모리 범위의 크기를 지정한다.	(9.8, p.423 / PDF p.363)
U09-207	프레임	(9.8, p.424 / PDF p.364)
U09-208	페이지	(9.8, p.424 / PDF p.364)
U09-209	페이징 논리 주소의 비트 구분: 논리 주소 공간 2^m, 페이지 크기 2^n바이트 → 페이지 번호 m−n비트 / 페이지 오프셋 n비트; 페이지 크기는 2의 거듭제곱이다.	(9.3.1, p.398 / PDF p.338)

제10장. 가상 메모리

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-001	가상 메모리는 물리 메모리를 매우 큰 균일한 저장장소 배열로 추상화한다.	(10.11, p.483 / PDF p.423)
U10-002	요구 페이징은 프로그램 실행 중 페이지가 필요할 때만 페이지를 적재하는 기법이다.	(10.11, p.484 / PDF p.424)
U10-003	가상 메모리는 프로세스 전체가 메모리에 없어도 실행하게 하며 실제 물리 메모리와 프로그래머의 논리 메모리를 분리한다.	(10.1, p.427-431 / PDF p.367-371)
U10-004	거의 실행되지 않는 오류 처리·사용되지 않는 배열 영역·드문 옵션까지 모두 올릴 필요가 없다.	(10.1, p.427-431 / PDF p.367-371)
U10-005	작은 메모리로 큰 주소 공간을 제공하고 한 프로그램의 점유를 줄여 더 많은 프로그램을 수행하며 적재·스왑 I/O를 줄일 수 있다.	(10.1, p.427-431 / PDF p.367-371)
U10-006	그러나 구현이 복잡하고 잘못 사용하면 성능이 크게 떨어진다.	(10.1, p.427-431 / PDF p.367-371)
U10-007	프로세스의 가상 공간은 연속처럼 보이지만 물리 프레임은 흩어질 수 있다.	(10.1, p.427-431 / PDF p.367-371)
U10-008	원문의 힙·스택 사이 성긴 주소 공간(sparse address space)은 공백도 주소 공간 일부이되 실제 확장 때만 물리 페이지를 요구한다.	(10.1, p.427-431 / PDF p.367-371)
U10-009	공유 라이브러리와 공유 메모리는 서로 다른 프로세스의 가상 페이지를 같은 물리 페이지에 사상하고 fork 생성에서도 페이지를 공유할 수 있다.	(10.1, p.427-431 / PDF p.367-371)
U10-010	요구 페이징(demand paging)은 페이지가 필요할 때만 메모리에 올린다.	(10.2, p.431-438 / PDF p.371-378)
U10-011	접근하지 않는 페이지는 적재하지 않는다.	(10.2, p.431-438 / PDF p.371-378)
U10-012	유효/무효 비트는 이 문맥에서 페이지가 메모리에 있는지 구별하며 무효는 논리적으로 잘못된 페이지이거나 유효하지만 보조저장장치에 있는 페이지를 뜻할 수 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-013	무효 페이지에 접근하면 페이지 폴트 트랩이 발생하므로 운영체제가 두 경우를 구별한다.	(10.2, p.431-438 / PDF p.371-378)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-014	프로세스 내부 테이블을 검사하여 참조가 유효한지 확인한다.	(10.2, p.431-438 / PDF p.371-378)
U10-015	무효 참조면 프로세스를 중단하고, 유효하지만 메모리에 없으면 페이지를 가져온다.	(10.2, p.431-438 / PDF p.371-378)
U10-016	가용 프레임을 찾는다.	(10.2, p.431-438 / PDF p.371-378)
U10-017	보조저장장치에 그 프레임으로 페이지 읽기를 요청한다.	(10.2, p.431-438 / PDF p.371-378)
U10-018	읽기가 끝나면 페이지 테이블과 내부 테이블을 갱신한다.	(10.2, p.431-438 / PDF p.371-378)
U10-019	중단된 명령어를 다시 수행한다.	(10.2, p.431-438 / PDF p.371-378)
U10-020	순수 요구 페이징은 시작할 때 한 페이지도 올리지 않고 첫 명령 인출의 폴트부터 시작할 수 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-021	한 명령이 명령·피연산자에 대해 여러 폴트를 낼 수 있지만 참조의 지역성 덕분에 특정 작은 부분을 집중적으로 사용하여 보통 만족할 성능을 낸다.	(10.2, p.431-438 / PDF p.371-378)
U10-022	유효/무효 표시가 가능한 테이블과 메모리에 없는 페이지를 유지할 스왑 장치가 필요하다.	(10.2, p.431-438 / PDF p.371-378)
U10-023	필수 조건은 폴트 후 명령을 정확히 재시작할 수 있다는 것이다.	(10.2, p.431-438 / PDF p.371-378)
U10-024	레지스터·조건 코드·명령 카운터 등 상태를 보존하여 같은 위치에서 실행해야 한다.	(10.2, p.431-438 / PDF p.371-378)
U10-025	여러 기억 장소를 바꾸고 원천·목적 영역이 겹칠 수 있는 이동 명령은 단순 재실행이 안 될 수 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-026	원문은 수정 전 양 끝을 먼저 접근하여 필요한 페이지를 확보하거나, 덮어쓸 값을 임시 레지스터에 보관하여 폴트 때 원상 복구하는 방법을 설명한다.	(10.2, p.431-438 / PDF p.371-378)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-027	운영체제는 가용 프레임 리스트를 유지하며 페이지 폴트·스택·힙 확장에 프레임을 준다.	(10.2, p.431-438 / PDF p.371-378)
U10-028	Zero-fill-on-demand는 할당 전 프레임을 0으로 채워 이전 내용을 지운다.	(10.2, p.431-438 / PDF p.371-378)
U10-029	프레임의 이전 데이터를 남기는 것은 보안 취약점이 될 수 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-030	리스트가 0 또는 임계값 아래가 되면 다시 채울 방법이 필요하다.	(10.2, p.431-438 / PDF p.371-378)
U10-031	원문 수식: 실질 접근 시간 = (1−p) × ma + p × 페이지 폴트 시간; p: 페이지 폴트 확률, $0 \le p \le 1$; ma: 메모리 접근 시간; 폴트가 없을 때의 메모리 시간과 폴트가 있을 때의 처리 시간을 확률로 가중한다.	(10.2, p.431-438 / PDF p.371-378)
U10-032	원문 상세 처리에는 트랩·상태 저장·유효성 및 디스크 위치 확인·읽기 대기와 전송·다른 프로세스 실행·I/O 완료 인터럽트·테이블 갱신·CPU 재배정·명령 재실행이 포함된다.	(10.2, p.431-438 / PDF p.371-378)
U10-033	주요 요소는 인터럽트 처리, 페이지 읽기, 프로세스 재시작이다.	(10.2, p.431-438 / PDF p.371-378)
U10-034	다른 프로세스를 실행하면 CPU 활용은 높지만 원래 처리 재개까지 추가 지연이 생긴다.	(10.2, p.431-438 / PDF p.371-378)
U10-035	인쇄437쪽의 200 ns·8 ms 가정 계산식.	(10.2, p.437 / PDF p.377)
U10-036	위 예에서 1,000번에 한 번 폴트이면 8.2 μs로 약 40배 느려지고, 성능 저하를 10% 이내로 하려면 p<0.0000025라는 원문의 조건이 나온다.	(10.2, p.431-438 / PDF p.371-378)
U10-037	폴트율을 낮추는 것이 매우 중요하다.	(10.2, p.431-438 / PDF p.371-378)
U10-038	이는 HDD 기반 원문 가정의 결과이며 다른 장치에 새 수치를 대입하지 않는다.	(10.2, p.431-438 / PDF p.371-378)
U10-039	스왑 공간은 파일 탐색·간접 할당을 거치지 않는 등 파일 시스템보다 I/O가 빠를 수 있다.	(10.2, p.431-438 / PDF p.371-378)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-040	실행 이미지를 미리 복사하면 시작 비용이 들고, 먼저 실행 파일에서 요구 페이징한 뒤 교체할 때 스왑에 쓰는 절충이 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-041	수정되지 않는 실행 페이지는 버렸다가 파일에서 다시 읽을 수도 있다.	(10.2, p.431-438 / PDF p.371-378)
U10-042	파일과 관련 없는 스택·힙의 익명 메모리는 별도 백업이 필요하다.	(10.2, p.431-438 / PDF p.371-378)
U10-043	원문의 모바일 시스템은 읽기 전용 페이지를 방출·재적재하고 실행 중 생긴 페이지는 다른 정책으로 관리한다.	(10.2, p.431-438 / PDF p.371-378)
U10-044	쓰기 시 복사(copy-on-write, COW)는 fork 때 부모의 모든 페이지를 바로 복사하지 않고 부모·자식이 먼저 공유하게 한다.	(10.3, p.438-440 / PDF p.378-380)
U10-045	수정 가능한 공유 페이지를 COW로 표시하고 둘 중 하나가 쓰면 복사본을 만들어 그 프로세스 주소 공간에 사상한다.	(10.3, p.438-440 / PDF p.378-380)
U10-046	수정하지 않는 페이지는 계속 공유하며 실행 코드처럼 변경 불가능한 페이지는 COW 표시 없이 공유할 수 있다.	(10.3, p.438-440 / PDF p.378-380)
U10-047	생성 직후 exec하는 자식을 위해 불필요한 복사를 줄이는 방법이다.	(10.3, p.438-440 / PDF p.378-380)
U10-048	원문의 vfork()는 부모를 보류하고 자식이 부모 주소 공간을 그대로 사용한다.	(10.3, p.438-440 / PDF p.378-380)
U10-049	COW를 쓰지 않으므로 자식의 변경이 나중에 부모에게 보인다.	(10.3, p.438-440 / PDF p.378-380)
U10-050	자식이 즉시 exec하는 용도이며 부모 페이지를 변경하지 않도록 주의해야 한다.	(10.3, p.438-440 / PDF p.378-380)
U10-051	페이지 복사가 없어 효율적이라는 원문 범위로 설명한다.	(10.3, p.438-440 / PDF p.378-380)
U10-052	프로세스 일부만 사용한다고 예상하여 과할당하면 실제로 더 많은 페이지가 필요해질 때 가용 프레임이 없어질 수 있다.	(10.4, p.440-454 / PDF p.380-394)
U10-053	운영체제 메모리와 I/O 버퍼도 프레임을 필요로 한다.	(10.4, p.440-454 / PDF p.380-394)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-054	논리적으로 사용자에게 투명해야 하는 페이징에서 단순 종료 대신 페이지 교체로 현재 프레임을 비워 요청 페이지를 올린다.	(10.4, p.440-454 / PDF p.380-394)
U10-055	필요한 페이지의 보조저장장치 위치를 찾는다.	(10.4, p.440-454 / PDF p.380-394)
U10-056	가용 프레임이 없으면 교체 알고리즘으로 희생 프레임을 고른다.	(10.4, p.440-454 / PDF p.380-394)
U10-057	필요하면 희생 페이지를 기록하고 관련 테이블을 갱신한다.	(10.4, p.440-454 / PDF p.380-394)
U10-058	빈 프레임에 새 페이지를 읽고 테이블을 바꾼 뒤 명령을 재개한다.	(10.4, p.440-454 / PDF p.380-394)
U10-059	희생 페이지 쓰기와 새 페이지 읽기가 모두 필요하면 저장장치 접근 두 번이 든다.	(10.4, p.440-454 / PDF p.380-394)
U10-060	변경/더티 비트는 메모리에서 쓴 적이 있는지 표시한다.	(10.4, p.440-454 / PDF p.380-394)
U10-061	변경되지 않았고 백업 사본이 유지되면 다시 쓸 필요가 없다.	(10.4, p.440-454 / PDF p.380-394)
U10-062	읽기 전용 코드도 버렸다가 다시 읽을 수 있다.	(10.4, p.440-454 / PDF p.380-394)
U10-063	프레임 할당 알고리즘은 프로세스별 수를, 교체 알고리즘은 희생 페이지를 정한다.	(10.4, p.440-454 / PDF p.380-394)
U10-064	교체 성능은 메모리 참조열(reference string)과 프레임 수를 정하고 폴트 횟수로 평가한다.	(10.4, p.440-454 / PDF p.380-394)
U10-065	페이지 크기가 정해지면 전체 주소 대신 페이지 번호를 쓰고 연속한 같은 페이지 접근을 줄일 수 있다.	(10.4, p.440-454 / PDF p.380-394)
U10-066	원문의 다음 비교는 처음 비어 있는 3프레임, 참조열 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1을 사용한다.	(10.4, p.440-454 / PDF p.380-394)
U10-067	FIFO는 메모리에 가장 먼저 들어온 페이지를 교체한다.	(10.4.2, p.444-446 / PDF p.384-386)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-068	적재 시간 또는 큐를 유지하여 머리에서 빼고 새 페이지를 끝에 넣는다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-069	이해·구현은 쉽지만 자주 쓰는 오래된 페이지도 내보낼 수 있다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-070	잘못된 교체는 이후 폴트를 늘릴 뿐 프로그램의 논리적 실행 자체를 틀리게 하지는 않는다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-071	그림 10.12. 3프레임 참조열 추적.	(10.4.2, p.445 / PDF p.385)
U10-072	Belady의 모순은 프레임을 더 주었는데 폴트가 오히려 늘어나는 현상이다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-073	원문 참조열 1 2 3 4 1 2 5 1 2 3 4 5에서 FIFO는 3프레임에 9회, 4프레임에 10회이다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-074	프레임 증가가 모든 알고리즘에서 항상 폴트를 줄인다고 단정하면 안 된다.	(10.4.2, p.444-446 / PDF p.384-386)
U10-075	OPT/MIN은 앞으로 가장 오랫동안 사용되지 않을 페이지를 교체한다.	(10.4.3, p.446-447 / PDF p.386-387)
U10-076	고정된 프레임 수에서 가장 낮은 폴트율을 보장하며 Belady의 모순이 없다.	(10.4.3, p.446-447 / PDF p.386-387)
U10-077	그러나 미래 참조를 미리 알아야 하므로 실제 구현이 어렵고 비교 연구의 기준으로 사용한다.	(10.4.3, p.446-447 / PDF p.386-387)
U10-078	그림 10.14. 같은 3프레임·참조열의 OPT.	(10.4.3, p.447 / PDF p.387)
U10-079	LRU(least-recently-used)는 가장 오랫동안 사용되지 않은 페이지를 교체한다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-080	가까운 과거를 가까운 미래의 근사로 본다.	(10.4.4, p.447-449 / PDF p.387-389)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-081	FIFO의 적재 시점, OPT의 미래 사용 시점, LRU의 마지막 사용 시점을 구별한다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-082	그림 10.15. 같은 3프레임·참조열의 LRU.	(10.4.4, p.448 / PDF p.388)
U10-083	계수기	(10.4.4, p.447-449 / PDF p.387-389)
U10-084	페이지마다 마지막 사용 시간을 기록한다. 메모리 참조마다 갱신하고 희생자 선택 때 최소 시간을 탐색한다. 문맥 교환 관리·계수 overflow도 고려한다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-085	스택	(10.4.4, p.447-449 / PDF p.387-389)
U10-086	참조된 페이지를 중간에서 제거해 꼭대기로 옮긴다. 맨 아래가 가장 오래 안 쓴 페이지이다. 연결 리스트 갱신 비용이 있지만 희생자 탐색은 필요 없다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-087	LRU·OPT는 스택 알고리즘이다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-088	n프레임에 남는 페이지 집합이 n+1프레임 집합의 부분집합이므로 Belady의 모순이 없다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-089	두 정확한 LRU 구현은 매 참조의 갱신을 위해 하드웨어 지원이 필요하며 이를 소프트웨어 인터럽트로 수행하면 큰 지연이 든다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-090	원문 확인 필요: 인쇄448쪽의 스택 설명 첫 문장은 "LFU"라고 표기하지만 해당 절·그림·후속 설명은 LRU를 다룬다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-091	표기를 조용히 수정한 인용으로 제시하지 않았다.	(10.4.4, p.447-449 / PDF p.387-389)
U10-092	정확한 사용 순서를 기록하는 하드웨어가 부족해도 참조 비트로 그동안 사용했는지는 알 수 있다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-093	운영체제가 0으로 두고 페이지를 읽거나 쓸 때 하드웨어가 1로 만든다.	(10.4.5, p.449-452 / PDF p.389-392)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-094	이것은 정확한 순서는 아니지만 근사 교체의 기반이다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-095	부가적 참조 비트는 일정 간격마다 참조 비트를 8비트 기록의 맨 위에 넣고 나머지를 오른쪽으로 민다. 00000000은 최근 8구간 미사용, 11111111은 구간마다 최소 한 번 사용을 뜻한다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-096	정수로 작은 기록을 우선 교체하고 동점은 FIFO 등으로 처리한다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-097	비트 수와 간격은 조절할 수 있다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-098	FIFO 순서의 다음 희생 후보에서 참조 비트를 본다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-099	0이면 교체한다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-100	1이면 0으로 바꾸어 한 번 더 기회를 주고 다음 후보로 간다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-101	순환 큐와 포인터로 반복한다. 모두 1이면 한 바퀴 돌며 0으로 만들고 다음 바퀴에서는 FIFO처럼 선택한다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-102	(0,0)	(10.4.5, p.449-452 / PDF p.389-392)
U10-103	최근 사용·변경이 없어 교체하기 가장 좋다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-104	(0,1)	(10.4.5, p.449-452 / PDF p.389-392)
U10-105	최근 미사용이지만 변경되어 먼저 기록해야 한다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-106	(1,0)	(10.4.5, p.449-452 / PDF p.389-392)
U10-107	최근 사용되어 다시 쓸 가능성이 있다.	(10.4.5, p.449-452 / PDF p.389-392)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-108	(1,1)	(10.4.5, p.449-452 / PDF p.389-392)
U10-109	최근 사용·변경되어 재사용 가능성과 기록 비용이 모두 있다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-110	개선된 알고리즘은 순환 탐색하며 가장 낮은 등급의 첫 페이지를 고른다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-111	목적은 최근 참조뿐 아니라 변경 페이지의 I/O 비용도 함께 고려하는 것이다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-112	여러 바퀴 탐색할 수 있다는 조건도 남긴다.	(10.4.5, p.449-452 / PDF p.389-392)
U10-113	LFU	(10.4.6, p.452 / PDF p.392)
U10-114	참조 횟수가 가장 적은 페이지를 교체한다. 초기에 집중 사용했지만 나중에 안 쓰는 페이지가 남을 수 있어 주기적으로 계수를 오른쪽으로 이동하여 오래된 영향력을 줄일 수 있다.	(10.4.6, p.452 / PDF p.392)
U10-115	MFU	(10.4.6, p.452 / PDF p.392)
U10-116	횟수가 적은 페이지는 최근 들어왔고 앞으로 사용할 것이라는 판단에 근거한다.	(10.4.6, p.452 / PDF p.392)
U10-117	원문은 두 방식 모두 구현 비용이 크고 OPT를 잘 근사하지 못해 일반적으로 널리 쓰이지 않는다고 설명한다.	(10.4.6, p.452 / PDF p.392)
U10-118	특정 응용에서 유리할 가능성과 범용 기본 정책으로 선호되는지는 구별한다.	(10.4.6, p.452 / PDF p.392)
U10-119	교체 알고리즘과 함께 가용 프레임 풀을 유지하여 희생 페이지를 다 쓰기 전에 새 페이지부터 읽어 실행을 빨리 재개할 수 있다.	(10.4.7, p.452-453 / PDF p.392-393)
U10-120	희생 페이지 기록이 끝나면 그 프레임을 풀에 넣는다.	(10.4.7, p.452-453 / PDF p.392-393)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-121	변경 페이지 리스트를 여유 시간에 미리 기록하면 나중에 교체 때 쓰기 비용을 줄인다.	(10.4.7, p.452-453 / PDF p.392-393)
U10-122	가용 풀 프레임의 원래 페이지를 기억하고 내용을 덮어쓰지 않았다면 재참조 때 디스크 I/O 없이 다시 줄 수 있어 잘못된 희생자 선택의 불이익을 줄인다.	(10.4.7, p.452-453 / PDF p.392-393)
U10-123	자체 버퍼링·메모리 관리가 있는 데이터베이스는 범용 운영체제보다 자기 접근을 잘 알아 이중 버퍼링이 오히려 메모리를 두 번 사용할 수 있다.	(10.4.8, p.453-454 / PDF p.393-394)
U10-124	연속 대량 읽기를 반복하는 데이터웨어하우스는 새 페이지보다 옛 페이지를 다시 쓸 수 있어 원문은 MFU가 LRU보다 효율적일 수 있는 예로 든다.	(10.4.8, p.453-454 / PDF p.393-394)
U10-125	Raw I/O는 단순 논리 블록 배열을 사용해 파일 시스템의 요구 페이징·잠금·선반입·이름 등의 서비스를 거치지 않는다.	(10.4.8, p.453-454 / PDF p.393-394)
U10-126	이런 특수 응용과 달리 대부분은 정상 파일 시스템이 더 좋은 성능이라고 원문은 제한한다.	(10.4.8, p.453-454 / PDF p.393-394)
U10-127	가용 프레임을 프로세스에 얼마나 줄지는 교체 정책과 별개의 선택이다.	(10.5, p.454-460 / PDF p.394-400)
U10-128	가용 프레임보다 많이 줄 수 없고 명령을 끝낼 수 있는 최소 수도 보장해야 한다.	(10.5, p.454-460 / PDF p.394-400)
U10-129	명령 인출·간접 참조·피연산자 등 한 명령이 필요한 페이지가 동시에 있어야 재실행이 끝난다.	(10.5, p.454-460 / PDF p.394-400)
U10-130	최소 수는 아키텍처가, 최대 수는 가용 물리 메모리가 정한다.	(10.5, p.454-460 / PDF p.394-400)
U10-131	균등 할당	(10.5.2, p.455-456 / PDF p.395-396)
U10-132	운영체제 몫을 제외한 m프레임을 n프로세스에 m/n씩 준다. 원문 93프레임·5프로세스는 각각18, 나머지3은 풀이다.	(10.5.2, p.455-456 / PDF p.395-396)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-133	비례 할당	(10.5.2, p.455-456 / PDF p.395-396)
U10-134	프로세스 크기 s_i와 전체 크기 합 S에 비례하여 $a_i = s_i / S \times m$을 준다. 최소 수 이상 정수로 조정하고 합이 m을 넘지 않게 한다.	(10.5.2, p.455-456 / PDF p.395-396)
U10-135	우선순위 반영	(10.5.2, p.455-456 / PDF p.395-396)
U10-136	크기 대신 우선순위 또는 크기·우선순위 조합으로 비례시킬 수 있다.	(10.5.2, p.455-456 / PDF p.395-396)
U10-137	원문 수식: $S = \Sigma s_i$, $a_i = (s_i / S) \times m$; s_i: 프로세스 p_i의 크기; m: 가용 프레임 전체 수; a_i: p_i에 할당하는 근사 프레임 수; 원문의 비례 할당식. 10페이지·127페이지에 62프레임을 나누는 예는 각각 약4·57프레임으로 제시된다.	(10.5.2, p.455-456 / PDF p.395-396)
U10-138	균등·비례 모두 다중 프로그래밍 정도가 높아지면 각자 몫이 줄고 낮아지면 늘어난다.	(10.5.2, p.455-456 / PDF p.395-396)
U10-139	단순 크기 기준은 우선순위를 똑같이 취급하므로 우선순위를 반영할지는 별도 정책이다.	(10.5.2, p.455-456 / PDF p.395-396)
U10-140	전역 교체	(10.5.3, p.456-459 / PDF p.396-399)
U10-141	다른 프로세스 프레임도 대상이다. 할당 수가 바뀌고 한 프로세스 성능이 다른 프로세스 행태에 영향을 받는다. 원문은 일반적으로 시스템 성능이 더 좋아 많이 쓰인다고 설명한다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-142	지역 교체	(10.5.3, p.456-459 / PDF p.396-399)
U10-143	자기 프레임에서만 선택한다. 외부 영향이 적지만 다른 곳의 안 쓰는 프레임을 쓰지 못해 낭비할 수 있다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-144	주요(하드) 폴트는 페이지가 메모리에 없어 백업 저장장치에서 읽어야 한다.	(10.5.3, p.456-459 / PDF p.396-399)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-145	사소한(소프트) 폴트는 페이지가 이미 메모리에 있지만 그 프로세스 매핑이 없을 때 발생할 수 있다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-146	공유 라이브러리 매핑 또는 아직 덮어쓰지 않은 가용 풀 페이지 재할당은 테이블 갱신 등으로 해결하여 일반적으로 시간이 덜 든다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-147	전역 회수 전략은 가용 리스트가 0이 되기를 기다리지 않고 최소 임계값 아래에서 커널 리퍼(reaper)를 깨워 최대 임계값까지 회수한다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-148	충분해지면 멈추고 다시 부족하면 반복한다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-149	보통 LRU 근사를 사용하지만 심각한 부족에서는 더 공격적인 정책이나 원문의 Linux OOM 킬러로 프로세스를 종료할 수 있다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-150	원문은 메모리 백분율 기반 점수와 임계값 조절을 설명하며 현행 정책을 새로 보충하지 않는다.	(10.5.3, p.456-459 / PDF p.396-399)
U10-151	NUMA에서는 CPU가 자기 로컬 메모리를 다른 CPU의 메모리보다 빨리 접근한다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-152	따라서 모든 프레임을 같게 보는 대신 실행 CPU에서 지연시간이 짧은 메모리를 할당해야 한다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-153	스케줄러가 직전 CPU와 가까운 위치에서 실행시키고 가상 메모리가 가까운 프레임을 주면 캐시 적중과 지연시간에 유리하다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-154	여러 스레드가 여러 보드에서 실행되면 더 복잡하다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-155	원문은 Linux의 스케줄링 도메인·NUMA 노드별 가용 리스트, Solaris의 CPU·메모리를 묶는 lgroup 계층을 예로 든다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-156	Solaris는 그룹 안에서 충족하지 못하면 가까운 그룹에서 나머지를 찾아 전체 지연을 줄인다.	(10.5.4, p.459-461 / PDF p.399-401)
U10-157	스래싱(thrashing)은 실제 실행보다 페이징에 더 많은 시간을 쓰는 과도한 페이징이다.	(10.6, p.461-467 / PDF p.401-407)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-158	현재 활발히 쓰는 페이지를 담을 프레임이 부족하면 어떤 것을 교체해도 곧 다시 필요해져 폴트가 반복된다.	(10.6, p.461-467 / PDF p.401-407)
U10-159	프레임이 더 필요한 프로세스가 다른 프로세스의 페이지를 빼앗는다.	(10.6, p.461-467 / PDF p.401-407)
U10-160	그쪽도 폴트하여 페이징 장치 대기열이 늘고 준비 큐·CPU 이용률이 줄어든다.	(10.6, p.461-467 / PDF p.401-407)
U10-161	운영체제가 낮은 CPU 이용률을 보고 프로세스를 더 넣으면 프레임 경쟁이 심해진다.	(10.6, p.461-467 / PDF p.401-407)
U10-162	추가 폴트와 더 긴 대기가 다시 이용률을 낮추어 처리율이 급격히 떨어진다.	(10.6, p.461-467 / PDF p.401-407)
U10-163	이 구간에서는 다중 프로그래밍 정도를 낮춰야 스래싱을 멈춘다.	(10.6, p.461-467 / PDF p.401-407)
U10-164	지역 교체는 다른 프로세스의 프레임을 빼앗는 전파는 막지만 같은 페이징 장치 큐를 길게 하므로 영향을 완전히 없애지 못한다.	(10.6, p.461-467 / PDF p.401-407)
U10-165	필요한 프레임 수를 확보하는 기준이 필요하다.	(10.6, p.461-467 / PDF p.401-407)
U10-166	지역성(locality)은 함께 집중적으로 참조되는 페이지 집합이다.	(10.6, p.461-467 / PDF p.401-407)
U10-167	프로세스는 함수·지역 변수·일부 전역 변수 등으로 형성된 한 지역에서 다른 지역으로 이동하며 지역들은 겹칠 수 있다.	(10.6, p.461-467 / PDF p.401-407)
U10-168	현재 지역을 담으면 적재 후 안정되지만 작게 주면 계속 폴트한다.	(10.6, p.461-467 / PDF p.401-407)
U10-169	원문은 이 모델을 캐싱의 기본 원리와도 연결한다.	(10.6, p.461-467 / PDF p.401-407)
U10-170	작업 집합은 최근 Δ번 페이지 참조에 등장한 서로 다른 페이지들의 집합이며 지역성의 근사이다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-171	활발히 사용하면 포함되고 마지막 참조 후 Δ가 지나면 제외된다. Δ가 너무 작으면 지역을 빠뜨리고 너무 크면 여러 지역을 함께 담는다.	(10.6.2, p.464-465 / PDF p.404-405)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-172	무한히 크면 실행 중 본 모든 페이지가 된다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-173	그림 10.22. Δ와 두 시점의 작업 집합.	(10.6.2, p.464 / PDF p.404)
U10-174	원문 수식: $D = \Sigma WSS_i$; WSS_i: 프로세스 i의 작업 집합 크기; D: 전체 요구 프레임 수; m: 시스템의 전체 메모리 프레임 수; D>m이면 지역성을 담을 프레임이 부족해 스래싱을 유발한다. 원문은 작업 집합에 맞게 할당하고 부족하면 한 프로세스를 연기하여 프레임을 나눈다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-175	매 참조마다 집합을 정확히 추적하기는 어렵다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-176	일정 간격 타이머와 참조 비트의 과거 기록으로 근사할 수 있다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-177	원문의 Δ=10,000·5,000참조 간격 예는 현재와 두 과거 비트를 검사한다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-178	더 많은 비트·더 잦은 인터럽트는 정확도를 높이지만 오버헤드도 늘린다.	(10.6.2, p.464-465 / PDF p.404-405)
U10-179	PFF(page-fault frequency)는 폴트율의 상·하한으로 직접 제어한다.	(10.6.3, p.465-467 / PDF p.405-407)
U10-180	높으면 프레임을 늘리고 낮으면 줄인다.	(10.6.3, p.465-467 / PDF p.405-407)
U10-181	높아졌는데 줄 프레임이 없으면 프로세스를 스왑아웃하여 나눈다.	(10.6.3, p.465-467 / PDF p.405-407)
U10-182	지역 전이 때 폴트율이 높아졌다 작업 집합 적재 후 낮아지는 원문 그래프의 맥락도 함께 본다.	(10.6.3, p.465-467 / PDF p.405-407)
U10-183	원문이 제시하는 일반적인 실용 대책은 극한 상황을 제외하고 작업 집합을 함께 담을 충분한 물리 메모리를 갖추는 것이다.	(10.6.3, p.465-467 / PDF p.405-407)
U10-184	메모리 압축은 수정 프레임을 스왑으로 내보내는 대신 여러 페이지를 압축하여 적은 프레임에 저장한다.	(10.7, p.467-469 / PDF p.407-409)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-185	원문은 15·3·35의 세 프레임을 하나의 프레임7에 압축하여 원래 프레임들을 가용 리스트로 옮기고, 나중 참조 시 폴트로 압축을 풀어 복원하는 예를 든다.	(10.7, p.467-469 / PDF p.407-409)
U10-186	압축을 유지할 프레임도 필요하지만 전체 사용량을 줄일 수 있다.	(10.7, p.467-469 / PDF p.407-409)
U10-187	원문은 Android·iOS·Windows10·macOS의 지원을 설명하고 macOS는 먼저 압축한 뒤 부족하면 페이징한다고 기술한다.	(10.7, p.467-469 / PDF p.407-409)
U10-188	압축률과 계산 시간 사이에 절충이 있고 여러 코어의 병렬 압축으로 개선할 수 있다.	(10.7, p.467-469 / PDF p.407-409)
U10-189	제품명·압축 알고리즘 수치는 원문 범위를 넘어 갱신하지 않는다.	(10.7, p.467-469 / PDF p.407-409)
U10-190	커널은 다양한 작은 자료구조를 할당하고 일부 하드웨어가 물리적으로 연속된 메모리를 요구한다.	(10.8, p.469-473 / PDF p.409-413)
U10-191	사용자 프로세스의 비연속 프레임 배정과 달리 별도 풀을 사용하는 이유이다.	(10.8, p.469-473 / PDF p.409-413)
U10-192	많은 운영체제가 커널 코드·데이터를 페이징하지 않으므로 단편화 낭비를 줄이는 것이 특히 중요하다.	(10.8, p.469-473 / PDF p.409-413)
U10-193	버디 시스템	(10.8, p.469-473 / PDF p.409-413)
U10-194	연속 세그먼트를 2의 거듭제곱으로 나누고 요청을 가까운 위 크기로 올린다.	(10.8, p.469-473 / PDF p.409-413)
U10-195	반환한 인접 버디를 쉽게 병합하지만 올림에 따른 내부 단편화가 있다.	(10.8, p.469-473 / PDF p.409-413)
U10-196	슬랩 할당	(10.8, p.469-473 / PDF p.409-413)
U10-197	자료구조별 캐시에 미리 만든 동일 종류 객체를 유지한다. 슬랩은 하나 이상 연속 페이지이다.	(10.8, p.469-473 / PDF p.409-413)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-198	필요한 객체 크기로 할당·반환하여 원문은 단편화 낭비를 피하고 빈번한 요청을 빠르게 처리한다고 설명한다.	(10.8, p.469-473 / PDF p.409-413)
U10-199	그림 10.26. 256 KB에서 21 KB 요청을 위한 32 KB 버디까지 분할.	(10.8, p.470 / PDF p.410)
U10-200	슬랩의 상태는 full(모두 사용), empty(모두 가용), partial(혼합)이다.	(10.8, p.469-473 / PDF p.409-413)
U10-201	먼저 partial의 free 객체, 없으면 empty, 그것도 없으면 새 연속 슬랩을 사용한다.	(10.8, p.469-473 / PDF p.409-413)
U10-202	반환한 객체를 free로 표시하여 재사용하므로 매번 새 생성·해제 비용을 줄인다.	(10.8, p.469-473 / PDF p.409-413)
U10-203	원문은 Solaris의 도입, Linux의 SLAB·제한 메모리용 SLOB·오버헤드를 줄이는 SLUB를 비교한다.	(10.8, p.469-473 / PDF p.409-413)
U10-204	SLOB는 크기별 세 목록에 최초 적합을 쓰며 SLUB는 메타데이터와 CPU별 큐 비용을 줄여 많은 처리기에서 개선한다고 설명한다.	(10.8, p.469-473 / PDF p.409-413)
U10-205	페이지 교체·할당뿐 아니라 프리페이징, 페이지 크기, TLB reach, 테이블 정보, 프로그램 구조 및 I/O 고정이 실제 성능과 정확성에 영향을 준다.	(10.9, p.473-479 / PDF p.413-419)
U10-206	프리페이징은 시작·재시작의 연속 폴트를 줄이려고 필요한 일부·전체 페이지를 미리 가져온다.	(10.9.1, p.473 / PDF p.413)
U10-207	이전 작업 집합을 기억해 재개 전에 올리는 방법이 있다.	(10.9.1, p.473 / PDF p.413)
U10-208	s개를 가져와 실제로 α만큼 쓰면 s×α개의 폴트를 줄이는 이익과 s×(1−α) 불필요 반입 비용을 비교한다. α가 0에 가까우면 불리하고 1에 가까우면 유리하다.	(10.9.1, p.473 / PDF p.413)
U10-209	실행 코드의 필요 페이지는 예측이 어렵지만 순차 파일은 더 확실하다고 원문은 설명한다.	(10.9.1, p.473 / PDF p.413)
U10-210	마지막 페이지 내부 단편화를 줄인다.	(10.9.2, p.473-475 / PDF p.413-415)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-211	페이지 수와 테이블 크기를 줄인다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-212	필요한 지역을 더 정밀하게 가져와 메모리·불필요 I/O를 줄일 수 있다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-213	HDD의 탐색·회전 비용을 분산하고 한 번에 더 많이 전송한다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-214	정확한 필요 부분을 선택한다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-215	페이지 폴트·인터럽트·상태 저장·테이블 갱신 횟수를 줄인다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-216	한 크기가 모든 시스템에 최적이지 않다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-217	원문은 페이지가 커지면 같은 양의 데이터 전송을 한 번에 처리할 수 있지만 필요하지 않은 주변도 가져오며, 작으면 메모리는 알차지만 많은 폴트 오버헤드가 든다고 비교한다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-218	기존 시스템에서는 하드웨어 때문에 크기를 바꾸기 어렵다는 조건도 중요하다.	(10.9.2, p.473-475 / PDF p.413-415)
U10-219	원문 수식: TLB reach = TLB 항목 수 × 페이지 크기; TLB reach: TLB로 직접 접근 가능한 메모리 공간의 크기; 작업 집합을 담을 수 있으면 좋다. 항목 수 증가는 연관 메모리의 가격·전력 비용이 있고 페이지 확대는 내부 단편화를 늘릴 수 있다.	(10.9.3, p.475-476 / PDF p.415-416)
U10-220	여러 페이지 크기·거대 페이지는 필요한 응용에서 reach를 높이는 대안이다.	(10.9.3, p.475-476 / PDF p.415-416)
U10-221	ARMv8의 contiguous 비트는 인접 블록을 하나의 TLB 항목으로 묶어 reach를 늘린다.	(10.9.3, p.475-476 / PDF p.415-416)
U10-222	여러 크기를 다루는 소프트웨어 관리가 성능 비용을 더할 수 있지만 높은 적중률로 상쇄할 수 있다고 원문은 설명한다.	(10.9.3, p.475-476 / PDF p.415-416)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-223	역 테이블은 물리 프레임의 현재 가상 페이지 정보만 유지하므로 메모리에 없는 페이지의 디스크 위치를 모른다.	(10.9.4, p.476 / PDF p.416)
U10-224	이를 위해 프로세스별 확장 페이지 테이블이 필요하다.	(10.9.4, p.476 / PDF p.416)
U10-225	이 테이블은 매 메모리 참조가 아닌 폴트 때만 쓰므로 자체를 페이징할 수 있다.	(10.9.4, p.476 / PDF p.416)
U10-226	폴트 처리 중 확장 테이블 때문에 다시 폴트가 날 수 있어 커널이 조심스럽게 처리해야 한다.	(10.9.4, p.476 / PDF p.416)
U10-227	자료·프로그램 구조는 지역성·작업 집합·폴트율에 영향을 준다.	(10.9.5, p.476-477 / PDF p.416-417)
U10-228	다음은 원문의 행 중심 128×128 배열과 128워드 페이지 비교이다.	(10.9.5, p.476-477 / PDF p.416-417)
U10-229	선언·변수·루프를 그대로 보존하며 제공되지 않은 프레임 수나 새 결과를 만들어 넣지 않았다.	(10.9.5, p.476-477 / PDF p.416-417)
U10-230	원문의 배열 순회: 열을 먼저 진행 (원문 코드 발췌) int i, j; int[128][128] data; for (j = 0; j < 128; j++) for (i = 0; i < 128; i++) data[i][j] = 0;	(10.9.5, p.477 / PDF p.417)
U10-231	원문의 배열 순회: 행을 먼저 진행 (원문 코드 발췌) int i, j; int[128][128] data; for (i = 0; i < 128; i++) for (j = 0; j < 128; j++) data[i][j] = 0;	(10.9.5, p.477 / PDF p.417)
U10-232	원문은 한쪽 끝을 쓰는 스택의 지역성은 높고 참조를 분산하는 해시 테이블은 낮다고 설명한다.	(10.9.5, p.476-477 / PDF p.416-417)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-233	단, 지역성만이 효율 기준은 아니며 탐색 속도·총 참조·페이지 접근 수도 본다.	(10.9.5, p.476-477 / PDF p.416-417)
U10-234	컴파일러의 코드·데이터 분리와 재진입 코드, 로더의 페이지 경계 내 루틴 배치·자주 호출하는 루틴 묶기도 도움이 된다.	(10.9.5, p.476-477 / PDF p.416-417)
U10-235	I/O 처리기가 사용자 버퍼의 물리 프레임에 전송하는 동안 다른 프로세스의 전역 교체로 그 프레임이 재할당되면 잘못된 메모리에 입출력할 수 있다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-236	원문은 두 방법을 제시한다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-237	시스템 버퍼에서만 I/O한 뒤 사용자 버퍼로 옮기면 안전하지만 이중 이동 비용이 든다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-238	또는 잠금 비트로 I/O 페이지를 교체 대상에서 빼고 완료 시 해제한다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-239	커널의 일부·전체나 데이터베이스의 특수 버퍼도 고정할 수 있다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-240	악용하면 메모리 관리에 부담을 주므로 원문은 응용의 특권을 요구한다고 설명한다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-241	새로 읽었지만 아직 한 번도 못 쓴 낮은 우선순위 프로세스 페이지를 첫 재실행까지 지키는 용도도 있다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-242	잠금은 언젠가 풀려야 하며 버그로 영구 유지되면 사용할 수 없는 프레임이 된다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-243	Solaris의 잠금 힌트는 메모리 상황에 따라 무시할 수 있다.	(10.9.6, p.478-479 / PDF p.418-419)
U10-244	다음은 제공된 원문이 기술하는 Linux·Windows10·Solaris의 가상 메모리 사례이다.	(10.10, p.480-483 / PDF p.420-423)
U10-245	현행 버전의 사실을 외부 자료로 보충하지 않는다.	(10.10, p.480-483 / PDF p.420-423)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-246	공통으로 요구 페이징·COW와 LRU 근사 클록의 변형을 사용하지만 회수 구조와 임계값 정책은 다르다.	(10.10, p.480-483 / PDF p.420-423)
U10-247	Linux	(10.10, p.480-483 / PDF p.420-423)
U10-248	active_list는 사용 중인 페이지, inactive_list는 최근 안 써 회수할 수 있는 페이지를 유지한다. accessed 비트·리스트 위치를 갱신하고 inactive의 재참조는 active로 옮긴다. kswapd가 주기적으로 가용량을 확인하여 부족하면 회수한다.	(10.10, p.480-483 / PDF p.420-423)
U10-249	Windows10	(10.10, p.480-483 / PDF p.420-423)
U10-250	인접 페이지까지 읽는 클러스터 요구 페이징과 작업 집합 한계를 사용한다. 지역·전역 LRU 근사를 조합하고 부족하면 자동 작업 집합 트리밍을 수행한다.	(10.10, p.480-483 / PDF p.420-423)
U10-251	Solaris	(10.10, p.480-483 / PDF p.420-423)
U10-252	lotsfree 임계값 아래에서 pageout이 2차 기회와 유사한 두 바늘 스캔을 한다. 앞 바늘은 참조 비트를 지우고 뒤 바늘은 여전히0인 페이지를 회수한다.	(10.10, p.480-483 / PDF p.420-423)
U10-253	Windows 원문의 데이터 클러스터는 3페이지, 나머지는 7페이지이며 작업 집합 초기 최소50·최대345를 제시한다.	(10.10, p.480-483 / PDF p.420-423)
U10-254	hard working-set limits가 없으면 여유 때 최대를 넘거나 부족 때 최소 아래로 갈 수 있다.	(10.10, p.480-483 / PDF p.420-423)
U10-255	여유 메모리가 부족할 때 최대에 도달한 프로세스는 자기 집합의 지역 교체를 하고 전역 트리밍은 큰 유휴 프로세스를 먼저 줄인다.	(10.10, p.480-483 / PDF p.420-423)
U10-256	Solaris는 scanrate·handspread로 두 바늘 간 검사 간격을 정한다.	(10.10, p.480-483 / PDF p.420-423)
U10-257	가용량에 따라 slowscan에서 fastscan으로 늘리고 desfree가 부족하면 더 자주 수행한다.	(10.10, p.480-483 / PDF p.420-423)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-258	계속 부족하면 프로세스 페이지를 회수하고 minfree도 못 채우면 새 요청마다 pageout을 수행한다.	(10.10, p.480-483 / PDF p.420-423)
U10-259	공유 라이브러리 페이지를 건너뛰고 일반 파일 페이지와 프로세스 페이지를 구별하는 우선순위 페이징도 원문에 나온다.	(10.10, p.480-483 / PDF p.420-423)
U10-260	가상 메모리의 유연성은 폴트·입출력·교체 비용 관리와 함께 이해해야 한다.	(10.11, p.483-488 / PDF p.423-428)
U10-261	교체의 미래·과거·참조 정보, 프로세스별 프레임 수, 지역성과 작업 집합을 연결하고 커널·I/O의 별도 요구를 남긴다.	(10.11, p.483-488 / PDF p.423-428)
U10-262	원문 연습 문제는 폴트 처리·비교 알고리즘·Belady 모순·요구 하드웨어·배열 순회·CPU와 디스크 이용률 등을 점검한다.	(10.11, p.483-488 / PDF p.423-428)
U10-263	별도 풀이와 새 수치는 추가하지 않았다.	(10.11, p.483-488 / PDF p.423-428)
U10-264	현재 메모리에 없는 페이지에 액세스하면 페이지 폴트가 발생하며, 백업 저장장치에서 가용 페이지 프레임으로 가져와야 한다.	(10.11, p.484 / PDF p.424)
U10-265	쓰기 시 복사는 자식과 부모가 동일 주소 공간을 공유하다가, 어느 쪽이든 페이지를 수정하면 페이지 복사본을 만드는 기법이다.	(10.11, p.484 / PDF p.424)
U10-266	스래싱은 시스템이 실행보다 페이징하는 데 더 많은 시간을 소비할 때 발생한다.	(10.11, p.484 / PDF p.424)
U10-267	지역성은 함께 사용되는 일련의 페이지를 나타낸다.	(10.11, p.484 / PDF p.424)
U10-268	작업 집합은 지역성에 기반을 두며 프로세스에서 현재 사용 중인 페이지 집합이다.	(10.11, p.484 / PDF p.424)
U10-269	TLB reach는 TLB에서 액세스할 수 있는 메모리 양이며, TLB 항목 수에 페이지 크기를 곱한 값이다.	(10.11, p.484 / PDF p.424)
U10-270	가상 메모리는 물리 메모리보다 큰 프로그램, 프로그램 일부만의 적재, 프로세스 메모리 공유와 효율적인 프로세스 생성을 가능하게 한다.	(10.11, p.483 / PDF p.423)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-271	페이지 교체 알고리즘에는 FIFO·최적·LRU가 있으며, 순수한 LRU는 구현하기에 비실용적이므로 대부분의 시스템은 LRU 근사를 사용한다.	(10.11, p.484 / PDF p.424)
U10-272	전역 교체는 시스템의 모든 프로세스를, 지역 교체는 폴트를 발생시킨 프로세스를 대상으로 한다.	(10.11, p.484 / PDF p.424)
U10-273	전역 교체	(10.11, p.484 / PDF p.424)
U10-274	전역 페이지 교체는 시스템의 모든 프로세스에서 교체할 페이지를 선택한다.	(10.11, p.484 / PDF p.424)
U10-275	지역 교체	(10.11, p.484 / PDF p.424)
U10-276	지역 페이지 교체는 폴트를 발생시킨 프로세스에서 페이지를 선택한다.	(10.11, p.484 / PDF p.424)
U10-277	TLB reach: TLB 항목 수 × 페이지 크기; TLB에서 액세스할 수 있는 메모리 양이다.	(10.11, p.484 / PDF p.424)

제11장. 대용량 저장장치 구조

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-001	하드 디스크 드라이브와 비휘발성 메모리 장치는 대부분의 컴퓨터에서 주요 보조저장장치 I/O 장치이며, 논리 블록의 1차원 배열로 구성된다.	(11.9, p.530 / PDF p.469)
U11-002	RAID 알고리즘은 드라이브 고장 시에도 지속적인 작업과 자동 복구를 가능하게 하며, 각 레벨은 신뢰성과 높은 전송 속도의 조합을 제공한다.	(11.9, p.531 / PDF p.470)
U11-003	대용량 저장장치는 비휘발성 정보를 보관한다.	(11.1, p.491-499 / PDF p.430-438)
U11-004	원문은 HDD와 NVM을 주요 보조저장장치로, 자기 테이프·광디스크·클라우드 저장장치 등을 느리고 큰 3차 저장장치의 예로 다룬다.	(11.1, p.491-499 / PDF p.430-438)
U11-005	여러 종류를 함께 지칭할 때 NVS(nonvolatile storage) 또는 드라이브라고 한다.	(11.1, p.491-499 / PDF p.430-438)
U11-006	물리 장치의 특성은 운영체제의 주소 매핑을 거쳐 논리 저장장치로 제공된다.	(11.1, p.491-499 / PDF p.430-438)
U11-007	HDD는 자기 물질로 덮인 원형 플래터에 정보를 기록하고, 각 표면 위의 읽기-쓰기 헤드가 자기 패턴을 감지한다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-008	헤드들은 디스크 암에 붙어 함께 이동한다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-009	원형 트랙은 다시 섹터로 나뉘며, 같은 암 위치의 트랙 집합이 실린더이다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-010	섹터는 크기가 고정된 최소 전송 단위이다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-011	전송률은 드라이브와 컴퓨터 사이의 데이터 흐름 속도이며, 위치 지정 시간(임의 접근 시간)은 원하는 실린더로 암을 옮기는 탐색 시간과 원하는 섹터가 헤드 아래까지 도는 회전 지연시간으로 구성된다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-012	컨트롤러의 DRAM 버퍼는 성능을 높인다.	(11.1.1, p.492-493 / PDF p.431-432)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-013	공표된 전송률은 매체에서 비트를 읽는 비율일 수 있으므로 운영체제에 블록이 실제 전달되는 전송률과 구별해야 한다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-014	헤드는 매우 얇은 공기나 가스층 위에서 움직인다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-015	표면에 닿는 헤드 충돌은 자기 표면을 손상시키며 일반적으로 수리할 수 없다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-016	드라이브 전체를 교체해야 할 수 있고, 다른 저장장치나 RAID로 보호하지 않은 데이터는 손실된다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-017	일부 섀시는 시스템 종료 없이 드라이브를 제거·교체할 수 있다.	(11.1.1, p.492-493 / PDF p.431-432)
U11-018	NVM 장치는 기계식이 아닌 전기식이다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-019	원문은 컨트롤러와 플래시 NAND 다이로 구성되는 장치에 초점을 맞춘다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-020	디스크와 유사한 외형이면 SSD라고 하며, USB 드라이브나 메모리 스틱 형태 또는 기판에 직접 실장하는 형태도 같은 원리로 취급한다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-021	움직이는 부분이 없어 HDD보다 안정적이고 탐색·회전 지연이 없으며 전력 소비도 적다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-022	반면 원문이 비교하는 시점에서는 메가바이트당 가격이 비싸고 대형 HDD보다 용량이 작다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-023	버스가 성능을 제한할 수 있어 시스템 버스에 직접 연결하기도 한다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-024	NAND는 페이지 단위로 읽고 쓰지만 기존 데이터를 그대로 덮어쓸 수 없다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-025	덮어쓰려면 먼저 지워야 하며, 지우기는 여러 페이지로 된 블록 단위이다.	(11.1.2, p.494-496 / PDF p.433-435)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-026	읽기가 가장 빠르고, 쓰기는 읽기보다 느리지만 지우기보다는 훨씬 빠르다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-027	여러 다이와 데이터 경로를 이용한 병렬 처리가 가능하다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-028	지우기마다 셀이 열화되므로 쓰기 마모가 수명을 제한한다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-029	DWPD(Drive Writes Per Day)는 보증 기간 동안 하루에 드라이브 전체 용량만큼 데이터를 몇 번 쓸 수 있는지를 나타낸다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-030	블록이 다시 기록되면 새 페이지가 현재 데이터를 보유하고 이전 페이지의 데이터는 무효가 된다. 플래시 변환 계층(FTL)은 유효한 논리 블록을 실제 페이지로 매핑하고 물리 블록 상태를 추적한다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-031	공간이 부족하면 유효하지 않은 페이지만 든 블록을 지운다. 유효·무효 페이지가 섞인 블록은 유효 데이터를 다른 위치로 복사한 뒤 지운다. 이것이 가비지 수집이다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-032	이 복사를 위한 여유 공간과 쓰기 성능을 확보하려고 장치는 과잉 공급 영역을 따로 둔다. 지운 블록은 장치 상태에 따라 과잉 공급 공간이나 가용 풀로 돌아간다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-033	삭제 횟수가 적은 블록에 데이터를 배치하여 후속 지우기를 분산한다. 이러한 마모 평준화는 일부 블록만 일찍 마모되어 전체 수명이 줄어드는 것을 막는다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-034	컨트롤러는 데이터와 함께 오류 수정 코드를 저장하고 읽을 때 검사한다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-035	수정 가능한 오류가 반복되는 페이지는 불량으로 표시하여 쓰지 않는다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-036	그래도 장치 전체 고장이나 데이터 오염이 가능하므로 RAID로 보호할 수 있다.	(11.1.2, p.494-496 / PDF p.433-435)
U11-037	이 알고리즘들은 대체로 컨트롤러 내부에서 구현되지만 그 동작에 따라 성능이 달라진다.	(11.1.2, p.494-496 / PDF p.433-435)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-038	RAM 드라이브는 시스템 DRAM의 일부를 장치 드라이버가 저장장치처럼 제공하는 것이다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-039	raw 블록 장치로도 쓸 수 있지만 보통 파일 시스템을 만들어 표준 파일 연산으로 사용한다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-040	캐시·버퍼가 운영체제나 프로그래머에 의해 할당되는 것과 달리 사용자는 파일을 생성·읽기·쓰기·삭제하는 방식으로 임시 데이터를 보관한다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-041	매우 빠른 임시 파일·프로그램 간 데이터 공유·부팅 중 임시 루트 파일 시스템에 유용하다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-042	휘발성이므로 크래시·종료·전원 차단 뒤에는 내용이 지속되지 않는다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-043	원문의 별도 설명에서 자기 테이프는 비휘발성과 큰 용량을 제공하지만 임의 위치를 찾는 데 오래 걸리므로 보조저장장치보다는 백업·잘 쓰지 않는 정보의 보관·시스템 간 전송에 적합하다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-044	위치를 찾은 뒤에는 HDD와 비슷한 전송 속도로 읽고 쓸 수 있다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-045	따라서 임의 위치 접근의 지연과 위치를 찾은 후의 순차 전송 속도를 구별해야 한다.	(11.1.3, p.496-498 / PDF p.435-437)
U11-046	보조저장장치는 시스템 버스나 I/O 버스에 연결된다.	(11.1.4, p.498 / PDF p.437)
U11-047	원문은 ATA·SATA·eSATA·SAS·USB·FC를 예로 들고, NVM에는 PCI 버스에 직접 연결하여 처리량을 높이고 지연을 줄이는 NVMe를 설명한다.	(11.1.4, p.498 / PDF p.437)
U11-048	호스트 컨트롤러(HBA)는 컴퓨터 쪽에서 버스 전송을 맡고, 장치 내부 컨트롤러는 실제 하드웨어를 작동시킨다.	(11.1.4, p.498 / PDF p.437)
U11-049	컴퓨터는 보통 메모리 매핑 I/O 포트로 호스트 컨트롤러에 명령을 놓는다.	(11.1.4, p.498 / PDF p.437)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-050	호스트 컨트롤러는 장치 컨트롤러에 명령을 전달하고, 장치 컨트롤러는 드라이브를 작동시킨다.	(11.1.4, p.498 / PDF p.437)
U11-051	매체와 내장 캐시 사이에서 데이터가 이동하고, 캐시와 호스트 DRAM 사이에는 DMA로 데이터가 전송된다.	(11.1.4, p.498 / PDF p.437)
U11-052	운영체제는 저장장치를 논리 블록의 1차원 배열로 다룬다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-053	논리 블록은 최소 전송 단위이며 물리 섹터나 반도체 페이지로 매핑된다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-054	실린더·헤드·섹터 또는 칩·블록·페이지의 조합보다 LBA(논리 블록 주소)가 다루기 쉽다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-055	현대 드라이브에서 LBA를 실제 위치로 단순 환산하기는 어렵다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-056	손상 섹터는 예비 섹터로 대체되고, 트랙당 섹터 수가 일정하지 않을 수 있으며, 제조업체가 내부 매핑을 관리하기 때문이다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-057	따라서 스케줄러는 보통 가까운 LBA가 가까운 물리 블록에 해당한다는 경향을 이용한다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-058	CLV(고정 선형 속도)	(11.1.5, p.498-499 / PDF p.437-438)
U11-059	트랙의 비트 밀도를 일정하게 유지한다. 바깥 트랙은 길어 더 많은 섹터를 담으며, 안쪽으로 갈수록 회전 속도를 높여 데이터 비율을 유지한다. CD-ROM·DVD-ROM의 예가 제시된다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-060	CAV(고정 각 속도)	(11.1.5, p.498-499 / PDF p.437-438)
U11-061	회전 속도를 일정하게 유지하며 안쪽에서 바깥쪽으로 갈수록 비트 밀도를 줄인다. 원문은 HDD에서 사용하는 방법으로 설명한다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-062	원문은 실린더를 구역으로 나누어 같은 구역 안에서는 트랙당 섹터 수를 일정하게 하고 구역 사이에서는 다르게 구성하는 방법도 설명한다.	(11.1.5, p.498-499 / PDF p.437-438)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-063	HDD와 NVM을 결합한 장치·볼륨 관리자는 일반 장치와 특성이 달라 캐싱과 스케줄링도 달라질 수 있다.	(11.1.5, p.498-499 / PDF p.437-438)
U11-064	HDD 관리의 목표는 접근 시간을 줄이고 전송 대역폭을 높이는 것이다.	(11.2, p.500 / PDF p.439)
U11-065	장치 대역폭은 첫 요청부터 마지막 전송 완료까지의 전체 시간으로 총 전송 바이트 수를 나눈 값이다.	(11.2, p.500 / PDF p.439)
U11-066	I/O 시스템 콜은 읽기·쓰기 여부, 열린 파일 핸들, 메모리 주소, 데이터 양을 전달한다.	(11.2, p.500 / PDF p.439)
U11-067	장치나 컨트롤러가 바쁘면 요청은 큐에 대기하고, 드라이버는 큐를 재정렬할 수 있다.	(11.2, p.500 / PDF p.439)
U11-068	현대 드라이브는 실제 헤드·트랙 위치를 호스트에 완전히 공개하지 않는다.	(11.2, p.500 / PDF p.439)
U11-069	따라서 디스크 스케줄링은 물리적 탐색 최소화뿐 아니라 공정성·적시성·순차 요청의 병합을 고려한다.	(11.2, p.500 / PDF p.439)
U11-070	특정 요청 집합의 최적 순서를 계산하는 비용이 얻는 이익보다 클 수도 있다.	(11.2, p.500 / PDF p.439)
U11-071	FCFS는 요청이 도착한 순서대로 처리한다.	(11.2.1, p.501 / PDF p.440)
U11-072	본질적으로 공평해 보이지만 빠른 서비스를 보장하지 않는다.	(11.2.1, p.501 / PDF p.440)
U11-073	원문 큐는 98, 183, 37, 122, 14, 124, 65, 67이고 시작 헤드는 53이다.	(11.2.1, p.501 / PDF p.440)
U11-074	이 순서 그대로 이동한 총 거리는 원문에 640 실린더로 제시된다. 122→14→124처럼 가까운 두 요청 사이를 크게 왕복하는 이동이 발생한다.	(11.2.1, p.501 / PDF p.440)
U11-075	그림 11.6 FCFS 디스크 스케줄링 — 원문 그림 부분 발췌	(11.2.1, p.501 / PDF p.440)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-076	SCAN은 한 방향으로 이동하면서 경로상의 요청을 모두 처리한 뒤 반대로 이동한다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-077	원문은 한쪽 끝 또는 그 방향에 더 이상 대기 요청이 없는 경우 방향을 바꾼다고 설명하며, 엘리베이터 알고리즘이라고도 한다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-078	현재 위치뿐 아니라 진행 방향도 알아야 한다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-079	같은 큐에서 53에서 0 방향으로 출발하면 37·14를 처리하고 0에서 반전하여 65·67·98·122·124·183을 처리한다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-080	헤드 앞에 새 요청이 생기면 진행하며 처리할 수 있지만 뒤에 생기면 돌아올 때까지 기다린다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-081	모든 실린더에 요청이 균일하게 도착한다고 가정하면 막 통과한 쪽 끝보다 오래 방문하지 않은 반대 끝에 대기 요청이 더 많다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-082	이 관찰이 C-SCAN의 출발점이다.	(11.2.2, p.501-502 / PDF p.440-441)
U11-083	C-SCAN은 대기 시간을 더 균등하게 하려는 SCAN의 변형이다.	(11.2.3, p.502-503 / PDF p.441-442)
U11-084	한 방향으로 움직이며 요청을 처리하지만 끝에 도달하면 반대 방향의 요청을 처리하며 돌아오지 않고 시작 쪽으로 돌아간다.	(11.2.3, p.502-503 / PDF p.441-442)
U11-085	실린더들을 마지막에서 처음으로 이어지는 순환 리스트처럼 취급한다.	(11.2.3, p.502-503 / PDF p.441-442)
U11-086	원문 예는 시작 53에서 199 방향으로 이동하여 큰 실린더의 요청들을 처리하고 0으로 돌아간 뒤 14·37을 처리한다.	(11.2.3, p.502-503 / PDF p.441-442)
U11-087	그림 11.8 C-SCAN 디스크 스케줄링 — 원문 그림 부분 발췌	(11.2.3, p.503 / PDF p.442)
U11-088	성능은 요청 수와 유형에 좌우된다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-089	큐에 요청이 하나뿐이면 모든 알고리즘이 사실상 FCFS처럼 동작한다.	(11.2.4, p.503-504 / PDF p.442-443)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-090	SCAN과 C-SCAN은 부하가 큰 시스템에서 기아 가능성을 줄이지만 완전히 없애지는 않는다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-091	원문의 Linux 마감시간 스케줄러는 읽기·쓰기마다 LBA 순서 큐와 FCFS 큐를 두어 총 네 큐를 유지한다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-092	읽을 때 프로세스가 봉쇄될 가능성이 더 높아 읽기에 우선순위를 준다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-093	LBA 순서로 배치를 처리한 뒤 FCFS 큐에서 오래 기다린 요청을 검사하고, 그런 요청이 있으면 그 요청이 든 LBA 큐를 다음 배치로 선택한다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-094	원문의 기본 대기 기준은 500ms이다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-095	원문에서 NOOP는 빠른 NVM을 쓰는 CPU 중심 시스템에 적합한 단순 스케줄러이고, CFQ는 실시간·최선 노력·유휴의 세 큐를 우선순위대로 관리한다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-096	CFQ는 이전 기록으로 같은 프로세스의 후속 I/O를 예측하여 기다림으로써 참조 지역성과 탐색 감소를 이용하지만 배타적 우선순위로 기아가 생길 수 있다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-097	이 설명은 책에서 제시한 배포판·버전의 사례이며 현재 설정으로 확대하지 않는다.	(11.2.4, p.503-504 / PDF p.442-443)
U11-098	NVM에는 움직이는 헤드가 없어 HDD의 이동 최소화 알고리즘의 이익이 작다.	(11.3, p.504-505 / PDF p.443-444)
U11-099	보통 간단한 FCFS를 쓰며 Linux NOOP는 인접 요청을 병합한다.	(11.3, p.504-505 / PDF p.443-444)
U11-100	읽기 서비스 시간은 일정하지만 플래시 쓰기 시간은 일정하지 않으므로 일부 SSD 스케줄러는 인접 쓰기만 병합하고 읽기는 FCFS로 처리한다.	(11.3, p.504-505 / PDF p.443-444)
U11-101	무작위 접근에서 HDD는 헤드 이동 때문에 제약을 받지만 NVM은 더 많은 IOPS를 처리할 수 있다.	(11.3, p.504-505 / PDF p.443-444)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-102	순차 전송에서는 헤드 탐색이 적어 NVM의 상대적 이득이 작아진다.	(11.3, p.504-505 / PDF p.443-444)
U11-103	NVM 쓰기는 읽기보다 느리며, 장치가 얼마나 찼는지와 얼마나 마모되었는지에 따라 성능이 달라진다.	(11.3, p.504-505 / PDF p.443-444)
U11-104	응용의 쓰기를 위해 새 페이지를 기록해야 하지만 유효·무효 데이터가 섞인 블록에서 공간 회수가 필요하다.	(11.3, p.504-505 / PDF p.443-444)
U11-105	유효 페이지를 읽어 과잉 공급 공간으로 복사하고, 무효 데이터만 남은 블록을 지운다.	(11.3, p.504-505 / PDF p.443-444)
U11-106	이 블록을 다시 여유 공간에 포함한다. 응용의 한 번 쓰기에 추가 읽기와 추가 쓰기가 생긴다.	(11.3, p.504-505 / PDF p.443-444)
U11-107	이처럼 가비지 수집과 공간 관리를 위해 장치 자체가 생성하는 추가 I/O가 쓰기 증폭이다.	(11.3, p.504-505 / PDF p.443-444)
U11-108	최악에는 매 쓰기마다 여러 I/O가 촉발된다.	(11.3, p.504-505 / PDF p.443-444)
U11-109	파일 삭제 사실을 장치에 알려 불필요한 블록을 지우게 하는 방법은 수명과 성능 개선에 도움이 된다.	(11.3, p.504-505 / PDF p.443-444)
U11-110	오류 감지는 기록·전송·보관 중 데이터가 변했는지를 알아내어 전파를 막거나 경고하는 것이다.	(11.4, p.505-506 / PDF p.444-445)
U11-111	바이트와 함께 패리티 비트를 저장하면 모든 단일 비트 오류를 감지하지만 이중 비트 오류는 놓칠 수 있다.	(11.4, p.505-506 / PDF p.444-445)
U11-112	패리티는 XOR로 계산할 수 있고 추가 저장 공간이 필요하다.	(11.4, p.505-506 / PDF p.444-445)
U11-113	체크섬은 계산한 값을 저장·비교하는 방식이며 CRC도 다중 비트 오류를 감지하는 방법으로 제시된다.	(11.4, p.505-506 / PDF p.444-445)
U11-114	ECC(오류 수정 코드)는 감지뿐 아니라 일부 오류를 보정할 만큼 정보를 추가로 저장한다.	(11.4, p.505-506 / PDF p.444-445)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-115	디스크는 섹터별, 플래시는 페이지별 ECC를 사용한다.	(11.4, p.505-506 / PDF p.444-445)
U11-116	기록할 때 값을 계산해 저장하고 읽을 때 다시 계산하여 비교한다.	(11.4, p.505-506 / PDF p.444-445)
U11-117	손상 비트가 적어 수정할 수 있으면 소프트 오류이고, ECC로 수정할 수 없으면 하드 오류이다.	(11.4, p.505-506 / PDF p.444-445)
U11-118	가능한 수정 범위는 알고리즘과 저장한 추가 정보의 양에 좌우된다.	(11.4, p.505-506 / PDF p.444-445)
U11-119	컨트롤러가 통상적인 읽기·쓰기 때 이 처리를 자동 수행한다.	(11.4, p.505-506 / PDF p.444-445)
U11-120	저수준(물리적) 포매팅은 섹터·페이지를 컨트롤러가 읽고 쓸 수 있는 구조로 초기화한다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-121	헤더·데이터 영역·트레일러가 마련되며 번호와 오류 탐지·수정 정보가 저장된다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-122	NVM은 페이지 초기화와 FTL 생성이 필요하다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-123	HDD는 대체로 공장에서 포맷되어 나온다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-124	섹터가 커지면 헤더·트레일러 비중이 줄지만 운영체제가 지원하는 섹터 크기는 제한될 수 있다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-125	파티션: 블록·페이지 그룹을 나누어 별도 장치처럼 취급한다. 파티션 정보는 정해진 형식과 위치에 기록된다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-126	볼륨 생성·관리: 파티션 하나를 바로 볼륨으로 쓰거나 여러 장치·파티션을 RAID 등으로 묶는다. 마운트 가능한 파일 시스템이나 파일 시스템을 담은 이미지 파일도 볼륨이라고 할 수 있다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-127	논리적 포매팅: 가용·할당 공간의 맵과 빈 디렉터리 등 초기 파일 시스템 자료구조를 기록한다.	(11.5.1, p.506-508 / PDF p.445-447)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-128	마운트하면 파일 시스템을 사용할 수 있다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-129	Windows는 C:·D:처럼 볼륨을 구분하고 Linux는 루트에서 시작하는 트리 내부에 다른 파일 시스템을 마운트한다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-130	파일 시스템은 블록을 클러스터로 묶어 순차 I/O를 늘리고 파일 데이터와 메타데이터를 가까이 배치하여 탐색을 줄인다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-131	raw I/O는 파티션을 논리 블록의 배열로 직접 쓰며 버퍼 캐시·잠금·선반입·공간 할당·이름·디렉터리 같은 파일 시스템 서비스를 우회한다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-132	데이터베이스나 스왑처럼 전용 관리를 원하는 경우 유용하지만 대부분의 응용은 파일 시스템을 사용한다.	(11.5.1, p.506-508 / PDF p.445-447)
U11-133	펌웨어의 작은 부트스트랩 로더가 CPU 레지스터·컨트롤러·메모리 등을 초기화한다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-134	아직 전체 장치 드라이버가 없을 때에도 고정 위치의 부트 블록을 읽을 만큼의 기능으로 더 큰 부트스트랩 프로그램을 적재한다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-135	부트스트랩 프로그램이 커널을 찾아 시작하고, 커널이 서브시스템과 서비스를 적재한다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-136	원문의 Windows 예에서는 첫 논리 블록·페이지인 MBR에 부트 코드, 파티션 표, 부트 파티션 표시가 있다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-137	펌웨어가 MBR을 읽고 부트 파티션의 첫 섹터·페이지를 읽으면 그 부트 섹터가 커널로 안내한다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-138	부트 파티션이 있는 장치를 부트 디스크 또는 시스템 디스크라고 한다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-139	펌웨어는 갱신 가능하지만 악성 수정으로 감염될 수도 있다는 주의가 제시된다.	(11.5.2, p.508-509 / PDF p.447-448)
U11-140	단순한 방식은 포맷 중이나 검사 프로그램으로 손상 블록을 찾아 파일 시스템이 할당하지 않게 표시하는 것이다.	(11.5.3, p.509-511 / PDF p.448-450)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-141	컨트롤러가 관리하는 방식에서는 손상 목록과 운영체제에 보이지 않는 예비 섹터를 유지하고 손상 섹터를 교체한다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-142	이것이 섹터 예비/포워딩이다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-143	논리 주소는 같지만 실제 위치가 달라져 스케줄링 최적화를 약화시킬 수 있어 같은 실린더의 예비 섹터를 우선 사용할 수 있다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-144	섹터 밀어내기는 손상 위치부터 예비 섹터까지 여러 섹터를 옮겨 논리적 순서를 유지하는 방법이다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-145	원문은 논리 블록 17의 결함과 202 뒤의 예비 섹터를 이용해 202부터 차례로 뒤로 복사하는 예를 든다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-146	복구 가능한 소프트 오류는 사본을 만든 뒤 교체할 수 있지만 하드 오류의 데이터는 백업 등의 개입이 필요하다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-147	NVM은 탐색 위치를 고려할 필요가 없어 예비 페이지·과잉 공급 영역과 불량 페이지 표로 관리한다.	(11.5.3, p.509-511 / PDF p.448-450)
U11-148	가상 메모리는 보조저장장치를 메모리의 확장으로 쓰지만 훨씬 느리므로 스왑 설계가 성능에 크게 작용한다.	(11.6, p.511-513 / PDF p.450-452)
U11-149	원문은 초기의 프로세스 전체 스와핑과 현대의 페이지 단위 스와핑을 구별한다.	(11.6, p.511-513 / PDF p.450-452)
U11-150	스왑 공간의 크기는 메모리 관리 방식·물리/가상 메모리 크기에 좌우된다.	(11.6, p.511-513 / PDF p.450-452)
U11-151	부족하면 프로세스 중단이나 시스템 크래시가 가능하고, 너무 크면 일반 파일 공간을 줄인다.	(11.6, p.511-513 / PDF p.450-452)
U11-152	여러 장치의 스왑 영역으로 I/O 부하를 분산할 수도 있다.	(11.6, p.511-513 / PDF p.450-452)
U11-153	파일 시스템의 스왑 파일	(11.6, p.511-513 / PDF p.450-452)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-154	일반 파일 루틴으로 생성·삭제·관리하기 편하다.	(11.6, p.511-513 / PDF p.450-452)
U11-155	파일 시스템을 거친다.	(11.6, p.511-513 / PDF p.450-452)
U11-156	전용 raw 파티션	(11.6, p.511-513 / PDF p.450-452)
U11-157	파일·디렉터리를 저장하지 않고 속도에 초점을 맞춘 스왑 루틴으로 관리한다. 내용이 짧게 유지되고 부팅 때 초기화된다.	(11.6, p.511-513 / PDF p.450-452)
U11-158	고정 크기를 늘리려면 재파티션하거나 다른 스왑 공간을 추가해야 한다.	(11.6, p.511-513 / PDF p.450-452)
U11-159	11.6.3의 Solaris 예는 실행 파일의 코드 페이지를 교체할 때 스왑에 쓰지 않고 버린 뒤 필요하면 파일 시스템에서 다시 읽는다.	(11.6, p.511-513 / PDF p.450-452)
U11-160	스왑은 파일로 뒷받침되지 않는 스택·힙·미초기화 데이터 같은 익명 메모리의 백업에 쓴다.	(11.6, p.511-513 / PDF p.450-452)
U11-161	이후 Solaris는 페이지 생성 시점이 아니라 실제 퇴출 시점에 스왑 공간을 할당한다.	(11.6, p.511-513 / PDF p.450-452)
U11-162	Linux도 익명 메모리에 스왑을 사용하며 파일이나 전용 파티션의 여러 영역을 허용한다.	(11.6, p.511-513 / PDF p.450-452)
U11-163	원문 예의 각 영역은 연속된 4KB 페이지 슬롯으로 구성되고 스왑 맵의 정수 카운터가 슬롯 사용을 나타낸다. 0이면 가용, 양수이면 사용 중이며 값은 그 스왑 페이지를 공유하는 수이다.	(11.6, p.511-513 / PDF p.450-452)
U11-164	원문 그림의 카운터 3은 세 프로세스가 같은 페이지를 쓰는 경우이다.	(11.6, p.511-513 / PDF p.450-452)
U11-165	호스트 연결	(11.7, p.513-517 / PDF p.452-456)
U11-166	로컬 I/O 포트로 특정 장치의 논리 블록을 읽고 쓴다. USB·SATA·FC 등의 연결이 사용된다.	(11.7, p.513-517 / PDF p.452-456)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-167	NAS	(11.7, p.513-517 / PDF p.452-456)
U11-168	NFS·CIFS 등의 RPC로 네트워크상의 파일을 공유한다. 로컬 파일 시스템 같은 이름·접근을 제공하지만 데이터 네트워크 대역폭과 성능 비용이 있다.	(11.7, p.513-517 / PDF p.452-456)
U11-169	iSCSI	(11.7, p.513-517 / PDF p.452-456)
U11-170	IP 네트워크로 SCSI 명령과 논리 블록을 전달한다. 파일 시스템을 제공하는 NFS·CIFS와 달리 클라이언트가 raw 블록을 쓰거나 파일 시스템을 만든다.	(11.7, p.513-517 / PDF p.452-456)
U11-171	클라우드 저장장치	(11.7, p.513-517 / PDF p.452-456)
U11-172	WAN·인터넷을 거쳐 원격 데이터 센터의 저장장치에 접근하며 대체로 API를 사용한다. LAN용 NAS보다 지연·연결 장애의 가능성을 고려한다.	(11.7, p.513-517 / PDF p.452-456)
U11-173	SAN	(11.7, p.513-517 / PDF p.452-456)
U11-174	서버와 저장장치 유닛을 저장장치 프로토콜로 연결하는 사유 네트워크이다. 동적 할당·공유가 유연하고 일반 데이터 네트워크와의 경쟁을 줄인다.	(11.7, p.513-517 / PDF p.452-456)
U11-175	NAS의 NFS·CIFS는 파일 공유를 위해 잠금 기능을 제공한다.	(11.7, p.513-517 / PDF p.452-456)
U11-176	SAN 스위치는 호스트의 접근을 허용·금지하고 여러 호스트에 저장장치를 배정한다.	(11.7, p.513-517 / PDF p.452-456)
U11-177	SAN은 포트가 많아 비용이 크지만 근거리 연결이므로 NAS가 더 많은 호스트를 연결할 수도 있다.	(11.7, p.513-517 / PDF p.452-456)
U11-178	저장장치 배열은 드라이브·컨트롤러·캐시·관리 소프트웨어를 갖춘 독립 장치이며 RAID·스냅숏·복제·압축·중복 제거·암호화 등을 구현할 수 있다.	(11.7, p.513-517 / PDF p.452-456)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-179	SSD만 쓰거나 HDD와 조합하여 용도에 맞게 배치·캐싱할 수도 있다.	(11.7, p.513-517 / PDF p.452-456)
U11-180	RAID는 여러 드라이브를 이용하여 성능과 신뢰성을 높이는 구성이다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-181	원문은 I가 과거 inexpensive에서 independent로 해석되는 추세도 소개한다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-182	드라이브 수만 늘리고 데이터 사본은 하나만 유지하면 전체의 고장 가능성이 커진다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-183	중복 정보는 보통 불필요하지만 고장 시 분실 정보를 재구성하는 데 쓰인다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-184	원문 수식: 100,000 / 100 = 1,000시간; ; 원문의 단일 드라이브 MTBF가 100,000시간인 100개 드라이브 집합의 고장 간 평균 시간 예이다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-185	미러링은 하나의 논리 드라이브를 두 물리 드라이브로 만들고 모든 쓰기를 양쪽에 수행한다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-186	하나가 고장 나면 나머지에서 읽지만, 복구 전에 두 번째도 고장 나면 데이터가 손실된다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-187	신뢰성은 드라이브 MTBF뿐 아니라 교체·복구의 평균 시간에도 좌우된다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-188	원문 수식: $100{,}000^2 / (2 \times 10) = 500 \times 10^6$시간; ; 드라이브 고장이 독립적이고 단일 MTBF가 100,000시간, 평균 수리 시간이 10시간이라는 원문의 가정에서 얻는 평균 데이터 분실 시간이다. 원문은 약 57,000년으로 표현한다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-189	이 독립성은 현실에서 보장되지 않는다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-190	전력·재해·제조 결함은 두 드라이브에 함께 영향을 주고, 노후화·복구 중 부하도 실패 가능성을 바꾼다.	(11.8.1, p.517-519 / PDF p.456-458)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-191	두 미러 블록을 동시에 쓰다가 전원이 끊기면 둘 다 불완전할 수 있다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-192	순서대로 쓰거나 보호된 NVRAM 쓰기 지연 캐시를 두어 완료할 수 있게 하는 대응이 제시된다.	(11.8.1, p.517-519 / PDF p.456-458)
U11-193	미러의 두 드라이브로 읽기 요청을 분산하면 개별 읽기의 전송률 자체와 별도로 여러 읽기 처리 능력을 높일 수 있다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-194	스트라이핑은 데이터를 드라이브에 나누어 저장한다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-195	비트 수준 예는 바이트의 각 비트를 여러 드라이브에 나누어 같은 시간에 더 많은 데이터를 읽는 것이다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-196	블록 수준에서는 블록을 여러 드라이브에 순환 배치하며 이것이 일반적으로 사용된다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-197	원문 수식: (i mod n) + 1; ; 원문은 n개 드라이브에서 파일의 i번째 블록을 배치하는 드라이브 번호를 이 형태로 제시한다. 원문의 인덱스 표현을 보존한다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-198	병렬성의 두 목적은 여러 작은 접근의 처리량을 부하 균등화로 높이는 것과 큰 접근의 응답 시간을 줄이는 것이다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-199	스트라이핑 자체는 중복이 아니므로 신뢰성을 높이지 않는다.	(11.8.2, p.519-520 / PDF p.458-459)
U11-200	0	(11.8.3, p.520-525 / PDF p.459-464)
U11-201	중복 없는 블록 수준 스트라이핑	(11.8.3, p.520-525 / PDF p.459-464)
U11-202	전송률은 높이지만 오류 복구용 중복 정보가 없다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-203	1	(11.8.3, p.520-525 / PDF p.459-464)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-204	미러링	(11.8.3, p.520-525 / PDF p.459-464)
U11-205	높은 신뢰성과 간단한 복구 대신 드라이브 사본이 필요하다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-206	4	(11.8.3, p.520-525 / PDF p.459-464)
U11-207	블록 스트라이핑과 전용 패리티 드라이브	(11.8.3, p.520-525 / PDF p.459-464)
U11-208	고장 섹터를 식별할 수 있을 때 나머지 데이터와 패리티로 복구한다. 작은 쓰기는 데이터와 패리티의 읽기-수정-쓰기가 필요하다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-209	5	(11.8.3, p.520-525 / PDF p.459-464)
U11-210	데이터·패리티를 모든 드라이브에 분산	(11.8.3, p.520-525 / PDF p.459-464)
U11-211	RAID 4의 전용 패리티 드라이브 집중을 줄인다. 같은 블록과 그 패리티는 동일 드라이브에 두지 않는다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-212	6	(11.8.3, p.520-525 / PDF p.459-464)
U11-213	P+Q의 두 중복 정보	(11.8.3, p.520-525 / PDF p.459-464)
U11-214	두 드라이브 고장을 허용한다. 같은 XOR 결과를 두 번 저장하는 것만으로는 추가 보호가 되지 않는다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-215	다차원 6	(11.8.3, p.520-525 / PDF p.459-464)
U11-216	논리적 행·열 방향에 RAID 6 적용	(11.8.3, p.520-525 / PDF p.459-464)
U11-217	여러 위치의 중복 정보를 이용한다. 그림은 단순화를 위해 전용 패리티 드라이브로 보이지만 실제 블록은 분산된다.	(11.8.3, p.520-525 / PDF p.459-464)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-218	0+1	(11.8.3, p.520-525 / PDF p.459-464)
U11-219	스트라이프 집합을 다시 미러링	(11.8.3, p.520-525 / PDF p.459-464)
U11-220	단일 고장으로 한 스트라이프가 사용 불가능해질 수 있다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-221	1+0	(11.8.3, p.520-525 / PDF p.459-464)
U11-222	미러 쌍들을 스트라이핑	(11.8.3, p.520-525 / PDF p.459-464)
U11-223	한 드라이브 고장 뒤에도 해당 미러와 다른 쌍이 계속 사용될 수 있다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-224	그림 11.15 RAID 레벨 — 원문 그림 부분 발췌	(11.8.3, p.521 / PDF p.460)
U11-225	RAID 4의 작은 쓰기는 이전 데이터·패리티 읽기 두 번과 새 데이터·패리티 쓰기 두 번, 총 네 드라이브 접근을 요구한다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-226	XOR 계산·기록도 비용이다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-227	전용 패리티 하드웨어와 NVRAM 캐시는 CPU 부담을 줄이고 전체 스트라이프가 모이면 병렬 기록하여 읽기-수정-쓰기를 줄일 수 있다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-228	WAFL은 0으로 초기화한 드라이브를 추가해도 패리티가 바뀌지 않는 RAID 4의 확장 특성을 이용한다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-229	그림 11.16 RAID 0+1과 1+0의 단일 장애 — 원문 그림 부분 발췌	(11.8.3, p.524 / PDF p.463)
U11-230	RAID는 커널·볼륨 관리 소프트웨어, HBA, 저장장치 배열, SAN 내부 연결 계층에서 구현할 수 있다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-231	위치에 따라 비용·유연성·공유 범위가 다르다.	(11.8.3, p.520-525 / PDF p.459-464)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-232	스냅숏은 갱신 전 파일 시스템 모습이고 복제는 다른 위치에 같은 내용을 자동 기록하는 것이다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-233	동기 복제는 로컬·원격 기록 뒤 완료하고, 비동기 복제는 묶어서 기록하므로 빠르고 거리에 덜 제한되지만 주 사이트 고장 시 미전달 데이터가 손실될 수 있다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-234	hot spare는 데이터를 담지 않고 대기하다 고장 드라이브를 자동 대체하여 복구 지연을 줄인다.	(11.8.3, p.520-525 / PDF p.459-464)
U11-235	선택 기준에는 정상 성능뿐 아니라 고장 후 데이터를 계속 공급할 능력과 복구 시간·비용·공간 오버헤드가 포함된다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-236	RAID 1은 복사로 복구하여 단순하고, 패리티 방식은 다른 드라이브에 접근해 재구성한다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-237	큰 RAID 5의 복구는 오래 걸릴 수 있다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-238	레벨 0은 데이터를 다시 적재할 수 있는 고성능 용도, 레벨 1은 빠른 복구, 0+1·1+0은 성능과 신뢰성이 함께 중요한 용도의 예로 제시된다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-239	패리티 하나로 더 많은 데이터를 보호하면 공간 비용은 줄지만 복구 전 추가 고장에 노출될 수 있다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-240	드라이브 수를 늘리는 것도 전송률과 비용을 함께 바꾼다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-241	별도 사례인 InServ는 드라이브를 256MB chunklet으로 나누어 서로 다른 볼륨과 RAID 레벨에 쓸 수 있게 한다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-242	논리적으로 큰 공간을 먼저 제공하고 실제 쓰임에 따라 물리 저장장치를 할당하는 방식과 쓰기 시 복사 스냅숏도 설명된다.	(11.8.4, p.525-527 / PDF p.464-466)
U11-243	RAID의 중복·분산 개념은 테이프와 무선 데이터 전송에도 적용된다.	(11.8.5, p.527 / PDF p.466)
U11-244	여러 테이프에 스트라이핑하면 처리량을 높이고 백업 시간을 줄일 수 있다.	(11.8.5, p.527 / PDF p.466)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-245	데이터 조각과 패리티를 함께 전송하면 일부 조각을 수신하지 못해도 다른 조각에서 복구할 수 있다.	(11.8.5, p.527 / PDF p.466)
U11-246	RAID는 물리 매체 오류를 보호하지만 모든 데이터 손상을 막지는 않는다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-247	잘못된 포인터, 미완료 쓰기(torn writes), 잘못된 쓰기, 컨트롤러 고장, RAID 소프트웨어 버그는 여전히 위험하다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-248	원문 ZFS는 데이터뿐 아니라 메타데이터도 체크섬으로 검사하며, 체크섬을 해당 블록 자체가 아니라 그 블록을 가리키는 포인터와 함께 둔다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-249	미러에 올바른 사본이 있으면 손상 블록을 갱신할 수 있다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-250	그림 11.17 ZFS의 메타데이터·데이터 체크섬 — 원문 그림 부분 발췌	(11.8.6, p.528 / PDF p.467)
U11-251	별도 볼륨·파일 시스템 크기는 수요가 변할 때 부적절해질 수 있고, 볼륨을 바꿔도 파일 시스템이 확장·축소를 지원하지 않을 수 있다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-252	ZFS는 파일 시스템 관리와 볼륨 관리를 통합하고 RAID 드라이브를 저장장치 풀로 묶어 여러 파일 시스템에 필요에 따라 할당·반환한다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-253	쿼터는 파일 시스템 크기를 제한하고 예약은 지정 크기까지 확장을 보장한다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-254	이 변수들은 변경 가능하다.	(11.8.6, p.527-529 / PDF p.466-468)
U11-255	풀에 객체를 생성하고 객체 ID를 받는다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-256	필요할 때 객체 ID로 접근한다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-257	객체 ID로 삭제한다.	(11.8.7, p.529-530 / PDF p.468-469)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-258	객체 저장소는 일반 파일 시스템처럼 사용자가 디렉터리를 탐색하는 대신 프로그램이 ID로 접근하도록 설계된다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-259	관리 소프트웨어가 위치와 복제를 결정한다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-260	원문의 HDFS 예는 서로 다른 컴퓨터에 여러 사본을 두고 사본이 있는 컴퓨터에서 계산한 결과를 전송한다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-261	다른 컴퓨터의 데이터 접근에는 네트워크가 필요하여 고속 임의 접근보다는 대량 저장에 사용된다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-262	컴퓨터와 디스크를 더해 풀을 늘리는 수평 확장성과 내용의 설명을 포함하는 자체 설명성이 특징이다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-263	원문은 콘텐츠를 기반으로 검색할 수 있어 콘텐츠 주소지정가능 저장소라고도 하며 형식이 정해지지 않은 구조화되지 않은 데이터를 저장한다고 설명한다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-264	이는 원문의 객체 저장소 설명 범위이다.	(11.8.7, p.529-530 / PDF p.468-469)
U11-265	원문 요약은 논리 블록·연결 방식·스케줄링·오류 처리·파티션/볼륨·스왑·RAID·객체 저장소를 다시 연결한다.	(11.9, p.530-534 / PDF p.469-473)
U11-266	장말 연습문제(인쇄532~533쪽)는 스케줄링의 적용 조건, 회전 지연, 장치 간 I/O 균형, 코드 페이지와 스왑, 안정 저장장치, 전송 크기와 유효 전송률, RAID 읽기, HDD/NVM 선택을 점검한다.	(11.9, p.530-534 / PDF p.469-473)
U11-267	원문에 제시되지 않은 풀이·계산 결과를 추가하지 않는다.	(11.9, p.530-534 / PDF p.469-473)
U11-268	추가 자료와 참고문헌은 존재를 확인했으며 외부 내용을 요약에 보충하지 않았다.	(11.9, p.530-534 / PDF p.469-473)
U11-269	미러링(mirroring)은 모든 드라이브의 복사본을 만드는 중복 도입 방법이다.	(11.8.1, p.518 / PDF p.457)
U11-270	데이터 스트라이핑(data striping)은 여러 드라이브에 걸쳐 데이터를 나누어 저장하여 전송 비율을 향상하는 방법이다.	(11.8.2, p.519 / PDF p.458)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-271	오류 감지는 저장·전송 문제를 발견하여 시스템에 보고하고 수정 조치를 하게 하며 오류 전파를 방지한다.	(11.9, p.531 / PDF p.470)
U11-272	오류 수정은 사용 가능한 수정 데이터 양과 손상된 데이터 양에 따라 문제를 감지하고 복구한다.	(11.9, p.531 / PDF p.470)
U11-273	객체는 파일 이름이 아닌 객체 ID로 지정되는 자체 정의 데이터 모음이다.	(11.9, p.531 / PDF p.470)
U11-274	MTBF(mean time between failure)는 고장 사이의 평균 시간이다.	(11.8.1, p.518 / PDF p.457)
U11-275	디스크 스케줄링은 HDD의 유효 대역폭·평균 응답 시간·응답 시간 편차를 향상할 수 있으며, SCAN과 C-SCAN은 디스크 큐 순서를 위한 전략이다.	(11.9, p.531 / PDF p.470)
U11-276	저장장치는 하나 이상의 공간으로 파티션되고, 각 파티션은 볼륨을 보유하거나 다중 장치 볼륨의 일부가 되며, 파일 시스템은 볼륨 안에 생성된다.	(11.9, p.531 / PDF p.470)
U11-277	오류 감지는 문제를 발견·보고하여 전파를 막고, 오류 수정은 수정 데이터와 손상 데이터의 양에 따라 문제를 감지·복구한다.	(11.9, p.531 / PDF p.470)
U11-278	오류 감지	(11.9, p.531 / PDF p.470)
U11-279	오류 수정	(11.9, p.531 / PDF p.470)
U11-280	원문의 미러드 디스크 평균 데이터 분실 시간: $100{,}000^2/(2\times10)=500\times10^6$시간, 57,000년; 단일 MTBF 100,000시간·평균 수리 10시간·두 고장의 독립성을 가정한 예이며, 원문은 실제 고장을 독립적이라 가정할 수 없다는 점도 지적한다.	(11.8.1, p.519 / PDF p.458)

제12장. 입출력 시스템

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-001	입출력의 가장 기본적인 하드웨어 장치는 버스·장치 컨트롤러·장치이다.	(12.8, p.574 / PDF p.513)
U12-002	장치와 메모리 사이의 자료 이동은 CPU의 프로그램된 입출력이나 DMA 컨트롤러에 의해 이루어진다.	(12.8, p.574 / PDF p.513)
U12-003	운영체제는 다양한 속도·기능의 입출력 장치를 관리·제어한다.	(12.1, p.535-536 / PDF p.474-475)
U12-004	인터페이스 표준화는 새 장치의 결합을 쉽게 하지만 장치의 다양성도 계속 커진다.	(12.1, p.535-536 / PDF p.474-475)
U12-005	포트·버스·컨트롤러라는 하드웨어 요소와 장치 드라이버라는 소프트웨어 모듈이 이 문제를 다룬다.	(12.1, p.535-536 / PDF p.474-475)
U12-006	드라이버는 구체적인 장치 차이를 감추고 상위 커널 I/O 서브시스템에 일관된 인터페이스를 제공한다.	(12.1, p.535-536 / PDF p.474-475)
U12-007	포트는 장치가 연결되는 접점이다.	(12.2, p.536-538 / PDF p.475-477)
U12-008	버스는 여러 장치가 공유하는 선과 메시지를 교환하는 프로토콜을 포함한다.	(12.2, p.536-538 / PDF p.475-477)
U12-009	장치가 차례로 연결되어 포트로 이어지는 데이지 체인은 통상 버스처럼 동작한다.	(12.2, p.536-538 / PDF p.475-477)
U12-010	컨트롤러는 포트·버스·장치를 동작시키는 전자장치이며, 단순 칩부터 프로세서·마이크로코드·메모리를 갖춘 HBA까지 다양하다.	(12.2, p.536-538 / PDF p.475-477)
U12-011	원문의 PC 구조는 프로세서·메모리와 고속 장치를 PCI 계열 버스로 연결하고 느린 장치는 확장 버스에 연결한다.	(12.2, p.536-538 / PDF p.475-477)
U12-012	PCIe는 수신·송신 신호 쌍으로 구성된 레인을 사용하여 양방향 전송을 제공한다.	(12.2, p.536-538 / PDF p.475-477)
U12-013	디스크 자체의 컨트롤러는 프로토콜뿐 아니라 불량 섹터 매핑·선반입·버퍼링·캐싱도 담당한다.	(12.2, p.536-538 / PDF p.475-477)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-014	CPU는 컨트롤러 레지스터에 비트 패턴을 읽고 써서 I/O를 수행한다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-015	특수 I/O 명령으로 포트를 지정하는 방법과, 장치 제어 레지스터를 프로세서 주소 공간에 대응시키는 메모리 맵드 I/O를 구별한다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-016	후자는 보통의 데이터 전송 명령으로 레지스터를 접근한다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-017	그래픽 출력의 큰 매핑 영역처럼 많은 데이터를 다룰 때 유용하다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-018	입력(data-in)	(12.2.1, p.538-539 / PDF p.477-478)
U12-019	입력 데이터를 읽는다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-020	출력(data-out)	(12.2.1, p.538-539 / PDF p.477-478)
U12-021	출력할 데이터를 쓴다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-022	상태(status)	(12.2.1, p.538-539 / PDF p.477-478)
U12-023	명령 완료·입력 준비·오류 여부를 읽는다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-024	제어(control)	(12.2.1, p.538-539 / PDF p.477-478)
U12-025	명령을 내리거나 통신 방식·검사·속도 등의 모드를 지정한다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-026	데이터 레지스터가 작더라도 FIFO 칩으로 버퍼를 확장하여 돌발적으로 많은 데이터가 들어오는 상황에 대응할 수 있다.	(12.2.1, p.538-539 / PDF p.477-478)
U12-027	호스트는 busy 비트가 소거될 때까지 검사한다.	(12.2.2, p.539-540 / PDF p.478-479)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-028	명령 레지스터의 쓰기 비트를 설정하고 data-out에 출력 바이트를 쓴다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-029	command-ready 비트를 설정한다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-030	컨트롤러는 command-ready를 알아차리고 busy를 설정한다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-031	컨트롤러는 쓰기 명령과 데이터를 읽어 장치로 출력한다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-032	완료하면 command-ready·오류·busy 비트를 소거한다. 이 과정은 바이트마다 반복된다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-033	첫 단계의 반복 검사가 폴링(바쁜 대기)이다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-034	기본 검사 자체는 짧지만 준비까지 오래 걸리면 CPU가 다른 유용한 일을 못 한다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-035	반대로 입력 장치를 너무 늦게 서비스하면 버퍼가 넘쳐 데이터를 잃을 수 있다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-036	상태가 바뀌었을 때 장치가 CPU에 알리는 인터럽트는 긴 반복 대기를 피하는 방법이다.	(12.2.2, p.539-540 / PDF p.478-479)
U12-037	드라이버가 I/O를 시작하고 CPU는 다른 명령을 수행한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-038	컨트롤러가 입력 준비·출력 완료·오류를 알리는 인터럽트 요청을 보낸다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-039	CPU는 상태를 저장하고 인터럽트 핸들러로 제어를 옮긴다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-040	핸들러가 원인을 조사하여 장치를 서비스하고 복귀 명령으로 이전 실행 상태를 회복한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-041	그림 12.3 인터럽트 구동식 입출력 사이클 — 원문 그림 부분 발췌	(12.2.3, p.541 / PDF p.480)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-042	현대 시스템은 인터럽트의 연기, 발생 장치 식별, 여러 우선순위, 예외·트랩 처리를 지원한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-043	마스크 불가 인터럽트는 회복 불가능한 메모리 오류 등의 긴급 상황에, 마스크 가능 인터럽트는 필요하면 일시 중단할 수 있는 장치 요청에 사용된다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-044	인터럽트 벡터는 핸들러 주소 테이블을 선택하고, 여러 장치가 한 항목을 공유하면 사슬의 핸들러를 차례로 조사한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-045	큰 테이블과 탐색 비용 사이의 절충이다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-046	우선순위는 높은 인터럽트가 낮은 인터럽트 처리를 선점하게 한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-047	시간·자원이 제한된 처리기를 1차와 2차로 나누어 1차는 상태 저장·문맥 교환·큐 삽입 등을, 별도로 스케줄되는 2차는 나머지 작업을 수행할 수 있다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-048	디스크 예에서 다음 I/O 시작은 급하지만 사용자 버퍼로 복사하는 작업은 덜 급하므로 우선순위를 달리한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-049	커널 스레드와 우선순위로 이를 구현하면 다중 처리기에서 병렬 핸들링도 가능하다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-050	페이지 폴트는 현재 프로세스를 대기시키고 페이지 입력을 스케줄한 뒤 다른 프로세스를 실행하게 한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-051	입력 완료 인터럽트는 대기 프로세스를 준비 큐로 옮긴다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-052	시스템 콜의 트랩은 인자를 마련하고 사용자 상태를 저장한 뒤 커널 모드의 서비스를 실행하는 경로이다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-053	원문은 장치 인터럽트와 하드웨어 오류·시스템 콜을 모두 커널 루틴을 촉발하는 용도로 연결한다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-054	인터럽트 처리에도 상태 저장·복귀 등의 비용이 있다.	(12.2.3, p.540-545 / PDF p.479-484)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-055	I/O 발생률이 낮을 때 인터럽트를 쓰다가 매우 높아져 폴링이 더 효율적이면 전환하는 혼합 방식도 있다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-056	따라서 인터럽트가 언제나 폴링보다 빠르다고 단정하지 않는다.	(12.2.3, p.540-545 / PDF p.479-484)
U12-057	CPU가 상태를 검사하며 바이트마다 직접 전송하는 PIO는 대용량 장치에 CPU 비용이 크다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-058	DMA는 이 전송을 특수 컨트롤러에 맡긴다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-059	호스트는 출발·도착 포인터와 바이트 수를 담은 명령 블록을 메모리에 쓰고 그 주소를 DMA에 알려준다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-060	비연속 주소 목록을 사용하는 분산-수집 방식은 한 명령으로 여러 전송을 수행한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-061	장치 컨트롤러가 DMA-request로 전송을 요청한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-062	DMA 컨트롤러가 메모리 버스를 확보하고 주소를 놓은 뒤 DMA-acknowledge를 보낸다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-063	장치가 워드를 전송하고 요청 신호를 제거한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-064	전체 전송이 끝나면 DMA가 CPU에 인터럽트로 완료를 알린다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-065	그림 12.6 DMA 전송의 단계 — 원문 그림 부분 발췌	(12.2.4, p.546 / PDF p.485)
U12-066	DMA가 버스를 쓰는 동안 CPU는 캐시에는 접근할 수 있어도 주메모리 접근이 잠시 제한된다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-067	이 사이클 스틸링은 CPU를 늦추지만 전체 I/O 성능을 개선한다.	(12.2.4, p.545-547 / PDF p.484-486)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-068	커널 버퍼를 거치면 사용자 공간으로 다시 복사하는 이중 버퍼링 비용이 있고, 직접 사용자 공간을 이용하면 전송 도중 수정에 주의해야 한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-069	DVMA는 가상 주소로 메모리 맵드 장치 사이의 전송을 지원한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-070	일반 프로세스의 직접 I/O는 성능상 이익이 있어도 보안·안정성을 해칠 수 있어 범용 운영체제는 메모리와 장치를 보호한다.	(12.2.4, p.545-547 / PDF p.484-486)
U12-071	컨트롤러·포트·레지스터가 장치와 통신하는 기반이다.	(12.2.5, p.547-548 / PDF p.486-487)
U12-072	호스트와 컨트롤러는 핸드셰이킹하고, 준비 상태는 폴링 또는 인터럽트로 확인하며, 큰 전송은 DMA로 위임한다.	(12.2.5, p.547-548 / PDF p.486-487)
U12-073	이 하드웨어 차이를 상위의 일관된 인터페이스에 어떻게 수용하는지가 다음 절의 주제이다.	(12.2.5, p.547-548 / PDF p.486-487)
U12-074	추상화·캡슐화·계층화로 장치를 대표 범주와 표준 함수 집합으로 표현한다.	(12.3, p.548-551 / PDF p.487-490)
U12-075	장치 드라이버가 이 함수를 구체적으로 구현하므로 상위 I/O 서브시스템은 하드웨어 차이에 덜 의존한다.	(12.3, p.548-551 / PDF p.487-490)
U12-076	새 하드웨어는 기존 컨트롤러 인터페이스를 구현하거나 새 드라이버를 제공할 수 있다.	(12.3, p.548-551 / PDF p.487-490)
U12-077	다만 드라이버 인터페이스는 운영체제마다 달라 각 운영체제용 드라이버가 필요하다.	(12.3, p.548-551 / PDF p.487-490)
U12-078	전송 단위	(12.3, p.548-551 / PDF p.487-490)
U12-079	문자 스트림 / 블록	(12.3, p.548-551 / PDF p.487-490)
U12-080	접근 순서	(12.3, p.548-551 / PDF p.487-490)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-081	순차 / 임의 접근	(12.3, p.548-551 / PDF p.487-490)
U12-082	다른 사건과의 조율	(12.3, p.548-551 / PDF p.487-490)
U12-083	동기식 / 비동기식 장치	(12.3, p.548-551 / PDF p.487-490)
U12-084	사용자 수	(12.3, p.548-551 / PDF p.487-490)
U12-085	공유 가능 / 전용	(12.3, p.548-551 / PDF p.487-490)
U12-086	속도	(12.3, p.548-551 / PDF p.487-490)
U12-087	지연·탐색 시간·전송률·연산 간 지연	(12.3, p.548-551 / PDF p.487-490)
U12-088	입출력 방향	(12.3, p.548-551 / PDF p.487-490)
U12-089	읽기·쓰기 / 읽기 전용 / 한 번 쓰기	(12.3, p.548-551 / PDF p.487-490)
U12-090	운영체제는 블록·문자 스트림·메모리 맵드 파일·네트워크 소켓·클록/타이머 등의 범주를 제공한다.	(12.3, p.548-551 / PDF p.487-490)
U12-091	범주에 없는 고유 명령은 UNIX의 ioctl() 같은 호출로 전달할 수 있다.	(12.3, p.548-551 / PDF p.487-490)
U12-092	원문의 세 인자는 장치 식별자, 명령을 나타내는 정수, 임의 자료구조를 가리키는 포인터이다.	(12.3, p.548-551 / PDF p.487-490)
U12-093	UNIX/Linux의 주(major) 번호는 드라이버·장치 유형을, 부(minor) 번호는 구체적 인스턴스를 선택한다.	(12.3, p.548-551 / PDF p.487-490)
U12-094	블록 인터페이스는 read·write·seek로 블록 장치를 제어하며 응용은 보통 파일 시스템을 거쳐 사용한다.	(12.3.1, p.551-552 / PDF p.490-491)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-095	자체 버퍼·잠금을 쓰는 데이터베이스에는 운영체제의 중복 버퍼링·잠금이 부담이나 충돌이 될 수 있다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-096	raw 접근은 장치 제어를 응용에 맡기고, direct I/O는 파일을 유지하되 버퍼링·잠금을 하지 않는 절충으로 소개된다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-097	메모리 맵드 파일은 파일을 가상 주소에 대응시켜 메모리 읽기·쓰기로 접근한다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-098	해당 주소가 참조될 때 데이터가 실제로 메모리에 올라와 요구 페이징을 이용한다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-099	실행 파일 적재와 스왑 접근에도 이러한 인터페이스가 쓰인다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-100	문자 장치는 put·get처럼 한 문자씩 전송하고 상위 라이브러리가 줄 단위·편집·버퍼링을 제공할 수 있다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-101	키보드·마우스·모뎀이나 선형 바이트 스트림 출력 장치에 적합하다.	(12.3.1, p.551-552 / PDF p.490-491)
U12-102	네트워크는 디스크의 read-write-seek와 다른 소켓 인터페이스를 사용한다.	(12.3.2, p.552 / PDF p.491)
U12-103	소켓 생성, 원격 주소와 연결, 연결 완료 확인, 패킷 교환을 제공하여 하드웨어·프로토콜 내부를 감춘다.	(12.3.2, p.552 / PDF p.491)
U12-104	select()는 여러 소켓 가운데 수신할 데이터나 송신 여유가 있는 소켓을 알려 주어 응용의 바쁜 대기를 피한다.	(12.3.2, p.552 / PDF p.491)
U12-105	원문은 FIFO·STREAMS·메시지 큐·소켓 등 다양한 IPC 방식의 존재도 설명한다.	(12.3.2, p.552 / PDF p.491)
U12-106	클록·타이머는 현재 시각, 지난 시간, 지정 시각의 연산 실행을 제공한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-107	프로그램 가능 인터벌 타이머는 한 번 또는 주기적으로 인터럽트를 만들며 타임 슬라이스 종료, 더티 버퍼 기록, 네트워크 시간 제한 등에 쓰인다.	(12.3.3, p.553-554 / PDF p.492-493)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-108	소프트웨어 가상 클록은 제한된 하드웨어 채널로 더 많은 요청을 지원한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-109	요청을 마감시간순으로 정렬한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-110	하드웨어 타이머를 가장 이른 마감시간으로 설정한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-111	인터럽트가 발생하면 요청자에게 알리고 다음 마감시간으로 다시 설정한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-112	타이머 해상도에는 하드웨어뿐 아니라 가상 클록 유지 비용도 작용한다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-113	시스템 시각 오차는 NTP 같은 프로토콜로 보정할 수 있다는 것이 원문의 설명이다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-114	읽을 수 있는 고정밀 카운터는 정확한 간격 측정에 사용되지만 반드시 인터럽트 생성에도 쓰이는 것은 아니다.	(12.3.3, p.553-554 / PDF p.492-493)
U12-115	봉쇄형(blocking)	(12.3.4, p.554-556 / PDF p.493-495)
U12-116	호출 스레드를 대기 큐로 옮기고 I/O가 끝나면 실행 가능한 상태로 돌린다. 코드 작성이 간단하다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-117	비봉쇄형(nonblocking)	(12.3.4, p.554-556 / PDF p.493-495)
U12-118	오래 멈추지 않고 그 시점에 가능한 데이터 전송량을 반환한다. 전체 또는 일부 데이터일 수 있다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-119	비동기식(asynchronous)	(12.3.4, p.554-556 / PDF p.493-495)
U12-120	요청을 제출한 뒤 즉시 돌아와 다른 일을 한다. 나중에 전체 작업 완료를 변수·시그널·소프트웨어 인터럽트·콜백 등으로 알린다.	(12.3.4, p.554-556 / PDF p.493-495)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-121	원문은 비봉쇄형 읽기가 지금 얻을 수 있는 데이터를 반환하는 반면 비동기식 읽기는 나중에 완전한 데이터를 제공하도록 요청한다는 차이를 강조한다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-122	여러 스레드를 사용하면 일부는 봉쇄형 호출로 기다리고 나머지는 연산할 수도 있다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-123	select()에 대기시간 0을 주면 봉쇄 없이 상태만 검사하지만 실제 전송에는 뒤이은 읽기·쓰기가 필요하다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-124	운영체제 내부에서도 쓰기나 네트워크 전송을 버퍼링한 뒤 먼저 응용에 복귀할 수 있다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-125	미기록 요청은 크래시 때 손실될 수 있어 기록 시간 제한과 명시적 플러시 방법을 둔다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-126	읽기는 아직 매체에 쓰지 않은 버퍼 내용과도 일관되어야 하며, 같은 파일을 여러 스레드가 접근하면 구현에 따라 잠금 프로토콜이 필요하다.	(12.3.4, p.554-556 / PDF p.493-495)
U12-127	벡터형 I/O는 한 시스템 콜로 여러 버퍼에 대한 작업을 수행하는 분산-수집 인터페이스이다.	(12.3.5, p.556 / PDF p.495)
U12-128	원문의 UNIX readv 예처럼 여러 개별 호출과 중간 대형 버퍼 복사를 줄여 문맥 교환·시스템 콜 비용을 낮춘다.	(12.3.5, p.556 / PDF p.495)
U12-129	일부 구현은 전체 작업의 원자성을 보장하여 같은 버퍼의 동시 I/O에 의한 오염을 막는다.	(12.3.5, p.556 / PDF p.495)
U12-130	원자성은 모든 버전에 무조건 있다고 확대하지 않는다.	(12.3.5, p.556 / PDF p.495)
U12-131	I/O 스케줄링은 요청 실행 순서를 정한다.	(12.4.1, p.556-557 / PDF p.495-496)
U12-132	장치별 대기 큐를 재정렬하여 성능·공정성·평균 응답 시간을 개선한다.	(12.4.1, p.556-557 / PDF p.495-496)
U12-133	원문 예는 헤드가 시작 부분에 있고 요청들이 끝·시작·중간 순서로 들어오면 2→3→1로 바꾸는 것이다.	(12.4.1, p.556-557 / PDF p.495-496)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-134	긴급한 페이징 요청이 응용 I/O보다 우선할 수도 있다.	(12.4.1, p.556-557 / PDF p.495-496)
U12-135	비동기 요청은 동시에 여러 개가 진행되므로 장치 상태 테이블의 종류·주소·상태와 해당 큐를 함께 추적한다.	(12.4.1, p.556-557 / PDF p.495-496)
U12-136	버퍼는 두 장치 또는 장치와 프로그램 사이에서 전송 중인 데이터를 임시 저장하는 메모리이다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-137	첫째 목적은 생산자·소비자의 속도 차이를 완충하는 것이다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-138	이중 버퍼링에서는 한 버퍼를 장치에 기록하는 동안 다른 버퍼를 수신 데이터로 채워 두 동작의 타이밍을 분리한다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-139	둘째는 전송 크기 차이이다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-140	메시지를 작은 패킷으로 나누거나 수신 패킷을 조합할 때 버퍼를 쓴다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-141	셋째는 복제 시맨틱이다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-142	write() 이후 응용이 자기 버퍼를 수정해도 매체에는 호출 시점의 버전이 기록되어야 한다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-143	간단한 구현은 응용에 복귀하기 전에 커널 버퍼로 복사하고 그 사본을 기록하는 것이다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-144	복사 비용이 있지만 의미가 간단하다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-145	원문은 가상 메모리 매핑과 쓰기 시 복사 보호로 비슷한 효과를 더 효율적으로 얻을 수도 있다고 설명한다.	(12.4.2, p.557-559 / PDF p.496-498)
U12-146	캐시는 자주 사용할 자료의 사본을 빠른 저장 공간에 두는 것이다.	(12.4.3, p.559 / PDF p.498)
U12-147	원문의 구분에서 버퍼는 전송 데이터의 유일한 보관 장소일 수 있고 캐시는 다른 곳의 원본에 대한 사본이다.	(12.4.3, p.559 / PDF p.498)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-148	기능은 다르지만 한 메모리 영역이 두 역할을 겸할 수 있다.	(12.4.3, p.559 / PDF p.498)
U12-149	파일 I/O는 먼저 버퍼 캐시를 검사하여 실제 I/O를 피하거나 미룰 수 있고, 지연된 쓰기를 모아 큰 단위로 스케줄할 수 있다.	(12.4.3, p.559 / PDF p.498)
U12-150	스풀은 프린터처럼 여러 작업을 뒤섞어 출력할 수 없는 장치의 출력 버퍼이다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-151	프로그램별 출력을 별도 보조저장장치 파일에 모으고 완성된 작업을 큐에 넣어 하나씩 출력한다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-152	디먼이나 커널 스레드가 관리하며 작업 열거·취소·중지 기능을 제공한다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-153	전용 장치는 한 프로그램에 할당하고 명시적으로 반납하게 하거나 열린 핸들을 하나로 제한할 수도 있다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-154	프로세스 간 협조적 배타 접근을 지원하는 방식도 있다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-155	이러한 장치 예약에서 생길 수 있는 교착 상태는 원문의 해당 시스템 예에서 응용의 책임이다.	(12.4.4, p.559-560 / PDF p.498-499)
U12-156	일시적 오류는 읽기·전송 재시도 등으로 복구할 수 있지만 핵심 장치의 영구 고장은 완전히 극복하기 어렵다.	(12.4.5, p.560-561 / PDF p.499-500)
U12-157	시스템 콜은 성공·실패와 추가 오류 정보를 반환하며 UNIX errno가 예로 제시된다.	(12.4.5, p.560-561 / PDF p.499-500)
U12-158	SCSI의 sense key·sense code·qualifier·오류 로그는 더 세부적인 원인을 표현하지만 운영체제가 모든 하드웨어 정보를 응용에 전달하는 것은 아니다.	(12.4.5, p.560-561 / PDF p.499-500)
U12-159	입출력 명령을 특권 명령으로 두어 사용자가 직접 실행하지 못하게 하고, 시스템 콜을 받은 운영체제가 요청의 유효성을 검사한다.	(12.4.6, p.561-562 / PDF p.500-501)
U12-160	메모리 맵드 I/O·포트 영역도 메모리 보호 대상이다.	(12.4.6, p.561-562 / PDF p.500-501)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-161	다만 그래픽 성능처럼 직접 접근을 허용할 필요가 있으면 커널은 영역을 적절히 할당하고 잠금으로 중복 사용을 제어해야 한다.	(12.4.6, p.561-562 / PDF p.500-501)
U12-162	커널은 열린 파일·네트워크 연결·문자 통신 등의 상태를 자료구조로 관리한다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-163	UNIX의 파일 인터페이스는 일반 파일·raw 장치·프로세스 주소 공간을 모두 read()로 접근하게 하지만 실제 처리는 다르다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-164	열린 파일 레코드의 디스패치 테이블은 객체 유형에 맞는 읽기·쓰기·select·ioctl·close 함수를 가리킨다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-165	원문은 독립적인 I/O 관리자와 메시지 전달 방식도 비교한다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-166	메시지 횟수가 늘어 비용이 생기지만 커널을 작게 하고 구조·설계의 유연성을 높일 수 있다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-167	이 비교는 원문이 설명하는 구현 사례의 범위에서 이해한다.	(12.4.7, p.562-563 / PDF p.501-502)
U12-168	운영체제는 부하에 맞춰 프로세스를 이동시키거나 장치·CPU 코어를 유휴 상태로 만들어 전력·발열·냉각 비용을 줄인다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-169	일시 중단한 코어의 상태는 저장하고 재개 시 복원한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-170	모바일 운영체제에서는 배터리 수명이 특히 중요하다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-171	전원 축소(power collapse)	(12.4.8, p.563-565 / PDF p.502-504)
U12-172	개별 구성요소를 끄고 CPU를 깊은 슬립 상태로 둔다. 외부 자극·인터럽트에는 반응하여 빠르게 재개한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-173	구성요소 수준 전원 관리	(12.4.8, p.563-565 / PDF p.502-504)
U12-174	장치 트리의 부모·자식 관계와 드라이버의 사용 여부를 추적한다. 자식들이 모두 유휴이면 버스 등 상위 구성요소도 끌 수 있다.	(12.4.8, p.563-565 / PDF p.502-504)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-175	wakelock	(12.4.8, p.563-565 / PDF p.502-504)
U12-176	필요한 동안 응용이 획득하여 전원 축소 진입을 막고 작업이 끝나면 해제한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-177	전체 시스템은 부팅 때뿐 아니라 실행 중 장치 추가·제거와 상태·오류·전력 변화를 관리해야 한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-178	ACPI는 커널이 호출할 수 있는 하드웨어 관리 루틴을 제공한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-179	원문의 정지 예에서는 커널→장치 드라이버→ACPI 루틴→장치의 순서로 지시한다.	(12.4.8, p.563-565 / PDF p.502-504)
U12-180	커널 I/O 서브시스템은 이름·접근 제어·연산 허용 여부·파일 공간·장치 할당·버퍼/캐시/스풀·스케줄링·상태 감시·오류/복구·드라이버 초기화·전원 관리를 조정한다.	(12.4.9, p.565 / PDF p.504)
U12-181	상위 계층은 드라이버가 제공하는 공통 인터페이스로 장치를 사용한다.	(12.4.9, p.565 / PDF p.504)
U12-182	파일 이름은 파일 시스템의 디렉터리와 할당 정보로 연결된다.	(12.5, p.566-568 / PDF p.505-507)
U12-183	FAT는 이름을 할당 테이블 항목과 블록으로, UNIX는 이름을 inode와 할당 정보로 매핑한다.	(12.5, p.566-568 / PDF p.505-507)
U12-184	UNIX에서는 마운트 테이블의 가장 긴 일치 접두어가 장치를 정하고, 주 장치 번호는 드라이버를, 부 장치 번호는 장치 테이블 항목을 선택하며, 여기에서 컨트롤러 포트·메모리 주소를 얻는다.	(12.5, p.566-568 / PDF p.505-507)
U12-185	여러 검색 테이블은 유연성과 동적 드라이버 적재를 제공하지만 잠금·오류 처리도 복잡해진다.	(12.5, p.566-568 / PDF p.505-507)
U12-186	열린 파일의 디스크립터로 read를 요청하고 커널이 인자를 검사한다.	(12.5, p.566-568 / PDF p.505-507)
U12-187	버퍼 캐시에 데이터가 있으면 바로 반환한다. 없으면 프로세스를 대기시키고 I/O를 스케줄한다.	(12.5, p.566-568 / PDF p.505-507)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-188	드라이버가 커널 버퍼를 확보하고 컨트롤러 레지스터에 명령을 기록한다.	(12.5, p.566-568 / PDF p.505-507)
U12-189	장치가 데이터를 전송한다. 원문의 가정에서는 DMA가 완료 인터럽트를 발생시킨다.	(12.5, p.566-568 / PDF p.505-507)
U12-190	벡터를 통해 핸들러가 드라이버에 알리고 드라이버는 완료 요청을 식별하여 I/O 서브시스템에 통지한다.	(12.5, p.566-568 / PDF p.505-507)
U12-191	커널이 사용자 버퍼로 데이터를 옮기고 프로세스를 준비 큐로 옮긴다.	(12.5, p.566-568 / PDF p.505-507)
U12-192	스케줄러가 CPU를 할당하면 프로세스가 시스템 콜에서 복귀한다.	(12.5, p.566-568 / PDF p.505-507)
U12-193	그림 12.14 입출력 요청의 생애 주기 — 원문 그림 부분 발췌	(12.5, p.567 / PDF p.506)
U12-194	STREAMS는 응용이 드라이버 코드의 파이프라인을 동적으로 조립하는 UNIX의 기법이다.	(12.6, p.568-570 / PDF p.507-509)
U12-195	스트림은 사용자 프로세스와 장치 드라이버 사이의 완전 양방향 연결이며, 스트림 헤드·0개 이상의 모듈·드라이버 엔드로 구성된다.	(12.6, p.568-570 / PDF p.507-509)
U12-196	각 요소에는 읽기·쓰기 큐가 있고 메시지 전달로 데이터를 교환한다.	(12.6, p.568-570 / PDF p.507-509)
U12-197	ioctl()로 모듈을 삽입할 수 있어 편집·네트워크 처리 등을 재사용한다.	(12.6, p.568-570 / PDF p.507-509)
U12-198	그림 12.15 STREAMS 구조 — 원문 그림 부분 발췌	(12.6, p.569 / PDF p.508)
U12-199	흐름 제어를 사용하는 큐는 메시지를 버퍼링하고 여유가 없으면 새 메시지를 거절하여 인접 큐의 오버플로를 막는다.	(12.6, p.568-570 / PDF p.507-509)
U12-200	write/read는 바이트 스트림을, putmsg/getmsg는 메시지를 응용에 제공한다.	(12.6, p.568-570 / PDF p.507-509)
U12-201	내부는 비동기적으로 동작하지만 프로세스가 헤드에 쓸 공간이나 읽을 데이터가 없으면 봉쇄된다.	(12.6, p.568-570 / PDF p.507-509)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-202	드라이버 엔드는 외부 인터럽트에 응답해야 하므로 기다릴 수 없고 장치 버퍼가 차면 들어오는 메시지를 버릴 수 있다.	(12.6, p.568-570 / PDF p.507-509)
U12-203	메시지 경계·제어 정보와 모듈 재사용이 원문의 장점이다.	(12.6, p.568-570 / PDF p.507-509)
U12-204	I/O 비용은 장치 시간에만 있지 않다.	(12.7, p.570-574 / PDF p.509-513)
U12-205	드라이버 실행·봉쇄/해제·문맥 교환·인터럽트·캐시 영향·장치/메모리/커널/사용자 사이의 복사가 CPU와 메모리 버스에 부하를 준다.	(12.7, p.570-574 / PDF p.509-513)
U12-206	원격 로그인에서 문자 하나를 전달하고 다시 반향하는 예는 작은 데이터도 여러 계층과 문맥 교환을 거칠 수 있음을 보인다.	(12.7, p.570-574 / PDF p.509-513)
U12-207	프런트 엔드 처리기와 I/O 채널은 이 부담을 주 CPU에서 덜어 준다.	(12.7, p.570-574 / PDF p.509-513)
U12-208	문맥 교환·복사·인터럽트 빈도를 줄인다.	(12.7, p.570-574 / PDF p.509-513)
U12-209	큰 전송·지능형 컨트롤러를 사용하거나, 바쁜 대기를 최소화할 수 있다면 폴링을 지혜롭게 사용한다.	(12.7, p.570-574 / PDF p.509-513)
U12-210	DMA·채널로 CPU 부담을 낮추고 I/O와 연산을 중첩한다.	(12.7, p.570-574 / PDF p.509-513)
U12-211	원시 연산을 장치 하드웨어에서 병렬 수행하고 CPU·메모리·버스·I/O 부하가 균형을 이루게 한다.	(12.7, p.570-574 / PDF p.509-513)
U12-212	응용 코드	(12.7, p.570-574 / PDF p.509-513)
U12-213	신축적이며 실험·디버깅이 쉽고 버그가 시스템 전체를 중단시킬 가능성이 작다.	(12.7, p.570-574 / PDF p.509-513)
U12-214	문맥 교환 비용, 커널 내부 서비스 이용의 제약	(12.7, p.570-574 / PDF p.509-513)
U12-215	커널	(12.7, p.570-574 / PDF p.509-513)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-216	성능을 높이고 커널 서비스와 결합할 수 있다.	(12.7, p.570-574 / PDF p.509-513)
U12-217	커널이 커지고 복잡해지며 오류가 크래시·데이터 손상으로 이어질 수 있다.	(12.7, p.570-574 / PDF p.509-513)
U12-218	장치·컨트롤러 하드웨어	(12.7, p.570-574 / PDF p.509-513)
U12-219	가장 높은 성능을 지향한다.	(12.7, p.570-574 / PDF p.509-513)
U12-220	개발이 오래 걸리고 수정·개선·부하에 따른 정책 변경이 어렵다.	(12.7, p.570-574 / PDF p.509-513)
U12-221	원문의 저장장치 성능 그림은 용량과 지연시간이라는 서로 다른 축에서 캐시·DRAM·NVM·SSD·HDD와 네트워크 지연을 비교한다.	(12.7, p.570-574 / PDF p.509-513)
U12-222	큰 용량이 낮은 지연을 뜻하지 않으며 네트워크를 통한 접근은 추가 지연을 가진다는 관계를 읽는다.	(12.7, p.570-574 / PDF p.509-513)
U12-223	수치가 아닌 도표상의 관계 범위에서 정리한다.	(12.7, p.570-574 / PDF p.509-513)
U12-224	장말 문제는 기능을 컨트롤러에 둘 때의 장단점, 핸드셰이킹 비트, 폴링·인터럽트·혼합 전략, DMA의 병행성, CPU/버스/장치 속도의 균형, STREAMS의 엔드와 모듈을 점검한다.	(12.8, p.574-576 / PDF p.513-515)
U12-225	새 프로토콜·풀이를 만들지 않는다.	(12.8, p.574-576 / PDF p.513-515)
U12-226	추가 자료·참고문헌의 범위는 확인했으나 외부 설명을 가져오지 않았다.	(12.8, p.574-576 / PDF p.513-515)
U12-227	봉쇄형	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)
U12-228	완료를 기다려 결과를 받는다.	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-229	비봉쇄형	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)
U12-230	지금 가능한 전송량을 반환한다.	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)
U12-231	비동기식	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)
U12-232	먼저 돌아온 후 전체 요청 완료를 별도로 알린다.	(봉쇄형·비봉쇄형·비동기식 I/O, p.554-556 / PDF p.493-495)
U12-233	버퍼	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-234	전송 중 임시 보관, 속도·크기 차이와 복제 시맨틱 지원	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-235	캐시	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-236	다른 곳에 있는 데이터의 빠른 사본으로 재사용	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-237	스풀	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-238	섞어 처리할 수 없는 장치에 작업별 출력을 모아 순차 전달	(버퍼·캐시·스풀, p.557-560 / PDF p.496-499)
U12-239	폴링(polling)은 호스트가 루프를 돌면서 비지 비트가 소거될 때까지 검사를 반복하는 바쁜 대기이다.	(12.2.2, p.540 / PDF p.479)
U12-240	DMA는 CPU의 PIO 작업 일부를 특수 프로세서인 DMA 컨트롤러에 위임하여 CPU의 일을 줄인다.	(12.2.4, p.546 / PDF p.485)
U12-241	버퍼는 두 장치 또는 장치와 응용 프로그램 사이에 데이터가 전송되는 동안 전송 데이터를 임시로 저장하는 메모리 영역이다.	(12.4.2, p.557 / PDF p.496)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-242	캐시는 자주 사용될 자료의 복사본을 저장하는 빠른 메모리 영역이다.	(12.4.3, p.559 / PDF p.498)
U12-243	스풀(spool)은 인터리브하게 동작할 수 없는 프린터 같은 장치를 위해 출력 데이터를 보관하는 버퍼이다.	(12.4.4, p.559 / PDF p.498)
U12-244	봉쇄형 시스템 콜을 하면 호출 스레드는 봉쇄 상태로 들어가며, 입출력이 끝나면 실행 큐로 되돌아와 실행을 재개할 수 있다.	(12.3.4, p.554 / PDF p.493)
U12-245	비봉쇄형 호출은 스레드를 오래 멈춰 두지 않고 즉각 복귀하며, 그 시점에서 가져올 수 있는 데이터를 반환한다.	(12.3.4, p.554 / PDF p.493)
U12-246	비동기식 호출은 즉각 복귀하여 코드를 계속 수행하게 하며, 입출력이 완료되면 운영체제가 완료 사실을 알려준다.	(12.3.4, p.554 / PDF p.493)
U12-247	커널 입출력 서브시스템은 입출력 스케줄·버퍼링·스풀링·오류 처리·장치 예약과 이름 변환 등의 서비스를 제공한다.	(12.8, p.574 / PDF p.513)
U12-248	STREAMS는 장치 드라이버와 네트워크 프로토콜 작성을 위한 모듈화된 프레임워크를 제공하며, 쌓인 층을 따라 데이터가 순차적으로 양방향 전송된다.	(12.8, p.574-575 / PDF p.513-514)
U12-249	버퍼는 전송 중 데이터를 임시 저장하고, 캐시는 자주 사용될 자료의 복사본을 빠른 메모리에 저장한다.	(12.4.2, p.557,559 / PDF p.496,498)
U12-250	비봉쇄형 읽기는 현재 가져올 수 있는 데이터를 가지고 즉시 복귀하고, 비동기식 읽기는 입력이 완전히 끝난 후 완전한 데이터 전송을 요청한다.	(12.3.4, p.554 / PDF p.493)
U12-251	봉쇄형	(12.3.4, p.554 / PDF p.493)
U12-252	비봉쇄형	(12.3.4, p.554 / PDF p.493)
U12-253	비동기식	(12.3.4, p.554 / PDF p.493)

제13장. 파일 시스템 인터페이스

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-001	파일은 운영체제에 의하여 정의되고 구현되는 추상적인 자료형이며, 논리 레코드의 연속이다.	(13.6, p.613 / PDF p.551)
U13-002	파일 보호는 읽기·쓰기·실행·덧붙이기·디렉터리 열거 등의 접근 유형에 따라 파일 접근을 별도로 통제한다.	(13.6, p.613 / PDF p.551)
U13-003	파일은 보조저장장치에 기록된 관련 정보의 집합이며, 운영체제는 저장장치의 물리 특성을 추상화하여 일관된 논리적 저장 단위로 제공한다.	(13.1, p.579-580 / PDF p.517-518)
U13-004	일반적으로 비휘발성 장치에 사상되므로 재부팅 뒤에도 내용이 존속한다.	(13.1, p.579-580 / PDF p.517-518)
U13-005	사용자에게는 논리적 보조저장장치의 최소 할당 단위이다.	(13.1, p.579-580 / PDF p.517-518)
U13-006	내용의 의미는 작성자·사용자가 정하며 비트·바이트·행·레코드의 연속이나 프로그램·데이터 등으로 해석한다.	(13.1, p.579-580 / PDF p.517-518)
U13-007	파일 시스템은 실제 파일의 집합과 모든 파일의 정보를 제공하는 디렉터리 구조로 이루어진다.	(13.1, p.579-580 / PDF p.517-518)
U13-008	파일 인터페이스는 원래의 저장 용도를 넘어 proc 파일 시스템의 시스템 정보 접근에도 이용된다.	(13.1, p.579-580 / PDF p.517-518)
U13-009	이름	(13.1.1, p.580-581 / PDF p.518-519)
U13-010	사람이 읽는 기호형 이름	(13.1.1, p.580-581 / PDF p.518-519)
U13-011	식별자	(13.1.1, p.580-581 / PDF p.518-519)
U13-012	파일 시스템 안에서 파일을 고유하게 구별하는 숫자 등의 꼬리표	(13.1.1, p.580-581 / PDF p.518-519)
U13-013	유형	(13.1.1, p.580-581 / PDF p.518-519)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-014	여러 파일 유형을 지원할 때 필요한 정보	(13.1.1, p.580-581 / PDF p.518-519)
U13-015	위치	(13.1.1, p.580-581 / PDF p.518-519)
U13-016	장치와 그 장치 안의 위치를 나타내는 포인터	(13.1.1, p.580-581 / PDF p.518-519)
U13-017	크기	(13.1.1, p.580-581 / PDF p.518-519)
U13-018	현재 크기와 최대 허용 크기	(13.1.1, p.580-581 / PDF p.518-519)
U13-019	보호	(13.1.1, p.580-581 / PDF p.518-519)
U13-020	읽기·쓰기·실행 등의 접근 통제	(13.1.1, p.580-581 / PDF p.518-519)
U13-021	타임스탬프·사용자 식별	(13.1.1, p.580-581 / PDF p.518-519)
U13-022	생성·변경·사용 시각과 보호·보안·감시 정보	(13.1.1, p.580-581 / PDF p.518-519)
U13-023	이름의 대소문자 구별은 시스템마다 다르다.	(13.1.1, p.580-581 / PDF p.518-519)
U13-024	파일은 만든 프로세스와 독립적으로 존속하고 복사·전송될 수 있지만 공유·동기화 수단이 없다면 사본은 서로 독립적으로 바뀐다.	(13.1.1, p.580-581 / PDF p.518-519)
U13-025	확장 속성에는 문자 인코딩·체크섬 같은 정보가 포함될 수 있다.	(13.1.1, p.580-581 / PDF p.518-519)
U13-026	디렉터리 항목은 보통 이름과 식별자를 두고 식별자로 다른 속성을 찾는다.	(13.1.1, p.580-581 / PDF p.518-519)
U13-027	디렉터리도 비휘발성으로 유지해야 하므로 저장장치에 보관하고 필요한 부분만 메모리에 반입한다.	(13.1.1, p.580-581 / PDF p.518-519)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-028	생성	(13.1.2, p.582-586 / PDF p.520-524)
U13-029	공간을 확보하고 디렉터리 항목을 만든다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-030	열기	(13.1.2, p.582-586 / PDF p.520-524)
U13-031	이름·요청 모드·권한을 확인하고 후속 호출에 쓸 핸들을 반환한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-032	쓰기·읽기	(13.1.2, p.582-586 / PDF p.520-524)
U13-033	핸들로 데이터를 전달하며 현재 파일 위치를 갱신한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-034	위치 재설정(seek)	(13.1.2, p.582-586 / PDF p.520-524)
U13-035	현재 위치를 설정한다. 실제 I/O가 반드시 필요한 것은 아니다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-036	삭제	(13.1.2, p.582-586 / PDF p.520-524)
U13-037	항목과 공간을 해제한다. 하드 링크가 있으면 마지막 링크가 없어질 때까지 내용은 남는다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-038	절단	(13.1.2, p.582-586 / PDF p.520-524)
U13-039	다른 속성을 유지하며 길이를 0으로 하고 공간을 해제할 수 있다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-040	매번 이름을 검색하지 않도록 열린 파일 테이블을 사용한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-041	대부분의 시스템은 명시적 open을 요구하며 접근 모드를 권한과 비교한다.	(13.1.2, p.582-586 / PDF p.520-524)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-042	닫으면 관련 테이블 항목과 락을 해제할 수 있다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-043	여러 프로세스가 동시에 여는 시스템은 프로세스별 테이블에 현재 위치·접근 권한 등을, 범 시스템 테이블에 장치 위치·크기·접근 날짜 등 프로세스 독립 정보를 둔다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-044	프로세스별 항목이 범 시스템 항목을 가리킨다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-045	열린 계수는 마지막 사용자가 닫아 0이 될 때 항목을 회수하도록 한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-046	현재 위치 포인터는 각 프로세스의 사용 상태이므로 디스크상의 고유 속성과 분리한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-047	열린 계수는 하드 링크 참조 계수와 구별하여 문맥에 맞게 읽는다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-048	전자는 열린 테이블의 수명에, 후자는 같은 파일에 대한 이름 참조와 파일 내용의 삭제에 관련된다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-049	공유 / 배타	(13.1.2, p.582-586 / PDF p.520-524)
U13-050	공유 락은 여러 프로세스가 함께 획득할 수 있고, 배타 락은 한 프로세스만 가진다. 모든 운영체제가 둘 다 지원하지는 않는다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-051	강제적(mandatory)	(13.1.2, p.582-586 / PDF p.520-524)
U13-052	운영체제가 잠긴 파일에 다른 프로세스가 접근하지 못하게 한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-053	권고적(advisory)	(13.1.2, p.582-586 / PDF p.520-524)
U13-054	응용이 협조하여 락을 획득·해제해야 하며 운영체제가 모든 접근을 차단하지는 않는다.	(13.1.2, p.582-586 / PDF p.520-524)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-055	락은 접근할 때만 유지하도록 하며 여러 락의 획득으로 교착 상태가 생기지 않게 해야 한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-056	원문의 Java 예는 FileChannel을 얻고 처음 부분의 배타 락과 나머지 부분의 공유 락을 획득한 뒤 release로 해제한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-057	아래 코드는 원문을 이미지로 보존한다.	(13.1.2, p.582-586 / PDF p.520-524)
U13-058	원문에 실행 결과 미제시.	(13.1.2, p.582-586 / PDF p.520-524)
U13-059	그림 13.2 Java의 파일 락킹 예제 — 원문 그림 부분 발췌	(13.1.2, p.586 / PDF p.524)
U13-060	운영체제가 파일 유형을 인식하면 유형에 맞는 연산을 선택하거나 부적절한 연산을 막을 수 있다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-061	이름의 확장자, 작성 프로그램 속성, 시작 부분의 매직 넘버가 원문의 식별 방법이다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-062	확장자는 실행 가능·소스·목적·배치·문서·라이브러리·보관·멀티미디어 등의 용도를 암시한다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-063	그러나 모든 시스템에서 확장자가 강제되는 것은 아니며 응용의 힌트일 수도 있다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-064	원문의 macOS 예는 작성 프로그램을 파일 속성으로 기록하여 데이터 파일을 열 때 해당 프로그램을 실행한다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-065	UNIX는 일부 파일의 시작 부분 매직 넘버를 쓰지만 모든 파일이 이를 가지지는 않는다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-066	확장자와 작성 프로그램의 취급도 시스템마다 다르므로 특정 식별법을 보편 규칙으로 확대하지 않는다.	(13.1.3, p.587-588 / PDF p.525-526)
U13-067	파일 유형은 내부 구조를 암시한다.	(13.1.4, p.588-589 / PDF p.526-527)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-068	운영체제가 여러 구조를 직접 지원하면 편리한 연산을 제공할 수 있지만 코드가 커지고 복잡해지며 새로운 구조의 응용을 제약할 수 있다.	(13.1.4, p.588-589 / PDF p.526-527)
U13-069	원문의 텍스트·실행 파일만 지원하는 가정에서는 암호화 데이터가 어느 유형에도 맞지 않는 문제가 생긴다.	(13.1.4, p.588-589 / PDF p.526-527)
U13-070	반대로 UNIX처럼 바이트열로 다루면 유연하지만 해석과 운용은 응용의 책임이 된다.	(13.1.4, p.588-589 / PDF p.526-527)
U13-071	다만 프로그램 적재·실행을 위해 실행 파일의 구조는 운영체제가 이해해야 한다.	(13.1.4, p.588-589 / PDF p.526-527)
U13-072	논리 레코드 크기와 물리 블록 크기는 다를 수 있어 여러 레코드를 한 블록에 패킹하거나 다시 풀어야 한다.	(13.1.5, p.589 / PDF p.527)
U13-073	UNIX의 바이트 스트림은 파일 시작부터의 오프셋으로 바이트를 지정하고 파일 시스템이 물리 블록으로 묶는다.	(13.1.5, p.589 / PDF p.527)
U13-074	이 변환은 응용이나 운영체제가 담당할 수 있다.	(13.1.5, p.589 / PDF p.527)
U13-075	공간은 블록 단위로 할당하므로 마지막 블록 일부가 남는 내부 단편화가 발생한다.	(13.1.5, p.589 / PDF p.527)
U13-076	원문 예는 512바이트 블록에서 1,949바이트 파일에 네 블록(2,048바이트)을 할당하여 99바이트가 남는 경우이다.	(13.1.5, p.589 / PDF p.527)
U13-077	블록을 크게 하면 이 낭비가 커질 수 있다.	(13.1.5, p.589 / PDF p.527)
U13-078	순차 접근은 저장된 레코드 순서로 파일을 처리한다.	(13.2.1, p.590 / PDF p.528)
U13-079	read-next는 다음 부분을 읽고 현재 위치를 증가시키며, 쓰기는 끝에 추가하고 새 끝으로 포인터를 옮긴다.	(13.2.1, p.590 / PDF p.528)
U13-080	되감기나 정해진 레코드 수만큼의 이동을 제공할 수도 있다.	(13.2.1, p.590 / PDF p.528)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-081	테이프 모델에 기반하지만 순차 장치와 임의 접근 장치 모두에 사용할 수 있고 편집기·컴파일러가 예로 제시된다.	(13.2.1, p.590 / PDF p.528)
U13-082	직접(상대) 접근은 고정 길이 레코드·블록을 번호로 지정하여 읽기·쓰기를 임의 순서로 수행한다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-083	read(n)·write(n) 또는 position-file(n) 후 순차 연산으로 표현할 수 있다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-084	사용자가 보는 상대 블록 번호는 실제 디스크 주소와 다르며 운영체제가 위치를 할당하고 다른 파일 영역의 접근을 막는다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-085	번호의 시작이 0인지 1인지는 시스템에 따라 다르다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-086	원문 수식: L × N 위치에서 L바이트; L: 논리 레코드 길이; N: 0부터 세는 레코드 번호; 0번을 첫 레코드로 세고 논리 레코드의 길이가 L일 때 N번째 레코드의 요청 위치이다. 고정 길이라는 전제를 유지한다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-087	reset	(13.2.2, p.590-591 / PDF p.528-529)
U13-088	cp = 0;	(13.2.2, p.590-591 / PDF p.528-529)
U13-089	read next	(13.2.2, p.590-591 / PDF p.528-529)
U13-090	read cp; 이후 cp = cp+1	(13.2.2, p.590-591 / PDF p.528-529)
U13-091	write next	(13.2.2, p.590-591 / PDF p.528-529)
U13-092	write cp; 이후 cp = cp+1	(13.2.2, p.590-591 / PDF p.528-529)
U13-093	그림 13.5의 cp는 현재 위치이다.	(13.2.2, p.590-591 / PDF p.528-529)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-094	직접 접근에 현재 위치 추적을 더하면 다음 레코드 순서로 처리할 수 있음을 보이는 원문의 표를 정리했다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-095	이는 별도의 새 알고리즘이 아니다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-096	직접 접근으로 순차 접근을 구현하는 것은 간단하지만 순차 접근만으로 직접 접근을 흉내 내는 것은 비효율적이다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-097	원문의 항공편 번호로 예약 블록을 찾는 사례는 원하는 블록을 먼저 결정한 뒤 직접 읽는 의미를 보여준다.	(13.2.2, p.590-591 / PDF p.528-529)
U13-098	색인 접근은 먼저 레코드의 색인에서 포인터를 찾은 뒤 해당 블록을 직접 읽는다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-099	파일이 커 색인도 메모리에 못 들어가면 색인 파일에 대한 상위 색인을 둘 수 있다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-100	원문의 ISAM은 작은 마스터 색인→2차 색인 블록→실제 파일 블록의 순서로 찾고 블록 안에서 검색한다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-101	원문의 UPC 가격 예는 10자리 코드와 6자리 가격의 16바이트 레코드, 1,024바이트 블록당 64레코드, 블록의 첫 UPC로 구성한 메모리 색인을 설명한다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-102	다만 120,000레코드를 2,000블록·색인 2,000항목으로 설명하는 수치 연결은 원문 확인 필요이다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-103	원문 수치를 다른 값으로 고쳐 계산하지 않고, 색인에서 블록을 정하여 적은 I/O로 탐색한다는 구조를 보존한다.	(13.2.3, p.591-593 / PDF p.529-531)
U13-104	디렉터리는 파일 이름을 파일 제어 블록에 대응시키는 심볼 테이블로 볼 수 있다.	(13.3, p.593 / PDF p.531)
U13-105	필요한 연산은 이름·패턴 검색, 생성·삭제, 항목 나열, 이름 변경, 전체 순회이다.	(13.3, p.593 / PDF p.531)
U13-106	이름이 커져 항목에 맞지 않거나 삭제로 빈 공간이 생기면 공간 관리도 필요하다.	(13.3, p.593 / PDF p.531)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-107	순회는 검색뿐 아니라 백업·보관과 공간 회수에도 이용된다.	(13.3, p.593 / PDF p.531)
U13-108	모든 파일을 한 디렉터리에 두므로 개념이 단순하지만 모든 이름이 고유해야 한다.	(13.3.1, p.594 / PDF p.532)
U13-109	사용자나 파일 수가 늘면 이름 충돌과 기억·관리 문제가 커진다.	(13.3.1, p.594 / PDF p.532)
U13-110	긴 이름을 허용해도 파일을 조직하는 문제 자체가 없어지지는 않는다.	(13.3.1, p.594 / PDF p.532)
U13-111	마스터 파일 디렉터리(MFD)가 사용자별 파일 디렉터리(UFD)를 가리킨다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-112	각 UFD 안에서만 이름이 고유하면 되므로 다른 사용자는 같은 파일명을 쓸 수 있다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-113	생성·삭제는 자신의 UFD만 검색하고 UFD의 생성·삭제는 적절한 관리 권한이 필요하다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-114	사용자 간 격리는 되지만 다른 사용자의 파일을 공유하려면 접근 허용과 사용자명+파일명의 경로가 필요하다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-115	시스템 프로그램을 모든 UFD에 복사하면 공간을 낭비한다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-116	대신 현재 UFD에서 찾지 못하면 시스템 파일 디렉터리 등 정해진 순서를 찾는 탐색 경로를 둔다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-117	사용자가 자신의 경로를 가질 수도 있다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-118	볼륨 문자·디렉터리·버전의 경로 구문은 시스템마다 다르다.	(13.3.2, p.594-596 / PDF p.532-534)
U13-119	2단계를 임의 높이로 확장하면 사용자가 하위 디렉터리를 만드는 트리가 된다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-120	최상위 루트와 파일별 고유 경로를 갖는다.	(13.3.3, p.596-598 / PDF p.534-536)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-121	디렉터리도 특별 취급되는 파일로 구현할 수 있으며 일반 파일과 구별하고 별도 생성·삭제 연산을 사용한다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-122	프로세스의 현재 디렉터리를 기준으로 파일을 찾고, 자식 프로세스는 보통 부모의 현재 디렉터리를 따른다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-123	절대 경로	(13.3.3, p.596-598 / PDF p.534-536)
U13-124	루트부터 목적 파일까지 명시	(13.3.3, p.596-598 / PDF p.534-536)
U13-125	상대 경로	(13.3.3, p.596-598 / PDF p.534-536)
U13-126	현재 디렉터리부터 목적 파일까지 명시	(13.3.3, p.596-598 / PDF p.534-536)
U13-127	원문 예에서 현재 디렉터리가 /spell/mail이면 prt/first와 /spell/mail/prt/first는 같은 파일을 가리킨다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-128	디렉터리 삭제는 비어 있을 때만 허용하는 정책과 하위 내용을 함께 재귀 삭제하는 정책이 있다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-129	후자는 편하지만 한 명령의 실수로 많은 내용을 잃을 수 있다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-130	트리에서는 경로와 접근 허용을 통해 다른 사용자 파일에도 접근할 수 있다.	(13.3.3, p.596-598 / PDF p.534-536)
U13-131	비순환 그래프는 파일·하위 디렉터리를 여러 곳에서 공유하도록 트리를 일반화한다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-132	물리적으로 같은 파일을 공유하면 수정이 다른 참조에도 반영되지만 독립 사본 둘은 그렇지 않다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-133	링크는 다른 파일의 경로를 가진 간접 포인터로 구현할 수 있다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-134	동일 항목 내용을 여러 디렉터리에 복사하는 방식은 속성 변경을 일치시켜야 하는 문제가 있다.	(13.3.4, p.599-601 / PDF p.537-539)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-135	그림 13.10 비순환 그래프 디렉터리 구조 — 원문 그림 부분 발췌	(13.3.4, p.599 / PDF p.537)
U13-136	여러 절대 경로가 한 파일을 가리키므로 탐색·통계·백업에서 같은 대상을 중복 처리하지 않아야 한다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-137	공유 파일을 한 참조만 지웠다고 제거하면 dangling pointer가 생긴다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-138	심볼릭 링크는 원래 파일이 삭제되면 없는 이름을 가리킬 수 있으며, 접근 시 해석 실패로 처리하거나 따로 관리할 수 있다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-139	같은 이름의 새 파일이 생기는 문제도 고려해야 한다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-140	다른 방식은 모든 참조가 없어질 때까지 파일을 유지하는 것이다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-141	가변 길이 참조 목록 대신 참조 계수를 두어 링크 추가 때 증가, 삭제 때 감소시키고 0일 때 파일을 지울 수 있다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-142	원문 UNIX의 하드 링크는 inode에 이 계수를 유지한다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-143	디렉터리에 대한 다중 참조를 제한하는 것은 비순환성을 지키는 데 도움이 된다.	(13.3.4, p.599-601 / PDF p.537-539)
U13-144	기존 구조에 링크를 추가하면 순환이 생길 수 있다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-145	일반 그래프는 공유의 유연성이 크지만 잘못된 탐색은 무한 반복할 수 있고 순환 내부의 참조 계수가 0이 되지 않아 도달 불가능한 공간을 회수하지 못할 수 있다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-146	파일 시스템을 순회하면서 접근 가능한 모든 대상을 표시한다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-147	두 번째 탐색에서 표시되지 않은 대상을 수집하여 가용 공간 목록에 넣는다.	(13.3.5, p.601-603 / PDF p.539-541)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-148	디스크 기반 전체 탐색은 비용이 커 드물게 쓰인다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-149	링크 추가 시 순환 검사도 비싸다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-150	디렉터리 순회에서 링크를 따라가지 않거나 탐색 수를 제한하는 등의 방안을 고려한다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-151	비순환 구조는 검색과 마지막 참조 판단이 상대적으로 간단하다는 점과 대조된다.	(13.3.5, p.601-603 / PDF p.539-541)
U13-152	신뢰성은 물리 손상·사고·오류에서 내용을 보존하는 문제이고 보호는 부적절한 접근을 통제하는 문제이다.	(13.4.1, p.603-604 / PDF p.541-542)
U13-153	복사본·백업은 신뢰성에, 인증·암호화·방화벽과 파일별 접근 권한은 보호에 관계된다.	(13.4.1, p.603-604 / PDF p.541-542)
U13-154	모든 접근 금지나 완전 개방 대신 필요한 연산을 통제한다.	(13.4.1, p.603-604 / PDF p.541-542)
U13-155	통제할 수 있는 기본 연산: 읽기, 쓰기, 실행, 끝에 추가, 삭제, 이름·속성 나열, 속성 변경.	(13.4.1, p.603-604 / PDF p.541-542)
U13-156	이름 변경·편집·복사 같은 고급 작업은 기본 연산의 조합이므로 기본 연산에서 보호를 적용할 수 있다.	(13.4.1, p.603-604 / PDF p.541-542)
U13-157	규모와 목적에 따라 보호 요구가 다르므로 모든 환경에 같은 정책을 무조건 적용하지 않는다.	(13.4.1, p.603-604 / PDF p.541-542)
U13-158	ACL(접근 제어 리스트)은 파일·디렉터리에 누가 어떤 연산을 할 수 있는지 연결한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-159	요청 사용자의 신원과 연산을 목록에서 검사한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-160	세밀하지만 사용자를 일일이 열거하면 목록이 길어지고 디렉터리 공간 관리가 복잡해진다.	(13.4.2, p.604-607 / PDF p.542-545)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-161	이를 줄이기 위해 소유자·그룹·기타 사용자로 분류하고 필요한 대상에 ACL을 더하는 결합 방식을 쓸 수 있다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-162	r	(13.4.2, p.604-607 / PDF p.542-545)
U13-163	파일 읽기. 디렉터리에서는 내용 열람을 제어한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-164	w	(13.4.2, p.604-607 / PDF p.542-545)
U13-165	쓰기 접근을 제어한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-166	x	(13.4.2, p.604-607 / PDF p.542-545)
U13-167	파일 실행. 디렉터리에서는 해당 디렉터리로 이동하는 데 필요하다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-168	세 사용자 필드	(13.4.2, p.604-607 / PDF p.542-545)
U13-169	소유자·그룹·기타에 각 rwx 3비트, 총 9비트	(13.4.2, p.604-607 / PDF p.542-545)
U13-170	원문의 Sara 사례는 책 전체를 공동 작업하는 그룹과 한 장만 접근시킬 방문자의 요구를 구분한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-171	그룹 하나만으로 부족할 때 해당 장의 ACL에 방문자를 추가한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-172	권한·ACL을 바꾸는 행위 자체도 엄격히 통제해야 한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-173	두 허가가 충돌할 때의 우선순위도 정책으로 정해야 하며, 원문의 Solaris 사례는 더 구체적으로 부여된 ACL을 우선한다.	(13.4.2, p.604-607 / PDF p.542-545)
U13-174	명시한 시스템의 규칙을 모든 시스템에 확대하지 않는다.	(13.4.2, p.604-607 / PDF p.542-545)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-175	파일마다 암호를 요구할 수도 있지만 암호가 너무 많으면 기억하기 어렵고 하나를 공통 사용하면 유출 시 전부 노출된다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-176	디렉터리별 암호나 파일·파티션 암호화가 대안으로 제시되며 암호 관리가 중요하다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-177	디렉터리는 파일과 다른 연산을 가지므로 생성·삭제·목록 조회 등을 별도로 보호해야 한다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-178	이름이나 존재 자체가 중요한 정보일 수 있다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-179	경로로 파일에 접근하려면 파일뿐 아니라 그 경로의 디렉터리 접근도 허용되어야 한다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-180	여러 경로가 있으면 어떤 경로를 쓰는가에 따라 접근 권한이 다를 수 있다.	(13.4.3, p.607-608 / PDF p.545-546)
U13-181	메모리 사상 파일은 프로세스 가상 주소의 일부를 파일 블록과 대응시킨다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-182	처음 참조할 때 요구 페이징으로 페이지 폴트가 발생하여 해당 내용을 메모리로 읽고, 이후에는 메모리 접근으로 처리한다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-183	매 접근마다 read/write 시스템 콜을 하는 부담을 줄인다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-184	사상했다고 파일 전체가 즉시 적재되는 것은 아니다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-185	메모리에 쓴 내용이 보조저장장치에 즉시 동기 기록되는 것은 아니다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-186	기록·닫기·메모리 확보 시 처리는 원문이 설명한 시스템의 정책에 따른다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-187	어떤 시스템은 명시적 사상 호출을 요구하며 Solaris는 일반 파일 I/O도 커널 주소 공간의 매핑을 통해 처리하는 사례로 소개된다.	(13.5.1, p.608-610 / PDF p.546-548)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-188	원문의 기록 시점 설명에서는 일반적으로 파일을 닫을 때 메모리 이미지의 변경을 파일에 반영하고 가상 주소의 사상을 제거한다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-189	메모리 부족으로 공간을 확보할 때는 변경을 잃지 않도록 스왑 공간에 반영한다고 설명한다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-190	이 처리와 공유 프로세스가 같은 물리 페이지의 변경을 보는 시점은 별개의 관계이다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-191	여러 프로세스의 페이지 테이블이 같은 물리 페이지를 가리키면 같은 파일의 변경을 공유할 수 있다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-192	쓰기 시 복사를 사용하면 읽기 전용으로 공유하다 수정할 때 별도 페이지를 만든다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-193	공유 내용의 동시 수정이 필요하면 상호 배제 기법도 사용해야 한다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-194	같은 파일을 여러 주소 공간에 사상하는 방법은 공유 메모리 통신을 구현하는 데 쓰인다.	(13.5.1, p.608-610 / PDF p.546-548)
U13-195	생산자가 CreateFile()로 temp.txt를 열고 HANDLE을 얻는다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-196	CreateFileMapping()으로 SharedObject라는 이름의 공유 메모리 객체를 만든다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-197	MapViewOfFile()로 자신의 가상 주소 공간에 뷰를 만들고 반환 포인터로 메시지를 기록한다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-198	소비자는 OpenFileMapping()으로 같은 이름의 객체를 열고 MapViewOfFile()로 뷰를 만든다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-199	소비자가 메시지를 읽고, 각 프로세스는 UnmapViewOfFile()과 CloseHandle()로 뷰·핸들을 정리한다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-200	뷰는 전체 파일 또는 오프셋·크기로 지정한 일부일 수 있다.	(13.5.2, p.610-613 / PDF p.548-551)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-201	원문은 마지막 세 인자를 0으로 두어 전체 파일의 뷰를 지정한다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-202	실제 페이지는 요구 페이징으로 접근할 때 적재된다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-203	아래 생산자·소비자 코드는 원문 그대로 이미지로 보존한다.	(13.5.2, p.610-613 / PDF p.548-551)
U13-204	본문에 메시지 읽기가 설명되어 있지만 별도 실행 출력 화면은 원문에 실행 결과 미제시.	(13.5.2, p.610-613 / PDF p.548-551)
U13-205	그림 13.15 공유 메모리에 기록하는 Windows 생산자 — 원문 그림 부분 발췌	(13.5.2, p.611 / PDF p.549)
U13-206	그림 13.16 공유 메모리를 읽는 Windows 소비자 — 원문 그림 부분 발췌	(13.5.2, p.612 / PDF p.550)
U13-207	파일은 논리 레코드를 물리 저장장치에 대응시키는 추상 자료형이며 디렉터리는 이름과 조직을 제공한다.	(13.6, p.608,613-615 / PDF p.546,551-553)
U13-208	구조가 1단계→2단계→트리→공유 그래프로 확장될수록 유연성과 함께 탐색·삭제·공간 회수의 복잡성이 증가한다.	(13.6, p.608,613-615 / PDF p.546,551-553)
U13-209	파일 보호는 사용자뿐 아니라 연산의 종류와 경로·디렉터리를 함께 고려한다.	(13.6, p.608,613-615 / PDF p.546,551-553)
U13-210	장말 문제는 파일 보관 정책, 유형·구조 지원, 이름과 디렉터리 조직, open/close, 디렉터리 보호, 권한 표현과 제어 리스트를 점검한다.	(13.6, p.613-615 / PDF p.551-553)
U13-211	본문에 없는 문제 풀이를 추가하지 않는다.	(13.6, p.613-615 / PDF p.551-553)
U13-212	추가 자료의 다중 레벨 디렉터리·NFS 소개와 참고문헌 존재를 확인했고 외부 내용을 보충하지 않았다.	(13.6, p.613-615 / PDF p.551-553)
U13-213	1단계	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-214	단순한 한 디렉터리	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-215	모든 파일명 고유	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-216	2단계	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-217	사용자별 같은 이름	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-218	공유와 경로 표현 필요	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-219	트리	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-220	사용자 하위 디렉터리·계층화	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-221	물리적 파일·하위 디렉터리 공유 제한	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-222	비순환 그래프	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-223	공유 허용	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-224	중복 순회·참조·삭제 관리	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-225	일반 그래프	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-226	순환까지 허용하는 유연성	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-227	탐색 종료·도달성·가비지 수집 문제	(디렉터리 구조의 변화, p.594-603 / PDF p.532-541)
U13-228	순차 접근은 저장되어 있는 레코드 순서로 접근하는 방법이다.	(13.2.1, p.590 / PDF p.528)
U13-229	직접 접근은 파일을 고정 길이의 논리 레코드 집합으로 정의하며, 임의의 블록을 읽거나 쓸 수 있어 순서에 제약이 없다.	(13.2.2, p.590 / PDF p.528)
U13-230	파일 포인터는 읽기·쓰기 시스템 콜에 오프셋을 포함하지 않는 시스템에서 가장 최근의 읽기·쓰기 위치를 추적하는 현재 파일 위치 포인터이다.	(13.1.2, p.584 / PDF p.522)
U13-231	파일 열린 계수는 파일의 총 open()과 close() 수를 추적하며, 값이 0이면 열린 파일 테이블 항목을 제거할 수 있다.	(13.1.2, p.584 / PDF p.522)
U13-232	비순환 그래프 디렉터리는 서브디렉터리와 파일의 공유를 허용하지만 탐색과 삭제가 복잡하다.	(13.6, p.613 / PDF p.551)
U13-233	물리 레코드와 논리 레코드의 크기는 보통 다르므로 논리 레코드를 물리 레코드에 연관시켜야 하며, 운영체제 또는 사용자 응용 프로그램이 이 작업을 할 수 있다.	(13.6, p.613 / PDF p.551)
U13-234	다중 사용자 시스템의 1단계 디렉터리는 모든 파일에 고유 이름을 요구하며, 2단계 디렉터리는 각 사용자에게 독자적인 디렉터리를 할당하여 이름 문제를 해결한다.	(13.6, p.613 / PDF p.551)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-235	트리 구조는 서브디렉터리 구성을, 비순환 그래프는 공유를 허용하며, 일반 그래프는 사용하지 않는 공간 회수를 위해 쓰레기 수집이 필요하다.	(13.6, p.613 / PDF p.551)
U13-236	순차 접근은 저장된 레코드 순서로 접근하고, 직접 접근은 임의 블록을 읽고 쓸 수 있어 순서에 제약이 없다.	(13.2.1, p.590 / PDF p.528)
U13-237	순차 접근	(13.2.1, p.590 / PDF p.528)
U13-238	직접 접근	(13.2.2, p.590 / PDF p.528)
U13-239	고정 길이 논리 레코드의 위치: 파일의 L × N 위치에서 L바이트 요청; 논리 레코드 길이는 L이며, N=0을 첫 번째 레코드로 가정한다.	(13.2.2, p.591 / PDF p.529)

제14장. 파일 시스템 구현

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-001	대부분의 파일 시스템은 보조저장장치에 상주하여 많은 양의 데이터를 영구적으로 보유하도록 설계된다.	(14.9, p.650 / PDF p.587)
U14-002	파일 시스템은 종종 계층·모듈 구조로 구현되며, 하위 수준은 저장장치의 물리 속성과 통신을, 상위 수준은 심볼릭 파일 이름과 논리 특성을 처리한다.	(14.9, p.650 / PDF p.587)
U14-003	파일 시스템의 두 설계 문제는 사용자에게 보이는 파일·속성·연산을 정의하는 일과 이를 실제 보조저장장치에 대응시키는 알고리즘·자료구조를 만드는 일이다.	(14.1, p.618-620 / PDF p.555-557)
U14-004	원문은 재기록·직접 접근이 가능한 HDD와 덮어쓰기 특성이 다른 NVM을 구별하며, 효율을 위해 블록 단위로 I/O한다.	(14.1, p.618-620 / PDF p.555-557)
U14-005	논리 파일 시스템	(14.1, p.618-620 / PDF p.555-557)
U14-006	디렉터리·메타데이터·보호·FCB를 관리하고 이름을 하위 계층의 정보로 변환한다.	(14.1, p.618-620 / PDF p.555-557)
U14-007	파일-구성 모듈	(14.1, p.618-620 / PDF p.555-557)
U14-008	파일의 논리 블록과 실제 공간의 관계를 알고 가용 공간 관리자를 포함한다.	(14.1, p.618-620 / PDF p.555-557)
U14-009	기본 파일 시스템	(14.1, p.618-620 / PDF p.555-557)
U14-010	LBA 기반의 블록 읽기·쓰기, I/O 스케줄링, 버퍼·캐시를 관리한다.	(14.1, p.618-620 / PDF p.555-557)
U14-011	I/O 제어	(14.1, p.618-620 / PDF p.555-557)
U14-012	드라이버·인터럽트 핸들러가 일반 명령을 특정 하드웨어 명령으로 바꾸고 전송한다.	(14.1, p.618-620 / PDF p.555-557)
U14-013	메타데이터는 파일 내용 자체를 제외한 파일 시스템 구조이다.	(14.1, p.618-620 / PDF p.555-557)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-014	FCB는 소유·권한·내용의 위치 같은 파일 정보를 담고 UNIX에서는 inode가 그 역할을 한다.	(14.1, p.618-620 / PDF p.555-557)
U14-015	계층화는 코드 중복을 줄여 여러 파일 시스템이 하위 코드를 공유하게 하지만 계층 통과 비용을 늘릴 수 있다.	(14.1, p.618-620 / PDF p.555-557)
U14-016	파일 시스템마다 기능·성능·신뢰성·목적이 달라 여러 구현이 존재하고 FUSE는 사용자 수준 구현의 유연성을 제공하는 사례이다.	(14.1, p.618-620 / PDF p.555-557)
U14-017	부트 제어 블록	(14.2.1, p.620-622 / PDF p.557-559)
U14-018	해당 파티션에서 운영체제를 부팅하는 정보	(14.2.1, p.620-622 / PDF p.557-559)
U14-019	볼륨 제어 블록	(14.2.1, p.620-622 / PDF p.557-559)
U14-020	전체·가용 블록 수, 블록 크기, 가용 공간과 FCB의 수·포인터	(14.2.1, p.620-622 / PDF p.557-559)
U14-021	디렉터리	(14.2.1, p.620-622 / PDF p.557-559)
U14-022	이름과 파일 식별·제어 정보의 연결	(14.2.1, p.620-622 / PDF p.557-559)
U14-023	파일별 FCB	(14.2.1, p.620-622 / PDF p.557-559)
U14-024	고유 식별자와 자세한 파일 정보	(14.2.1, p.620-622 / PDF p.557-559)
U14-025	메모리에는 마운트된 파티션 정보, 최근 디렉터리 정보, FCB 사본을 담은 범 시스템 열린 파일 테이블, 이를 가리키는 프로세스별 열린 파일 테이블, 전송 중 블록 버퍼를 둔다.	(14.2.1, p.620-622 / PDF p.557-559)
U14-026	마운트 때 적재하고 동작 중 갱신하며 해제 때 제거한다.	(14.2.1, p.620-622 / PDF p.557-559)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-027	파일 생성은 새 FCB를 할당하고 디렉터리에 이름·FCB 연결을 추가한 뒤 기록하는 작업이다.	(14.2.1, p.620-622 / PDF p.557-559)
U14-028	디렉터리를 특별 파일로 구현하는지 별개 객체로 보는지는 시스템에 따라 다르다.	(14.2.1, p.620-622 / PDF p.557-559)
U14-029	open이 먼저 범 시스템 테이블을 확인한다. 이미 열려 있으면 그 항목을 가리키는 프로세스별 항목을 만든다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-030	처음 열 때는 디렉터리에서 찾아 FCB를 범 시스템 테이블에 복사한다. 프로세스별 항목에는 현재 위치와 접근 모드 등을 둔다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-031	프로세스는 반환된 파일 디스크립터(UNIX) 또는 핸들(Windows)로 후속 연산을 한다. 매번 파일 이름을 다시 검색하지 않는다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-032	close는 프로세스별 항목을 없애고 열린 계수를 줄인다. 마지막 사용자가 닫으면 갱신 정보를 디스크에 반영하고 범 시스템 항목을 회수한다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-033	이름·디렉터리·FCB를 캐싱하면 실제 데이터 외의 파일 정보를 메모리에서 해결하여 저장장치 접근을 줄일 수 있다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-034	테이블은 파일별 공통 정보와 프로세스별 사용 상태를 구별하는 구조이다.	(14.2.2, p.622-623 / PDF p.559-560)
U14-035	파일 이름과 데이터 블록 포인터의 선형 리스트는 구현이 쉽지만 검색 시간이 길다.	(14.3.1, p.624 / PDF p.561)
U14-036	생성은 같은 이름이 없는지 검색한 뒤 추가하고, 삭제는 공간을 돌려주고 항목을 비워 재사용한다.	(14.3.1, p.624 / PDF p.561)
U14-037	사용 비트·특별 이름·잘못된 inode 번호 표시, 마지막 항목 복사, 연결 리스트 등으로 빈 항목을 관리할 수 있다.	(14.3.1, p.624 / PDF p.561)
U14-038	정렬하면 이진 검색과 정렬된 목록 출력이 쉬워지지만 삽입·삭제 때 정보 이동이 필요하다.	(14.3.1, p.624 / PDF p.561)
U14-039	트리 구조나 최근 디렉터리 캐시가 보완할 수 있다.	(14.3.1, p.624 / PDF p.561)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-040	디렉터리 검색은 빈번하므로 단순한 구현의 속도 비용이 중요하다.	(14.3.1, p.624 / PDF p.561)
U14-041	해시 테이블은 파일 이름을 값으로 바꾸어 항목 위치를 빠르게 찾는다.	(14.3.2, p.624-625 / PDF p.561-562)
U14-042	충돌 처리가 필요하며 크기가 고정되면 확장 때 함수와 기존 항목을 재구성해야 할 수 있다.	(14.3.2, p.624-625 / PDF p.561-562)
U14-043	원문의 64항목에서 65번째 항목을 추가하는 예는 더 큰 테이블과 재해싱의 필요를 보여준다.	(14.3.2, p.624-625 / PDF p.561-562)
U14-044	체인 오버플로 방식은 각 해시 항목의 연결 리스트에 충돌 항목을 넣어 확장하지만 그 리스트를 탐색하는 비용이 있다.	(14.3.2, p.624-625 / PDF p.561-562)
U14-045	연속 할당은 파일이 연속한 블록을 차지하게 하고 디렉터리에 시작 블록과 길이를 기록한다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-046	시작이 b이고 길이가 n이면 b부터 b+n−1까지이며 i번째 블록은 b+i이다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-047	순차·직접 접근이 모두 쉽고 HDD의 헤드 이동을 줄일 수 있다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-048	이 이익은 가까운 논리 주소가 물리적으로도 가깝다는 전제를 가진다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-049	가용 연속 영역 선택은 동적 공간 할당 문제이다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-050	최초 적합·최적 적합 등을 사용할 수 있지만 외부 단편화로 전체 빈 공간은 충분해도 요구 크기의 한 덩어리가 없을 수 있다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-051	다른 장치로 복사했다 돌아오거나 밀집화하면 개선되지만 시간·비용·정상 운영 중 성능 저하가 크다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-052	파일 최종 크기도 예측하기 어렵다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-053	작게 예약하면 인접 공간이 없어 확장할 수 없고, 종료 후 다시 실행하거나 더 큰 공간에 전체를 복사해야 한다.	(14.4.1, p.625-627 / PDF p.562-564)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-054	지나치게 선할당하면 오랫동안 사용하지 않는 공간이 낭비된다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-055	익스텐트는 초기의 연속 영역이 부족할 때 연속 블록 묶음을 추가하여 위치·길이·다음 묶음 포인터를 기록하는 절충이다.	(14.4.1, p.625-627 / PDF p.562-564)
U14-056	연결 할당은 파일을 장치에 흩어질 수 있는 블록의 연결 리스트로 저장한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-057	각 블록이 다음 블록을 가리키고 디렉터리는 시작·끝을 기억한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-058	파일 크기를 미리 정하지 않고 가용 블록을 계속 붙일 수 있어 외부 단편화·밀집화 문제를 피한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-059	반면 직접 접근은 앞의 링크를 따라가야 하므로 비효율적이다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-060	포인터가 데이터 공간을 차지한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-061	원문 예는 512바이트 블록의 4바이트 포인터로 508바이트만 데이터를 담는 경우이다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-062	여러 블록을 클러스터로 묶으면 포인터·할당 관리와 HDD 탐색을 줄이지만 내부 단편화와 작은 임의 요청의 전송량은 늘어난다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-063	링크 하나의 손상은 후속 데이터 접근을 잃게 할 수 있고 이중 링크·추가 식별 정보는 더 많은 비용을 요구한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-064	FAT(파일 할당 테이블)는 링크들을 파티션의 별도 연속 테이블로 모은 변형이다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-065	디렉터리의 시작 블록을 인덱스로 FAT를 찾으면 다음 블록 번호를 얻고 특수 값으로 끝을 표시한다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-066	원문 예의 사슬은 217→618→339이다.	(14.4.2, p.627-630 / PDF p.564-567)
U14-067	FAT가 메모리에 캐시되지 않으면 테이블과 데이터 위치 사이의 왕복 탐색이 늘 수 있다.	(14.4.2, p.627-630 / PDF p.564-567)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-068	그림 14.6 파일 할당 테이블(FAT) — 원문 그림 부분 발췌	(14.4.2, p.629 / PDF p.566)
U14-069	색인 할당은 파일마다 블록 주소 배열인 색인 블록을 두고 i번째 항목이 i번째 데이터 블록을 가리키게 한다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-070	디렉터리는 색인 블록의 주소를 갖는다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-071	새 파일은 포인터를 null로 시작하고 실제 블록을 쓸 때 주소를 넣는다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-072	외부 단편화 없이 직접 접근을 지원하지만 작은 파일에도 색인 블록 전체가 필요하여 포인터 오버헤드가 크다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-073	색인을 작게 하면 큰 파일을 표현할 별도 방법이 필요하다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-074	연결 기법	(14.4.3, p.630-632 / PDF p.567-569)
U14-075	색인 블록의 마지막 포인터로 다음 색인 블록을 연결한다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-076	다중 수준 색인	(14.4.3, p.630-632 / PDF p.567-569)
U14-077	상위 색인이 하위 색인 블록을 가리키고 최하위 색인이 데이터를 가리킨다. 필요하면 수준을 더한다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-078	결합 기법	(14.4.3, p.630-632 / PDF p.567-569)
U14-079	inode 안의 직접 포인터와 단일·이중·삼중 간접 포인터를 함께 사용한다. 작은 파일은 추가 색인을 피하고 큰 파일은 간접 수준을 쓴다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-080	원문 다중 색인 예는 4,096바이트 블록에 4바이트 포인터 1,024개를 넣고, 두 수준으로 1,048,576 데이터 블록, 4GB 파일을 표현한다.	(14.4.3, p.630-632 / PDF p.567-569)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-081	결합 예의 15개 포인터 중 12개는 직접 블록이고 나머지 3개가 단일·이중·삼중 간접 블록이다. 4KB 블록이면 처음 48KB는 직접 포인터로 접근한다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-082	색인을 캐시해도 데이터 자체는 볼륨 전역에 흩어질 수 있다.	(14.4.3, p.630-632 / PDF p.567-569)
U14-083	그림 14.8 UNIX의 inode — 원문 그림 부분 발췌	(14.4.3, p.632 / PDF p.569)
U14-084	연속	(14.4.4, p.632-633 / PDF p.569-570)
U14-085	시작 주소로 i번째 블록의 주소를 즉시 계산하여 한 번의 접근으로 읽는다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-086	연결	(14.4.4, p.632-633 / PDF p.569-570)
U14-087	다음 블록의 순차 접근은 쉽지만 i번째 직접 접근에 여러 링크 읽기가 필요하다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-088	색인	(14.4.4, p.632-633 / PDF p.569-570)
U14-089	색인의 메모리 상주 여부·구조·파일 크기·목표 위치에 따라 추가 I/O가 달라진다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-090	시스템의 주 사용 패턴이 선택 기준이다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-091	순차·직접 파일을 다른 방식으로 저장하거나 작은 파일은 연속, 커지면 색인으로 전환하는 결합도 있다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-092	HDD에서는 탐색을 줄이기 위한 추가 CPU 연산이 타당할 수 있으나, 헤드가 없는 NVM에 같은 최적화를 그대로 쓰면 불필요한 비용이 된다.	(14.4.4, p.632-633 / PDF p.569-570)
U14-093	NVM은 명령 수와 접근 경로를 줄이는 최적화가 필요하다는 것이 원문의 설명이다.	(14.4.4, p.632-633 / PDF p.569-570)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-094	가용 공간 관리는 할당 시 빈 블록을 제거하고 삭제 시 되돌려 받는 역할이다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-095	목록이라는 이름이어도 반드시 연결 리스트일 필요는 없다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-096	비트맵은 블록당 한 비트를 사용한다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-097	이 원문의 표기에서는 1이 가용, 0이 할당이다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-098	워드를 검사하여 처음 0이 아닌 워드와 그 안의 첫 1을 찾으면 첫 가용 블록을 구할 수 있다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-099	연속 가용 블록 탐색도 쉽다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-100	원문 수식: (워드의 비트 수) × (값이 0인 워드의 수) + 첫 번째 1비트의 변위; ; 원문이 제시한 첫 가용 블록 번호의 계산식이다. 앞의 워드는 모두 할당 비트 0으로만 구성되어 있다는 검색 절차의 전제를 갖는다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-101	하드웨어 비트 연산은 효율적이지만 큰 비트맵을 메모리에 두기 어렵다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-102	원문은 4KB 블록의 1TB 디스크에 32MB 비트맵이 필요하다고 설명한다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-103	블록·클러스터가 커지면 관리 비트 수는 줄지만 공간 할당의 단위도 커진다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-104	복구를 위해 장치상의 기록도 유지해야 한다.	(14.5.1, p.634-635 / PDF p.571-572)
U14-105	모든 가용 블록을 다음 가용 블록의 포인터로 연결하고 첫 포인터를 정해진 위치와 메모리에 둔다.	(14.5.2, p.635 / PDF p.572)
U14-106	전체 순회는 디스크 접근이 많아 비싸지만 보통 할당에는 첫 가용 블록만 필요하다.	(14.5.2, p.635 / PDF p.572)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-107	FAT는 할당 사슬과 가용 블록의 관리를 같은 테이블에서 처리하여 별도 구조를 줄일 수 있다.	(14.5.2, p.635 / PDF p.572)
U14-108	첫 가용 블록에 n개의 주소를 넣고 처음 n−1개는 실제 빈 블록, 마지막은 같은 형식의 다음 주소 묶음을 가진 블록을 가리킨다.	(14.5.3, p.636 / PDF p.573)
U14-109	한 블록씩 따라가는 방식보다 여러 가용 주소를 쉽게 확보한다.	(14.5.3, p.636 / PDF p.573)
U14-110	연속 블록들이 함께 할당·반환되는 경향을 이용하여 첫 주소와 연속 길이만 기록한다.	(14.5.4, p.636 / PDF p.573)
U14-111	개별 항목은 주소 하나보다 크지만 대개 길이가 1보다 커 전체 목록이 짧아진다.	(14.5.4, p.636 / PDF p.573)
U14-112	효율적인 검색·삽입·삭제를 위해 균형 트리로 저장할 수 있다.	(14.5.4, p.636 / PDF p.573)
U14-113	익스텐트 관리와 유사한 구조이다.	(14.5.4, p.636 / PDF p.573)
U14-114	큰 파일 시스템에서 산재한 블록을 반환할 때 비트맵 여러 블록을 갱신하는 비용이 커질 수 있다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-115	ZFS는 장치를 메타슬랩으로 나누고 각 영역의 공간맵에 할당·반환 활동을 계수 형식으로 시간순 기록한다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-116	필요한 메타슬랩의 공간맵을 오프셋으로 색인된 메모리의 균형 트리로 적재한다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-117	로그를 재실행하여 현재의 할당·가용 상태를 반영한다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-118	인접 가용 구간을 결합하여 항목을 압축한다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-119	트랜잭션 작업의 일부로 장치의 공간 정보를 갱신한다. 처리 중 새 요청은 로그를 통해 충족시킬 수 있다.	(14.5.5, p.636-637 / PDF p.573-574)
U14-120	로그와 균형 트리가 함께 가용 리스트 역할을 수행하여 구조 크기와 I/O를 제어한다.	(14.5.5, p.636-637 / PDF p.573-574)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-121	HDD는 해제된 블록을 다음 사용 때 덮어쓰면 되지만 NAND는 먼저 큰 블록을 지워야 한다.	(14.5.6, p.637 / PDF p.574)
U14-122	파일 시스템이 어느 페이지가 더 이상 사용되지 않는지 장치에 알려야 미리 가비지 수집·삭제하여 쓰기 가능한 공간을 유지할 수 있다.	(14.5.6, p.637 / PDF p.574)
U14-123	원문은 ATA의 TRIM, NVMe의 unallocate 명령을 제시한다.	(14.5.6, p.637 / PDF p.574)
U14-124	이 기능이 없으면 거의 가득 찼을 때 쓰기 중 정리가 집중되어 성능이 급락하는 쓰기 절벽(write cliff)이 생길 수 있다.	(14.5.6, p.637 / PDF p.574)
U14-125	공간 효율과 성능은 함께 판단한다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-126	inode 선할당은 공간을 쓰지만 데이터 근처에 분산하면 탐색을 줄인다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-127	큰 클러스터는 전송에 유리하지만 내부 단편화가 커, 작은 파일·마지막 부분에 작은 클러스터를 쓰는 변형이 있다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-128	마지막 접근 시각을 유지하려면 읽기 때도 메타데이터 기록이 필요하여 각 속성의 실제 가치와 비용을 고려해야 한다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-129	큰 포인터는 표현 가능한 파일·공간을 늘리지만 색인·가용 공간 구조의 크기도 키운다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-130	고정 크기 테이블은 가득 차면 자원이 남아도 새 프로세스·파일을 만들지 못하고 크기 변경에 재부팅·재컴파일이 필요할 수 있다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-131	동적 할당은 이런 인위적 제한을 줄이지만 관리 알고리즘이 복잡해지고 할당·반환 비용이 생긴다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-132	원문의 과거 MS-DOS·Solaris 사례는 초기 구조 선택이 향후 확장에 미치는 영향을 보여준다.	(14.6.1, p.637-639 / PDF p.574-576)
U14-133	컨트롤러 캐시는 트랙·블록을 보관하고 운영체제는 메모리에 다시 캐시한다.	(14.6.2, p.639-642 / PDF p.576-579)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-134	페이지 캐시는 가상 메모리의 페이지로 파일 데이터를 관리하고 프로세스 페이지와 함께 이용할 수 있다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-135	메모리 사상 접근은 페이지 캐시, read/write는 별도 버퍼 캐시를 쓰면 동일 데이터가 두 번 복사되는 이중 캐싱이 생겨 공간·CPU·일관성 비용이 든다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-136	통합 버퍼 캐시는 두 접근 경로가 같은 캐시를 사용하게 한다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-137	그림 14.11 통합 버퍼 캐시를 사용하는 입출력 — 원문 그림 부분 발췌	(14.6.2, p.641 / PDF p.578)
U14-138	프로세스 메모리와 파일 캐시가 가용 메모리를 공유하면 한쪽이 다른 쪽을 밀어낼 수 있어 교체 정책과 경계를 조정한다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-139	원문의 Solaris 발전은 단순한 범용 정책만으로 모든 부하를 만족시키기 어렵다는 사례이다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-140	동기식 쓰기	(14.6.2, p.639-642 / PDF p.576-579)
U14-141	저장장치에 도달한 뒤 진행한다. 메타데이터·순서 보장이 필요한 작업에 사용한다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-142	비동기식 쓰기	(14.6.2, p.639-642 / PDF p.576-579)
U14-143	캐시에 쓰고 먼저 복귀한다. 모아서 스케줄할 수 있지만 실제 매체 기록은 뒤에 이루어진다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-144	free-behind	(14.6.2, p.639-642 / PDF p.576-579)
U14-145	순차 접근에서 다음 페이지를 요청하면 이전 페이지를 버퍼에서 제거한다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-146	read-ahead	(14.6.2, p.639-642 / PDF p.576-579)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-147	요청 페이지와 이어질 페이지를 한 번에 읽어 미리 캐시한다. 순차 접근에서 이후 요청 가능성을 이용한다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-148	순차 파일은 최근 페이지를 다시 쓸 가능성이 낮아 일반 LRU가 적합하지 않을 수 있다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-149	작은 비동기 출력은 캐싱 덕분에 입력보다 빠르게 보일 수 있지만 파일이 캐시 용량보다 크면 결국 장치 속도에 제약된다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-150	따라서 캐시가 대량 연속 I/O의 물리 병목을 없애는 것은 아니다.	(14.6.2, p.639-642 / PDF p.576-579)
U14-151	파일 생성 같은 작업은 디렉터리·FCB·데이터·가용 계수 등 여러 구조를 바꾼다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-152	일부만 기록되고 크래시가 나면 메모리와 장치 또는 자료구조 사이가 불일치한다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-153	일관성 검사기는 디렉터리와 메타데이터를 실제 할당 상태와 비교하여 복구한다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-154	전체 검사는 오래 걸리므로 갱신 중 상태 비트를 설정하고 완료 후 소거하여 필요할 때 검사할 수 있다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-155	복구 가능성은 할당 구조에 좌우된다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-156	원문은 데이터 블록의 링크가 있으면 재구성이 가능하지만 서로 관계를 모르는 색인 데이터 블록은 디렉터리 정보를 잃었을 때 어렵다고 설명한다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-157	메타데이터의 동기식 기록과 전원 보호 장치 버퍼는 일부 위험을 줄여도 크래시 도중 부분 갱신을 모두 없애지는 못한다.	(14.7.1, p.642-643 / PDF p.579-580)
U14-158	파일 시스템 메타데이터 변경을 로그에 순차 기록하고 한 작업의 연산 집합을 트랜잭션으로 묶는다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-159	로그 기록이 확약되면 호출자에게 복귀할 수 있다. 이후 실제 자료구조에 변경을 재실행하고 진행·완료를 기록한다.	(14.7.2, p.643-644 / PDF p.580-581)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-160	크래시 뒤에는 확약되었지만 미완료인 트랜잭션을 끝까지 적용한다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-161	확약 전에 중단된 트랜잭션의 변경은 원상 복구하여 일관성을 회복한다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-162	로그는 원형 버퍼로 구현될 수 있으므로 아직 필요한 정보를 새 기록으로 덮지 않게 관리한다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-163	전체 메타데이터를 검사·수리하는 대신 필요한 로그를 처리하므로 복구의 지연·불확실성·사람의 개입을 줄인다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-164	HDD에서는 동기식 임의 메타데이터 쓰기를 순차 로그 쓰기로 바꾸고 실제 구조 갱신을 비동기로 수행하여 성능도 높일 수 있다.	(14.7.2, p.643-644 / PDF p.580-581)
U14-165	WAFL과 ZFS의 다른 접근은 기존 블록을 새 내용으로 덮지 않고 데이터·메타데이터 변경을 새 블록에 기록하는 것이다.	(14.7.3, p.644-645 / PDF p.581-582)
U14-166	트랜잭션이 끝나면 포인터를 새 블록으로 바꾸고 옛 블록을 회수한다.	(14.7.3, p.644-645 / PDF p.581-582)
U14-167	옛 포인터·블록을 유지하면 특정 시점의 스냅숏이 된다.	(14.7.3, p.644-645 / PDF p.581-582)
U14-168	포인터 갱신의 정확성이 중요하며 WAFL도 일부 메타데이터 오염에 대응하는 검사기를 가진다고 원문은 설명한다.	(14.7.3, p.644-645 / PDF p.581-582)
U14-169	ZFS는 여기에 데이터·메타데이터 체크섬과 RAID를 결합하는 방식으로 소개된다.	(14.7.3, p.644-645 / PDF p.581-582)
U14-170	첫날 전체 파일을 전체 백업한다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-171	다음 날부터 직전 백업 이후 바뀐 파일만 점증적으로 백업한다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-172	일정 주기가 지나면 다시 전체 백업으로 돌아간다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-173	복원은 전체 백업에서 시작하여 이후 필요한 점증 백업을 차례로 적용한다.	(14.7.4, p.645-646 / PDF p.582-583)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-174	주기가 길면 백업량을 줄일 수 있지만 복원에 읽을 매체가 많아진다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-175	중간 전체 백업은 복원 부담을 줄이는 대신 저장 공간을 더 쓴다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-176	손상 발견이 늦을 수 있어 재사용하지 않는 영구 백업과 다른 장소의 보관도 필요할 수 있다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-177	백업은 시스템 전체 고장뿐 아니라 실수로 삭제한 파일의 복구에도 이용된다.	(14.7.4, p.645-646 / PDF p.582-583)
U14-178	WAFL(write-anywhere file layout)은 원문의 NetApp 파일 서버와 NVRAM 쓰기 캐시·랜덤 I/O 부하에 맞춰 설계되었다.	(14.8, p.646-650 / PDF p.583-587)
U14-179	블록 기반이며 inode를 사용하고, 루트 inode 아래에 inode 파일·가용 블록 맵·가용 inode 맵 등을 모두 파일로 둔다.	(14.8, p.646-650 / PDF p.583-587)
U14-180	메타데이터도 일반 파일이므로 고정 위치에 묶이지 않고 여러 디스크로 확장할 때 함께 커질 수 있다.	(14.8, p.646-650 / PDF p.583-587)
U14-181	루트 inode를 복사하여 특정 시점의 포인터를 남긴다.	(14.8, p.646-650 / PDF p.583-587)
U14-182	이후 변경은 기존 블록을 덮지 않고 새 공간에 기록한다. 현재 루트는 새 블록을 가리키고 스냅숏은 옛 블록을 계속 가리킨다.	(14.8, p.646-650 / PDF p.583-587)
U14-183	추가 맵 비트는 어느 스냅숏이 블록을 쓰는지 추적한다. 그 블록을 쓰는 모든 스냅숏이 없어지면 재사용할 수 있다.	(14.8, p.646-650 / PDF p.583-587)
U14-184	그림 14.13 WAFL의 스냅숏 — 원문 그림 부분 발췌	(14.8, p.648 / PDF p.585)
U14-185	스냅숏은 전체 파일을 복사하지 않아 적은 공간으로 여러 시점을 보관하고 백업·테스트·디버깅에 쓸 수 있다.	(14.8, p.646-650 / PDF p.583-587)
U14-186	클론은 읽기·쓰기 스냅숏으로 원본 스냅숏을 참조하다 변경을 새 블록에 기록한다.	(14.8, p.646-650 / PDF p.583-587)
U14-187	원본을 유지하므로 테스트·업그레이드 실패 때 클론만 제거할 수 있고, 필요하면 원본을 대신하도록 승격할 수 있다.	(14.8, p.646-650 / PDF p.583-587)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-188	원격 복제는 처음 스냅숏을 전송한 뒤 이후 스냅숏 사이에 바뀐 블록과 포인터를 적용하여 다른 시스템의 복사본을 유지한다.	(14.8, p.646-650 / PDF p.583-587)
U14-189	원문의 별도 APFS 설명은 공간 공유, 복제·스냅숏, 쓰기 시 복사, 암호화, 디렉터리 크기 관리, 원자적 안전 저장, I/O 병합을 소개한다.	(14.8, p.646-650 / PDF p.583-587)
U14-190	컨테이너의 가용 공간을 여러 파일 시스템이 공유하고, 작은 쓰기를 큰 쓰기로 묶어 NVM 성능을 돕는 관계를 설명한다.	(14.8, p.646-650 / PDF p.583-587)
U14-191	이 기능 목록의 상세 동작을 외부 지식으로 보충하지 않는다.	(14.8, p.646-650 / PDF p.583-587)
U14-192	원문 요약은 계층·할당·가용 공간·디렉터리와 안정성의 관계를 다시 정리한다.	(14.9, p.650-653 / PDF p.587-590)
U14-193	연습문제는 블록 추가·제거에 따른 할당 방식별 I/O, 비트맵의 저장, 할당 선택 기준, 확장 영역, 캐시의 한계, 동적 테이블의 이익·비용을 다룬다.	(14.9, p.650-653 / PDF p.587-590)
U14-194	원문에 없는 횟수 계산이나 풀이를 생성하지 않았다.	(14.9, p.650-653 / PDF p.587-590)
U14-195	추가 자료와 참고문헌의 존재를 확인하고 외부 설명은 사용하지 않았다.	(14.9, p.650-653 / PDF p.587-590)
U14-196	연속	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-197	빠른 순차·직접 접근	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-198	외부 단편화·성장·선할당	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-199	연결	(세 가지 파일 할당, p.625-633 / PDF p.562-570)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-200	동적 확장·외부 단편화 회피	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-201	직접 접근·포인터 비용과 손상	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-202	색인	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-203	외부 단편화 없이 직접 접근	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-204	색인 공간·색인 접근 I/O	(세 가지 파일 할당, p.625-633 / PDF p.562-570)
U14-205	일관성 검사	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-206	현재 구조를 검사해 불일치를 복구한다.	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-207	로그 기반	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-208	확약 로그의 재실행·미확약 변경의 복구를 수행한다.	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-209	새 블록 기록·스냅숏	(원문의 복구 방식, p.642-646 / PDF p.579-583)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-210	옛 내용은 남기고 포인터를 새 내용으로 전환한다.	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-211	백업 복원	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-212	다른 매체의 전체·점증 사본에서 내용을 되살린다.	(원문의 복구 방식, p.642-646 / PDF p.579-583)
U14-213	연속 할당은 각 파일이 저장장치의 연속적인 공간을 차지하도록 한다.	(14.4.1, p.625 / PDF p.562)
U14-214	익스텐트는 연속된 블록의 집합이며, 연속 할당에서 초기 공간이 충분하지 않을 때 추가로 할당되는 단위이다.	(14.4.1, p.627 / PDF p.564)
U14-215	연결 할당은 파일을 저장장치 블록의 연결 리스트로 구성하며, 블록들은 장치에 흩어져 있을 수 있다.	(14.4.2, p.627 / PDF p.564)
U14-216	FAT(파일 할당 테이블)는 각 파티션의 시작 부분에 두며, 블록 번호로 색인되는 항목이 파일의 다음 블록 번호를 갖는 테이블이다.	(14.4.2, p.629-630 / PDF p.566-567)
U14-217	색인 할당은 모든 포인터를 색인 블록에 모아 파일 블록 주소의 배열로 관리하며, i번째 항목은 파일의 i번째 블록을 가리킨다.	(14.4.3, p.630 / PDF p.567)
U14-218	비트 벡터는 각 블록을 한 비트로 나타내며, 이 원문에서는 1이 가용 블록이고 0이 할당된 블록이다.	(14.5.1, p.634 / PDF p.571)
U14-219	저장장치를 파티션으로 나누면 단일 장치에서 다양한 파일 시스템을 사용할 수 있고, 파일 시스템은 논리적 파일 시스템 아키텍처에 마운트하여 사용한다.	(14.9, p.650 / PDF p.587)
U14-220	연결 할당의 직접 접근은 비효율적이고 색인 할당은 색인 블록 오버헤드가 필요할 수 있으며, 연속 할당은 익스텐트로 확장할 수 있다.	(14.9, p.650 / PDF p.587)
U14-221	일관성 검사기는 손상된 파일 시스템 구조를 복구하고, 백업 도구로 복사한 데이터는 하드웨어·운영체제·사용자 오류에 따른 데이터 또는 장치 손실의 복구에 사용된다.	(14.9, p.650-651 / PDF p.587-588)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-222	연속 할당은 연속 공간, 연결 할당은 블록 연결 리스트, 색인 할당은 색인 블록의 주소 배열로 관리한다.	(14.4.1, p.625,627,630 / PDF p.562,564,567)
U14-223	연속 할당	(14.4.1, p.625 / PDF p.562)
U14-224	연결 할당	(14.4.2, p.627 / PDF p.564)
U14-225	색인 할당	(14.4.3, p.630 / PDF p.567)
U14-226	비트 벡터의 첫 가용 블록 번호: (워드의 비트 수) × (값이 0인 워드의 수) + 첫 번째 1비트의 변위; 처음으로 0이 아닌 워드를 찾은 뒤 그 안의 첫 1비트를 찾는다. 원문에서 가용 블록은 1이다.	(14.5.1, p.634 / PDF p.571)

제15장. 파일 시스템 내부구조

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-001	가상 파일 시스템 인터페이스는 서로 다른 파일 시스템에 대한 접근을 구분 없이 동일한 형식으로 만든다.	(15.9, p.676 / PDF p.612)
U15-002	일관성 의미는 한 사용자가 파일을 변경했을 때 다른 사용자가 그 변경을 언제 관찰할 수 있는지 명시한다.	(15.7, p.668 / PDF p.604)
U15-003	저장장치는 파티션을, 파티션·장치들의 볼륨은 파일 시스템을 보유한다.	(15.1, p.655-657 / PDF p.591-593)
U15-004	볼륨 관리자는 여러 파티션에 걸친 볼륨을 제공할 수 있다.	(15.1, p.655-657 / PDF p.591-593)
U15-005	한 컴퓨터에는 여러 종류의 파일 시스템이 함께 존재한다.	(15.1, p.655-657 / PDF p.591-593)
U15-006	범용 저장뿐 아니라 메인 메모리의 임시 tmpfs, 커널 심볼을 노출하는 objfs, 프로세스 정보를 제공하는 procfs, 다른 파일 시스템을 다시 보이게 하는 lofs 등 특수 목적도 있다.	(15.1, p.655-657 / PDF p.591-593)
U15-007	일부 가상 파일 시스템은 실제 디스크 파일이 아니라 커널 인터페이스를 파일처럼 보이게 한다.	(15.1, p.655-657 / PDF p.591-593)
U15-008	장치 이름과 기존 이름 구조에 부착할 마운트 포인트를 지정한다. 시스템에 따라 파일 시스템 유형도 지정하거나 구조를 검사하여 알아낸다.	(15.2, p.657-659 / PDF p.593-595)
U15-009	장치 드라이버를 통해 디렉터리를 읽어 유효한 파일 시스템 형식인지 확인한다.	(15.2, p.657-659 / PDF p.593-595)
U15-010	지정 지점에 마운트되었다는 정보를 기록한다. 경로 순회는 그 지점에서 알맞은 파일 시스템으로 넘어간다.	(15.2, p.657-659 / PDF p.593-595)
U15-011	마운트 전에는 새 볼륨의 파일을 기존 이름 공간에서 사용할 수 없다.	(15.2, p.657-659 / PDF p.593-595)
U15-012	원문의 /users 예는 기존 하부 구조 대신 새 볼륨의 하부 트리가 보이며 언마운트하면 원래 구조가 다시 보이는 것을 설명한다.	(15.2, p.657-659 / PDF p.593-595)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-013	비어 있지 않은 디렉터리의 마운트를 금지할지, 원래 내용을 가릴지, 같은 파일 시스템을 여러 곳에 붙일지는 시스템의 의미와 정책에 따른다.	(15.2, p.657-659 / PDF p.593-595)
U15-014	그림 15.4 /users에 마운트 된 볼륨 — 원문 그림 부분 발췌	(15.2, p.658 / PDF p.594)
U15-015	원문은 macOS의 자동 발견·마운트, Windows의 드라이브 문자와 디렉터리 마운트, UNIX의 구성 파일과 명시적 명령을 비교한다.	(15.2, p.657-659 / PDF p.593-595)
U15-016	자동 여부와 이름 표시 방식은 운영체제에 따라 달라진다.	(15.2, p.657-659 / PDF p.593-595)
U15-017	파티션은 파일 시스템을 가진 cooked 형태 또는 파일 시스템 없이 고유 형식을 사용하는 raw 형태일 수 있다.	(15.3, p.659-660 / PDF p.595-596)
U15-018	스왑·일부 데이터베이스·RAID 관리 정보가 raw 사용의 예이다.	(15.3, p.659-660 / PDF p.595-596)
U15-019	부팅할 때는 아직 일반 파일 시스템 코드가 적재되지 않았으므로 부트 정보가 별도 형식이어야 하며 부트스트랩 로더가 커널을 찾아 적재할 만큼 파일 시스템을 이해해야 한다.	(15.3, p.659-660 / PDF p.595-596)
U15-020	여러 파티션에 여러 운영체제를 둘 수 있어 부트 로더가 어느 것을 시작할지 선택한다.	(15.3, p.659-660 / PDF p.595-596)
U15-021	로더가 이해하지 못하는 파일 시스템의 운영체제는 시작할 수 없어 루트 파일 시스템 선택에 제약이 된다.	(15.3, p.659-660 / PDF p.595-596)
U15-022	루트 파티션은 부팅 때 마운트하고 다른 파티션은 자동 또는 나중에 수동으로 마운트한다.	(15.3, p.659-660 / PDF p.595-596)
U15-023	마운트 테이블은 유형과 장치를 연결한다.	(15.3, p.659-660 / PDF p.595-596)
U15-024	원문의 Windows 예는 드라이브 문자에 파일 시스템 포인터를 연관시킨다.	(15.3, p.659-660 / PDF p.595-596)
U15-025	UNIX는 마운트 포인트 디렉터리의 메모리 inode에 플래그와 마운트 테이블 포인터를 두고 그 항목이 해당 파일 시스템의 슈퍼블록을 가리킨다.	(15.3, p.659-660 / PDF p.595-596)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-026	경로 순회는 이 정보를 통해 파일 시스템을 전환한다.	(15.3, p.659-660 / PDF p.595-596)
U15-027	공유는 공동 작업을 돕지만 이름·보호·상충되는 쓰기를 조정해야 한다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-028	파일·디렉터리의 소유자와 그룹 ID를 저장하고 요청한 사용자의 ID·그룹 ID와 비교하여 연산을 허용·거부한다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-029	소유자는 속성·허가를 제어하며 그룹은 공유 사용자 집합을 정한다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-030	로컬에서 여러 파일 시스템을 마운트할 때도 ID와 허가를 검사한다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-031	이동식 장치를 다른 컴퓨터로 가져가면 ID가 다른 사용자를 가리킬 수 있으므로 ID가 일치하거나 소유권이 적절히 재설정되도록 해야 한다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-032	원격 시스템에서도 같은 신원 대응 문제가 확장된다.	(15.4.1, p.660-661 / PDF p.596-597)
U15-033	VFS(virtual file system)는 기본 시스템 콜을 구체적 파일 시스템 구현에서 분리한다.	(15.5, p.662-664 / PDF p.598-600)
U15-034	위에는 open·read·write·close와 디스크립터의 인터페이스, 가운데는 VFS, 아래에는 유형별 로컬 파일 시스템 또는 원격 프로토콜 구현을 둔다.	(15.5, p.662-664 / PDF p.598-600)
U15-035	서로 다른 구현이 함께 존재하면서 사용자는 통합 이름 공간을 탐색할 수 있다.	(15.5, p.662-664 / PDF p.598-600)
U15-036	그림 15.5 가상 파일 시스템의 개략적 모습 — 원문 그림 부분 발췌	(15.5, p.663 / PDF p.599)
U15-037	VFS의 vnode는 활성 파일·디렉터리를 나타내며 원문은 개별 파일 시스템 안에서 유일한 inode와 네트워크에서 파일을 구별해야 하는 vnode의 역할을 구분한다.	(15.5, p.662-664 / PDF p.598-600)
U15-038	로컬 요청은 해당 파일 시스템 함수로, 원격 요청은 NFS 등의 프로토콜로 보낸다.	(15.5, p.662-664 / PDF p.598-600)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-039	객체의 연산 함수 테이블이 실제 구현의 주소를 담으므로 상위 층은 적절한 함수를 선택한다.	(15.5, p.662-664 / PDF p.598-600)
U15-040	Inode object	(15.5, p.662-664 / PDF p.598-600)
U15-041	각 파일	(15.5, p.662-664 / PDF p.598-600)
U15-042	File object	(15.5, p.662-664 / PDF p.598-600)
U15-043	열린 파일	(15.5, p.662-664 / PDF p.598-600)
U15-044	Superblock object	(15.5, p.662-664 / PDF p.598-600)
U15-045	전체 파일 시스템	(15.5, p.662-664 / PDF p.598-600)
U15-046	Dentry object	(15.5, p.662-664 / PDF p.598-600)
U15-047	각 디렉터리 항목	(15.5, p.662-664 / PDF p.598-600)
U15-048	파일 객체의 함수 집합에는 open·close·read·write·mmap 등이 있다.	(15.5, p.662-664 / PDF p.598-600)
U15-049	VFS는 파일의 구체적 종류마다 처리 코드를 직접 중복하기보다 해당 객체의 테이블에서 연산을 찾아 호출한다.	(15.5, p.662-664 / PDF p.598-600)
U15-050	원격 공유의 방법으로 사용자가 ftp 등으로 직접 전송하는 방식, 원격 디렉터리를 통합하여 접근하는 DFS(분산 파일 시스템), 웹·클라우드를 통한 접근이 소개된다.	(15.6, p.664 / PDF p.600)
U15-051	익명 전송은 서버 계정 없이 가능할 수 있으나 DFS는 클라이언트와 서버의 더 긴밀한 통합을 요구하므로 복잡성이 커진다.	(15.6, p.664 / PDF p.600)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-052	파일을 가진 컴퓨터는 서버, 접근하려는 컴퓨터는 클라이언트이다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-053	서버는 제공하는 볼륨·디렉터리와 허용 클라이언트를 명시하며 다수 서버·다수 클라이언트 관계가 가능하다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-054	한 컴퓨터가 동시에 양쪽 역할을 할 수 있다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-055	네트워크 이름·IP만의 식별은 스푸핑에 취약하다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-056	암호화된 키를 쓰는 인증도 알고리즘 호환과 안전한 키 교환이 필요하다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-057	원문의 UNIX/NFS 사례에서는 클라이언트·서버의 사용자 ID가 일치해야 파일 접근 검사도 올바르다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-058	같은 사용자가 양쪽에서 서로 다른 ID를 가지면 서버가 다른 사람의 권한을 검사할 수 있다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-059	마운트 뒤 파일 연산은 사용자 신원과 함께 프로토콜로 전달되고 서버가 권한을 판단한다.	(15.6.1, p.664-665 / PDF p.600-601)
U15-060	분산 정보·네이밍 서비스는 원격 컴퓨팅에 필요한 정보를 통일된 방식으로 제공한다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-061	DNS는 호스트 이름을 네트워크 주소로 변환한다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-062	사용자 이름·암호·UID·GID·프린터 등의 정보도 중앙에서 관리할 수 있다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-063	원문의 NIS는 정보 집중을 제공하지만 인증의 안전성에 문제가 있고 NIS+는 더 안전하되 복잡한 사례이다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-064	CIFS의 네트워크 로그인과 Active Directory, Kerberos 방식 인증, LDAP가 원문의 다른 사례이다.	(15.6.2, p.665-666 / PDF p.601-602)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-065	통일된 사용자·자원 이름 공간은 시스템마다 흩어진 정보를 합쳐 관리 부담을 줄이고 한 번의 인증으로 조직 자원에 접근하는 단일 사인온을 가능하게 한다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-066	이 설명은 원문의 서비스 관계에 한정하며 새로운 구성 절차를 추가하지 않는다.	(15.6.2, p.665-666 / PDF p.601-602)
U15-067	로컬 시스템도 매체·컨트롤러·메타데이터·케이블·관리 실수로 실패할 수 있다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-068	원격 시스템은 여기에 네트워크 고장·잘못된 구성·서버 중단·분할 등이 더해진다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-069	서버를 잃었을 때 요청을 종료할지 재개까지 미룰지는 프로토콜의 오류 의미이다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-070	즉시 종료하면 데이터 손실이 생길 수 있어 많은 DFS는 기다리거나 연기하는 방식을 제공한다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-071	상태성 방식은 열린 파일·활동을 양쪽이 기억하여 복구에 이용한다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-072	원문의 NFS 버전 3 설명은 각 요청에 필요한 정보를 담는 무상태 방식을 채택하여 구현과 재시작을 단순하게 하지만 신원·거짓 요청의 보안 문제가 남는다고 설명한다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-073	버전 4는 보안·성능·기능 개선을 위해 상태를 도입한 것으로 소개된다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-074	뒤의 상세 NFS 절 역시 책이 명시한 버전 3 중심의 범위를 유지한다.	(15.6.3, p.666-667 / PDF p.602-603)
U15-075	일관성 의미는 공유 파일의 한 사용자 변경이 다른 사용자에게 언제 보이는지 규정한다.	(15.7, p.668 / PDF p.604)
U15-076	파일 시스템 코드로 구현되며 프로세스 동기화와 관련된다.	(15.7, p.668 / PDF p.604)
U15-077	그러나 디스크·네트워크의 지연 때문에 복잡한 원자 연산을 모두 제공하면 성능 비용이 크다.	(15.7, p.668 / PDF p.604)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-078	원문의 비교는 open과 close 사이의 접근을 하나의 파일 세션으로 가정한다.	(15.7, p.668 / PDF p.604)
U15-079	열린 파일에 대한 쓰기는 같은 파일을 연 다른 사용자에게 즉시 보일 수 있다.	(15.7.1, p.668 / PDF p.604)
U15-080	현재 위치 포인터를 공유하는 모드에서는 한 사용자의 위치 전진이 모든 공유자에 영향을 준다.	(15.7.1, p.668 / PDF p.604)
U15-081	파일은 접근이 서로 엇갈려 적용되는 단일 물리 이미지를 가지며 이 배타적 자원에 대한 경쟁은 대기를 낳을 수 있다.	(15.7.1, p.668 / PDF p.604)
U15-082	원문의 Andrew/OpenAFS 세션 의미에서는 한 사용자의 쓰기가 동시에 열어 둔 다른 사용자에게 즉시 보이지 않는다.	(15.7.2, p.668-669 / PDF p.604-605)
U15-083	닫은 뒤에 시작되는 세션에서 변경이 보이고 이미 열린 인스턴스에는 반영되지 않는다.	(15.7.2, p.668-669 / PDF p.604-605)
U15-084	한 파일에 일시적으로 서로 다른 이미지가 존재할 수 있어 여러 사용자가 각 이미지에서 병행하여 읽고 쓸 수 있다.	(15.7.2, p.668-669 / PDF p.604-605)
U15-085	공유된다고 선언한 파일은 더 이상 변경하지 못한다.	(15.7.3, p.669 / PDF p.605)
U15-086	파일 이름도 재사용하지 않고 내용도 바꾸지 않아 이름이 고정된 내용을 가리킨다.	(15.7.3, p.669 / PDF p.605)
U15-087	공유 규칙이 읽기 전용으로 명확하여 분산 환경에서 구현이 단순하다.	(15.7.3, p.669 / PDF p.605)
U15-088	NFS는 원격 파일 접근의 소프트웨어 구현과 명세를 뜻한다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-089	원문은 Solaris 구현의 예와 일반 명세 설명을 함께 사용하며 버전 3을 중심으로 다룬다고 명시한다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-090	책의 “최신 버전”·배포 빈도 설명을 현재 상황으로 갱신하지 않는다.	(15.8.1, p.669-671 / PDF p.605-607)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-091	독립된 시스템들이 클라이언트·서버 관계로 투명하게 파일을 공유하는 것이 목표이며 서버 고장은 관련 클라이언트에 영향을 준다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-092	마운트 명령에는 원격 호스트·디렉터리를 명시해야 하지만 완료 뒤에는 로컬 하부 트리처럼 투명하게 접근한다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-093	클라이언트의 기존 마운트 포인트 내용은 가려진다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-094	일부 구현은 원격 마운트 안에 다른 시스템을 다시 마운트하는 연속 마운트를 허용한다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-095	그러나 서버가 이미 마운트한 다른 원격 파일 시스템까지 자동으로 보이는 이행성은 없다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-096	각 기계는 자신이 수행한 마운트에 의해서만 영향을 받는다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-097	그림 15.7 NFS에서의 마운팅 — 원문 그림 부분 발췌	(15.8.1, p.671 / PDF p.607)
U15-098	서로 다른 하드웨어·운영체제·네트워크를 지원하기 위해 RPC와 XDR(외부 자료 표현)을 이용한다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-099	서비스는 초기 연결의 마운트 프로토콜과 실제 원격 연산의 NFS 프로토콜로 나뉜다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-100	공유 홈 디렉터리를 여러 기계에서 마운트하면 사용자 이동성도 제공할 수 있다.	(15.8.1, p.669-671 / PDF p.605-607)
U15-101	클라이언트가 서버 이름과 원격 디렉터리를 지정한 마운트 요구를 RPC로 보낸다.	(15.8.2, p.672 / PDF p.608)
U15-102	서버는 수출 리스트의 디렉터리·허가된 기계·접근 권리를 확인한다.	(15.8.2, p.672 / PDF p.608)
U15-103	허용하면 파일 시스템 식별자와 디렉터리 inode 번호 등 접근에 필요한 정보를 담은 파일 핸들을 반환한다.	(15.8.2, p.672 / PDF p.608)
U15-104	클라이언트는 이 핸들을 이후 원격 파일 접근에 사용한다.	(15.8.2, p.672 / PDF p.608)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-105	서버는 관리용 클라이언트·마운트 목록도 유지할 수 있으나 이는 실제 파일 연산의 정확성을 위한 상태와 다르다.	(15.8.2, p.672 / PDF p.608)
U15-106	마운트 구성은 부팅 때 정할 수 있고 변경도 가능하며 언마운트·수출 리스트 조회 절차도 제공된다.	(15.8.2, p.672 / PDF p.608)
U15-107	NFS 프로토콜은 디렉터리 검색·항목 읽기·링크/디렉터리 조작·속성 접근·파일 읽기/쓰기의 RPC를 제공한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-108	원문이 설명하는 무상태 서비스에는 open·close가 의도적으로 없다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-109	각 요청이 고유 파일 식별자와 절대 오프셋 등 필요한 매개변수를 모두 가져야 한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-110	재시도에서 같은 연산을 여러 번 적용해도 한 번 적용한 결과와 같도록 멱등성을 고려하고 일련번호로 중복·누락을 구별한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-111	마운트의 관리 목록은 크래시 뒤 복원하지 않아도 정확한 연산이 가능하므로 무상태 원칙과 모순되는 필수 상태로 보지 않는다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-112	원문은 클라이언트에 결과를 돌려주기 전에 서버의 수정 데이터·간접/상태 블록이 안전한 저장장치에 기록되어야 한다고 설명한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-113	동기 기록의 비용을 줄이기 위해 배터리로 보호된 비휘발성 캐시가 완료를 먼저 보장하고 나중에 디스크에 기록할 수 있다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-114	하나의 NFS 쓰기 RPC는 다른 쓰기와 섞이지 않지만 사용자 write 시스템 콜은 여러 RPC로 나뉠 수 있다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-115	따라서 전체 시스템 콜의 데이터가 다른 사용자의 쓰기와 섞일 가능성이 남는다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-116	원문은 별도의 락 서비스·협조적 접근이 필요하다고 설명한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-117	원문의 전송 크기와 프로토콜 조건은 해당 설명 범위의 예이며 현재 네트워크의 일반 제한으로 확대하지 않는다.	(15.8.3, p.672-674 / PDF p.608-610)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-118	클라이언트의 일반 시스템 콜을 vnode에 대한 VFS 연산으로 변환한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-119	VFS가 원격 파일임을 확인해 NFS 프로시저를 호출한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-120	RPC/XDR로 서버의 NFS 서비스에 전송한다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-121	서버 VFS가 로컬 파일 시스템 연산으로 처리하고 같은 경로를 되돌아 결과를 보낸다.	(15.8.3, p.672-674 / PDF p.608-610)
U15-122	그림 15.8 NFS 구조의 개략도 — 원문 그림 부분 발췌	(15.8.3, p.674 / PDF p.610)
U15-123	경로를 구성요소 이름으로 나누고 각 이름과 디렉터리 vnode 쌍에 대해 lookup한다.	(15.8.4, p.675 / PDF p.611)
U15-124	마운트 포인트를 넘은 뒤의 각 구성요소는 별도 서버 RPC를 요구할 수 있다.	(15.8.4, p.675 / PDF p.611)
U15-125	경로 전체를 한꺼번에 서버에 주면 빠를 수 있지만 클라이언트마다 다른 마운트 이름 공간과 중간의 추가 마운트를 무상태 서버가 알지 못한다는 제약이 있다.	(15.8.4, p.675 / PDF p.611)
U15-126	클라이언트의 디렉터리 이름 조회 캐시는 공통 접두어를 가진 경로의 반복 참조를 빠르게 한다.	(15.8.4, p.675 / PDF p.611)
U15-127	반환된 속성이 캐시된 vnode와 맞지 않으면 캐시를 버린다.	(15.8.4, p.675 / PDF p.611)
U15-128	클라이언트가 명시적으로 연속 마운트한 경로는 여러 서버를 지날 수 있지만 서버 자체의 다른 마운트를 자동으로 따라가는 것은 아니다.	(15.8.4, p.675 / PDF p.611)
U15-129	개념상 파일 연산과 RPC가 대응하지만 실제로는 파일 블록·속성을 지역에 캐시하여 여러 연산이 RPC 없이 처리될 수 있다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-130	원문은 열린 시점에 속성의 최신 여부를 서버에 확인하고, 속성이 최신일 때만 관련 블록 캐시를 쓰며, 속성 캐시는 일정 시간 뒤 버린다고 설명한다.	(15.8.5, p.675-676 / PDF p.611-612)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-131	미리 읽기와 지연 쓰기를 함께 사용하되 서버 기록 완료를 확인하기 전까지 지연 쓰기 블록을 해제하지 않는다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-132	원문의 수치 사례에서 속성 캐시는 기본 60초 뒤 버리며, 한 기계에서 만든 새 파일은 다른 기계에서 30초 동안 보이지 않을 수 있다고 설명한다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-133	이 수치는 본문의 NFS 구현·성능 조정 사례를 그대로 기록한 것이며 모든 버전과 설정의 공통값이라는 의미로 사용하지 않는다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-134	캐시와 지연 쓰기로 인해 다른 기계의 새 파일·수정이 곧바로 보이지 않을 수 있다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-135	이미 열린 파일에서 보이는 내용과 새 open에서 보이는 내용도 구별해야 한다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-136	따라서 원문의 NFS는 엄격한 UNIX 의미도 Andrew의 세션 의미도 그대로 제공하지 않는다고 결론짓는다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-137	성능을 위한 조정이 일관성 의미에 미치는 영향이 핵심이다.	(15.8.5, p.675-676 / PDF p.611-612)
U15-138	다양한 파일 시스템은 마운트와 VFS를 통해 통합되고, 파일 공유에는 신원·권한·일관성 의미가 필요하다.	(15.9, p.676-678 / PDF p.612-614)
U15-139	원격 공유는 통신·인증·고장·경로 변환과 캐시 일관성까지 고려한다.	(15.9, p.676-678 / PDF p.612-614)
U15-140	장말 문제는 VFS의 역할, 여러 파일 시스템 유형의 필요, procfs, 이름 공간 통합과 별도 표시, ftp와 NFS의 차이를 점검한다.	(15.9, p.676-678 / PDF p.612-614)
U15-141	추가 자료·참고문헌은 확인했지만 원문에 없는 실습 명령이나 풀이를 작성하지 않았다.	(15.9, p.676-678 / PDF p.612-614)
U15-142	UNIX	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-143	같이 열린 파일의 다른 사용자에게 즉시 보일 수 있다. 단일 이미지이다.	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)
U15-144	세션	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)
U15-145	닫은 이후 새로 시작한 세션에서 보이며 기존 열린 인스턴스에는 반영되지 않는다.	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)
U15-146	불변 공유	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)
U15-147	공유 선언 후 내용 변경·이름 재사용이 없다.	(원문의 파일 공유 일관성 의미, p.668-669 / PDF p.604-605)
U15-148	마운트 포인트는 파일 시스템을 부착할 기존 파일 구조의 위치이며, 일반적으로 빈 디렉터리이다.	(15.2, p.657 / PDF p.593)
U15-149	vnode는 전체 네트워크에서 유일한 수치 지정자를 포함하는 파일 표현 구조로, 네트워크 파일 시스템에 필요한 파일 구별을 제공한다.	(15.5, p.662 / PDF p.598)
U15-150	파일 세션은 open()과 close() 사이의 파일 접근이다.	(15.7, p.668 / PDF p.604)
U15-151	불변 공유 파일은 공유된다고 선언한 이후 변경할 수 없는 파일이며, 파일 이름은 재사용될 수 없고 내용도 변경할 수 없다.	(15.7.3, p.669 / PDF p.605)
U15-152	NFS는 LAN 또는 WAN을 통한 원격 파일 접근용 소프트웨어 시스템 구현과 명세를 모두 가리킨다.	(15.8, p.669 / PDF p.605)
U15-153	범용 운영체제는 특수 목적에서 일반까지 다양한 파일 시스템 유형을 제공한다.	(15.9, p.676 / PDF p.612)
U15-154	파일 시스템이 포함된 볼륨은 컴퓨터의 파일 시스템 공간에 마운트될 수 있다.	(15.9, p.676 / PDF p.612)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-155	다중 사용자 시스템에는 파일 공유와 보호 방법이 필요하며, 파일과 디렉터리에는 소유자·사용자·그룹 접근 권한 등의 메타데이터가 포함된다.	(15.9, p.676 / PDF p.612)
U15-156	동일 파일에 대한 동시 접근을 조정하는 일관성 모델에는 UNIX·세션·불변 공유 파일 시맨틱 등이 있다.	(15.9, p.677 / PDF p.613)
U15-157	쓰기는 같은 파일을 동시에 연 다른 사용자에게 즉시 보이지 않고, 파일이 닫힌 뒤 시작하는 세션에만 변경 내용이 보인다.	(15.7.2, p.668-669 / PDF p.604-605)
U15-158	UNIX 의미	(15.7.1, p.668 / PDF p.604)
U15-159	열린 파일에 쓴 내용은 같은 파일을 동시에 연 다른 사용자에게 즉시 보일 수 있다.	(15.7.1, p.668 / PDF p.604)
U15-160	세션 의미	(15.7.2, p.668-669 / PDF p.604-605)
U15-161	불변 공유 파일	(15.7.3, p.669 / PDF p.605)

제7부 도입. 보안과 보호

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-001	보안은 저장된 정보의 무결성과 물리 자원의 보호를 위해 시스템 사용자의 인증을 보장한다.	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-002	보호는 프로그램·프로세스·사용자가 컴퓨터 시스템의 자원에 접근하는 것을 제어하는 기법으로 제공된다.	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-003	보안은 저장된 자료와 코드의 무결성뿐 아니라 물리 자원의 보호를 위해 사용자 인증을 보장한다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-004	인가되지 않은 접근, 접근 후의 악의적인 파괴·자료 변경, 우연한 비일관성 도입을 막는 역할을 한다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-005	보호는 사용자의 파일 접근 유형을 제한해 제어된 접근을 제공한다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-006	운영체제에서 적절한 권한을 얻은 프로세스만 메모리 세그먼트·CPU 등의 자원에 작업할 수 있도록 한다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-007	프로그램·프로세스·사용자의 자원 접근을 제어하는 기법에는 부과할 제어를 지정하는 방법과 시행하는 방법이 필요하다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-008	이 페이지는 제7부의 도입이다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-009	뒤의 제16장 보안과 제17장 보호 본문은 제공되지 않아, 구체적인 위협·암호 기법·접근 행렬 등의 설명은 원문에서 확인 불가으로 남긴다.	(P7.1 도입에서 확인되는 보안과 보호의 역할, p.679 / PDF p.615)
U16-010	보안	(도입에서 구별한 보안과 보호, p.679 / PDF p.615)
U16-011	사용자 인증, 무결성과 물리 자원 보호, 인가되지 않은 접근과 파괴·변경 방지	(도입에서 구별한 보안과 보호, p.679 / PDF p.615)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-012	보호	(도입에서 구별한 보안과 보호, p.679 / PDF p.615)
U16-013	파일 접근 유형 제한과 권한에 따른 자원 접근 제어, 제어의 지정과 시행	(도입에서 구별한 보안과 보호, p.679 / PDF p.615)
U16-014	보안 시스템은 인가되지 않은 접근과 접근 이후의 악의적인 파괴·자료 변경 및 비일관성의 우연한 도입을 막는다.	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-015	보호는 파일 접근 유형을 제한하여 제어된 접근을 제공하고, 적절한 권한을 얻은 프로세스만 메모리 세그먼트·CPU 등의 자원에 작업하도록 보장한다.	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-016	보호 기법은 부과할 제어를 지정하는 방법과 시행하는 방법을 제공해야 한다.	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-017	보안	(제7부 보안과 보호 도입, p.679 / PDF p.615)
U16-018	보호	(제7부 보안과 보호 도입, p.679 / PDF p.615)